

IDM TOKENS

Written by,
**GEOFFREY
CARMAN**



*A definitive guide
to NetIQ's IDM tokens*

FIRST EDITION

Author

Geoffrey Carman

This Book has been released by Ninja-Tools, Inc. and all right reserved.

Copyright 2012

Additional information by be found at <http://www.ninja-tools.com>

Table of Contents

Conventions used in this book	10
Introduction	11
Tokens of Identity Manager	18
Noun Tokens.....	19
String Builder or Argument Builder	19
Text.....	20
Entitlement and Attribute tokens	21
Character	25
Class Name	26
Destination DN	26
Destination Name.....	28
Document	28
Generate Password	30
Global Configuration Value	31
Local Variable	33
Named Password	35
Operation.....	35
Operation Property	37
Password	38
Query	40
Resolve.....	42
Source DN.....	43
Source Name	44
Time	44
Unique Name.....	46
Unmatched Source DN	48
XPATH:.....	49
Verb Tokens.....	51
Base64 Encode and Decode.....	51
Convert Time.....	52
Escape Destination DN, Escape Source DN	54
Join and Split	55
Lowercase and Uppercase.....	56
Map.....	57
Parse DN.....	60
Replace All.....	62
Replace First.....	62
Substring.....	65
XML Parse.....	66
XML Serialize.....	66
Condition Tokens	70

If Association.....	74
If Attribute.....	75
If Destination Attribute	75
If Operation Attribute.....	75
If Source Attribute	75
If Class Name.....	78
If Destination DN.....	78
If Destination Name	78
If Source DN.....	78
If Source Name	78
If Entitlement	79
If Global Configuration Value.....	82
If Local Variable	84
If Named Password	89
If Operation.....	91
If Operation Property	93
If Password	94
If XML Attribute	94
If XPATH expression	95
Action Tokens.....	104
Add Association	104
Remove Association.....	104
Add Destination Attribute Value	108
Add Source Attribute Value	108
Set Destination Attribute Value	108
Set Source Attribute Value	108
Set Local Variable.....	114
Add Destination Object.....	119
Add Source Object.....	119
Add Role	121
Remove Role.....	121
Start Workflow	121
Append XML Element.....	124
Append XML text.....	124
Set XML attribute	124
Break.....	128
Clear Destination Attribute Value	128
Clear Source Attribute Value.....	129
Clear Operation Property	131
Clear SSO Credential	131
Set SSO Credential.....	131
Set SSO Passphrase.....	131
Clone By XPATH Expression	133
Clone Operation Attribute.....	134
Delete Destination Object.....	135
Delete Source Object.....	136
Find Matching Object.....	136
For Each.....	139
Generate Event.....	143

If.....	146
Implement Entitlement	147
Move Destination Object.....	151
Move Source Object.....	151
Reformat Operation Attribute	153
Rename Destination Object	154
Rename Source Object	154
Rename Operation Attribute.....	159
Send Email.....	161
Send Email From Template	161
Set Default Attribute Value	165
Set Destination Password.....	167
Set Source Password.....	167
Set Operation Association	169
Set Operation Class Name	170
Set Operation Destination DN.....	171
Set Operation Property.....	172
Set Operation Source DN	173
Set Operation Template DN.....	173
Status	174
Strip Operation Attribute	175
Strip By XPATH	176
Trace Message	177
Veto	180
Veto If Operational Attribute Not Available	181
While.....	182
Filter and Schema	184
Schema Map.....	184
Filter	185
Conclusions	186
Appendix A: ECMAScript functions.....	188
Function Reference	188
print(arg)	188
println(arg).....	188
getUniqueTimeString()	188
appendLog(log, append, delimiter, b64, maxsize).....	188
checkStringDiff(oldValue, newValue, orgValue, gcValue)	189
stripChars(input , validChars)	189
getDocumentFromString(xml).....	189
serialize(node, indent).....	189
getStringFromNodeSet(nodeset, indent)	190
readImage(imageFileName, wrap)	190
rollTime(time, units, unitID)	190
rollDays(time, days).....	190
rollHours(time, hours).....	190
rollSeconds(time, seconds).....	190
dayOfWeek(yyyy, mm, dd).....	191
getXmlHeader()	191
replaceLeft(name)	191

getSeconds(time, pattern).....	191
getTime(seconds, pattern).....	191
numericCompare(x, y).....	191
sortNumbersAscending(numbersIn, delimiter).....	192
moveFile(oldname, newname)	192
doWait(interval).....	192
readLineFromFile(filename, encoding)	192
makeSpecName(name)	192
getUTPTime(time).....	192
match(str, pattern)	192
octet2AsciiHex(s).....	193
guid2Association(s)	193
guid2string(s).....	193
getHostname()	193
dn2xml(dn, delimiter).....	193
Base64encode(str, encoding)	194
Base64decode(b64, encoding).....	194
execString(eStr).....	194
xml2dn(slashDN, type1, type2).....	194
NodeSetToString(nodeset)	194
fixLengthRight(inStr, size, padStr)	194
traceMsg(msg)	195
xdsQschedule(secs).....	195
interString(input)	195
getTimeString()	195
createPassword(input).....	195
createPassword2(minLength)	195
createPassword3(minLength)	195
createPassword4(passwordLength, numOfAlphabets, numOfDigits).....	195
CreatePasswd(input).....	195
sendMail(from, to, subj, msgText, host, debugging).....	196
sendMailWithAttachment(from, to, subj, msgText, file, host, debugging).....	196
sendMailC(from, to, subj, msgText, host, debugging, charSet).....	196
sendMailCWithAttachment(from, to, subj, msgText, file, host, debugging, charSet)	196
makeAscii(s0).....	196
makeAsciiAlpha(s0).....	196
writeLog(filename, msg, encoding).....	196
upperString(input).....	196
normalizeSpace(input).....	196
capitalizeWords(input)	197
lowerString(input)	197
createUsername(s1, c1, s2, c2, delimiter)	197
convertDate(inDate, inFormat, outFormatString)	197
getPrefix(input, delim)	197
getSuffix(input, delim).....	197
getSubstring(input , start, end).....	198
patternCount(input, pattern)	198
fixLength(input, len, pad)	198
replaceChar(inString, oldChar, newChar)	198

removeChar(inString, oldChar)	198
removeApostrophe(inString)	198
removeQuote(inString)	198
removeLesser(inString)	198
removeGreater(inString)	198
replaceString(inString, oldString, newString)	198
reverseString(inString, delimiter)	198
convertDateBetweenLocales(ds, style, fromLC, toLC)	199
convertTimeBetweenLocales(ts, style, fromLC, toLC)	199
convertTimeOrDate(useTime, style, ds, fromLC, toLC)	199
convertTimeDateBetweenLocales(ds, style, fromLC, toLC)	199
convertDateToDirFormat(ds, fromLC)	199
simpleDateFormat(ds, fromLC, format)	199
getPortFromHexNetAddr(hexStr)	199
getIPFromHexNetAddr(hexStr)	199
getProtocolFromURL(urlStr)	199
getPortFromURL(urlStr)	199
getIPFromURL(urlStr)	199
getEntParamField(param, field)	200
getEntParamFieldBackwardsCompatible(param, idm4field, legacyfield)	200
getIDM4EntParamField(jsonstr, field)	200
getParamValue(paramString, paramName)	200
hex2dec(hexStr)	200
endsWith(str1, str2)	200
tokenize(source, delimiter, escapeChar)	200
dump(data) internal function	200
hex(data) internal function	200
decodeAsciiHex(s) internal function	201
encodeAsciiHex(abyte0) internal function	201
Appendix B: Article reference	201
General Articles	201
OS Related	201
Netware	201
Linux	201
Windows	201
Macintosh	201
iPrint	202
Data Center	202
eDirectory	202
LDAP as a service of eDirectory	202
IDM Overview	202
IDM 4 Specific	202
IDM Engine	203
Designer Packages	203
Dstrace	204
DirXML Script Tokens	204
XPATH	204
Concepts	204
Cool tips	205

IDM Tutorials	205
Drivers	205
General.....	205
AD.....	205
AS400	206
Composer.....	206
Data Collection Service	206
Delimited Text	206
eDirectory.....	206
Google Apps.....	207
GW	207
JDBC	207
Jobs.....	207
Linux/Unix Bidir.....	207
Managed System Gateway.....	207
Notes.....	207
Password Notifier	208
Reporting.....	208
Remote Loaders	208
SAP.....	208
SAP Business Logic.....	208
SAP HR.....	208
Secret Store.....	209
SOAP	209
Student Interoperability Framework (SIF)	209
User Application	209
Work Order.....	209
Tools	209
Designer	209
iManager	210
DXCMD	210
ConsoleOne	210
Zenworks	210
ZAM	210
ZCM	210
ZfH	210

Conventions used in this book

- Token names are in italics

Token Name

- Field Names or parameters are noted in bold

Field Name

- Function Names are noted with gray background

Function Name

- Blocks of code are contained within a table

Code Block

- Attribute names are noted with gray background

Attribute name

- Program names are in italics

Program name

Introduction

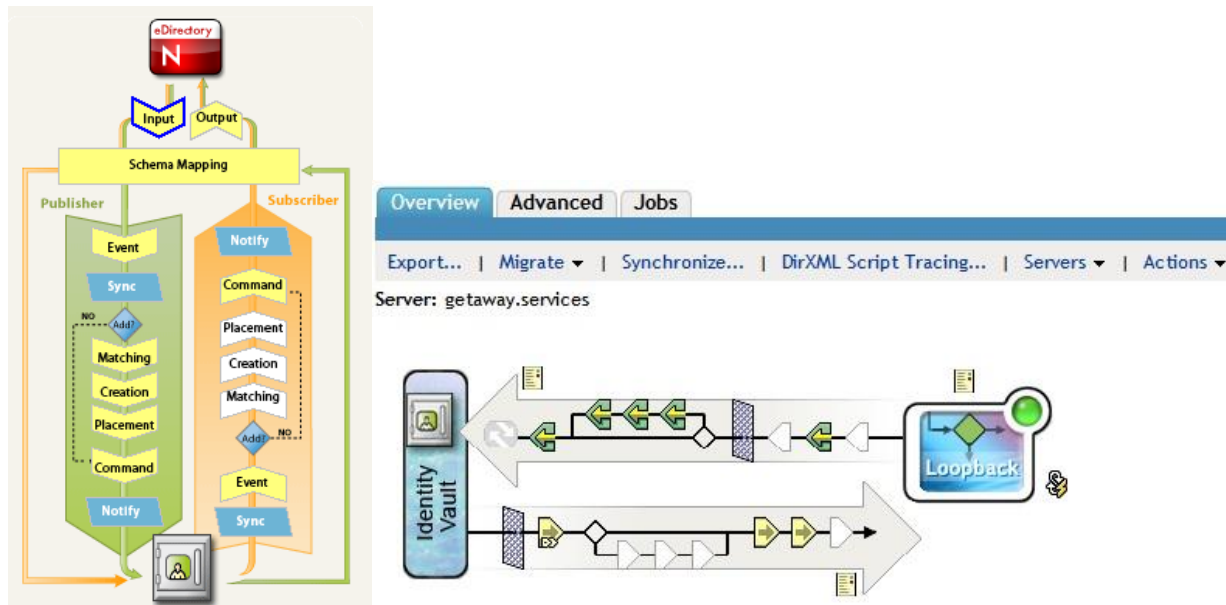
NetIQ (formerly Novell) Identity Manager is a very powerful product that uses an internal XML based language, DirXML Script, to handle identity synchronization events.

The basic design of Identity Manager is that of a hub and spoke. At the center, the hub is eDirectory acting as a Metadirectory, often called an Identity Vault. This runs the various connectors (called drivers). Coming out of the hub are spokes that connect various other systems, via drivers.

A driver can run as a Java class in the engine's JVM or in a Remote Loader. It is also possible to have native shims. (The Active Directory driver uses ADDriver.dll, a native Windows shim, which runs on a Remote Loader on a Windows server or else local on eDirectory running on a Windows server.) The shim is the program that connects to the target system. As an example, the SAP HR driver shim knows how to read iDOC files and talk BAPI (the SAP programming API) on the Publisher channel. The Sharepoint driver is a .NET application that knows how to talk to a Sharepoint server using .NET. The LDAP driver shim knows how to talk to an LDAP server.

Groups of drivers are all stored in driver sets. You can run as many driver sets as you want, but one eDirectory server can only host a single driver set. A driver set can span multiple servers and can be assigned multiple servers. But a driver can only be in a single driver set (the driver set being its parent object in the eDirectory tree). You can have two driver sets, each with ten drivers, and the ten drivers are distributed to run across three servers to distribute the load. Alas, one server cannot run drivers from two different driver sets. Generally, one driver set will suffice. Unless you are really high volume, one server will often suffice. As an example from the field, there is a large system with 80 drivers and two million objects in sync (five HR systems, 600,000 users, and over a million supporting objects) that runs on only three servers. Admittedly, large servers with eight cores each and 16GB of RAM but only three servers nonetheless.

Within each driver, there are two channels that are somewhat independent of each other, one for each direction of event flow. The Subscriber channel handles events flowing out of the Identity Vault and towards the connected system. The Publisher channel handles events coming from the connected system and flowing into the Identity Vault. This is represented in the infamous fishbone diagram you see in either iManager or Designer.



While most work needed for Identity Manager can be done in Designer or iManager, for the purposes of this book, Designer is the preferred tool. Any screen shots will be taken from Designer.

Each channel has a number of Policy Sets used at different times. Outside the channels themselves are the Input Transform (where data coming from the connected system should be normalized to eDirectory standards). For example, time/date fields might store in LDAP like format in the connected system as 201202231432Z. In eDirectory time fields are CTIME (32 bit integer storing a count of seconds since Jan 1, 1970). The Output Transform is where data coming out of eDirectory should be normalized into a format that the connected system can handle. For a variety of reasons, not worth going into right now, the Input and Output Transform policy sets are not actually part of either the Subscriber or Publisher channels. It is easy enough to think of the Input Transform as the first policy set of the Publisher channel and the Output Transform as the last step in the Subscriber channel even though it is technically incorrect.

The rough order of policy sets is Event Transform then, if not associated or an add event, Matching, Create, Placement, and finally, Command transform.

The basic idea is that the event gets into the Event Transform. Here you process event level things like blocking delete events or scoping events to a particular source container. If the document being processed does not have an association value after the Event Transform, then a modify would be considered a synthetic add. Imagine you send in a modify of email address alone. The driver checks and there is no associated object in the destination. This works much the same regardless of the channel, so the destination could be the connected system or eDirectory. In that case, you first need to build the object in order to send in the new email address value.

Thus, the modify gets converted to a synthetic add. This is done by checking the settings in the filter, specifying the attributes that are being sent in either direction, reading the attributes in play from the source directory, and building an add event with all the new stuff in it. Thus, the engine will try to create a fully complete object in the destination instead.

The first thing the engine does with a synthetic add is try to match an existing user in the destination. That is, if there is an existing object, the goal is to match it rather than create a new one. This avoids possibly causing a duplication error. Matching policies depend on each system. You might look for some unique value, perhaps uniqueID or email address, that is sufficiently unique; whereas, matching on Full Name or Given Name + Surname is a bad idea. (How many users named John Smith do you have in your organization?) Each driver needs to consider this problem specific to its data and usage, and it will differ for every implementation since there can be different amounts of data each time.

If a Match is found, the values are merged using the Filter settings. (In this case, Merge Authority as well as the settings for what synchronizes in each direction.) The source and destination are updated if needed. Perhaps nothing needs to be changed. Just the association is added to store the linkage for future events to use and save this processing time. Perhaps everything gets changed, or perhaps somewhere in between there are some changes in the source and some in the destination system that are necessary.

If there is no match, then the Create Transform policy set will process its policies. This is where criteria are set for things required to create the object. The most common requirement would be Surname for eDirectory since the eDirectory schema requires a Surname attribute to make a User object. Additionally, Active Directory really does not like making a User without a password (it is, after all, a big security hole). The Active Directory driver requires a password here before sending on the shim.

If the add event is vetoed here because it is missing an attribute, the next change on the object (perhaps adding the missing attribute) will cause another synthetic add and it will try again. Perhaps this time enough information will be available, or maybe it will take another try. It may never complete. IDM can be quite whimsical at times.

Next up is the Placement Policy set where a destination DN is generated. This is where the policies determine where to place the destination object in the connected system. Sometimes this is flat (like a Unix NIS or AS400 and simply a destination name) where the connected system has no notion of hierarchy. Sometimes this may need very complicated logic considering source container, group memberships, perhaps job code or location attribute values. Mapping tables come in really handy here. Map from the source container value to get the destination container value if there is no algorithmic way to determine it.

Now, the event rejoins the normal event flow for a modify of an associated object having finished the processing unique to add events. This is the Command Transform set where

changes needed for all types of events should be processed. From the discussion of synthetic adds, you can see that a modify in the Event Transform might change into an add. Any work done there could be thrown away. Thus, there is a class of work that should best be performed in the Command Transform rather than in the Event transform. Usually, this is changes to the data itself as opposed to the type of event as earlier.

In both channels, the filter is applied one more time to remove attributes not allowed by the filters list, notify only attributes, and allow those set to synchronize. On the Publisher channel, the event gets submitted to eDirectory and either succeeds or fails and should return a document of operation `<status level='success'>` where the level reports how the operation concluded. Did it succeed or fail. (Success, warning, error, and fatal are the possible values).

On the Subscriber side, the Schema Map rule is applied and converts the attributes and class names to the application naming standard.

Finally, the event is processed through the Output Transform. The Input Transform would have been applied on the Publisher channel before it got to the Schema Map, which is before the Event Transformation.

These two policy sets, which are not technically part of either channel (confusing to many), are meant to convert data formatting and normalize the attribute data. The common example cited is telephone number format. If the connected system uses (555) 555-1212 format, but in eDirectory you always store it as +1-555-555-1212, then you could implement policy on the Input transform to convert to eDirectory format and in the Output Transform to convert back.

This way, within the policy flow, you are always working with a single data format; and you can compare values that, although different (as stored literally), really mean the same thing.

That was a high level overview of most of what happens in the policy flow of an event through the engine. The details of what happens inside each of those steps is where the interesting work occurs, and that is the focus for the rest of this book.

There are several languages available to use. The original was XSLT (XML Stylesheets) that came with DirXML 1.0 and 1.1a. This has continued to be supported and still works perfectly fine. With NSure Identity Manager 2.0 a new language, DirXML Script, was introduced. This is a huge step forward. This language is XML based, in readable English, and consists of Conditions, Actions, and Tokens (Noun and Verb) with which events are processed.

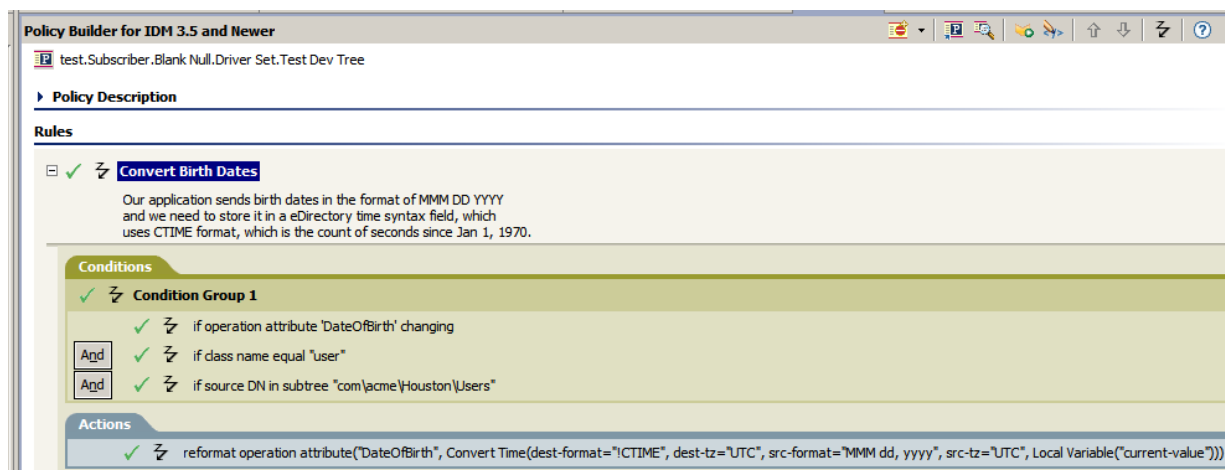
The focus of DirXML Script is to take an incoming XDS document (Identity Manager's XML dialect), detect something (using Condition tokens), then do something about it (with Actions that use Nouns and Tokens to build things as needed). This can all be seen by enabling Dstrace. The settings for Dstrace are set in the attribute `DirXML-TraceLevel` and can be set in Designer on the Trace tab of the driver properties or else in the Misc tab in iManager's properties of the driver. The best approach is to specify a file for the trace rather than watching

in iMonitor. The iMonitor view is nice since it is in colour and has useful visual hints, but it will interleave the results from all drivers making it impossible to read. Tracing each driver to its own file makes it much easier to follow an event.

A simple example would be a modify of a time related attribute coming out of the application on the Publisher channel that might look something like this XDS document:

```
[07/10/12 14:26:42.774]:Driver PT:
<nds dtdversion="8.7.2">
  <source>
    <product version="">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify class-name="user" src-dn="com\acme\Houston\Users\Jsmith">
      <modify-attr attr-name="DateOfBirth">
        <add-value>
          <value timestamp="1341049254#21" type="string">Jun 20, 2012</value>
        </add-value>
      </modify-attr>
    </modify>
  </input>
</nds>
```

Now a rule could be defined to reformat the date that looks like this somewhat contrived example in the Designer Policy Builder view:



When you look at the XML Source tab in Designer, you see the underlying DirXML Script XML that Policy Builder renders as above:

```
<rule>
  <description>Convert Birth Dates</description>
  <comment xml:space="preserve">Our application sends birth dates in the format of MMM DD YYYY and we need
```

```

to store it in a eDirectory time syntax field, which uses CTIME format, which is the count of seconds since Jan 1,
1970.</comment>
  <comment name="author" xml:space="preserve">Geoffrey Carman</comment>
  <comment name="version" xml:space="preserve">1</comment>
  <comment name="lastchanged" xml:space="preserve">Jul 13, 2012</comment>
  <conditions>
    <and>
      <if-op-attr name="DateOfBirth" op="changing"/>
      <if-class-name mode="nocase" op="equal">user</if-class-name>
      <if-src-dn op="in-subtree">com\acme\Houston\Users</if-src-dn>
    </and>
  </conditions>
  <actions>
    <do-reformat-op-attr name="DateOfBirth">
      <arg-value type="time">
        <token-convert-time dest-format="!CTIME" dest-tz="UTC" src-format="MMM dd,
yyyy" src-tz="UTC">
          <token-local-variable name="current-value"/>
        </token-convert-time>
      </arg-value>
    </do-reformat-op-attr>
  </actions>
</rule>

```

The `<conditions>` block is where the Condition tokens are used. They all start with "if-", and the three examples shown are testing:

If Examples

if-op-attr	<i>If Operation Attribute</i> is changing
if-class-name	If the class of the object is equal to user
if-src-dn	If the Source DN of the object is, in this case, in the subtree specified.

Then, in the `<actions>` block, there is one Action token that starts with "do-". The particular example is quite a powerful and complex token, `do-reformat-op-attr`, which is discussed later.

Inside the token, in the `<arg-value>` node, there are two tokens, a noun and a verb.

token-convert-time	A verb that changes time formats requires a nested noun to act upon.
token-local-variable	A noun that provides the value of a variable.

In this case, the *Convert Time* verb token uses the input of the *Local Variable* noun token to get a value to convert. If this event were to be processed, the whole thing would look something like this in DSTrace:

Blank Null :

```
<nds dtdversion="8.7.2">
  <source>
    <product version="">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify class-name="user" src-dn="com\acme\Houston\Users\Jsmith">
      <modify-attr attr-name="DateOfBirth">
        <add-value>
          <value timestamp="1341049254#21" type="string">Jun 20, 2012</value>
        </add-value>
      </modify-attr>
    </modify>
  </input>
</nds>
```

Blank Null :Applying policy: %+C%14CBirth Date Conversion%-C.

Blank Null : Applying to modify #1.

Blank Null : Evaluating selection criteria for rule 'Convert Birth Dates'.

Blank Null : (if-op-attr 'DateOfBirth' changing) = TRUE.

Blank Null : (if-class-name equal "user") = TRUE.

Blank Null : (if-src-dn in-subtree "com\acme\Houston\Users") = TRUE.

Blank Null : Rule selected.

Blank Null : Applying rule 'Convert Birth Dates'.

Blank Null : Action: do-reformat-op-attr("DateOfBirth",token-convert-time(dest-format="!CTIME",dest-tz="UTC",src-format="MMM dd, yyyy",src-tz="UTC",token-local-variable("current-value"))).

Blank Null : arg-string(token-convert-time(dest-format="!CTIME",dest-tz="UTC",src-format="MMM dd, yyyy",src-tz="UTC",token-local-variable("current-value")))

Blank Null : token-convert-time(dest-format="!CTIME",dest-tz="UTC",src-format="MMM dd, yyyy",src-tz="UTC",token-local-variable("current-value"))

Blank Null : token-convert-time(dest-format="!CTIME",dest-tz="UTC",src-format="MMM dd, yyyy",src-tz="UTC",token-local-variable("current-value"))

Blank Null : token-local-variable("current-value")

Blank Null : Token Value: "Jun 20, 2012".

Blank Null : Arg Value: "Jun 20, 2012".

Blank Null : Token Value: "1340150400".

Blank Null : Arg Value: "1340150400".

Blank Null :Policy returned:

Blank Null :

```
<nds dtdversion="8.7.2">
  <source>
    <product version="">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify class-name="user" src-dn="com\acme\Houston\Users\Jsmith">
      <modify-attr attr-name="DateOfBirth">
        <add-value>
          <value type="time">1340150400</value>
        </add-value>
      </modify-attr>
    </modify>
  </input>
</nds>
```

```
</input>  
</nds>
```

First, the event is shown before entering the policy so the starting conditions are known. As the rule is executed, the conditions are evaluated. Well, DateOfBirth is changing, so the first condition is true. Looks like it is a user object, so the second condition is true. The DN of the object in the source system (aka it's Source DN) is in the right source container, it meets the all conditions, and the rule is selected. The actions are fired, and the tokens are evaluated. Finally, the resulting document is shown.

This is the basic process that all the events run through in the Identity Manager engine. The complexity becomes more obvious when considering the number of different events that could come through and the 121 various tokens that are available to manipulate them.

Tokens of Identity Manager

I have been working with a client lately who has a large team of developers who have been learning about Identity Manager. They have been providing great fodder for what people might know, or not know, when getting started with IDM. This perspective has been really helpful in giving me ideas for a discussion that can help out beginners to IDM.

One of my favorite things about Novell Identity Manager is DirXML Script. This is a domain specific language that Novell wrote that turns out to be pretty powerful. You could probably do most everything in XSLT, but that is usually more complicated (to my mind). I had an example where this really interesting, somewhat long (two or three pages) XSLT stylesheet was used to perform a function. While we were looking at the XSLT and discussing how it worked, I rewrote it into 7 DirXML Script tokens. Now, the XML of the 7 DirXML Script tokens was several dozen lines due to the verbose nature of XML; it was very short in the Policy Builder view and somewhat easier to understand.

Now, I can understand that for those familiar with XSLT it is harder to switch over to DirXML Script. Since DirXML Script is meant to be the default approach in IDM, it seems like the better way to go. One of the things about the IDM product is that, over the years, with each new release, new and interesting tokens get added to make things easier. There are many well known examples of that, which I have addressed in articles before:

<http://www.novell.com/communities/node/6178/new-idm-36-enhancement-do-find-matching-objec>
<http://www.novell.com/communities/node/4337/parsedn-token-identity-manager-and-some-its-limitations>
<http://www.novell.com/communities/node/5687/examples-using-parsedn-token-identity-manager>
<http://www.novell.com/communities/node/5818/different-attribute-options-identity-manager>
<http://www.novell.com/communities/node/5920/ism-more-thoughts-sourcedestinationoperation-attribute-tokens-identity-manager>
<http://www.novell.com/communities/node/4906/the-query-token-identity-manager>

<http://www.novell.com/communities/node/2209/unique-name-token-functionality-idm-35>
<http://www.novell.com/communities/node/3034/reformat-operation-attribute>
<http://www.novell.com/communities/node/2572/using-time-tokens-idm-35>
<http://www.novell.com/communities/node/1903/idm-35-update-new-features>
<http://www.novell.com/communities/node/2198/troubleshooting-if-then-else-blocks-idm-code>
<http://www.novell.com/communities/node/4844/mapping-tables-and-render-browsed-dn-relative-policy-option>
<http://www.novell.com/communities/node/9351/join-and-split-tokens-identity-manager-35-and-higher>
<http://www.novell.com/communities/node/9356/using-and-copying-email-templates-identity-manager>
<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>
<http://www.novell.com/communities/node/11611/introducing-start-workflow-token-part-1>
<http://www.novell.com/communities/node/11612/introducing-start-workflow-token-part-2>
<http://www.novell.com/communities/node/13144/various-attribute-tokens-idm-policy>

While I had never before counted the total number of tokens available, I was sure I had only hit a small number with the listed articles. I thought I would start with the ones I missed. As I started doing that, I realized that the easiest way was to just go through them in order, so I started by counting and found that there are:

- 29 Noun Tokens
- 16 Verb Tokens
- 19 Conditions
- 57 Actions

Goodness, there are 121 different tokens! This might take awhile.

There are at least three distinct classes of tokens, depending on how you count them:

- Conditions
- Actions
- String or Arguments Builder (Verbs and Nouns)

Noun Tokens

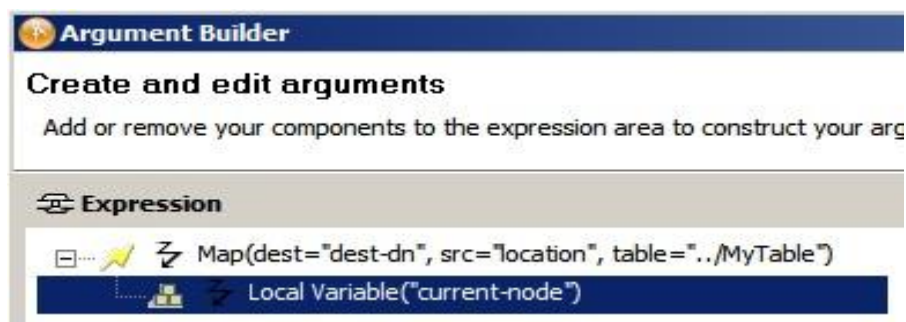
Let's start with the last category. Both of these (nouns and verbs) are most commonly used and, yet, contain some of the least commonly used tokens at the same time. A conflicted category at times.

String Builder or Argument Builder

There is a common builder made available from all sorts of tokens, usually anywhere you can provide a string value or an argument to a token that has a long list of two types of tokens, Nouns and Verbs.

In my series on Common Mistakes Beginners to IDM Make, part 1

(<http://www.novell.com/communities/node/13053/common-mistakes-newcomers-idm-make-part-1>), I discussed one of the most common mistakes about using Verbs. Verb tokens, like verbs in English, are action tokens. In the IDM context, they need some data to act upon. Thus, a verb needs a noun to provide some data. This is done in Argument Builder by using the up and down arrows to move the tokens in the list around. If you move something down, it first nests into the item below it. You can easily see how a verb with a nested noun is meant to apply. Of course, you can have many levels of nesting so that many verbs apply in sequence to the data in the originating noun. There is no specific limit to the depth of the nesting that is possible. I saw an odd example using the *Replace All* verb token that looked for non-US style characters, one per *Replace All* token, and nested them all on top of a *Operation Attribute* token so that all the various characters were replaced one by one. But it was nested 20 levels deep to accomplish this task. That may be a more extreme example than most. Usually, one or two layers of nesting is all that is needed.



In the image above, the *Map* token has a lightning bolt indicating an action (a verb). Below, but indented (indicating the nesting), is the *Local Variable* noun token using some funny icon of three boxes.

Text

Amusingly, the first item in the list of Noun tokens is *Text*, as opposed to something with the letter A. In case you are wondering, there is actually a good reason for this!

Define new action below

Do **set local variable**

Enter variable name: *

Select scope:

Select variable type:

Specify string:

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

Aside from the fact that *Text* is the most common noun token to use, it turns out that in Condition or Action tokens where there is a **Specify string** field that allows the String/Argument Builder to pop up if you click the icon at the end of the field, when you start typing before opening the String/Argument Builder, it defaults into the first noun token in the list that is the *Text* token (not to be confused with the XPATH `text()` function). Thus, in the example above, if you were to click on the String Builder icon on the right end of the **Specify string** line, it would show a *Text* token containing the value "Text Here". Thus, it actually saves time on a regular basis by allowing you to type in the **Specify string** line without the need to explicitly open the editor, find the *Text* token, and start typing.

An interesting side effect of this token is that if you were to enter XML in that field, perhaps to build a variable holding some XML, it will be stored in the underlying XML escaped. After all, XML, which uses less than and greater than signs, must handle embedded less than signs or else the XML parsing will get all broken.

If you need real XML stored in a variable, first make sure your *Set Local Variable* is type node-set. Then you could use a verb token discussed later, *XML Parse*, with the *Text* noun token nested underneath it. That will convert the XML text you enter into a DOM object that you can use XPATH upon or do all sorts of neat things with.

Entitlement and Attribute tokens

Added Entitlement

Removed Entitlement

Entitlement

Operation Attribute

Removed Attribute

Attribute

Destination Attribute

Source Attribute

There are two families of tokens that are very similar in general use and are sufficiently similar to be worth talking about together. The *Entitlement* and *Attribute* token families. Let's talk about these first.

The String Builder or Argument Builder interface is the same but used at different times with different names. Sometimes it is meant to build a literal string value or else it is meant to build a node-set, in which case it is really an Argument Builder. It can be used to select a node-set of values, often to be used to loop over that node-set in a *For Each* or *While* loop. Of course, a local variable can be a string or a node-set, so you get the same builder regardless.

Thus, there is a whole family of tokens (see the eight tokens in the title of this section) meant to provide a node-set of values to loop over. Interestingly enough, you can treat them as strings and get slightly different results as well.

Added Entitlement, Removed Entitlement, and Entitlement are different approaches to let you handle Entitlements, treating them as a node-set. You can read more about Entitlements in this series I wrote:

<http://www.novell.com/communities/node/12760/talking-about-entitlements-part-1>
<http://www.novell.com/communities/node/12908/talking-about-entitlements-part-2>
<http://www.novell.com/communities/node/12909/talking-about-entitlements-part-3>
<http://www.novell.com/communities/node/13027/talking-about-entitlements-part-4>

A typical XDS event might consist of any number of entitlement changes. Now, you might want a code block to do something with all the *Added Entitlements* and another to do something with all the *Removed Entitlements*. *Removed Entitlements* will also include revoked entitlements. It is called Removed instead of Revoked to maintain consistency with the *Removed Attribute* token naming. These are basically analogues. It's just that Entitlements have an extra state of revoked that differs slightly from removed. An entitlement can be present with a value (granted or revoked) or absent (implicitly not-granted, which is not the same as revoked). This is similar to how boolean attributes in eDirectory actually have two states but three possible values: true, false, and absent (which shares the same state as false).

Using the *Added* or *Removed Entitlements* tokens, you can implement whatever is supposed to happen when those entitlements are granted or revoked based on the payload data.

The *Entitlement* token is more like the *Source Attribute* token that will query back to eDirectory for the list of Entitlements currently on the object of reference. Alas, you cannot redirect this to another object (like in *Source Attribute*) since Entitlements are supposed to be object specific. However, you can always do the work 'by hand' the 'hard way' by reading back the `DirXML-EntitlementRef` attribute of the object of interest and checking each Entitlement to see if it is the one you are interested in. The volume component will be the DN of the `DirXML-Entitlement` object that is being granted to the user or object (no reason to limit entitlements to only User objects).

These tokens only refer to Entitlements specific to this driver, which also would be nice to have some flexibility around. Might be neat to be able to check not just if the object has a specific entitlement in this driver but maybe also in another driver? I can imagine some use cases that would make that useful. Again, you can do it by looking at the `DirXML-EntitlementRef`. So not that hard but would just be nicer if it was built in. It turns out that the attribute `DirXML-EntitlementRef` is a structured attribute of syntax Path. This means it has three components: `nameSpace` that holds the state (0 for revoked, 1 for granted), `volume` that holds the DN of the `DirXML-Entitlement` object being granted, and `path.xml` that holds the payload information. Thus, you could look at the volume component and look for an entitlement granted in another driver.

The *Entitlement* family of tokens take the Entitlements short name as their parameter. This is specific to this current driver, so although the picker is functionally a DN browser, it only lets you pick Entitlement objects contained within the current driver. This is neat as you can have a library based policy that is used in several drivers, and each driver has the same named Entitlement object (But the full DN of the entitlement is unique, of course, being in a different driver.). This should all work when reused.

There is much more that can be said about Entitlements. Check out the links to some other articles on the topic if you are interested in more information.

The other groups of similar tokens are the *Attribute* family.

The *Operation Attribute* should really be named *Added Attribute* to make it clearer. This is a little confusing when considering the condition token *If Operation Attribute*, which has a bunch of different modes. For example, *If Operation Attribute* can test for available, changing, equal, etc. Available means at least one value is being added (though some removes may be happening at the same time). Changing means at least one of a remove or add attribute is happening, thus, testing for *If Operation Attribute* changing and *If Operation Attribute* available. You can tell if there is only an add or only a remove.

You can read more about this neat ability in this article:

<http://www.novell.com/communities/node/13144/various-attribute-tokens-idm-policy>

The neat thing about *Operation* Token, when acting as a Noun in Argument Builder, is that it only means the attribute value being added in the current operation. As you can see, this is

slightly different than in the condition case. Thus, if you need a rule that loops over all added values of an attribute (say Member on a group), then, this is the token for you! In the example of Members of a group changing, the *Operation Attribute* token can be used in a loop to do something based on all the Member values being added. The *Removed Attribute* token can be looped over to do something based on any Member values removed in the current operation.

Removed Attribute is much clearer and does what the name says. It provides all the removed attributes as a node-set. (OK, the name does not imply a node-set, but if ever there is the possibility of more than one value, assume a node-set as the result).

The *Attribute* token is basically similar to the *Entitlements* token in that it will look at the operation attribute for values, and if none are found in the current operation document, it will look back at the *Source Attribute*. This means if the attribute you need is available, it does not need to query. If not available, it will do the query, which saves some code steps and unneeded queries in some cases.

The *Source* and *Destination Attribute* tokens let you look forwards or backwards in the event flow to get the value of an attribute. For these, it defaults to the current object. You can specify either a DN or association value to find a different objects attribute value. If you treat this like a node-set, you can handle one or more values returned. If you treat it like a string, it will work fine for a single value. If there are two or more values, you are likely to get the string values concatenated together into one big mush.

Many of these tokens are basically special cases of the *Query* token, those that do not just look at the current event document that is (*Source Attribute*, *Destination Attribute*, vs *Operation Attribute*, *Removed Attribute*). What that means is they query either the source or destination for the attribute you specified. In the case of the *Entitlement* family of tokens, they look specifically at a special case of a single attribute and know how to handle its Path syntax formatting to properly filter the results to just this driver's entitlements and handle granted, revoked, or absent values. This matters because if the data is not already cached, you will see a `<query>` event get sent (to source or destination) that looks just like the `<query>` you might see if you used a *Query* token.

Sample Query event:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product edition="Advanced" version="4.0.1.0">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <query class-name="Group" dest-dn="\ACME-TREE\acme\Users\TestGroup" scope="entry">
      <read-attr attr-name="Member"/>
    </query>
  </input>
```

```
</nds>
```

Sample Query response:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product edition="Advanced" version="4.0.1.0">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <output>
    <instance class-name="Group" qualified-src-dn="o=\ACME\ou=Users\CN=TestGroup" src-dn="\ACME-
TREE\acme\Users\TestGroup" src-entry-id="2012150">
      <association state="associated">SomeAssociationValue</association>
      <attr attr-name="Member">
        <value timestamp="1344366845#1" type="dn">\ACME-TREE\acme\Users\User1</value>
        <value timestamp="1344366845#1" type="dn">\ACME-TREE\acme\Users\OtherUser</value>
      </attr>
    </instance>
    <status level="success"></status>
  </output>
</nds>
```

Although the query looks the same, there is one major difference in how the results are handled. In a *Query* token case, the entire result document from the `<instance>` document is returned. If you treat it as a node-set (Store in a variable? Use as the subject of a loop?), then you can use XPATH upon it to get the `src-dn` XML attribute or other information.

When you use a *Source/Destination Attribute* token, you primarily get back the `<value>` nodes and cannot do much XPATH upon it. This is an important distinction since you can often do it either way. Sometimes one method is preferred.

Character

This is a neat token that is rarely needed, but when you do, it is mighty handy. This lets you specify a Unicode (double byte) character that you cannot otherwise type. You can use this to enter such a value into a string, or else in a condition test, such as *If Local Variable* is equal to some special character or the like.

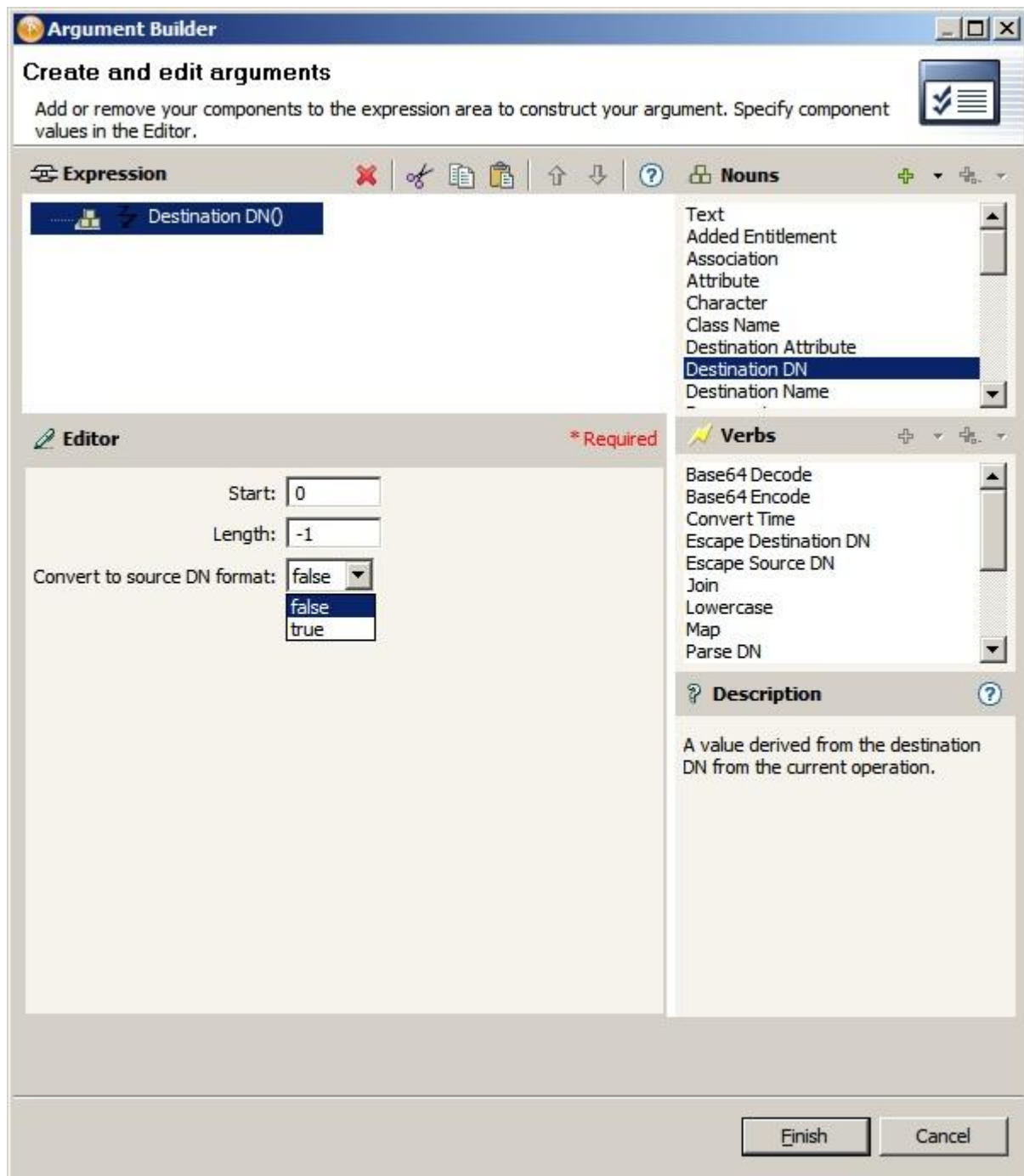
One example where this might be handy is in the *Find Matching Object* token, where if a single value is found, then, the `dest-dn` XML attribute (XPATH of `@dest-dn`, or use the *Destination DN* token) will hold the matched objects DN. However, if more than one match is found, the `dest-dn` is set to `xFFFD`. If the user is already associated, it gets set to `xFFFC`. Though, in actuality, you can literally test for the string `xFFFC` in a compare in this case. If you wanted to set a special Unicode character, you could use this token.

Class Name

This is an easy one to do without a token but is convenient to have available. XPATH of `@class-name` will do the same thing, but it is simpler to read and cleaner to use the *Class Name* token.

Destination DN

This is a basic token at first glance. It basically performs an XPATH of `@dest-dn`. Meaning if there is no `dest-dn` XML attribute in the current document, it is not going to return a value. Alas, it is not some magical token that does a *Resolve* from Association to DN under the covers if it needs to. That would be cool but, alas, is not the case. This only takes the `dest-dn` XML attribute from the current event. You could not do a query, store it in a node-set variable, then somehow use the *Destination DN* token against it. There you would have to use an XPATH of `$VarName/@dest-dn` to get the value.



However, it does have a built-in *Parse DN* function that is helpful. After all, a destination DN could be in many different formats depending on what the destination uses. However, it is a stripped-down *Parse DN* function, so you can specify the **Start** (0 is the root most element, usually the tree on an eDirectory to eDirectory driver) and then the **Length** where -1 (negative one) means the very end. You can read more about *Parse DN* examples and neat stuff in these articles:

<http://www.novell.com/communities/node/4337/parsedn-token-identity-manager-and-some-its-limitations>
<http://www.novell.com/communities/node/5687/examples-using-parsedn-token-identity-manager>
<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>

Basically, if you want to parse up `cn=Geoffc,ou=Users,ou=data,dc=acme,dc=com` to just get the `ou=Users` part, you would set the start to 3 (if 0 starts at `dc=com`, 1 at `dc=acme`, 2 at `ou=data`, 3 at `ou=Users`), and then length of just 1 to get one node that would work. Of course, you could approach it from the other end where you start at -2 for a length of 1 to get `ou=Users` as well.

As you can see, the *Parse DN* functionality exposed here, in *Source DN* and in the *Parse DN* token itself, is very powerful.

There is one more interesting option. **Convert to Source DN format** is part of the limited functionality since you can only specify **true** or **false**. *Parse DN*, on the other hand, allows you to pick from a variety of formats, for the source and destination formats, including Other, which allows you to define your own pattern. Here you can only choose whatever the source DN format is defined for this driver shim. Since this is *Destination DN*, the option is to convert to the Source format. The *Source DN* token will have an option to convert it to the Destination format.

Destination Name

This is a very simple token that is actually somewhat tricky to do in XPATH alone. It returns just the name of the object in the destination system. This is actually taking the *Destination DN* (Imagine an XPATH of `@dest-dn`), then applying *Parse DN* with a length of 1 and a start of -1 (getting the leaf most node), and then converting it to a format that does not have any qualifiers (like `cn=` or `ou=`). However, I am pretty sure this token, like *Destination DN* discussed in the previous article, does not perform magic under the covers. If there is no destination DN in this document (An XML attribute of `dest-dn` in the event node), then, it does not take the association value, do a `Resolve()` token call to convert the Association to a DN, and provide back the value. It would be cool if it did, but I do not think it will do this for you.

Document

I had never used this token until I was reading someone else's driver and saw it in use (the CMP SAP HR drivers, which is now the base SAP HR driver in IDM 4, and there is a Consulting Edition of the SAP drivers. Now that is even way better!). Basically, it is a way to read back an entire XML document from an object and attribute. In order to use it, you need to provide an XML document URI. This is reasonably tricky to generate and has a somewhat unique format.

You can read more about the format needed for this token in the article:

<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>

```

<do-set-local-variable name="configValues" scope="driver">
  <arg-node-set>
    <token-document>
      <arg-string>
        <token-text xml:space="preserve">vnd.nds.stream:</token-text>
        <token-text xml:space="preserve"></token-text>
        <token-parse-dn dest-dn-delims="00,/+=*" dest-dn-format="custom" src-dn-
format="slash">
          <token-global-variable name="dirxml.auto.driverdn"/>
        </token-parse-dn>
        <token-text xml:space="preserve">#</token-text>
        <token-text xml:space="preserve">DirXML-ConfigValues</token-text>
      </arg-string>
    </token-document>
  </arg-node-set>
</do-set-local-variable>

```

Basically, it uses *Parse DN* with a custom delimiter to convert the driver DN to the right custom format, preceded by the string `vnd.nds.stream:/`, followed by a pound sign (`#`), and the attribute value.

I personally would just use *Source Attribute* for the `DirXML-ConfigValues` of the driver object (using the Auto Global Configuration Variable they use as well). I always forget if I have to *Base64 Decode* it or not. As we will see later in this series, *Base64 Decode* is one of the available verb tokens so won't be a big deal!

It is not 100% clear to me why you would use the *Document* token in this context over the *Source Attribute* approach, but it clearly works both ways. In principle (though I have never thought of, nor seen an example) you ought to be able to get any valid XML document through its URI. I have no idea how well this will extend outside the IDM system (i.e. Can it go get a document from an external source? I just do not know.).

Now it's worth mentioning that both approaches to the same task above, getting the XML of the `DirXML-ConfigValues` attribute have a real use case. You can read more about what the various attributes of the `DirXML-Driver` object are in this series of articles:

<http://www.novell.com/communities/node/12517/what-do-various-idm-driver-object-attributes-represent-part-1>
<http://www.novell.com/communities/node/12520/what-do-various-idm-driver-object-attributes-represent-part-2>
<http://www.novell.com/communities/node/12528/what-do-various-idm-driver-object-attributes-represent-part-3>
<http://www.novell.com/communities/node/12529/what-do-various-idm-driver-object-attributes-represent-part-4>

The attribute `DirXML-ConfigValues` holds the Configuration XML. `DirXML-GlobalConfig` holds the Global Configuration Variable (hereafter referred to as "GCV") values, but there is no need for this approach since there is a `GCV` token in all the various contexts (Condition, Actions, and Argument Builder as a noun). However, you might need to know how the driver is configured in Policy in order to do something in your driver. Alas, since there is no Configuration Value token, this can be an issue. Thus, you read back the XML of the attribute it is stored in, then you can XPATH out the information using an XPATH as simple as `$VAR-NAME//@name='Config Value Name'`.

The Data Collection Service and Managed System Gateway drivers take advantage of this approach to go see how other drivers in the tree are configured. If this is something your policy will need to do, you could consider using an interesting configuration type in the driver called `<gcv-ref>`. The driver will now retrieve the specified GCV value. Since the same value is stored for Drive Configuration and GCV, you can now use the GCV tokens to get the data.

One interesting tidbit that you can learn from the tooltip that is shown in Designer is that it will, of course, return a node-set. If there is any error in your use of it, the returned value is an empty node-set and, thus, no error messages.

Every so often some useful tidbit can be gleaned from either the tooltips or even the context sensitive help. Alas, it is not very consistent, and you need to sometimes think about the consequences of what they are saying. For example, in the case of the *Destination DN* and *Destination Name*, the tooltip does suggest that the value will be derived from the destination DN of the current operation. The implied comment is what I described above. Should there not be a destination DN value in the event node (which is basically every `<modify>` event, only an `<add>` event would likely have a destination DN and only after the matching or placement policy sets the DN), then, it will not work. Be nice if that was called out more clearly. Once someone explains it, it makes sense, but without that extra little step, it looks like it just doesn't work when used on `<modify>` events.

Generate Password

This token is just plain the bees knees (Maybe the piano's strings? The chicken's knuckles?). When the time comes, and you have to generate a default password, it can be a pain to generate a sufficiently random one that also meets all the password policy's requirements. Over the years, I have seen many attempts to implement this functionality in DirXML Script policy. It is very doable but somewhat convoluted. However, with the addition of this token, there is no need to do any of that work.

Even better, this token does not need any complex interface to specify the password complexity rules because all it takes as a parameter is the DN of a password policy. A Password Policy (stored in the `Security\Password Policies` container in every eDirectory tree) is where Universal Password policies for complexity, synchronization (to Simple Password, NDS password, Distribution Password), expiration, and more are set up. Thus, you get a very nice, mature interface for describing a password complexity issue using the Password Policy editor in iManager (Alas, no way to do this in Designer yet.). Then, just point this token at it to generate a random password that meets that pattern.

It turns out the password policy does not even have to be assigned to the object of interest or even to any object. It is just basically a way to provide the configuration information for this token. I was working on an Active Directory driver where they use a GINA plugin on all their Windows machines, and thus, their actual password is in a Kerberos password in an MIT

Kerberos realm. Therefore, when we synchronized accounts into Active Directory, we had to set an initial password; but no one would ever use them. They decided to set some crazy password requirements like 20 special characters, 20 lowercase, 20 uppercase, and 20 numerics, to a maximum of 200 characters. So when you called *Generate Password* on such a policy, it returned 80 characters minimum to 200 characters of utter gibberish. Perfect for untypable random passwords.

Of course, the password policy you reference could be in use if you so desire (usually makes a smidgen more sense in that use case) but, as noted above, does not have to be. What this means is you could generate a really complex, random password for the first initial one, then, all future changes by the user would have a much simpler set of complexity rules.

The Consensus Student Interoperability Framework driver actually has an implementation of generating a random password that does not use this token. The function is probably legacy from earlier versions that predate the *Generate Password* token. (Much like the *Unique Name* token we will talk about later, lots of people implemented it over the years. Then, a simple token came along and replaced all that work. Both ways work, so why change what is working if it is working fine?) You can read more about the Student Interoperability Framework (hereafter referred to as “SIF”) driver in this series I wrote on walking through its policies:

<http://www.novell.com/communities/node/13159/walking-through-sif-driver-part-1>

<http://www.novell.com/communities/node/13160/walking-through-sif-driver-part-2>

<http://www.novell.com/communities/node/13327/walking-through-sif-driver-part-3>

<http://www.novell.com/communities/node/13336/walking-through-sif-driver-part-4>

<http://www.novell.com/communities/node/13381/walking-through-sif-driver-part-5>

<http://www.novell.com/communities/node/13382/walking-through-sif-driver-part-6>

<http://www.novell.com/communities/node/13384/walking-through-sif-driver-part-7>

Global Configuration Value

This is probably one of my favorite tokens. I love using GCVs. Especially when they are values that differ between the Development, QA, and production environments.

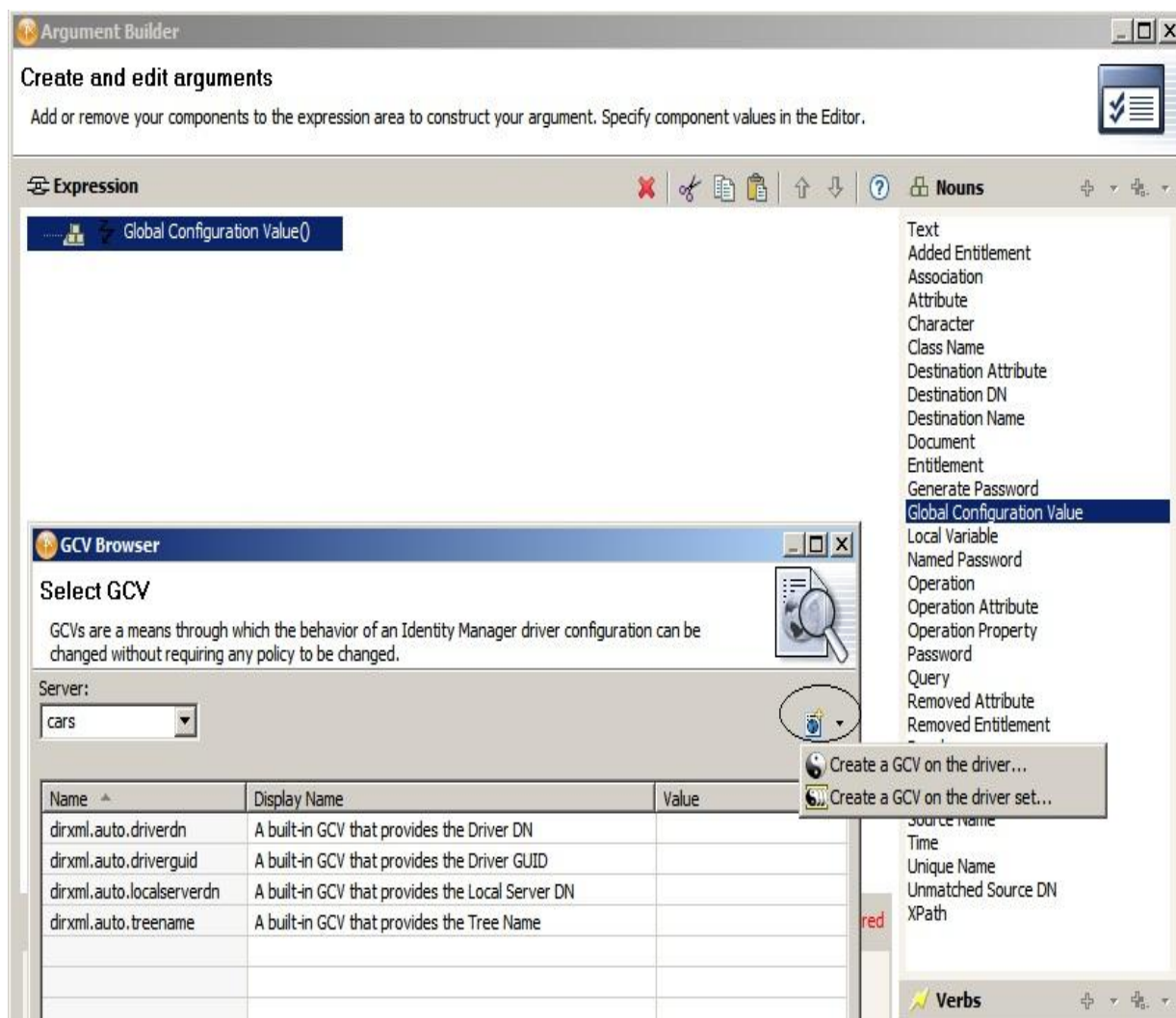
There are many GCV types, and they are quite powerful. You can read more about them in this series of articles:

<http://www.novell.com/communities/node/11344/explaining-gcvs-part-1>

<http://www.novell.com/communities/node/11471/explaining-gcvs-part-2>

<http://www.novell.com/communities/node/11480/structured-global-configuration-values-idm>

The really interesting thing with this token is actually not the token itself, but, rather, the selector!



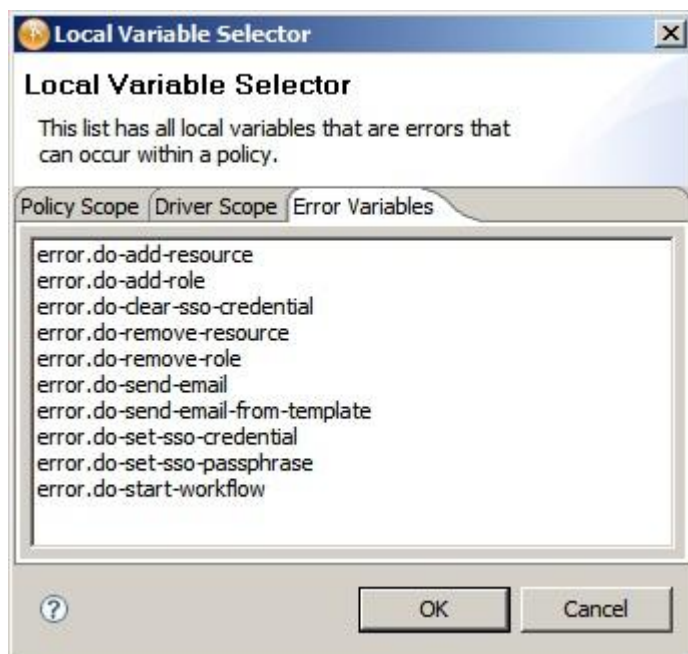
If you go to the selector, a couple of neat things are in the upper row. First off, on the left there is a server selector. It is critical to remember that GCVs are non-replicated attributes. That is, if you have 3 servers in the driverset, each can have different values. This makes sense if the GCV held an IP Address of a server to contact for some reason, and each server is in a different data center and has a different target. With these non-replicating attributes, you can have different settings on different servers, but it can still make your head hurt at times.

Now, this selection only affects what is shown in the list in terms of available defined GCVs, and the values of the specific GCVs since both can differ based on servers. So if your GCV is not visible, keep in mind it might not be defined on the first server in the list. I wish we could specify a default first server for each driver, but alas, that is not there. In Designer 4.x there is a setting for a Preferred server at the driver set level, but it does not seem to affect this selector. (It is meant more for Packaging and which server to suggest as the Preferred server as the install method.)

My favorite option, though, is the little icon in the upper right hand that allows you to add a new GCV while you are selecting it! In fact, it is even better! If you hit the tiny down arrow, you get two options in a drop down to add a GCV to the driver or to the driver set. This is great if you need a new GCV on the fly, and you are buried many nested layers down. Otherwise, you would have to remember to go visit the GCV tab after you finish and add the GCV. If you are nested in pretty deep, that could be a lot of mousing and clicking. I usually forget to do it if I let it slide. So I just use this little option to set it on the fly. Alas, once set, there is no edit GCV. I would probably use this alot as I often change my mind right after I click OK. Of course, with Packages you should really be adding the GCV into a GCV Object added to your package. At least here, you can add them before you forget. Before you release or build your package, remember to copy it over from the driver attribute GCV to the Packaged GCV object. Alas, you cannot select a package to add them to like you can select which server it will be added to.

Local Variable

I assume (but cannot prove) this is probably the most common noun token, maybe after Text, since it is so darn useful. Remember that local variables have two scopes, Policy or Driver scoped. Driver scoped means it stays around and is available until the driver is restarted. Policy scoped means it only lasts the duration of this policy object.



There are some built-in variables, like the error variables, for some wrapper tokens that call out to external functions and can return errors. Like the *Send Email* tokens, SSO related, and SOAP calls (*Start Workflow*, *Add Role*, *Remove Role*). You can see the different variable types when you get the selector. There will be tabs for Policy Scoped, Driver Scoped, and Error Variables.

Then, there are three automatic variables that are often not well understood:

- `current-node`
- `current-value`
- `current-op`

I discussed these in these articles I wrote for beginners to IDM:

<http://www.novell.com/communities/node/13053/common-mistakes-newcomers-idm-make-part-1>

<http://www.novell.com/communities/node/13383/common-mistakes-newcomers-idm-make-part-9>

Basically, if you are in a loop (*For Each* loop), then the node you are currently looping over is available in the variable `current-node`. Very helpful. It is worth remembering that the loop has to be done over a node-set, so the `current-node` is pretty much always a node in a node-set. You can

perform XPATH operations on the `current-node` variable (`$current-node/@src-dn`, for example).



If you treat it like a string, you will get all the string values in the node crunched together into one.

It is also really important to understand what the node-set being looped over looks like, as the results will likely vary, if you do not take care. See the section on the *Attribute* and *Entitlement* tokens for an example of how a *Query* token's result would differ from a *Source Attribute* token's result.

The `current-value` variable is actually a really neat variable that is really meant for use in the *Reformat Operation Attribute* token. That means, in a single (non-looping) action, you can manipulate the `current-value` variable, and all the values of the `current-op` attribute will be modified as such.

Imagine you tried to *Reformat Operation Attribute* Member and, say, decided to *Parse DN* it to change the DN formatting. Well, you might get one member value, or you might get ten thousand values. Now, you might think you have to loop and handle all the values, but by applying your *Parse DN* verb token against the noun token *Local Variable* of `current-value`, you implicitly handle all the values without a loop. It takes a bit to wrap your head around, but it can be very powerful.

Now, to make things interesting, it turns out there are usually at least two other ways to get access to the local variables instead of using this token. You could use an XPATH statement of `$variable-name` or, if the field you are building supports it, you could use the double dollar sign notation and use `$variable-name$` to get the variable value.

,

Even harder to wrap your head around is that if you set a value into `current-op`, that becomes the current operation. Thus, on a `<modify>` event being processed, if the XDS for a `<rename>` event is set into local variable `current-op`, then the event will change from the `<modify>` to the `<rename>`. A very wild and wacky variable.

Named Password

In order to run a driver, a number of passwords (Driver Object password, Application Password, and Remote Loader password) need to be stored in a secure fashion. They use an encrypted store for passwords (Well, for Application and Remote Loader passwords, the Driver Object password is actually the Universal Password assigned to the driver object itself just like it was a user.). The *Named Password* token exposes this code so you can define and use your own encrypted passwords within your policy. Alas, I have yet to figure out a way to use *Named Password* to get one of those configuration passwords since there are times when it would be helpful to be able to pull out the password of one of them. However, for drivers that require additional passwords (Like the SOAP driver or Salesforce.Com driver that requires a secondary token.), they can be defined as *Named Passwords* making them available as needed securely.

With IDM 4.x, there is a new API call that allows more access to *Named Passwords*, and there is a tool called Console2 that allows you to read some of them.

<http://www.novell.com/communities/node/12387/console2>

However, this is really nice when you need to store a password (Like in the Lotus Notes driver where your Domino server potentially could have a password on the certifiers file, and each OU object in Domino needs its own certifier file and password.).

Operation

This gets the name of the current operation. There are quite the stack of possible operation event types that are pretty common: add, modify, rename, move, delete, and instance. But there are some less common ones. Clicking on the magnifying glass icon can retrieve the entire list. Normally, this is used to select an object; but in this case, it gives you a list of supported operations to select from.

- add-association
- check-object-password
- check-password
- generated-password
- get-named-password
- init-params
- instance
- modify-association

- modify-password
- query
- query-schema
- remove-association
- schema-def
- status
- sync
- trigger

Some of those are usually only used during driver startup processing: check-object-password, init-params, query-schema (and its response), and schema-def.

Others are association related: add, remove, and modify association.

Query sends a query into the target system, and instance is the type of response you get back. When an event succeeds or fails, you get a status event back.

A Job (a scheduled event local to a driver or driverset) sends a trigger event based on how you defined it.

If a call is made to the *Named Password* token, it will generate a `get-named-password` event.

When you use the iManager check password synchronization status, it is supposed to call a `check-password` event for this user to test that the password in the Identity vault is the same as in the connected system. This should try a password compare (if the connected system shim supports it, to save an intruder lockout attempt) to tell you if it worked or not. To be honest, I have had mixed success with this functionality, and do not usually rely on it.

When you try to migrate from an application or migrate from Identity Vault in iManager on a driver (Alas, not an option in Designer.), you are basically generating a sync event in one channel or another. Interestingly enough, if you want to cause a sync event to happen in the Publisher channel, you can change the operation type to sync or *Append XML Elements* to a sync event to the current document. This will only work and cause the engine to treat it properly if done in Event Transformation policy set.

There remains one last event type that is unclear: `generated-password`. I am not sure what that is used for.

To further confuse the issue, in something like a SOAP driver, the incoming document from the web service (if you have it configured to listen on the Publisher channel) might have a format where the event node is actually something completely different, like `<soapenv:Body>` or the like. This token should be able to select that as the operation as well. Thus, you are not limited to the listed set of operations if your driver adds additional operation types.

Operation Property

Although it seems pretty tempting to use a Driver scoped variable to carry data from one policy to another, that may or may not be the best approach. This is because you never know if something else has manipulated it in between your intended use of it. Also, while it can carry data from the Subscriber channel into the Publisher channel, the problem is that the two channels are independent. Therefore, an event on the Subscriber channel might come in while the Publisher channel is processing the previous event, and the value can be changed out from under you.

Instead, the 'approved' method is to use an Operation property. These are stored as XML attributes of the `<operation-data>` node that is appended to the end of the event document. When the event flows into the shim on the Subscriber channel, the operation data is removed from the event but tracked so when the response document returns, the engine will put the operation data back.

This way you can tag an event with some information knowing it will carry through the shim and back out when the event completes. Oddly, this does not seem to work on the Loopback driver. There the looped event is treated as a new event coming out the other end, as opposed to a response to a previous event.

Now, if you look at the packaged common Password Sync policies, you will notice that, in fact, they use the `<operation-data>` node but not as standard operation properties, which would be `<operation-data SomeOpProp='SomeValue'>`. Rather, they do it by adding XML elements to the `<operation-data>` node as children. This actually builds up quite the structure at the end of the event, looking something like this:

```
<nds dtdversion="3.5" ndsversion="8.x">
  <source>
    <product version="3.5.11.20080307 ">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify-password class-name="User" event-id="pwd-subscribe" qualified-src-dn="O=com\OU=acme\CN=jsmith"
src-dn="\ACME\com\acme\jsmith" src-entry-id="33458">
      <association>7b62bd352a06644a8f9e6d72fc827129</association>
      <password><!-- content suppressed --></password>
      <operation-data>
        <password-subscribe-status>
          <association>7b62bd352a06644a8f9e6d72fc827129</association>
        </password-subscribe-status>
      </operation-data>
    </modify-password>
  </input>
</nds>
```

That works, but is not accessible via the *Operation Property* token. You have to create it using *Clone By XPATH* or the *Append XML Element* tokens to add the information and then XPATH statements to get at it. Both approaches work. If you need more than just simple values, it is a perfectly acceptable way to do it.

Interestingly enough, the SOAP driver uses a couple of operation property values to control the driver. If you set a specific operation property `url` on any event you send into the SOAP driver shim, it will try and use that value as the connection string for the SOAP endpoint. The operation property `method` is used to change from the default POST method for your HTTP connection. The operation property `soap-action` is another example for web services that require something other than the default of `#batchRequest`. You can also add some nodes under `<operation-data>` in the SOAP driver to control the HTTP headers used. This is quite a useful feature even if somewhat unconventionally exposed. This makes it seem likely that the SOAP driver could be used for REST operations since the URL generated for each event could be manipulated, and the method changed from POST to GET as needed.

If you are interested in how the SOAP driver handles this, you can look at the docs:

<http://www.novell.com/documentation/idm401drivers/soapdriver/data/b1kgb3j.html>

If you have noticed any other driver that does something interesting like this with Operation Data, please let me know since the only one I know of offhand is the SOAP driver. I would be curious to see how others handle it.

Password

This token is another one of those shorthand tokens for simple XPATH statements. The tooltip says this is the value of the password in the current operation. What that neglects to explain is that this only applies to a `<password>` node, which you will possibly see in an `<add>` event or a `<modify-password>` event. This is basically a wrapper around an XPATH of `password` in that case.

You need to understand how passwords are synchronized from eDirectory using the `nspmDistributionPassword` as a `<modify>` node with a `<modify-attr attr-name='nspmDistributionPassword'>` node. The transformation occurs in both channels and happens in the Command Transformation policies on each channel slightly differently.

On the Subscriber channel, the `nspmDistributionPassword` is in the document until the password sync policies in the Command Transform follow the GCV settings (Server Variables on the Driver in iManager, and Password Synchronization settings in Designer) and convert it into a `<modify-password>` event that has a `<password>` node that this token can pick up.

Coming out of the shim, on the Publisher channel, you will probably see a `<modify-password>` event that this token can pick out. Once it hits, the Publisher Command Transform rules it will be

converted into a `<modify-attr attr-name='nspmDistributionPassword'>` if the GCVs are set that way. (If it is still using the NDS password, the event will remain a `<modify-password>` event.)



You can read more about these policies and how they work in these articles:

<http://www.novell.com/communities/node/1474/password-transformation-rule-sets>

<http://www.novell.com/communities/node/9661/password-transformation-rules-publisher-channel>

Thus, this is one of those odd tokens that has very limited and focused use cases.

It is, however, quite confusing if you expect it to work before the Subscriber Command Transform fires when the event is not yet a `<password>` node. This is the sort of detail that really ought to be in the documentation since this is one of those tokens that looks obvious but really is not. It requires a bit of an explanation of the underlying processes.

In many ways, this is similar to the *Destination Name* and *Destination DN* tokens. They look like they should just work, but really they only work in specific cases. The tooltips and documentation do not make it entirely clear. Again, to be fair, the tooltip does try to indicate in the case of the *Destination DN* that it is based on the destination DN value in the document. However, you need to recognize that specific detail and the limitations that it signifies.

Regardless, it is pretty clear these examples are simple to fix things that the documentation really ought to elaborate on at some length.

Query

This has to be one of the nicest tokens they added. Everything this token can do, you can do in an XPATH command using the Java `query()` class. However, you have to know all the parameters, number of parameters, and ordering. The class responds differently depending on the number and content of the parameters submitted, so with this token, it's all available in a GUI that is quite easy to use.

The screenshot shows the 'Argument Builder' window with the title 'Create and edit arguments'. Below the title is a description: 'Add or remove your components to the expression area to construct your argument. Specify component values in the'. The 'Expression' section shows a toolbar with icons for adding, removing, and editing components, and a text box containing the expression 'Query(dn(), match(), "")'. The 'Editor' section, marked with a red '* Required' label, contains several fields: 'Datastore:' with a dropdown menu set to 'Destination'; 'Scope:' with a dropdown menu set to 'Subtree'; 'Max result count:' with an empty text box; 'Class name:' with an empty text box and three small icons to its right; 'Select object:' with a dropdown menu set to 'DN'; 'Specify DN: *' with an empty text box and a small icon to its right; 'Match attributes:' with an empty text box and a small icon to its right; and 'Read attribute:' with an empty text box, a small icon to its right, and a green plus sign to its left.

I wrote fairly extensively about this token in an article, and you can read more about it here:

<http://www.novell.com/communities/node/4906/the-query-token-identity-manager>

I also noted that I have seen people make an amusing mistake about how the **Read Attribute** line works due to an odd design choice in the GUI. For whatever reason, they chose to provide an Argument Builder interface at that point. As a consequence, I have seen people try and use the *Source Attribute* token for the attribute `employeeNumber` instead of just a *Text* string of

employeeNumber because it sort of looks right when done that way. Of course, doing it with *Source Attribute* means it tries to find an attribute named by the value of the employeeNumber attribute of the referenced object. Thus, if the source employeeNumber was 12345, then the *Query* will end up looking for an attribute named 12345 and not employeeNumber as intended.

I never did understand why there is the full Argument Builder made available for the **Read Attribute** strings. It seems like it should just be a text field where you can stick in a variable if you so desire to calculate the name elsewhere.

What I was wondering the other day was how could I store the list of attribute names in a GCV and use that in a *Query* token as the list of attributes to read back. I was thinking of this issue because I had a driver where we loaded a set of variables into a node-set variable via a query. I wanted to later loop through a GCV of the attribute names to do some work. In the end, I decided to query for the attribute named * (asterisk) that returns all the attributes and store that. It eats a fair bit more memory but, in the grand scheme of things, not very much, assuming it is used for one user at a time.

However, someone noted a clever way to do it, which is quite contrary to my entire thesis for this book. They suggested that if I used the Java query command, it would be possible to accomplish the task with a list GCV and variable names. This would look something like this XPATH statement:

```
query:search($destQueryProcessor, "~SearchScope~", "", "~SearchBase~", "~Class~",
"~FilterAttr~", "~FilterValue~", "~ReadAttrList~
```

As you can see, in general using the *Query* token is much easier since it lets you make all those decisions in a much simpler fashion; however, you still have the choice to go back to the older functions if they make sense for your use case.

I do wish, since this basically generates the proper Java query command, that there would be a way to build up what you want and then display the resulting query command. This would help to build complicated query expressions when you still want to use the Java equivalent. Alas, the user interface does not do this.

The various options in the token are pretty simple. **Datastore** with Destination or Source is nice since it makes it simple to understand which one you are querying. After all, you could be aiming forwards or backwards in either channel. Thus, destination in the Subscriber channel is the connected system, as is Source in the Publisher channel.

The **scope** control lets you select entry, subtree, or container that limits the scope of the search as you would expect.

Max Result count is the reason I wanted to work through these settings since this one does not do what you would at first think! It does not return just the number specified as you might think. Instead, it returns pages of results with the number specified in max-result-count in each page. That is, if the system supports (eDirectory does, and many driver shims do) the query-ex function, then it can return results in pages. Thus, this control will return, say, 50 values in each page, until all the results are returned, as opposed to only the first 50 results.

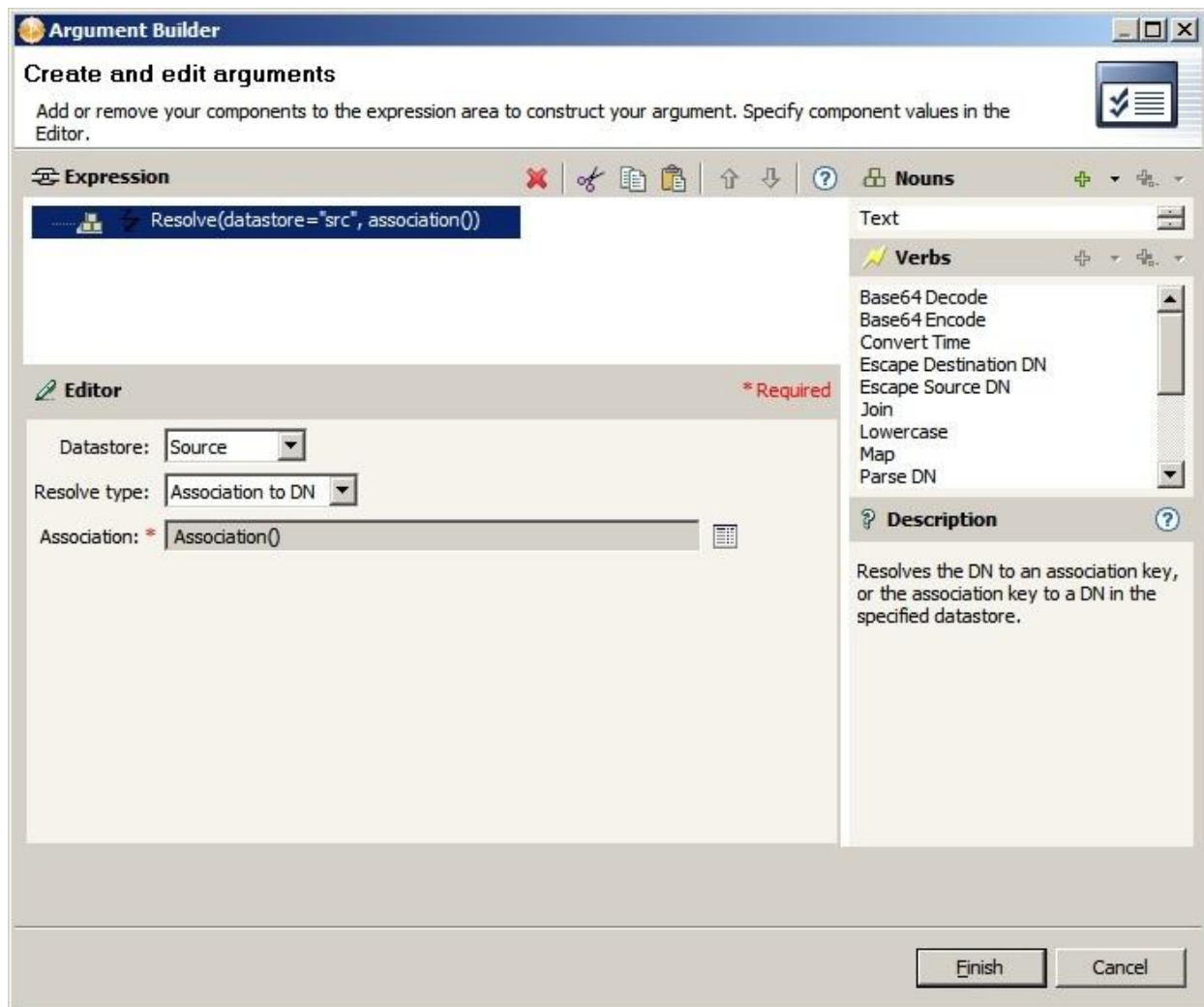
Class name is not required but is useful to scope down your query to a more targeted set of objects. The good news is that **Class name** is not mandatory; thus, there are useful cases to narrow down the query or leave it broadly based.

Select object can let you use three different ways to specify the focus. Root of datastore is useful when doing a subtree from the top down of the tree. DN allows you to specify a proper DN for the entry (exact DN of the object) or subtree to search down from. That DN is provided on the next line. Finally, Association allows the use of the association value to find the object. (Ultimately, the *Resolve* token is a special case of the use of this option.)

Match attributes can be used or not. If used, it lets you find objects that match your specific case. Alas, there is no presence, absence, nor time valued searches supported. In LDAP, you can query for a filter of (myAttr=*) to find all objects with any value for myAttr. By adding an exclamation, (!(myAttr=*)), all objects without values for myAttr will be returned. Alas, the *Query* token cannot perform a query of this sort, which would be really useful if it could. The time value searches referred to above means you cannot do a query with this token as you can do in LDAP, like (passwordExpiration<201209271111Z). Such a search would return those objects with password expirations less than September 27th 11:11 GMT. The ability to do better date range queries would be very helpful in the Work Order driver. Ideally, each polling cycle retrieved all existing work order objects and each object's DirXML-DueDate attribute is considered to see if that due date has come due yet. This works well for a few dozen or a hundred work orders, but I was working on a system with 25,000 work orders. It took almost four minutes on each iteration to work through them. I have a package that intercepts the shim's queries and replaces them with LDAP queries via ECMA functions. This takes about 20 seconds vs four minutes.

Resolve

This is a neat token that is really a special case of the *Query* token. You could reproduce it using the *Query* token or the Java query command quite easily. However, having it all wrapped up for you is very convenient since it is a nice common task. You can give it a DN or an Association value, and it will find the object and return the other thing. You can point it at the source or destination to resolve the value.



One odd thing. When you are resolving an association to a DN and go to specify the Association, you provide the *Association* token, which is really just a wrapper for the XPATH of 'association', but it looks kind of silly: **Association** of *Association()*. Anyway, it looks less silly when you use it to calculate an association value based on other data. If you look at the various possible association values for the various drivers, you can see that many of them are calculated or predictable. Thus, without knowing the actual association value, you might be able to recalculate it to target the appropriate object. It is helpful being able to build a string to provide the association value.

You can see the association values for the various drivers in this article:

<http://www.novell.com/communities/node/1696/open-call-idm-association-values-edirectory-objects>

Source DN

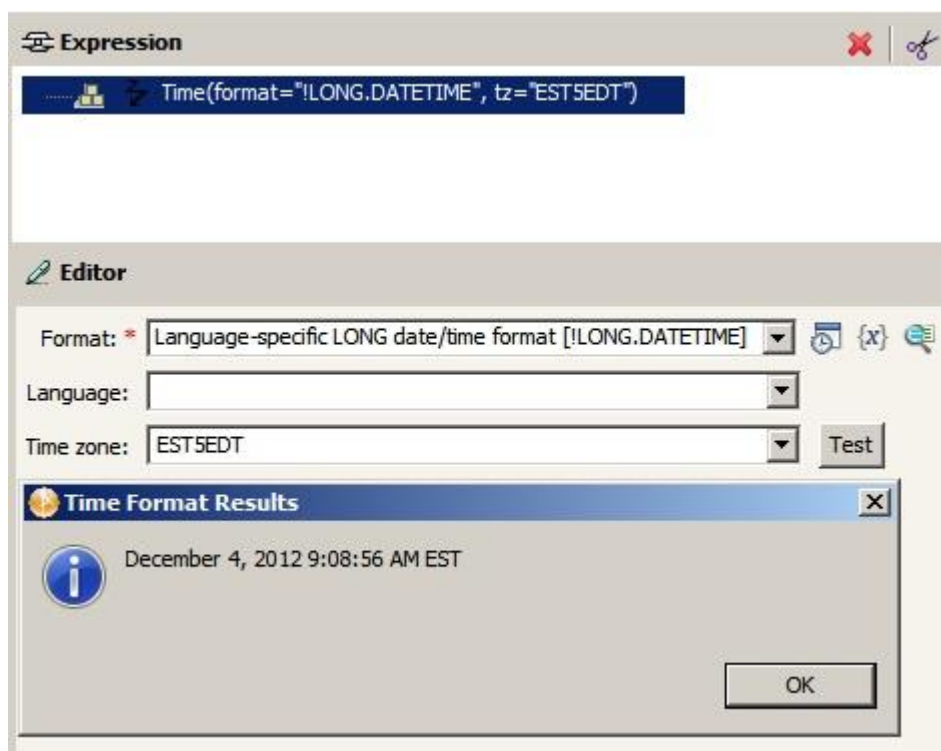
This token is basically the same as the *Destination DN*. It uses the `src-dn` XML attribute on the event node, so it only works if the `src-dn` is there. This is implied by the tooltip. It's not clear what happens when `src-dn` is not there, or why it might not be in the current event document.

Source Name

This is the analogue of the *Destination Name* and has the same issues as *Source DN*. If the `src-dn` is not available in the event document, then this just won't work. I am not entirely sure of all the circumstances that would generate an event document without a `src-dn`, but they definitely happen.

Time

I just plain love this token. The Java `time()` function is very cool and powerful, but it's syntax is reasonably arcane and complicated. This wrapper is a very clever and simple way of providing a GUI to all that awesome functionality. It is so easy to use to get exactly what you want. I have to admit, like with the *Query* token, I would love for an option in Designer to show the generated Java time command so I could use this elsewhere, but it does not provide that.



What is amusing is if you look at the Novell Consulting Advanced Java Class of 100 or more functions, a large number of the functions are actually time conversions functions of different types. These functions are basically made obsolete by this single token (or by the *Convert Time*

verb token, depending on the context). The Advanced Java Class was available as a Java library and, as of IDM 3.6.1, was ported to an ECMA library by Shon Vella. It is now included in pretty much every driver as of IDM version 4. The functions are included in Packages. (It's called NOVLLIBAJC-JS in your Library object at the root of your driver set.)

I wrote about these two tokens (*Time* and *Convert Time*) in this article:

<http://www.novell.com/communities/node/2572/using-time-tokens-idm-35>

In thinking about this token, I realized there are a couple of important things to know. There are a few time formats with names: CTIME, FILETIME, and JAVATIME. CTIME is a count of seconds, since the first second of 1970, and is a 32 bit integer, also known as Unix Time or Epoch time. This is actually a problem since 2 billion seconds or so from 1970 is somewhere in the year 2037, at which point CTIME 'runs out'. It's the Y2K37 server problem all over again! As an additional twist, it depends if you interpret the 32 bit integer as signed or unsigned. Either you can handle time before 1970 with a signed integer (Negative values are before 1970. and positive values are after.) or if you consider it unsigned, you are good until 2106. It turns out that there is some inconsistency in behavior of the tools and data in eDirectory that is being clarified in the latest patches of iManager. NetIQ/Novell will definitely need to do something to deal with this at some point in the future. Though, to be fair, we have 25 years to go. According to Peter Kuo, there is a 64 bit integer data type in eDirectory, even though I cannot see a way in the tools to use it. I never know where he gets his stuff, but he is usually reliable.

Other systems use CTIME as well. If not, the next major alternate time representation is FILETIME. This is a 64 bit integer, signed, that has a lot more room. It does not use seconds as the singular unit, since this would probably give us enough seconds for longer than the predicted lifetime of the universe. People sometimes forget how big 64 bits really is! FILETIME uses a count of 100 nanosecond intervals. A sufficiently small unit of time, that there ought to be 10,000 such units in a second. What this does mean is that when you calculate a time stamp in FILETIME, there are a lot of very insignificant zeros at the end. Especially if you are doing a date only time. In fact, it seems like the IDM engine is only resolving time down to the millisecond level, so the *Time* token cannot generate a time with better resolution than milliseconds. The *Convert Time* token can convert it if you have the data.

FILETIME starts counting its intervals at the very beginning of the year 1601, and this, with half of a 64 bit integer, runs out many many many millennia in the future. Microsoft uses this as the native time format in Active Directory. You will see this in the Active Directory driver where the attributes Account Expiration, Password Expiration, or Intruder Lockout time will synchronize across as a very large integer. This large integer is a timestamp in FILETIME format, and you need to convert it to CTIME. You will notice that an old Java class is still used in the default Active Directory driver. Alas, this interprets zero in a non-optimal way, so I usually modify the rule not to fire if the value coming in is zero.

I am not entirely sure where JTIME comes from. I assume J is for Java, but it is a count of milliseconds since 1970 in some data structure larger than 32 bit. This would have lots more room than even FILETIME in terms of calculating dates and time into the future or the past. I never really use this option.

Unique Name

This token is another convenience wrapper. One of the tasks most IDM projects will have to handle is some method of handling unique names to avoid collisions. The problem is that you set the naming attribute in the Placement rule, but you don't get the error that the name is already in use until it is submitted to eDirectory after the Command Transformation. This was such a common task that they wrote a very nice wrapper to try and do as much work as possible under the covers and give you a simple user interface to control it.

I wrote about this token when it came out in IDM 3.5, and you can read that article here:

<http://www.novell.com/communities/node/2209/unique-name-token-functionality-idm-35>

Argument Builder

Create and edit arguments

Add or remove your components to the expression area to construct your argument. Specify component values in the Editor.

Expression

Unique Name(counter-pattern = "last", counter-use = "fallback", on-unavailable = "error", test-all-objects = "false", "")

Editor * Required

Attribute name:

Scope:

Start search:

Pattern:

When to use counters:

Use counter with which pattern:

Counter start: digits: ☒ Pad counter with leading 0's

When unique name cannot be constructed:

Test all objects:

There was an issue I lamented back then, that it only queries for the current object class, and there is no control to suggest a class to consider. For example, you could, then, call it with one

value on Users, and then use that value to start on a test for Groups. Since there is no object class control, you cannot do that.

Now, as it turns out, you can accomplish this without re-implementing all the useful stuff by calling the *Unique Name* token first, which will run on the current object's object class. You use the *Set Operation Class* token to morph the object to the next class. Then, take the returned result and pass it through another *Unique Name* run. If you need another class, repeat. If not, use *Set Operation Class* to set it back to the original class.

With later IDM Engine and Designer versions, there is an item at the end of the token called **Test all objects** that defaults to false. If you set it to true, no object class is used in the queries generated. It will, thus, make sure the name is actually unique in the target container, regardless of object class, resolving this minor quibble.

This is another complicated (at first glance) token since it has a lot of controls.

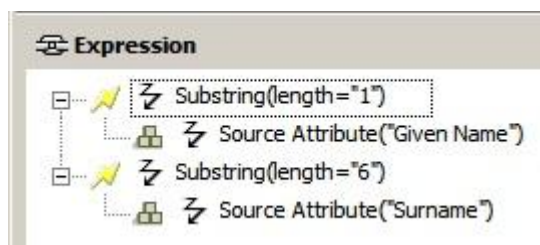
Attribute Name is the target attribute used for the naming attribute. Remember which side of the Schema Map this is executed on to decide which namespace to use. If it is not a mapped attribute, then, use the application name for the attribute.

Scope, Start Search, DN are the same as in the *Query* token and used to control how far to look for possible objects that share the namespace.

Pattern is where the fun begins with this token. You might want users to be named First Initial, then first six characters of the last name. Well, click on the Pattern Builder icon on the Pattern line, and try entering something like this image:



This is actually entered by using the Argument Builder icon on that line to build it using the tokens we have been discussing, like this image:



The possible patterns are limited only by your imagination.

But it gets better! There is no specific number of possible patterns to evaluate. After the above example, you could add a second pattern of perhaps the first initial, then five characters from the last name, and maybe again with four, and maybe again with three. They will be evaluated in order until a unique value is found (or not found since it is looking for the case where the query fails to find anything).

Counters are used when all the patterns fail to find a unique value. That is, append a digit to the end. Thus, you can end up with a user, such as jsmith27, if needed. However, there are actually three cases for using the counter: Fallback, Never, and Always. Fallback, as just described, means when no patterns work to generate a unique value, add a counter on the end to force it to be unique. Never and Always are pretty obvious as their value's suggest to never add a counter (I guess it just fails in that case.) or always add a counter.

When it hits a case to use a counter and there is more than one pattern, the control **Use counter with which pattern** comes into play. Here we see three choices: Last, First, and All. This way you can control which of your patterns (if more than one) is preferred for the appended counter value. First and Last are obvious, but All is interesting. This means it starts looking for values that match with the counter right away. Pretty much the same as first.

Counter Start and **Digits** are used to control the size of the appended number. That is, a start of 1 and digits of 2 means it will try from 1 to 99 as it appends values to find a unique naming attribute.

Pad counter with leading 0's is used to decide if it should append 01, 02, 03, or the like vs 1, 2, 3 when it is allowed to use more than 1 digit for the counter.

When unique name cannot be constructed has four options: Ignore, Warning, Error, and Fatal. These options tell the token what to do when no unique name value can be found. Fatal is the simplest. It generates a `<status level='fatal'>` event, which stops the driver. Error generates a `<status level='error'>` event, which ends the current transaction from being processed. Ignore generates nothing and moves on. Warning returns an empty value that can then be detected and perhaps tried again.

Unmatched Source DN

This is a mighty odd token. It only works if it comes after a *If Source DN* condition token. I am not sure how long the effect of the *If Source DN* lasts. I assume it persists for the duration of the policy object it is executed within, and until the next *If Source DN* token, but I have never seen its duration properly defined. It takes the source DN used in the *If Source DN* condition, and if the current object is further down the tree structure (further away from the root), then the Unmatched part of the Source DN (thus, the name) is the component after the DN tested.

That made even less sense written down. Maybe an example will help clarify. If you have a user who is `CN=geoffc,ou=Users,ou=Data,dc=domain,dc=acme,dc=com` and the *If Source DN* token is checking to see if the user is within the subtree `ou=Data,dc=domain,dc=acme,dc=com`, then the *Unmatched Source DN* is `CN=geoffc,ou=Users` since that is the part that did not match the *If Source DN* subtree. I think that example makes it clearer. If the source DN tested for was `ou=Users,ou=Data,dc=domain,dc=acme,dc=com`, then the *Unmatched Source DN* would just be the `CN=geoffc` in that case.

What may seem a bit confusing in the later Active Directory drivers, is that early in the Matching Policy Set, in fact in the first policy object, `NOVLADDCFG-sub-mp-Scoping` rule there is a rule called 'remember relative position in hierarchy'. This does the *If Source DN* test, and then it copies the *Unmatched Source DN* value into an operation property called `unmatched-src-dn`.

If you were not aware of this rule, it looks like the *Unmatched Source DN* is available as an operation property. In fact, it is just a way for the driver to get the value carried on to other policy objects for use and is NOT part of the token itself.

The Active Directory driver has been doing this for a while, and it is the most common driver in most implementations. This is often the one people see first. This rule is part of the reason why I think that the scope of the *Unmatched Source DN* is at least the policy object level, if not the immediately following action.

Anyway, this is one of those tokens that looks somewhat like magic without much explanation. I do wish the documentation had a lot more explanation of this one.

XPATH:

This is the final noun and is sort of an odd mix. This has a lot of the power packed into a five letter token. Identity Manager only supports XPATH 1.0 and not 2.0 (Lest there be some confusion).

XPATH is possibly one of the more complicated pieces of Identity Manager. Although it is a standard, there is still a fair bit of confusion about its usage. I have tried to address some of the concepts, issues, and ideas with using XPATH in a series of articles. You can read them here:

XPATH Concepts

<http://www.novell.com/communities/node/4833/some-thoughts-xpath-novell-identity-manager>

<http://www.novell.com/communities/node/6175/xpath-and-context-node>

<http://www.novell.com/communities/node/6109/xpath-and-math>

<http://www.novell.com/communities/node/6179/using-string-compares-xpath-statements>

<http://www.novell.com/communities/node/6910/another-attempt-explaining-xpath-context-node>

<http://www.novell.com/communities/node/9214/example-walk-through-using-xpath-identity-manager>

<http://www.novell.com/communities/node/12361/examples-using-xpath-identity-manager>

XPATH Cool tips

<http://www.novell.com/communities/node/5845/using-xpath-examine-association-values>
<http://www.novell.com/communities/node/5686/cool-tricks-using-xpath-nodesets>
<http://www.novell.com/communities/node/4825/using-global-configuration-values-xpath>
<http://www.novell.com/communities/node/6276/using-xpath-get-position-node-node-set>
<http://www.novell.com/communities/node/11637/xpath-do-schema-mapping-rule>
<http://www.novell.com/communities/node/12261/using-xpath-reproduce-map-token>

One thing I like to do when describing one of these tokens that have an easy way to do them in XPATH, is to call out how you could do it with XPATH. For example, while there is a *Class Name* token, the XPATH is simply `@class-name`. This way it provides examples along the way of how you might use XPATH in the IDM context. It turns out that most of the confusion people seem to have is using XPATH in the way IDM intends rather than issues with XPATH itself. Thus, this discussion of tokens also becomes a long set of examples of how XPATH can be used in an Identity Manager context.

Probably the single most important thing to know about XPATH in Identity Manager is that the current context starts at the current event. That is the `<modify>`, `<add>`, `<query>`, or `<instance>` node. In the defense of the docs, they mention this in a single line on a single page. But it needs to be surrounded by stars, in 40 point font, with examples given so people will realize how important that single sentence is in terms of things you need to know about IDM.

Thus, `@src-dn` suffices to get the `src-dn` XML attribute from current node that might be a `<modify>` node. Also `modify-attr[@attr-name='Something']` is sufficient, and there is no need to specify `/nds/input/modify/modify-attr`, for example. In fact, if you try it, it won't work. It is worth noting that if you are trying XPATH inside a Workflow (Provisioning Request Definition, PRD), in either ECMA or within an Integration Activity trying to manage a SOAP call, then you DO need to use the full XPATH path. The current context is different and is the root of the document.

Used as a string or node-set tool, this can select from a node-set, like `modify-attr[@attr-name]/add-value`, and return a single value or a node-set of values.

You can do string manipulation using all sorts of built-in functions. Things like:

- `substring-before()`
- `substring-after()`
- `contains()`
- `starts-with()`
- `concat()`
- `normalize-space()`
- `string-length()`
- `translate()`

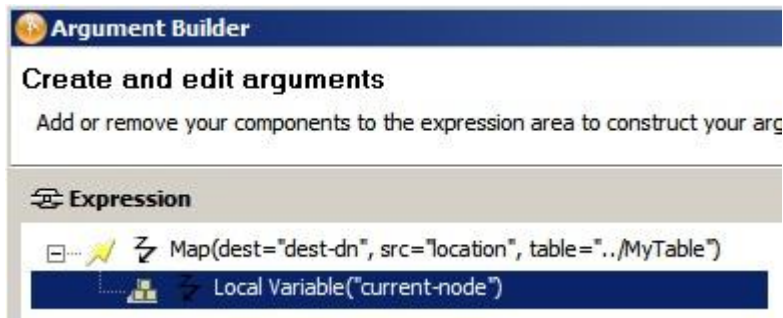
I use these often, since IDM's built-in verb tokens for *Substring* are position based and do not support before or after substring functionality.

You can do math in XPATH. If you need to add a day to a time value, you could use the *Convert Time* token. Version IDM 3.6.1 and later support an offset value. This allows you to convert an input time format to an output time format, and along the way, add some time to it in appropriate units. If you have time in CTIME, just add the correct number of seconds. Math works as you would expect, with normal math operators. You can use variables, and for safety, it is probably best to wrap any variable references (\$VarName) with a `number()` function. In other words, `(number($VarName))` will ensure that the data will be a number and not a string representation of a number. You get a NaN (not a number) error if you needed to do this and did not.

There is so much more to say about XPATH, but that is probably a topic for an entire other book by itself. Consider looking at the articles above for more information and inline for examples from the various tokens of how you might reproduce them in XPATH.

Verb Tokens

That wraps up the Noun tokens in Argument Builder. Now, on to the Verbs. Let's get some actions going! As mentioned at the beginning and in articles all over the place, one of the more common mistakes folk make as they try to use this, is not realizing that Verb tokens are verbs and operate upon 'something'. To use a *Map* token, you need to provide the input data to it, and it will return the output column data. You provide the input data by nesting something underneath the noun. Designer indicates this by indenting the next token inwards.



You can continue nesting, allowing you to start with a noun, and perform multiple actions upon it in one pass. You can get a bit silly with it if you so desire. There are up and down arrows in the Argument/String builder to help you nest things as desired.

Once you have seen it done this way, you will never make that mistake again. You need to see it once if it was not obvious to get you started.

Base64 Encode and Decode

This is a pair of actions that are great to have. Lots of data that cannot be represented as ASCII text needs to be encoded in some fashion to be transportable in a format that is functionally ASCII. In the old Usenet days, we all used uuencode that took 8 bit data and encoded it in a 7 bit format. It looked funny and was about an eighth bigger.

Base64 is the newer, more standardized format of the same thing and works nicely. Sometimes data in a SOAP message will be encoded. Sometimes you might need to encode data to make it transportable. Whatever you need to do, you can provide it via these tokens. Oddly, data ending with a trailing space, accessed via LDAP, is almost always returned Base64 encoded even if not stored that way in the directory. It is the only way that LDAP can preserve that trailing space.

As a funny, "when all you have is a hammer, everything looks like a nail" kind of story, I was offline running on Windows (so no native Linux tools), no access to the Internet, and I needed to play with some base64 data. So I wrote a policy that took an operation attribute from the input document, and base64 encoded and decoded the value. Then, I ran it through Simulator in Designer and made an add event with the operation attribute holding the data. There in the trace was the data I needed. If I had been connected to the network, I would have used one of the free online converters or connected to a Linux box to use some command line versions. Simulator and Designer worked well for me in this case! If you are not familiar with Simulator, you should be. It lets you feed in data, run it through your rule as if it was running on the engine, and see the results. This can be much faster as you do not have to make a change, push it out, restart the driver, generate a test event to fire your rule, find it in trace, and then examine the results. It can save you huge amounts of time.

You can read more about using Simulator in Designer in this article:

<http://www.novell.com/communities/node/12494/using-simulator-designer>

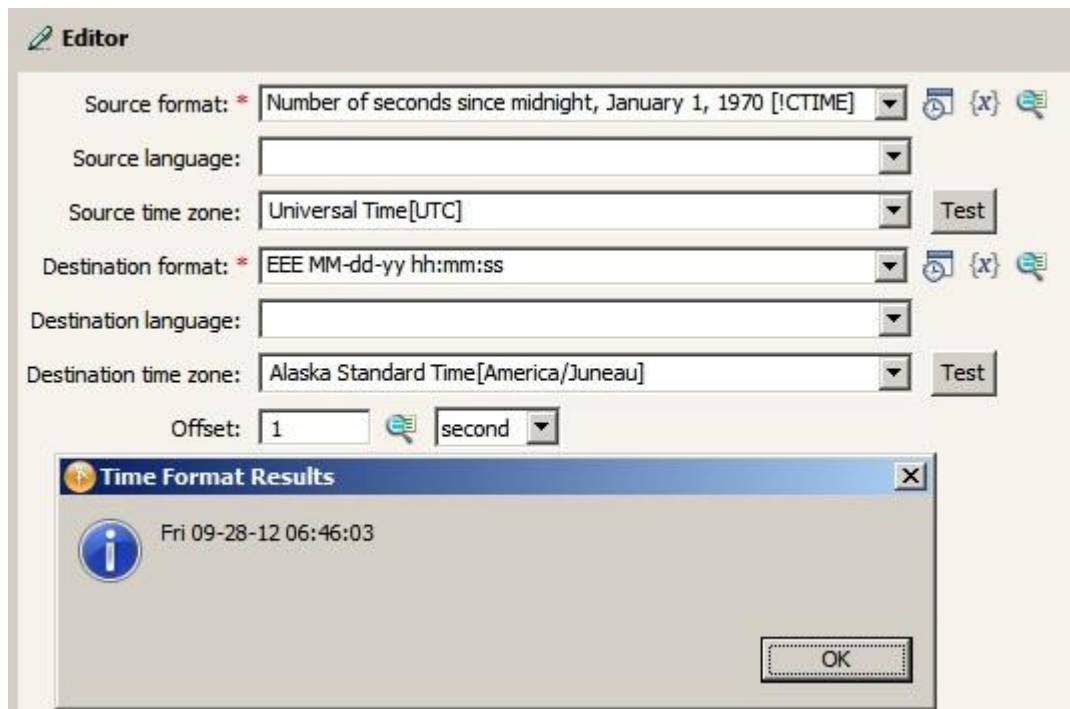
Convert Time

This is basically the same token as the *Time* token discussed earlier and in this article I wrote when it first came out:

<http://www.novell.com/communities/node/2572/using-time-tokens-idm-35>

This differs from the *Time* token, which is a noun, by being a verb. Thus, has a **source formatting** section, a **destination formatting** section, and needs some value passed in.

In fact, if you want to convert the current time format, you would normally use the *Time* token with the correct format. You could, of course, nest the *Time* token under the *Convert Time* token to get the current time in a format you would like.



You might want to do it this silly way since the *Convert Time* token also has an **offset** function. As you convert the time from CTIME to FILETIME, for instance, you might wish to add 90 days to it. In that case, you can use the offset settings. You specify a number to increment, and then a unit that that number represents. For example, you could increment 90 days, or $90 * 24$ hours, or $90 * 24 * 60$ minutes, and so on.

What I forgot to mention in the *Time* token section, and that I think is astonishingly useful is the **Test** button. It will show you the format you have built up so you can see that you got it just right. It uses the current time, since we need to pick some time value, and now is as good as any others.

As I mentioned in the discussion of the *Time* token, I do wish that next to the **Test** button was another button that would show the formatting string that the Java Time function needed. Then, it would be useful in other circumstances where you do not have access to a neat user interface to build the formatting. You can see the results of the test button in the screenshot above.

You can select from a bunch of predefined, pretty common, and useful formats. Or you can build a custom string item by item as there are all sorts of definitions available to build whatever kind of wacky string you need in your rule.



The three icons next to each pattern line give you the controls to get at these formatting strings. The first gives us raw items to use to provide definitions for things like year, days, minutes,

hours, seconds, and so on. The second gives us some common date patterning like yyyy-MM-dd and its ilk. The third icon is the common variable selector, since you could store a pattern in a variable or GCV, which would allow you to control the format in your driver.

The raw date items consist of things like, y for year. Two letter y (yy) means a two digit year value, and a four digit y (yyyy) gives you the four digit year. For month we have uppercase M (since lowercase m is minute) and h for hour. There is week in year as lowercase w, and week in month as uppercase W. Day in year is D, and day in month is lowercase d. This goes on for quite a while with 19 possible values, some more obscure than others.

One neat pattern that I found interesting is that EE is the short name for day of the week like Mon, Tue, Wed. Whereas, four uppercase E (EEEE) means the full name of the day.

Looking at the second selector, you can see some of these patterns made available. There are 16 listed there, and some are kind of neat.

Interestingly enough, if you want to include a raw string that should not be interpreted as a *Time* token, you can escape it with single quotes. It will come out as a raw string. For example, a format string of yyyy-'Testing' would render this year as 2012-Testing.

As always, remember the **Test** button will help you debug the display and get just what you want since you can build a string, test it, and keep iterating until you get it just as you want.

This is a great token, and I use it all the time in my policies. If you have ever used Java Time functions, you will greatly appreciate this token.

Escape Destination DN, Escape Source DN

This is another pair of tokens that do much the same thing, just in different directions. What this is meant to do is to understand the source and destination DN format structures. The connected systems DN format structure is specified in the driver manifest, and eDirectory is well understood. Thus, only one format actually has to be defined, and the choice of source or destination just depends on which channel this is used. You can see that the drivers know some of this information since *Parse DN* knows how to convert to and from the source and destination formats.

When you try and generate a DN, sometimes there are characters that are problematic. For example, in backslash notation (which is what IDM uses to send to eDirectory), how do you send a name for an object in the DN that contains a backslash? It is a delimiter, so if you had it, you probably would break something. Offhand I cannot recall if a backslash is even legal in an eDirectory naming attribute, but there is usually some escaping mechanism, usually a backslash. (So you might have a double backslash in that case.)

Now, you could try and implement something yourself using the *Replace All* tokens, to try replacing any instance of illegal characters with escaped versions, but that is a bit of work. These tokens come out of the box to do it for you and should know how to handle the various formats.

Probably the more common example is LDAP DN notation when you have a name that contains a comma. Commas, of course, are the basic delimiter in LDAP notation. A comma by itself should be expected to break all sorts of things. You are expected to replace any such commas with a backslash and then a comma, for Active Directory at least. This token should do all that for you. Active Directory is a pretty common case for this because the default behavior, when a user is created in Active Directory by Active Directory Users and Computers tool (ADUC, which can be run by loading `dsa.msc` from the file system) is to use the pattern “First Name, Last Name” as the `CN=` component of the DN. One of the most common variations is to make the Helpdesk folk either change it at create time or immediately rename it to a pattern of “Last Name, First Name”. That sorts better to find users in a long list of users. Accordingly, it is pretty common to have a comma in a DN in Active Directory.

Join and Split

This pair of tokens showed up around IDM 3.6 time frame, and they are actually kind of useful. Used together, these can convert a delimited string into a node-set (*Split*) and a node-set of values into a delimited string (*Join*).

If you have ever looked at the Account Tracking packages, used to generate information for Sentinel to use in its Identity Tracking component, it uses an attribute `DirXML-Accounts` that is a # delimited text string. It has the following components:

- Driver GUID
- Object Identifier attribute name for this driver (perhaps `CN`, or the like)
- Value of the Identifier
- Status of the user (A, I, U, -, which are Active, Inactive, Undefined, unknown)
- Realm, which is an identifier for this system.

To manipulate this string, it can be handy to break it up into a node-set of values. Use the *Split* token, specify the # sign as the delimiter, and the result is a node-set of values. Later, when you are done, you could return to the string formatting with the *Join* token.

One thing to keep in mind is that the delimiter field is actually expecting a Regular Expression (aka Regex). If your delimiter has meaning in Regular Expressions, you would need to escape it. Thus, to use a pipe symbol to delimit the string, you would use the Delimiter value of `\|` (Backslash pipe). Keep that in mind in case you start seeing crazy results. (You should try pipe by itself. The results are quite bizarre as it tries to interpret that as a logical OR rather than the literal pipe symbol.)

There is also an option to use the CSV quoting rules. Now, the data can contain commas even though the comma is a field delimiter. No doubt there are a lot of other ways to quote things, but CSV is probably the most common. Short of defining some method to describe a custom quoting format (like the *Parse DN* token has), this seems like a very reasonable compromise.

What is kind of neat is that the field where you are allowed to specify the delimiter itself allows variable replacement. I am not entirely sure I can think of a use case at the moment, but it is kind of useful to know that you could do it if needed.

Once you have a node-set of values, you can loop over it with a *For Each* or a *While* loop. This is slightly different than an XML node-set where you can use XPATH upon it. However, you can use positional notation to get at different nodes in the node-set. That is, you can say `$VarName[1]` as shorthand to get the `position()=1` of the node-set; the first node in the node-set.

Later in the section about the *Append XML* tokens, there is an example of another way to make such a node-set of values (as opposed to an XML document as your node-set). There I was trying to show how you need to be very specific when using the *Append XML element* tokens, since if you specified an XPATH target of just, say, `modify-attr`, then every `<modify-attr>` node in the document gets the new element appended instead of just the one you want. To demonstrate the issue, I needed a node-set of values, and I was too lazy to try and explain how to provide a multivalued input document to Simulator. Instead, in my *For Each* loop, each noun token I specified becomes a separate node when used that way. This is slightly different than in a *Set Local Variable* context where each token in the Argument is really concatenated together to make a string. Perhaps that is the true difference between when the seemingly identical builder should be called Argument Builder vs String Builder. When building an argument in a node-set context, each element you provide as a token is a node. When building a string in a string context, it is all one long string being put together.

It is a little confusing, since it is all the same basic interface, and there is little indication that the underlying rules have changed. Seems like it is left up to the user to understand the subtle distinctions.

Having node-set variables is very useful. You might want to examine them one after another (using a loop), or get a specific piece out of the string. Using the *Split* token to take a string and break it up into a node-set is a great way to get there.

Lowercase and Uppercase

These are very useful tokens since doing this otherwise is quite annoying. You can uppercase or lowercase a string in XPATH using the `translate()` function but wait till you see the format of that function!

```
translate($SourceValue,'ABCDEFGHIJKLMNOPQRSTUVWXYZ','abcdefghijklmnopqrstuvwxyz')
```

This would convert all uppercase characters to lowercase characters. You can see how unwieldy that is.

Another alternative approach would be to call out, via an XPATH token to an ECMA function, to do the task. This is much easier in ECMAScript since there is an uppercase and lowercase function for strings. Having it available in Argument Builder directly is much easier. Previously, Novell Consulting had their Advanced Java Class (now available as ECMAScript) that had a couple of functions to manage case strings.

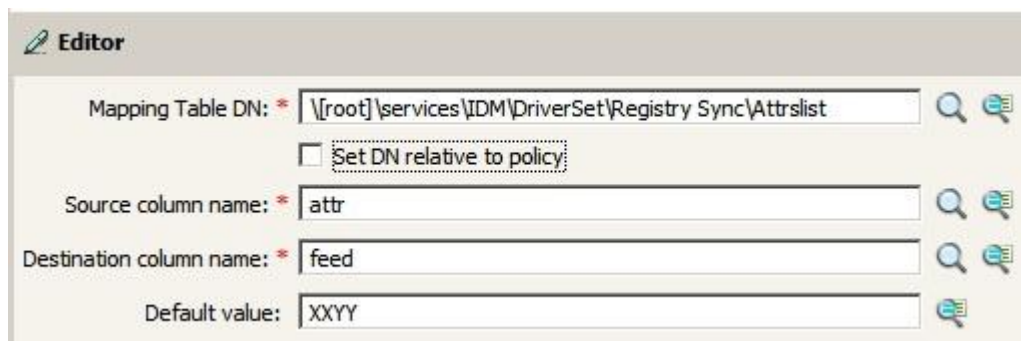
This is another one of those convenience tokens that I am quite glad to have, even if it's not entirely necessary, since it can be done several other ways.

Map

This is a great token. It allows you to use a Mapping Table to store table data. A Mapping Table is a DirXML-Resource object with a DirXML-ContentType attribute value that indicates it's a Mapping Table resource object. It's more of a MIME type definition string, like:

```
application/vnd.novell.dirxml.mapping-table+xml;charset=UTF-8)
```

There are a number of other possible DirXML-ContentType values that Designer supports. When you try to edit a DirXML-Resource, Designer looks at the DirXML-ContentType to determine which editor widget to open it with. Thus, the same underlying eDirectory object can be opened with an XML editor, a text editor, a Map editor, or a Package Prompt editor.



This is probably the verb where I see most people fall into the hole of not nesting any noun to provide data underneath it. They build a *Map* token, then it does nothing since there is no input data being passed in.

I wrote about this token in a couple of other articles that you can read here:

<http://www.novell.com/communities/node/1903/idm-35-update-new-features>

<http://www.novell.com/communities/node/4844/mapping-tables-and-render-browsed-dn-relative-policy-option>

In fact, I worked through an example of how you might implement the mapping token just using XPATH.

<http://www.novell.com/communities/node/12261/using-xpath-reproduce-map-token>

This helped the other day when I was looking at an XSLT policy and was trying to figure out what some XPATH was selecting. Then, I realized they were reimplementing the Map token in XSLT!

I have used Mapping Tables in all sorts of use cases, and I love them. If you can make it work, using them to hold configuration data can be very powerful. For example, phone numbers in Active Directory can be tricky because it is common to have more than one phone number. However, Active Directory stores the first value in `telephoneNumber`, and any additional values are stored in the `otherPhone` attribute as an overflow.

I have a nice and clever policy (if I dare say so myself) that works across the various attribute types. For example, the telephone number case, as discussed, is usually all numerics. There is also `labeledUri` in eDirectory mapped to `wwwHomePage` (single valued) and then overflowing into `url` (Multivalued in AD), which is usually a string. The linkage between the eDirectory attribute to the single valued, main attribute is easy, just use the schema map. But the secondary overflow attribute is not predictable. For example, while `pager` is overflowed into `otherPager`, `telephoneNumber` is overflowed into `otherPhone` not `otherTelephoneNumber` as you might have suspected. I use a mapping table to handle the mapping between attributes. This way I can fairly easily add new attributes to be supported as the customers' whims change like the wind.

On top of that, I can identify if the current attribute is one of those overflow case attributes just by trying to map the value and looking at the returned value. With IDM 3.6.1, a new feature was added, **Default Value**, that allows the token to return a well-known value when NO match is found. I use something silly like XXYY since that is unlikely to ever be real data.

You can see an example of such an implementation using the mapping table to handle load balancing across GroupWise post offices in these articles:

<http://www.novell.com/communities/node/6906/load-balancing-user-placement-groupwise-post-offices>

<http://www.novell.com/communities/node/6908/load-balancing-user-placement-groupwise-post-offices-part-2>

In that approach, I relied on the XXYY returning as the default value as a way of ending a *WHILE* loop. I visited a client who actually had used the same rule almost straight out the article, which was great to see! He had figured out a clever solution. Just adding a final line in the mapping table returned the XXYY value. Clever! I had not considered the pre-IDM 3.6.1

case for using the token, and his solution was trivial, elegant, and simple. The best kind of solution.

You can have as many columns as you would like in your Mapping Table and then can use as many *Map* token calls as you need using different source and destination column names. This lets you store information that changes reasonably often and is kind of hard to calculate the relationships, but by storing it in one location, it becomes reasonably easy to update.

One of the interesting things about the mapping tables is that changes are live, and you do not need a driver restart to pick up the changes. If you can get away with controlling the aspect you need with your mapping table, then you have a reasonably easy interface for non developers to add information without the need to nag someone with access to restart the driver. This allows you to make changes that affect functionality without needing a developer to make the changes, nor restart the driver.

The testing required for this kind of change will depend on your use case. Some things, like placement rules, which place users with a specific JobCode into a specific container, may be a simple change and would not require detailed testing. Whereas, my more complex example of mapping one single valued attribute to two attributes probably needs more testing just to be safe.

There is a very interesting side effect based on how you use the token. That is, if you store the result into a string, you get a string value. Perhaps as a *Set Local Variable* to a string type variable. If you set it to a node-set, you get a node-set with one value. If you, then, treat it as a string, it will concatenate all the values (all one of them) into a single string, thus, making the simple case of one value functionally identical with either approach.

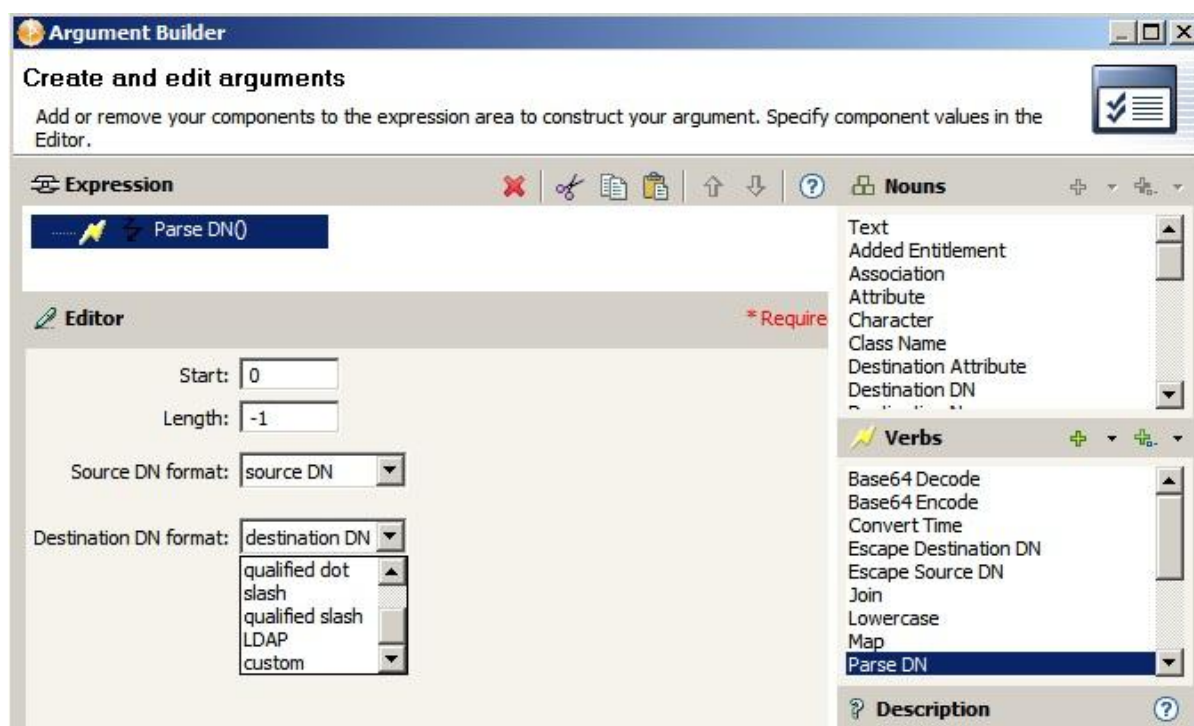
The interesting part comes in when the source column value occurs more than once. A *Map* token call stored in a node-set variable for a source column with multiple values returned looks very interesting. The result is a node-set of values that can be looped over. If you, then, treat the variable as a string, the values will be all squished together.

This is actually quite useful. You not only use a *Map* token to get a single value back, but also a set of values if that makes sense in your case. You might wish to have a set of data returned based on one input value. For example, you might have a bunch of attributes listed in one column with a value in the next column defining what type of case this is. Perhaps in my earlier example of phone attributes, the secondary column value might be 'phone' for all the phone related attributes, maybe 'web' for the web ones, and maybe some third case. Now you could map in the attribute name to see what kind of attribute type the second column says it belongs to. More interestingly, you could map in 'phone' as the source, and if you treat it as a node-set, the result will be all the attribute values that have 'phone' in the second column. This can be quite useful for looping over as needed.

This one came up inadvertently when I just tried it to see what happens and was pleasantly surprised at the results that came back. There are a number of tokens where the results differ depending on how they are treated.

Parse DN

This is one of the more powerful and useful tokens. This is very specific to an Identity Management solution, since DN's usually do not matter that much, and picking them apart and converting them is usually not all that relevant.



I have written more about the *Parse DN* token, including lots of examples, in the following articles:

<http://www.novell.com/communities/node/4337/parsedn-token-identity-manager-and-some-its-limitations>

<http://www.novell.com/communities/node/5687/examples-using-parsedn-token-identity-manager>

Parse DN can be used to change the length of a DN, convert it from one format to another, or both in the same action. It supports pretty much all the known formats that the various connected systems support. It even has a Custom option that allows you to specify a custom descriptor of a DN descriptor format. You can read more about how the Custom definition works in this article:

<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>

The supported formats are:

- Source DN
- Destination DN
- dot
- qualified dot
- slash
- qualified slash
- LDAP
- custom

Source and Destination DN types are shortcuts that allow you to avoid specifying and allow the shim to provide the information. That is, each driver knows what type of DN its system is expecting, and everyone knows what eDirectory is expecting. By default, therefore, *Parse DN* starts with Source DN to Destination DN conversion settings.

Dot syntax would be `geoffc.users.acme.com` or the like. I have not seen a system that specifically expects dot syntax, but I do know you can login into various eDirectory things with dot notation. If you need to pass a variable, it might be helpful to be able to generate it.

Qualified Dot would be something like `cn=geoffc.ou=user.dc=acme.dc=com` that looks a lot like LDAP syntax. It is using periods instead of commas. I am not aware of any specific system that uses this syntax either, but it is an easy one to handle so it is included.

Slash is the native format that IDM uses for the Identity Vault and is root most first `\Tree\com\acme\users\geoffc`. One important trick is that if you start with a leading backslash, then you need to specify the tree name. If you start without the leading backslash, then `com\acme\users\geoffc` will suffice without the tree name. It makes more sense if you think of it as similar to file paths in DOS. In a DOS path, a leading backslash starts you at the very root of the filesystem, whereas, just a path starts relative to where you currently are in the path.

It really should be called backslash format, but the assumption is that the slash in question is the backslash.

Qualified slash would look something like this example, `dc=com\dc=acme\ou=Users\cn=geoffc`. Qualified slash is an important format that the engine provides on events coming out of the Identity Vault.

If you need to convert an eDirectory DN to LDAP format, you cannot do it from backslash format since there is missing information. Is that `com\acme` really `ou=acme,o=com` or is it `dc=acme,dc=com`? Using just backslash syntax, there is no way to know since the information you need is just not there. So what you need to do is query for the object somehow, get the result into a node-set variable, and, then use an XPATH expression something like `$VarName/@qualified-src-dn` to get the qualified slash notation DN that will be returned in queries to eDirectory. Then, you can *Parse DN* that value to LDAP format. I do wish either the *Resolve* or *Parse DN* tokens would be

enhanced to support doing this under the covers for us since this is so common a need. Alas, not there yet.

Interestingly enough, Notes (or Domino) format `CN=geoffc/OU=Users/OU=Acme/O=com` is not directly supported. You should be able to use the Custom option to define a Notes compatible approach.

I think, at least to a first approximation, this custom delimiter string of: `10//+=*\` should work to handle Lotus Domino syntax. You can read my article on custom syntaxes to figure out why that might work.

<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>

LDAP syntax looks a lot like qualified dot, just using commas instead, and is probably the most common format people are used to seeing. Using our example, it would look something like:

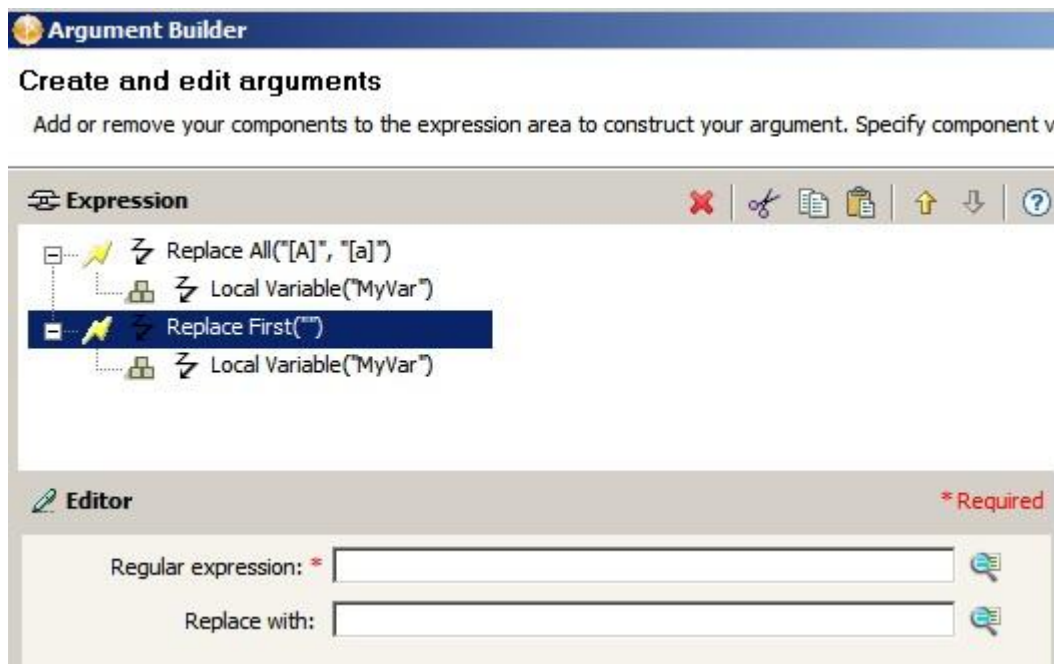
```
cn=geoffc,ou=user,dc=acme,dc=com
```

There is also a Java class `DNConverter()` that, I assume, this token calls under the covers. The JavaDoc does not make it clear how to accomplish all of the cool stuff the token can do. Only a few simpler functions seem exposed by the JavaDoc.

Replace All

Replace First

These two tokens are great tools since they allow you to replace text using Regular Expressions.



Regular Expressions are very powerful and way beyond the scope of what I can discuss here. There are tons of web resources and tools to help you cheat. I personally cheat with RegxBuddy, since it really is a nice Regex tool for Windows, that lets you build and test expressions, run them against sample data, and even has an explain mode. That is, you can take a complex Regex you got from somewhere, put it into the explain mode to get, at least an inkling, of what is happening item by item. Alas, it is not magic. For complicated expressions, it probably is not that great of a help. At least it shows you item by item what is happening and lets you try and figure it out.

I wrote a review of Regex Buddy that you can read here:

<http://www.novell.com/communities/node/4339/review-a-great-regular-expression-development-tool-regex-buddy>

The good news is that simple strings with no special characters replace as simple strings. So if all you need is a simple string based replacement, then you are good to go.

One major bug in these tokens is how the engine does variable replacement when there are special characters in the variable such as `|`, `\`, or `(,)`. The problem is when the engine expands the variable. It used to put in those little square boxes (At least that's how it showed up in trace.) like a binary, non-displayable character. They sort of fixed that bug, and instead it started escaping the funny characters, which breaks their usefulness in a Regular Expression.

However, the workaround is to use a GCV and use the `~GCV Name~` notation instead of `$GCV Name$` notation. The difference is in how the engine replaces these global variables. I discussed this in my series on things beginners in IDM ought to know:

<http://www.novell.com/communities/node/13126/common-mistakes-newcomers-idm-make-part-5>
<http://www.novell.com/communities/node/13493/common-mistakes-newcomers-idm-make-part-11>

Basically, the engine will replace all the squiggle (the ~GCV Name~ approach) notation instances of a GCV when the driver starts up. This is basically a search and replace on the driver configuration as it loads up. Thus, by the time it gets around to executing the *Replace All* token, it is not evaluating a variable. The text is already there.

When you're using a GCV surrounded by dollar signs (\$GCV Name\$, for example), at run time and each time through the policy (especially if you had looped it), the engine evaluates the value for use in the token. This is where the bug shows up. For the most part, this is tolerable. Just store your replacement string in a GCV and be done with it. However, there are cases where you want to use a variable, whose value you are calculating on the fly, a moment before in the rule. Alas, that may be harder to do if it contains any special characters. A special character is the sort of thing that might be above the number keys on your keyboard, or pipe (|), or slashes (forward or backslash). Alas, these are the key tools used in Regex to indicate something other than just a simple string.

I was working with a client whose IDM infrastructure predates the *Time* and *Convert Time* tokens (see earlier), and not knowing to use the Java time classes, the developer came up with a clever (but limited) way to transform dates from MM/dd/YYYY format to more of an LDAP time format like YYYYMMddhhmmssZ. It was such a clever use of Regex that I wanted to show this really out-of-the-box approach.

```
<do-set-local-variable name="newbirthdate">
  <arg-string>
    <token-replace-first regex="^(\\d\\d)/(\\d\\d)/(\\d\\d\\d\\d)" replace-with="$3$1$2000000Z">
      <token-local-variable name="MAIS_BDATE"/>
    </token-replace-first>
  </arg-string>
</do-set-local-variable>
```

The Regular Expression is:

```
^(\\d\\d)/(\\d\\d)/(\\d\\d\\d\\d)
```

We can break this up, where the caret (^) means look at the beginning of the string, and only match starting there. The brackets () make a capture group, which means find exactly this pattern, and capture it, functionally into a Regular Expression specific variable. Inside the brackets we have \\d, which is a character set shorthand for numerics (digits, thus the d). This means just 0-9. In Regex you could equally say [0-9] to define a set of values. Thus, uppercase letters could be [A-Z] and all letters, upper or lower, could be [A-Za-z].

This is going to get three groups captured: the first two digits (the month), the second set of two digits (the day), and the final four digits (the year). These get captured and are available in the

Copyright© 2013 Ninja-Tools

variables \$1, \$2, and \$3. These are not XPATH variables, rather, these are Regex variables useful only in this immediate token context.

Thus, you can see that the replacement string used is \$3\$1\$2000000Z. This says put the third capture group (\$3 representing year), then the first capture group (\$2 representing month), then the second (\$2 representing day), followed by 6 zeros for hours, minutes, seconds, and then Z for time zone of Zulu (aka GMT or zero time zone).

You can see this approach only works if the date formats are just string reformat of each other. This will not do anything if there needs to be conversions to other formats like CTIME. But it is very clever, and I had not realized, until I saw this, that you could use the capture groups to reformat text strings in this fashion.

In general the Regex defaults to either case insensitive or sensitive in IDM, and I can never remember which. Do you think it's in the docs? Ha, Ha, I say! Unlikely. But you can control it with some Regex inline control sequences that you can add to your replacement string. If you add (?i) to the beginning of your Regular Expression, it will force case sensitivity, and (?-i) will force case insensitivity.

You can do all sorts of fun stuff with Regex, which is great. Now, it turns out that you might need Regex in XPATH, which is a bit annoying since by default it does not support it. You can, of course, do your work in stages. That is, generate the first piece via XPATH. Then, apply a *Replace All* token on top of it or the converse, whichever works. However, David Gersic wrote a great article explaining how to implement an ECMA based Regular Expression compare function that can be called in XPATH. This is very useful, and you can read about it here:

<http://www.novell.com/communities/node/8434/adding-regular-expression-matching-xpath-identity-manager>

This is nice because the `contains()` function in XPATH is often not sufficient for the task you need to perform, and a Regular expression match would be nice. This solves that problem. Once you have it in an ECMA library in your project, you can link it into any driver that needs it. I wonder how one gets something added to the Advanced Java Class (LIBAJC-JS) library that Novell now provides as an ECMA library of functions?

The difference between *Replace All* and *Replace First* is primarily about how often the match is found. If you wish to replace only the first instance, then use the *Replace First* token. If there are possibly many such matches, then use *Replace All*.

Substring

This is a simple token that lets you select a segment of a string based on positions. Not much special here. You get to start from 0 (beginning) to the end (-1) of the string.

The length defaults to -1 designating the final character. In the context of *Parse DN*, the docs explain how to use the negative notation to figure out how to move backwards from the end. It was sufficiently complex. After I read it, I experimented a bit and realized that -1 is the last character. The second last character is -2. The third last is -3 and so on. I wish they could have just said it that simply so it would be easier to understand.

You can start at -1 and go for 1 length to get the last character. You can start at -3 and go for 3 characters to get the last three characters. Lots of possibilities with this token.

As it turns out, the XPATH equivalent string functions are probably more useful. XPATH has a `substring()` function that works much like this one, but there is a `substring-before()` and `substring-after()` set of functions that are quite useful.

For example, you can easily pick apart an email address with `substring-before('geoffc@acme.com','@')` to get the naming part, and `substring-after('geoffc@acme.com','@')` to get the acme.com mail domain.

One approach, prior to the release of the *Split* token, was to use `substring-before($VAR, '#')` and split a string with # into a node-set. If you looped through it and chopped the variable up, you could get the various parts you needed.

Thankfully, the *Split* token was added, which is way easier.

As you can see, this is a useful token but does not cover the complete space you might expect for substringing operations. Since we have XPATH as an alternative, we have a bunch more options that makes things much better.

Remember that a lot of the things you might wish to do with other substring-like functionality can probably be replicated using Regular Expressions in the *Replace All* or *Replace First* tokens that were discussed earlier.

XML Parse

XML Serialize

These two tokens, *XML Parse* and *XML Serialize*, are a matched pair much like *Base64 encode* and *Base64 decode*. These turn out to be really useful tokens, and in fact the other day I saw what it takes to implement them in XSLT and realized how much easier these tokens are!

To do this in XSLT, you need to do the following for *XML Parse* functionality.

Declare the proper namespaces in the first node:

```
<xsl:stylesheet exclude-result-prefixes="query cmd dncv exsl es sldr parser pfac" extension-element-prefixes="es"
version="1.0" xmlns:cmd="http://www.novell.com/nxsl/java/com.novell.nds.dirxml.driver.XdsCommandProcessor"
xmlns:dncv="http://www.novell.com/nxsl/java/com.novell.nds.dirxml.driver.DNConverter"
xmlns:es="http://www.novell.com/nxsl/ecmascript" xmlns:exsl="http://exslt.org/common"
xmlns:parser="http://www.novell.com/nxsl/java/com.novell.xml.parser.XMLParser"
xmlns:pfac="http://www.novell.com/nxsl/java/com.novell.xml.parser.XMLParserFactory"
xmlns:query="http://www.novell.com/nxsl/java/com.novell.nds.dirxml.driver.XdsQueryProcessor"
xmlns:sldr="http://www.novell.com/nxsl/java/java.io.StringReader" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

Then the following code:

```
<xsl:variable name="acadCareerDataXml" select="string(../value)"/>
```

Get your string value into a variable:

```
<xsl:variable name="parser" select="pfac:newParser()"/>
```

Initialize the parser object:

```
<xsl:variable name="acadCareerData" select="parser:parse($parser, sldr:new(string($acadCareerDataXml)))/>
```

Then use the StringReader function to get the proper type of string and pass it into the parse function.

Compare that approach with the simplicity of using a `<token-xml-parse>` node in Policy, and my point is made.

Now, there are lots of interesting use cases for this token. If you were to, say, read the `DirXML-ConfigInfo` attribute from the driver, you would get back text that contains XML. Alas, you cannot use XPATH to select stuff in text because it needs to be converted to a DOM object (Document Object Model). So while you could probably contrive a Regular expression to find the value nodes you need, knowing that there will be something like an XML attribute `name="Name of the Setting you want"`, doing it in XPATH is way easier. You might use an XPATH statement such as:

```
$CONFIG-VAR//[@name='Name of the Setting you want']/value
```

This example handles the cases of a single value, a list (a bunch of `<item>` nodes under the `<value>` node) or a Structured GCV. The result will be different in the three cases, so you should know in advance what is being selected.

What you would do in that case is read back the data; and if you used a *Query*, *Source*, or *Destination Attribute* token, you could nest that under an *XML Parse* token in one operation. Make sure to set the local variable to a node-set type for this to work. Or else, you could do it in a couple of steps. First, get the values into a string variable CONFIG-VAR. Second, *Set Local Variable* of type node-set, CONFIG-VAR-XML as *XML Parse* the *Local Variable* CONFIG-VAR. Third, you can start using XPATH upon it to get the bits and pieces you need out of the variable.

In Policy Builder this would look something like:

```
set local variable("CONFIG-VAR", scope="policy", Source Attribute("DirXML-ShimConfigInfo", dn(Global Configuration Value("dirxml.auto.driverdn"))))  
set local variable("CONFIG-VAR-XML", scope="policy", nodeset(XML Parse(Local Variable("CONFIG-VAR"))))
```

In DirXML Script this would look like:

```
<do-set-local-variable name="CONFIG-VAR" scope="policy">  
  <arg-string>  
    <token-src-attr name="DirXML-ShimConfigInfo">  
      <arg-dn>  
        <token-global-variable name="dirxml.auto.driverdn"/>  
      </arg-dn>  
    </token-src-attr>  
  </arg-string>  
</do-set-local-variable>  
<do-set-local-variable name="CONFIG-VAR-XML" scope="policy">  
  <arg-node-set>  
    <token-xml-parse>  
      <token-local-variable name="CONFIG-VAR"/>  
    </token-xml-parse>  
  </arg-node-set>  
</do-set-local-variable>
```

With *XML Serialize*, you can do the other direction. Perhaps you wish to catch the error messages that come back in an event. To do this, you would test for an operation of type status. You can also test for the *If XML Attribute* level and if it is equal to error. This is the same as XPATH of `@level='error'` since the 'level' XML attribute is in the `<status>` node. The `<status>` document you get back has a couple of nodes with error information. Well, if you just try to send that message as a string, treating the event document as a string, it does not send the info you are looking for. The problem is an XML node-set document does not work as a string directly; rather, you get the string parts strung together with no spaces. It just looks funny. However, you could *XML Serialize*, XPATH of `.` (period for the current operation), or local variable `current-op`; and you should get a text version of the event document XML. Sending this on can be helpful since it will also capture any data in the `<operation-data>` node (The operation properties in case you had any information there).

Another interesting use for these is to build up some arbitrary XML document. You could start by *Set Local Variable XML-DOC* to a node-set variable based on *XML Parse* of the text `<top-node/>`, and you have an XML node-set with just the parent node. Now, you can use the *Append XML Element*, *Set XML Attribute*, or *Append XML Text* tokens to target that variable; and add the nodes, attributes, and text into the document as you see fit.

The target XPATH expression would be something like `$XML-DOC/top-node` to start with and build up as appropriate.

Once this is done, you could *Clone by XPATH* to copy your variable's content into the event document or perhaps into the `<operation-data>` node, as discussed in the *Operation Property* noun earlier.

You might want to send XML in an attribute. In which case, you could *XML Serialize* the variable as the string you wish to send.

If you watch that in trace, the engine knows to escape the less than signs (`<`) with the appropriate escaping character, which if I type here, I am not sure how many times I need to escape it. Since to show the literal escape character in this article on a web page, I think I need to escape my escape characters. My head hurts.... Thus, your trace will look interesting, to say the least, as all the white space will be lost as well. It does work without much of a problem.

When you need to do this sort of thing, these tokens do a great job of making it a piece of cake.

In the SOAP driver, I was able to use the approach of the *Append XML* style tokens to build up a replacement SOAP document and copy it into the current event. Then *Strip by XPATH* to remove the original event. This allowed me to do a neat driver to Salesforce.com using SOAP without a stitch of XSLT required. That was a lot of fun, and you can read about the gory details in this series of articles:

<http://www.novell.com/communities/node/11778/getting-started-building-soap-driver-idm-part-1>
<http://www.novell.com/communities/node/11807/getting-started-building-soap-driver-idm-part-2>
<http://www.novell.com/communities/node/11810/getting-started-building-soap-driver-idm-part-3>
<http://www.novell.com/communities/node/11850/getting-started-building-soap-driver-idm-part-4>
<http://www.novell.com/communities/node/11982/starting-soap-driver-idm-part-5>
<http://www.novell.com/communities/node/12022/getting-started-building-soap-driver-idm-part-6>
<http://www.novell.com/communities/node/12052/getting-started-building-soap-driver-idm-part-7>
<http://www.novell.com/communities/node/12138/getting-started-building-soap-driver-idm-part-8>
<http://www.novell.com/communities/node/12144/getting-started-building-soap-driver-idm-part-9>

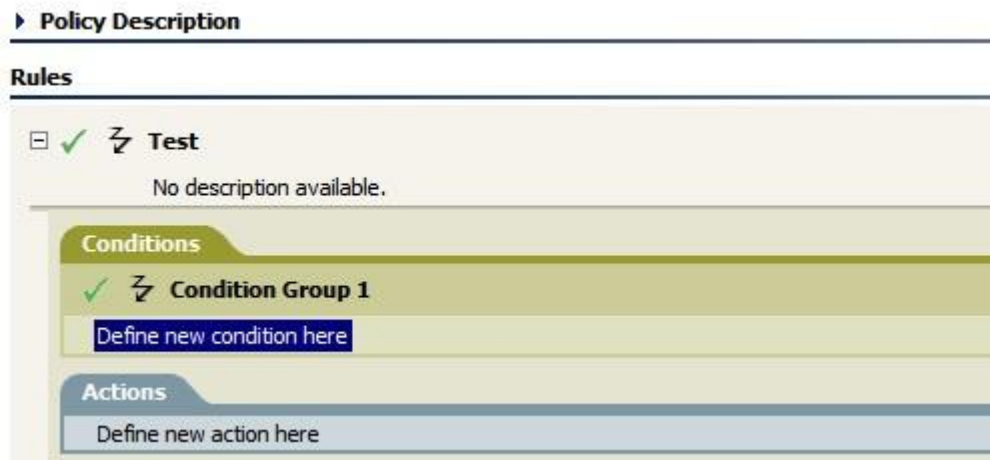
As you can see, there was a fair bit of detail involved, but once you get the basic premise, it is pretty easy. Having said that, my boss came up with a very clever approach to SOAP drivers that tries to use configuration information to handle building the SOAP documents instead of specific policies. It is very clever and complicated. I need to find some more time to muck about with it and convert my Salesforce.com driver into his interesting format.

That about wraps up the Verbs, with Nouns before that, so it seems like it is time to move on to the Conditions and all the tests that are available there!

Condition Tokens

This should be fun as there is a ton of hidden detail that is not really well explained in the documentation.

First off, there is the entire Condition block itself. Initially it only shows a single condition group, that conditions can be added to, in the **Define new condition here** area.



But you might not know about the ability to add a Condition Group as well as a new condition. If you right click on the **Condition Group 1** line, you get a menu with some options. There is a **New** submenu, with the option to **Insert Condition Group Before** or **Insert Condition Group After**. Of course, once they are inserted, there are up and down arrows in the upper right of the window to allow you to reorder each item. You can reorder the Conditions, Condition Groups, or later Actions as well.

► Policy Description

Rules

☐ ✓ ⚡ **Test**
No description available.

Conditions

✓ ⚡ **Condition Group 1**
Define new condition here

✓ ⚡ **Condition Group 2**
Define new condition here

Actions
Define new action here

- New
 - Insert Condition Group Before
 - Insert Condition Group After**
 - Append Condition...
- Expand All Conditions
- Collapse All Conditions
- Move Up
- Move Down
- Cut Ctrl+X
- Copy Ctrl+C
- Paste Ctrl+V
- Delete Delete
- Undo Ctrl+Z
- Redo Ctrl+Y
- Preferences...

That is, I could have an example such as:

	If Class name = User
AND	If operation attribute Login Disabled available

In Policy Builder that might look like this:

Policy Description

Rules

☐ ✓ ⚡ **Test**

No description available.

Conditions

✓ ⚡ **Condition Group 1**

✓ ⚡ if class name equal "User"

And

✓ ⚡ if operation attribute 'Login Disabled' available

Actions

Define new action here

That is two conditions that both must be true (AND'ed together). However, you might want an entire second case. In which case, you could add a condition block with a second set of conditions that is then OR'ed to the first one.

OR	AND	If Class name = User
		If operation attribute Login Disabled available
	AND	If Class name = Student
		If operation attribute Login Disabled available

In Policy Builder this might look like:

► Policy Description

Rules

☐ ✓ ⚡ Test

No description available.

Conditions

✓ ⚡ Condition Group 1

✓ ⚡ if class name equal "User"

And

✓ ⚡ if operation attribute 'Login Disabled' available

Or

✓ ⚡ Condition Group 2

✓ ⚡ if class name equal "Student"

And

✓ ⚡ if operation attribute 'Login Disabled' available

Actions

Define new action here

Of course, you can switch it so it is AND'ed condition blocks and OR'ed inside the conditions just by clicking on any of the AND or OR icons.

Many conditions support an equality test with a Regular Expression compare. For example, *If Operation* supports a neat way to test multiple conditions in one. When testing for equals with a regular expression instead of case insensitive as the compare type (shows in the XML as 'match' instead of 'equal'), you can use a value of `add|modify|delete`. The pipe symbol (`|`) is a Regular Expression OR test, thus, the test is for any of those three operation types.

► Policy Description

Rules

☐ ✓ ⚡ Test

No description available.

Conditions

✓ ⚡ Condition Group 1

✓ ⚡ if operation equal "add|modify|delete"

Actions

Define new action here

So you could have an OR nested in an AND'ed condition, and then the whole thing OR'ed to another condition group. If this does not offer you enough flexibility, you need to reconsider your requirements.

As before, it is just easiest to work through in order of the drop down, so let's start with the A's.

If Association

This is a condition that deals with the association value. When you get an event document from eDirectory, you will see an <association> node, if the object has a DirXML-Associations value, whose DN component is for this driver.

Conditions

✓ ⚡ Condition Group 1

Define new condition below

Condition: association ?

Operator: equal

Mode: case sensitive

Value:

OK Cancel

Condition modified. Click OK to update the policy or click Cancel to discard changes.

* Required

If there is one, you will probably see a state XML attribute. Something like one of these possibilities:

```
<association state='associated'>
<association state='migrate'>
<association state='pending'>
```

Alas, this token cannot test all these states entirely, though, you can get pretty close. *If Association* available will be true as long as there is a value in the <association> node. When you chose the **Operator** associated, if the current event document does not show it as associated, the engine will query the object for its DirXML-Associations values. This is sort of like how the *If Entitlement* is available token will work. If the current document does not have the proper entitlement in it, then it will query back to the source to be certain.

The various **Operators** are things like the following list:

- associated
- available
- equal
- not equal
- greater than
- less than
- not available
- not associated
- not greater than
- not less than

This set of operators are pretty much the same for most of the Condition tokens with some minor differences to make sense in context.

Equal makes a lot of sense. Once you get to the equal, greater than, less than, and the various negative versions, you get the usual equality modes. You can test for case sensitive, case insensitive, and numeric; but most importantly, Regular Expression is one of the choices. With a regular expression (regex), you should be able to select anything you need if it is predictable and follows a regular pattern. You can find web sites all over with samples of crazy Regular Expressions, but even simple ones should suffice.

The greater than and less than cases don't make a heck of a lot of sense in the context of an association unless your association value is a numeric, then you can imagine a case where it matters. Since the functionality was all there, I suppose they might as well leave it in for this condition.

When testing for equality, the **Value** field does not let you build a string. If you select the magnifying glass icon on the side  , you strangely get an eDirectory tree browser. Your options are to have a value ready as a literal string, or to use a previously calculated variable. In a previous rule, you might have set some value as the association you are looking for. (These are not the Associations you are looking for....) In the condition block, you could test *If Association* is associated (or available depending on what you need). Then, in the Actions, set the value you care about up in a variable. Then, in a nested *If-Then* block, test for *If Association* equals \$VarName\$. That is, use \$VarName\$ in the **Value** field to use a variable for the value.

If Attribute

If Destination Attribute

If Operation Attribute

If Source Attribute

I want to lump the four attribute tokens all together into one section because they are all basically the same concept and interrelated, just with interesting twists. First up is a major limitation of these condition tests compared to the noun token equivalents. When talking about the Nouns, you could specify as the target object for Source and Destination attribute the DN or Association of some other object. In those tokens, you can get data from another object. Here you can only work with the current object. That is, you can look at the operation attribute value of this object (Bad example as operational attribute does not make sense in the context of another object.). You can look at the source or destination data for this object in the condition but not point at a different object to test.

To do that you would need to, in a previous rule, read the data back into a variable, then use the *If Local Variable* test to compare it.

I have written fairly extensively about these tokens in some other articles. This is a good indication of how powerful and useful they are. You can find those articles here:

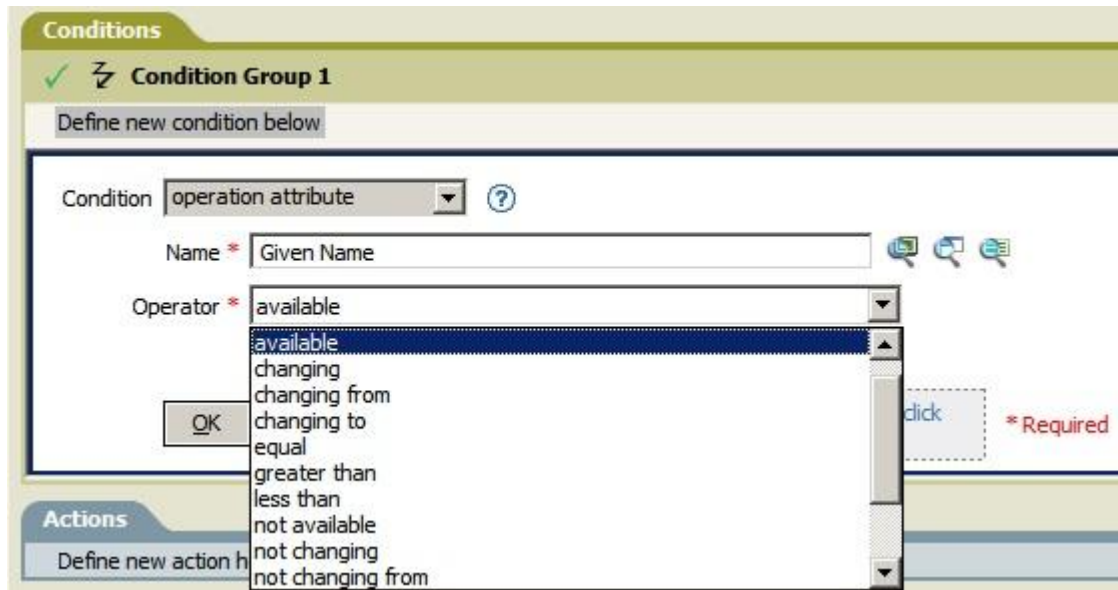
<http://www.novell.com/communities/node/5818/different-attribute-options-identity-manager>
<http://www.novell.com/communities/node/5920/ism-more-thoughts-sourcedestinationoperation-attribute-tokens-identity-manager>
<http://www.novell.com/communities/node/13144/various-attribute-tokens-idm-policy>

Additionally, in the first article in this series, I discussed the Noun versions of these articles. You can read that earlier in this book.

What is a little annoying is that the Value field in the interface gives you an object picker as the primary selector and secondary is the Variable picker. I never did understand this interface choice since it almost never occurs that I want to compare an attribute value to a DN. However, the Variable selector gives you a lot of possible options to select from: GCVs, Local Variables (policy scoped, driver scoped, error variables). It will insert the format properly for you. In this context, it is \$VarName\$ whether it is local or global variables. You could use ~GCV Name~ for a GCV value if you wanted to, but \$GCV Name\$ or \$Local VarName\$ will both work.

The *Source* and *Destination Attribute* tokens basically look in either direction. They will query the appropriate data source depending on the channel you are in. Source in the Subscriber channel is the IDV; Destination in the Publisher channel is also the IDV.

The *If Operation Attribute* token looks at the current event document, and here there is an interesting twist in terms of what changing vs available means in the operator field.



An operation attribute is changing if it is in the document. That means it could be a remove value by itself, a remove all values by itself, an add value by itself, or any supported combination of them. Very common is a remove all values and an add value in one event. (This would be caused by a *Set Destination Attribute* token, for example, whereas an *Add Destination Attribute* would just throw in an add value.)

An *If Operation Attribute* available condition is only for the case of there being an add value in there. So you could have a test for *If Operation Attribute* is changing then, if true, a nested test *If Operation Attribute* available. If it is available, then you know there is some kind of remove and add happening. (It could be remove two values and add one value, or a remove all values and an add value.) If the operation attribute is not available, then you know it is a remove only case. I think it is a bit trickier using these tokens to test for an add value only case. You can do it with XPATH, which you only need to really care about in a modify case, since an `<add>` event is always an add value and no remove values.

For example, this set of conditions would test for add only cases:

```
If XPATH of modify-attr[@attr-name='MyAttribute']/add-value/value is true
and if XPATH of modify-attr[@attr-name='MyAttribute']/remove-value/value is not true
and if XPATH of modify-attr[@attr-name='MyAttribute']/remove-all-values is not true
```

I am sure there is a more elegant way in XPATH to do it, but this works and is easy to read and understand.

The *If Attribute* condition token is kind of a neat compromise. If a value is found in the current event document, then it will return that. If not, it will query back to the source to get the current

values of the attribute to return. This can be nice for efficiency since it will try and minimize the querying if the data is already in hand.

Do be careful. Sometimes the data in the current event is not the complete picture, specifically in the case of a modify. In an add case, you can rely on this; in a modify case, you might want to still look back at the source. We will see later that the *If Entitlement* tokens will do something similar. If the event is an entitlement grant, then it will be satisfied with the event document; but if it is not, it will check back at the source.

If Class Name

This is a simple token that can be replicated in XPATH easily with a test of *If XPATH* is true, `@class-name='SomeClass'`, but it can be very convenient to have a token available. Often I find I need to use the class name value in an XPATH test later, so I find myself storing it in a variable. In which case, I could also test the variable value. Though, in a complex XPATH I could actually use the `@class-name` to get the value if needed. For example, `contains(@class-name,"User")` could use the XPATH of `@class-name` to provide the value to `contains()`. Perhaps in a variable replacement case, I would find the variable of the class name handy. For example, an XPATH of `modify-attr[@attr-name="Object Class"]/value[text()=$CLASS-NAME`.

If Destination DN

If Destination Name

If Source DN

If Source Name

These tokens suffer the same issues the noun tokens suffer, as discussed earlier. They rely on a `dest-dn` or `src-dn` XML attribute being available in the event document.

For example, the event node might look like:

```
<add class-name="User" src-dn="com\acme\users\JSmith">
```

That is, on an `<add>` event the `dest-dn` is set in the Placement policy set. In fact if you fail to do so, your event stops with an error that "No Placement DN" was generated, if you are in the Publisher channel, since eDirectory needs a placement context. On the Subscriber channel, if you do not set a `dest-dn`, it gets past and just errors later if it cannot create the object.

In a `<modify>` event, you usually have an `<association>` node to identify the target object instead of a `dest-dn` so the token would not work.

Destination Name also relies on the `dest-dn`. It implicitly performs a *Parse DN* to get just the leaf most node to use as the object name.

The same is true if the `src-dn` is not in the document. I cannot think of the exact circumstances that would cause this to happen other than the result of an *Add Destination Object* call that would have a `dest-dn` to identify the new object but no `src-dn`. I am sure there are other reasonable cases.

Just keep it in mind that you might want to be sure `dest-dn/src-dn` are available if you want to use these tokens. If all you have is an association, remember you can use the *Resolve* token to resolve an association to a DN in either the source or destination system. Amusingly, helping someone understand that there was such a thing as the *Resolve* token is what got me started on this series of articles.

Since this is a condition token, there are the usual sorts of Operators available: equals, less than, greater than, and so on. Within each token is the usual set of equality tests (case sensitive, numeric, Regular expressions, etc). Also in these tokens, there are the additional tests of In Subtree or In Container.

In Subtree really is a DN based compare that is nicely handled, so you don't have to figure out a method to do it. If the Destination (or Source) DN value is a child of the specified Subtree to any depth, then it will return true. The In Container would only look one level deep from the specified container like an LDAP scope of 'one'.

If you do an In Subtree equality test with an *If Source DN* token, there is one extra feature that was discussed earlier, the *Unmatched Source DN* noun token, that will now have the data it needs to proceed. That is, if my Source DN is `com\acme\data\users\us\Geoffc` and the test was for *If Source DN* is contained in subtree `com\acme\data`, then the *Unmatched Source DN* would be `users\us\geoffc`. That is, the part the *If Source DN* condition test did not match.

This is used in the various mirror placement configs since by using *Unmatched Source DN* appended to the base DN, you have the location that mirrors the source location.

Now, as a reminder, the Active Directory driver looks like it has an operation property named `unmatched-src-dn` that is used. In fact, the first rule in the Matching policy set adds this operation property to be used later based on the *Unmatched Source DN* token. Thus, it looks confusing. You might assume an `op prop` is also available, but in reality, this is just something done on purpose in the Active Directory driver.

If Entitlement

One slightly confusing aspect of entitlements is that they are linked into specific drivers since they exist inside a driver object. Thus, the same Entitlement name (UserAccount) can be used in every driver you have, but be specific to your driver when used within a driver context.

I wrote a fair bit about Entitlements in this series of articles that may be helpful to better understand them:

<http://www.novell.com/communities/node/12760/talking-about-entitlements-part-1>

<http://www.novell.com/communities/node/12908/talking-about-entitlements-part-2>

<http://www.novell.com/communities/node/12909/talking-about-entitlements-part-3>

<http://www.novell.com/communities/node/13027/talking-about-entitlements-part-4>

This token has the usual comparison operators, but the confusing part revolves around what you are comparing. To understand why this is an issue, you need to understand what an Entitlement looks like.

An entitlement, as part of an add event, might look something like this:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product version="?.?.?.?">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <add class-name="User">
      <add-attr attr-name="DirXML-EntitlementRef">
        <value type="structured">
          <component name="volume">\ACME\services\IDM\DriverSet\Directory Driver\Account</component>
          <component name="nameSpace">1</component>
          <component name="path.xml">
            <ref>
              <src>RBE</src>
              <id>services\IDM\DriverSet\Entitlement Policies\RBE-RoleTest1</id>
              <param>Test</param>
            </ref>
          </component>
        </value>
      </add-attr>
    </add>
  </input>
</nds>
```

The actual entitlement being added to the user is contained in the `DirXML-EntitlementRef` attribute. It is a structured attribute of syntax Path, which was originally meant to refer to paths to files on volumes, and has three parts to it.

There is a nameSpace component that is a 32 bit integer but usually 0 for revoked or 1 for granted in this case.

The volume component, which is a DN syntax field pointing at the DirXML-Entitlement object that defines the entitlement. This has to be contained under the driver object and is specific to the driver. Thus, every driver can have a UserAccount entitlement, but they would all have unique DN's being the child of different driver objects.

The final component is a bit strange. It can be named path, in which case it is a string. Or it can be named path.xml, in which case it must be valid XML.

Look at just the entitlement value below:

```
<add-attr attr-name="DirXML-EntitlementRef">
  <value type="structured">
    <component name="volume">\ACME\services\IDM\DriverSet\Dir\Account</component>
    <component name="nameSpace">1</component>
    <component name="path.xml">
      <ref>
        <src>RBE</src>
        <id>services\IDM\DriverSet\Entitlement Policies\TestPolicy</id>
        <param>Test</param>
      </ref>
    </component>
  </value>
</add-attr>
```

The path.xml component has its own XML structure.

```
<ref>
  <src>RBE</src>
  <id>services\IDM\DriverSet\Entitlement Policies\TestPolicy</id>
  <param>Test</param>
</ref>
```

It turns out that the compare operators are looking at the value of the <param> node inside that XML.

However, this is all for naught as prior to IDM 3.6.1, there was only a single valued parameter available. In IDM 3.6.1, they started using a pipe symbol (vertical bar) to separate values. In IDM 4.x, they started using JSON as the text inside the <param> node. There are now ECMA Script functions provided to handle extracting values, but equality tests in the *If Entitlement* condition token is no longer directly usable since it cannot really handle the actual values

You could use an equality test with Regular Expressions to try and pick the value of interest out of the JSON (which sort of defeats the purpose of using JSON).

Equality tests for all allow the usual types (case sensitive, numeric, Regular Expression) against the `<param>` node.

The changing operand behaves as usual where any event for `DirXML-EntitlementRef` in the event document for the named entitlement suffices. This is subtly different than the *If Operation Attribute* changing because this is actually filtered a bit to handle only the entitlements for this driver and, specifically, the named entitlement versus just the name of the attribute. You could do all this without this special condition, but having it built in, makes using it significantly easier. To do it 'by hand' you would probably:

```
if operation attribute DirXML-EntitlementRef changing,  
    then for each operation attribute DirXML-EntitlementRef  
        Parse DN to be sure it is for this driver  
        Parse DN to get its name.  
        Decide if we care about this one.
```

It is very doable, but this condition test makes it so much easier and more efficient.

The operand 'available' is also interesting here. If the entitlement is not being granted in the current event document, then it will query back to the source to get the current state of Entitlements on this object to properly answer the question of it's availability.

If Global Configuration Value

I really like using GCVs. One of the things that drove me nuts in Workflows and PRDs was the inability to use GCV values for stuff. Nuts to the point where we wrote a driver for a client that converted the GCVs, if they changed, into attributes we could read back with a DAL query in a PRD and then be used inside the PRD. This is an issue when, for example, you need to hardcode server name references for a SOAP endpoint, the User App URL, or the like. However, these kinds of values will most often change as you stage the project from the Development lab to the Quality Assurance lab, and finally get it ready to roll out to Production. After all, the server acting as the endpoint is most likely different in the different staging environments.

With the IDM 4 release, they added at least two new features to the User Application. First, support for using GCV values in many places within the workflow. Second, the ability to store ECMA functions in an ECMA library object, link it to the User App driver, then to the specific PRD, and have them be available. This is a much simpler model than was possible before, where you often had to reuse the same function in many locations since it was not always possible to reuse them.

GCVs come in many types and forms, and you can read more about that in this series of articles:

<http://www.novell.com/communities/node/11344/explaining-gcvs-part-1>

<http://www.novell.com/communities/node/11471/explaining-gcvs-part-2>

<http://www.novell.com/communities/node/11480/structured-global-configuration-values-idm>

As usual you get the normal set of operands with the normal set of equality tests. Alas, there is no direct support, at the operand level, for some of the fun GCV syntaxes. To handle a list GCV, you just have to know this test will represent it as a delimited string value when testing equality, and either use a Regular expression to match or some exact string match. The same holds true for a structured GCV. Just imagine the value as the string representation of the GCV. If you read my article on Structured GCVs, you will see that if you treat a structured GCV as a node-set, you get the XML node-set from the GCV definition. Using XPATH you can find the different values. If you treat it as a string, you will get the various bits strung together using the delimiters you specified when you defined the GCV. Be careful, things can quickly become messy.

The string you get can vary since between each instance of a Structured GCV there can be a delimiter. Inside each instance, there can be multiple GCVs, and they need to be delimited. Then, any of those could be a List GCV. In fact, I think you can even have another Structured GCV in there. The Designer UI will not let you do it, but if you edit the XML by hand, it can be done as the SIF driver does it.

Regardless, the point is know what you are getting as data. There are lots of possibilities if you want to get crazy. The SIF driver uses structured GCVs in very clever ways. You can read more about that driver here if you are interested:

<http://www.novell.com/communities/node/13159/walking-through-sif-driver-part-1>

<http://www.novell.com/communities/node/13160/walking-through-sif-driver-part-2>

<http://www.novell.com/communities/node/13327/walking-through-sif-driver-part-3>

<http://www.novell.com/communities/node/13336/walking-through-sif-driver-part-4>

<http://www.novell.com/communities/node/13381/walking-through-sif-driver-part-5>

<http://www.novell.com/communities/node/13382/walking-through-sif-driver-part-6>

<http://www.novell.com/communities/node/13384/walking-through-sif-driver-part-7>

One very common use case for this token is to allow the driver developers to build one driver with different approaches to certain things (like Placement with values of mirrored or flat, or perhaps using Entitlements or not), and then control the functionality with a GCV. I like using this sort of thing for sending emails on certain errors. This way I can turn off emailing, if necessary, in just one place for the entire system or for just the specific driver using the GCV. So you build a GCV, and if it does not have a certain value (send email is true), then you do not perform the action. You will see a LOT of this in modern driver configurations.

One issue of GCVs that is a bit confusing, specifically with IDM 4 and Package support, is two fold. First, the GCV values are stored in a Per Replica flagged attribute that, oddly enough, means that in this replicated database you do not replicate this value. That means ServerA can

Copyright© 2013 Ninja-Tools

have one value for the GCV defined and ServerB, using the same object and the same attribute, can have a different value defined. That is, one attribute can have different values on different servers. Thus, you must always be sure you are modifying the value on the right server when developing. Secondly, with the release of packages, the GCV value can be defined in several places. Initially this was two places, the driver set level, which all drivers inherit, or the driver level (which would override the driver set value). With Packages, while the driver level GCVs are still supported, the preferred approach is to use a `DirXML-GlobalConfig` object, which holds the values as part of the package. Values in the `DirXML-GlobalConfig` can be easily added and removed making it very useful.

The good news is that engine handles all this in the background, and the policies work with the effective values.

The Value field, for the name of the GCV is sort of amusing since you can use variables to define the name of the GCV! How crazy is that? Well, actually the SIF driver does a very nice job of using that functionality. It turns out that in SIF, the same object class can be multiple things depending on the value of an attribute. Therefore, the IDM engine cannot handle that kind of class mapping out of the box. Early on in the event flow they calculated the class based on the data and set an operation property. In each policy object that needs it, they set a local variable, `lv_sifClass`, to whatever value the operation property is set to.

When they need to look up configuration information, from the very cool structured GCV they store it all in, they actually use a GCV name of `gv_SifConfigFor_${lv_sifClass}` where the name of the GCV itself has part of it provided via another variable. (Ok, actually that example is a variable name, but they do the same thing with GCV names.) This is a pretty neat approach that allows them to more easily select the info they need.

If Local Variable

This is probably one of the more commonly used conditions since it is so useful.

Conditions

✓ **Condition Group 1**

Define new condition below

Condition: local variable ?

Name * MultiValued

Operator * equal

Mode case insensitive

Value \$SingleValued\$

OK Cancel

Condition modified. Click OK to update the policy or click Cancel to discard changes.

* Required

There are two scopes for variables, local and driver scoped. Local scope means it exists for the duration of the current policy object. Driver scope means that until it is changed or the driver restarted, the variable will be available. If you are testing for a policy scoped variable set in a different policy, it will, of course, not be available here.

One of the things that most people do not know is that this test can actually compare a node-set of values to a single value. It will return true if it matches ANY of them and false if it returns NONE of them.

This is a really big deal since you can easily test if your value is in a list. The trick to doing it is that the node-set of values has to be the local variable being compared (in the **Name** field) to the value of the single value (in the **Value** field). That can be a variable using the \$SingleValued\$ notation or some literal string. It does not matter.

The node-set has to have either a node-set of values (no XML, just a bunch of value nodes) or, if XML, then the values will be XML text strings.

A node-set of values is kind of neat and can be 'made' a couple of different ways. If you were inside a *For Each* loop, you could set a node-set local variable as itself, then the current-node local variable. Perhaps like this:

```

✓ for each
  XPath("$current-node/attr[@attr-name='Member']/value")
do
  set local variable("MEMBER-LIST", scope="policy", nodeset(Local Variable("MEMBER-LIST")+Parse DN(length="1", start="-1", Local Variable("current-node"))))

```

```

<do-for-each>
  <arg-node-set>

```

```

        <token-xpath expression='$current-node/attr[@attr-name="Member"]/value'/>
    </arg-node-set>
    <arg-actions>
        <do-set-local-variable name="MEMBER-LIST" scope="policy">
            <arg-node-set>
                <token-local-variable name="MEMBER-LIST"/>
                <token-parse-dn length="1" start="-1">
                    <token-local-variable name="current-node"/>
                </token-parse-dn>
            </arg-node-set>
        </do-set-local-variable>
    </arg-actions>
</do-for-each>

```

This will give you a node-set that is not XML but can be looped over or, in this context, compared against. This example could convert a group's member list that might be made up of proper full DN's and having used *Parse DN* to get the naming value, return a node-set of values of just the short names.

Of course, it would be much simpler to just use the *Reformat Operation Attribute* token, and have it reformat the `Member` attribute values from the full DN's to the shorter naming attribute value. The above example demonstrates the point by doing it the hard way.

A List GCV, if treated as a node-set, will return a bunch of `<item>` nodes with XML text inside, and the compare will work perfectly fine against that.

If you were to trace out a List GCV using the *XML Serialize* token perhaps using a code example like this:

```

<do-trace-message>
    <arg-string>
        <token-xml-serialize>
            <token-global-variable name="MyListGCV"/>
        </token-xml-serialize>
    </arg-string>
</do-trace-message>

```

You would get something like this back in your trace output:

```

<item>Value1</item>
<item>Value2</item>
<item>Value3</item>
<item>Value4</item>

```

If you set a local variable as a node-set with the value of that GCV (since we are really talking about the *Local Variable* token here), then you could compare with the *If Local Variable* token to find out if a particular value is in that list. Perhaps like this:

```
✓ ⚡ trace message(XML Serialize(Global Configuration Value("MyListGCV")))
✓ ⚡ set local variable("LIST-VALUES", scope="policy", nodeset(Global Configuration Value("MyListGCV")))
✓ ⚡ if
    if local variable 'MyListGCV' equal "Value3"
    then
        trace message("Found it!")
    else
        trace message("Not found.")
```

That looks as follows in the XML source:

```
<do-trace-message>
  <arg-string>
    <token-xml-serialize>
      <token-global-variable name="MyListGCV"/>
    </token-xml-serialize>
  </arg-string>
</do-trace-message>
<do-set-local-variable name="LIST-VALUES" scope="policy">
  <arg-node-set>
    <token-global-variable name="MyListGCV"/>
  </arg-node-set>
</do-set-local-variable>
<do-if>
  <arg-conditions>
    <and>
      <if-local-variable mode="nocase" name="MyListGCV" op="equal">Value3</if-local-variable>
    </and>
  </arg-conditions>
  <arg-actions>
    <do-trace-message>
      <arg-string>
        <token-text xml:space="preserve">Found it!</token-text>
      </arg-string>
    </do-trace-message>
  </arg-actions>
  <arg-actions>
    <do-trace-message>
      <arg-string>
        <token-text xml:space="preserve">Not found.</token-text>
      </arg-string>
    </do-trace-message>
  </arg-actions>
</do-if>
```

I use this all the time to decide if my attribute is one of a special set (for, say, FERPA or HIPPA protection) or anything like that.

You get the usual operands with their usual equality test types, and the **Value** line is allowed to be a variable as well as using the `$MyVarName$` notation.

One interesting tidbit to keep in mind, if you declare a local variable to no value (i.e., the value line was left blank), that local variable is available but empty of values. So it is best to test *If Local Variable* equal, Regular Expression compare, and the value `.+` (Period plus sign). Period means any character, and the plus sign means 1 to any number of characters. Whereas `*` (as in `.*`) would mean match 0 to any number of characters. In this case, `.*` (period asterisk) would not be useful as it would also match an empty value, the zero value case. This way you can make a more useful compare of whether there is any data in the variable or not. This is important since a very common way to clear a variable's value is to *Set Local Variable* to an empty **Value** field. There is no way to make it unavailable, once it is declared, in scope. Obviously, a policy scoped variable will go 'poof' after the policy is completed.

Like the GCV example, you can have a Variable name made up partly of a variable. That means you could define a GCV to be used based upon the value held in a variable. This can be done with the entire name as discussed many times before by the use of `$VarName$` style notation. However, you can also use the same approach to provide part of the name through a string and part through variable replacement. The same driver, SIF, uses this heavily as well. Really, it is quite a clever way of approaching a configuration data problem that I kind of like. This is why I like doing the driver walkthroughs. I get to see how other folk approach a problem in ways I never thought of, and I always learn something interesting usually not in the documentation. Sometimes it takes awhile to figure out what they were doing since the code looks wrong, but clearly, they had something in mind. This is where adding some comments into the rule would help. However, that is a long battle I am slowly fighting.

You can read more of my driver walkthroughs on this Wiki Page that I maintain with all of them collected:

http://wiki.novell.com/index.php/Detailed_driver_walk_through_collection

If you are interested in learning more about Identity Manager, I recommend walking through a driver. I guarantee you will learn something in the process no matter how advanced you might be in IDM. As long as you are doing that, take notes, write them up, and contribute them! I do not have enough spare time to walk through every driver, so it would help if others would do some for me!

Even if you just want to walk through a single common Policy object, I would appreciate it! For example, I did the Password Synchronization policies in the Command Transforms in this series:

<http://www.novell.com/communities/node/1474/password-transformation-rule-sets>
<http://www.novell.com/communities/node/9661/password-transformation-rules-publisher-channel>

I have a series in mind about the Identity Tracking rules. There are many Packages now available that are common across multiple drivers, and it would help to have someone walk through them and report back. These common packages are much smaller packages, chunks of work, to document than a full driver and more easily approachable. It is much easier to approach a small task than a huge task. Some of these Packages for common functionality are great targets to document.

Another thing to keep in mind, with this *If Local Variable* condition test, is whether your local variable is a string, a node-set, or an Object. We do not use objects much but these are meant to be Java objects. Objects may be required if you need to hand off the value to a Java class that is expecting an object. More likely some Java class returns a Java object, which you store in a local variable, to pass to another Java class. Since strings and node-sets are more commonly used, this token will cast your variable as a string should it happen to be a node-set. Be Careful as this could lead you into potentially odd circumstances.

When a node-set is treated as a string in Identity Manager, the string values within it are concatenated together and presented as the single value. This is common when using XPATH with the `text()` function. Now, sometimes this does actually work out well. For example, if your node-set has but a single string value in it, then the result will be the same. However, if you had a case where there were two values in your node-set, then you probably will not get your expected result. This is an example where good test cases would catch this if you forget to take it into account. That is, testing with single valued and multivalued data.

If you do need to test a node-set variable, then you are possibly better off with an *If XPATH* true (or not true) and then using XPATH to do the test you need. The shortcoming there is that Regular Expressions are not directly supported in an XPATH test. Instead, you could use a `contains()` function to see if your selection has a value in it. Be warned, the `contains()` function is case sensitive. You might wish to uppercase or lowercase all the data before comparing it. Worst case, you can always call out to an ECMA function that can implement the Regular Expression compare for use in XPATH. The LIBAJC-JS ECMA library that is the Novell Consulting Advanced Java Class ported from Java to ECMA Script has a `match()` function that will do a Regular Expression compare. David Gersic wrote an article about how to do this as well. I suspect he did not notice the LIBAJC-JS already had it; still valid. You can read about it here:

<http://www.novell.com/communities/node/8434/adding-regular-expression-matching-xpath-identity-manager>

If you have a string value, then the Regular Expression equality type can do all sorts of magic in terms of making sure you can express any condition you need to test for. I am constantly amazed what people can make a Regular Expression do!

If Named Password

Named Passwords were added around the IDM 3.5 or so time frame and are really handy. The engine already knew how to encrypt and store passwords. It already does three passwords on every driver. To be fair, Driver Object password is just the regular password for the driver object itself. In fact, you will notice you get a default password policy that enables Universal Password assigned to the driver set partition object when you install Identity Manager. This is another reason they want the driver set to be a partition by default since password policies are only inherited by one level unless assigned to a partition. This is required so that when you set a Driver Object password, it gets a value for its Universal Password so the driver can retrieve it as needed. Because the old style RSA Key pair password, aka the NDS password, cannot be recovered, you must use the policy to enable this. I am pretty sure that the Remote and Application passwords are stored in a similar fashion to the Universal Password but not visible in that API. There are tools that can retrieve Universal Password, but they cannot see these passwords. I suspect that the engine uses the same basic mechanism for *Named Passwords* as well. However, since the engine is already supporting these three passwords, the code to handle it must exist already.

It makes a lot of sense to add support for an arbitrary set of additional passwords as needed by a driver. For example, the Lotus Notes (Domino server) driver needs a password for each Certifier. If you want to build an OU structure in Domino, it turns out that each OU in that structure is a Certifier, like a Certificate Authority; and it has its own certificate used to sign everything below it. As you might imagine, private keys are very sensitive and are password protected. Thus, if you want to place users in multiple OUs in Domino, you will potentially need multiple passwords. This is where *Named Password* comes in handy as a lovely thing. There is no need to store the password in configuration as clear text. You can just use a *Named Password*.

Another common place to use it would be in a *Start Workflow*, *Add Role*, or *Remove Role* actions. Basically, these are wrappers to a SOAP call to the User Application web services. These need to know an endpoint, a user name (LDAP DN format), and password to login with. This would stink to hardcode a password for such a powerful use. *Named Password's* make this so much better. An amusing alternate is to make sure the User Application administrator user you use for your *Start Workflow* token calls has a Universal Password. Then, you can actually just read the `nspmDistributionPassword` attribute from that object for your *Start Workflow* call instead. This is also the best practice because, if you change the User App user password, no changes are needed at all.

This condition token does not let you do much WITH the *Named Password*, other than check for its existence or absence. You cannot compare the value, only that there is a set password (which theoretically could be blank). So the Operands are just available and not available.

However, this is useful to protect you from trying to use a *Named Password* that does not exist. Your rule that is dependent upon a password to work, could test for it, and skip it, or error if the password is not available.

If Operation

This condition lets you consider the events current operation. This might be useful to know that you are reacting to a status event. For example, send an email if there is an error. Perhaps it would be useful to know if the current event is an `<add>` instead of a `<modify>`.

The Operators are basically the usual set: equal, greater than, less than, and their negatives. I have no clue how one would use something like "greater than add" in a meaningful test, but I suspect it was easier to leave in the generic code than to exclude it.

The list of operations for the Value field is interesting:

- add
- add-association
- check-object-password
- check-password
- delete
- generated-password
- get-named-password
- init-params
- instance
- modify
- modify-association
- modify-password
- move
- password
- query
- query-schema
- remove-association
- rename
- schema-def
- status
- sync
- trigger

Some of those event types like: `check-object-password`, `init-params`, `query-schema` and its response `schema-def`, are usually only used during driver startup processing.

Others are association related like, `add`, `remove`, or `modify` association.

The *Query* token sends a query into the target system, and the `<instance>` is the type of response you get back. When an event succeeds or fails, you get a `<status>` event back. Interestingly enough, when you query for something and nothing is returned, you actually get an empty

<status> event with an XML attribute level of success. This is a little trickier than you might expect. Notably, in Simulator, there are actually two options in the response to a query. That is, if your policy makes a query, Simulator will show you the query, and there is a response tab in which you are expected to build the response document. Usually I copy and paste an actual response from the engine when doing this. However, you could start building an <instance> document if you intend an actual result. But if you need to return the absence of anything found, the operation tab in the upper left should be set to <No Object Found> instead of an <instance> document. Otherwise, your code will get an empty <instance> node that it might interpret as a result, as opposed to a null response.

A Job (a scheduled event local to a driver or driverset) sends a trigger event based on how you defined it. There are a number of jobs that ship with IDM, as they are really just Java classes that do something. If you define a job as a 'Subscriber channel trigger' type, it will send an event on the defined schedule, then it will be a <trigger> event.

As we discussed just a moment ago in the *If Named Password* token, you might see a <get-named-password> event when you use it. At least, if the shim is hardcoded to use a Named Password (as the Notes shim seems to be), then you might see this event.

When you use the iManager check password synchronization status, it is supposed to call a check-password event for this user, testing that the password in the Identity vault is the same as in the connected system. This should try a password compare (if the connected system shim supports it to save an intruder lockout attempt), and tell you if it worked or not. To be honest, I have had mixed success with this functionality and do not usually rely on it.

When you try to migrate from an application or migrate from Identity Vault in iManager (not an option in Designer) on a driver, you basically are generating a sync event in one channel or another. Interestingly enough, if you want to cause a sync event to happen in the Publisher channel, you can change the operation type to sync or append via XML elements a sync event to the current document. This will only work and cause the engine to treat it properly if done in Event Transformation policy set.

I do not know what a generated-password event is for, or do I know what the password event is used for. It does show up in the list.

As always, the **Value** field can be a variable, so you can get it from a local or Global variable. One common, little trick is to use an Equal operator with a mode of Regular Expression, and then test for something like:

```
add|modify|rename|delete
```

This lets you use a single condition test to handle 4 possible cases. In complicated condition blocks, this can be helpful to get an extra OR case. Of course, you could AND it, but that

makes no sense in this context since you can never have one event that is two or more types. Rather, you could have two or more events processed one at a time.

If Operation Property

Operation properties are the approved of method for carrying data both between policy objects and through the shim. When you use a Set Operation Property action, this adds an XML attribute to the `<operation-data>` node at the end of your event. It turns out you can Clone By XPath any XML you want into there, and it will work as well. However, the built-in tokens only work on the XML attributes of the `<operation-data>` node.

You can perform this test with XPATH as well since it is an XML attribute in a known node; however, it is nicer to use the built-in conditions and actions. You will, however, see that the Password Synchronization rules will actually add an XML structure under the `<operation-data>` node. Either way works, but the built-in tokens are much simpler.

```
<nds dtdversion="3.5" ndsversion="8.x">
  <source>
    <product version="3.5.11.20080307 ">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify-password class-name="User" event-id="pwd-subscribe" qualified-src-dn="O=com\OU=acme\CN=jsmith"
src-dn="\ACME\com\acme\jsmith" src-entry-id="33458">
      <association>7b62bd352a06644a8f9e6d72fc827129</association>
      <password><!-- content suppressed --></password>
      <operation-data>
        <password-subscribe-status>
          <association>7b62bd352a06644a8f9e6d72fc827129</association>
        </password-subscribe-status>
      </operation-data>
    </modify-password>
  </input>
</nds>
```

Once added to the event, the operation property will remain attached until the event hits the shim. At that point, the engine records the info, tracks it, and removes it before submitting. Then, when the response is received, the operation data is re-added to the event on its return. Thus, you can have data carried between rules and between channels.

This condition has the usual operators, including available, since it can be absent if not set. This means if you want to get rid of an operation property, you could Strip by XPATH to get rid of it (perhaps with XPATH of `operation-data/@opPropName` where your operation property is named `opPropName`) instead of setting it to an empty value. Or you could use the Clear Operation Property action.

If Password

This token allows you to detect a `<password>` node in your event. When you create a new user, the `<add>` event may have a `<password>` node. When the shim sends a `<modify-password>` event, it will have a `<password>` node at first.

The Password Synchronization rules (mostly residing in the Command Transform policy set, a different set of rules in Subscriber and Publisher channels) follow the GCV's settings to either leave the event as is (to modify the NDS password) or to transform it into a `<modify>` event for the attribute `nspmDistributionPassword`.

Thus, you have a limited span where this token will accomplish what you want. Stay aware of where the event is in terms of how it might be transformed, and then you can safely use this token. It is not magic and probably will not work much of the time in the Subscriber channel. It should work pretty well in much of the Publisher channel before the Command Transform.

If you want to use this, generate the events you are thinking of and follow it through in trace. You can see what happens to the password in the process to ensure your test will work when you wish to use it.

If XML Attribute

When you see your event in IDM's XML dialect, XDS, you will see an event node like: `<add>`, `<modify>`, `<status>`, or the like. Inside that node will be many XML attributes. In the case of an `<add>` or `<modify>` event, you will have `src-dn`, `qualified-src-dn`, `event-id`, `dest-dn`, and many others. In the case of a `<status>` event, you will have `level`, which is important to know if it was a success or an error.

The top of that event (rest snipped out for brevity) might look something like this:

```
<nds dtdversion="3.5" ndsversion="8.x">
  <source>
    <product version="4.0.0.1">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <add cached-time="20110518172910.608Z" class-name="User" event-id="SomeDriver+5567596:561fa9dd-2f28-4815-a7d6-707e75455267" qualified-src-dn="O=Acme\OU=Identities\cn=jsmith" src-dn="\Tree\Acme\Identities\jsmith" src-entry-id="2669108" timestamp="1305739746#35">
```

You can test for any of these XML attributes. Obviously, the test will differ based on the type of event. So you would be well advised to limit this to your specific event type of interest.

The Operators are the usual group: equal, greater than, less than, available, and their negatives. It is possible to have a blank XML attribute. In fact if you use the *Set XML Attribute* token and leave the value line blank, you can generate this case. This would look something like `<modify class-name="">`. If you tested *If XML Attribute* `class-name` is available, it would return true, but the value would be blank.

Now, it is worth mentioning that this only tests for the XML attribute in the current event node. You do not get to specify an XPATH expression to try and find an XML node on any other node.

For example, the test *If XML Attribute* `attr-name` available would not work. That is always contained on a child of the event node and not on the event node (the `<add>`, `<modify>`, `<status>`, or `<instance>` event).

To do anything more complicated, you will have to use XPATH (which, coincidentally, is the next condition in the list). In short, you select an XML attribute via XPATH with the `@` symbol so the XPATH or `@src-dn` will try and return the `src-dn` XML attribute of the current context. That is basically the same as this *If XML Attribute* token. To do anything more complex, you need to specify a path to the node, like `modify-attr/@attr-name` would get the `attr-name` XML attribute of the `modify-attr` node that is a child of the current context (which, of course, only works in a modify event since an add event won't have a `modify-attr` child node).

Nonetheless, this is a useful token to have since there are many current context XML attributes that are quite handy to get. Possibly the most useful is actually `qualified-src-dn`. If you intend to try and get an LDAP equivalent DN from the `src-dn`, you will not be able to. This is because important information is not available in a backslash formatted DN. In the DN `com\acme\users\jsmith`, how do you know if the 'com' part is `dc=com`, `o=com`, or even `c=com`, all of which are perfectly valid. The `qualified-src-dn` is qualified slash format and has the type information in it. *Parse DN* verb token, which was discussed earlier, can easily use that extra information to convert DN formats. Alas, it cannot make stuff up out of thin air, so it cannot convert from backslash format to LDAP format. Therefore, `qualified-src-dn` becomes much more useful since it contains the fully qualified information to use in the conversion. (Thus, this is why it is named `qualified-src-dn`. It is the qualified source DN.)

If XPATH expression

This is one of the more versatile conditions. As I have been spending the time detailing, you can reproduce most of the other tokens using XPATH. It also opens a whole new world of possibilities. Look back at the *XPATH* Noun token for links to other good XPATH references.

In most of my other articles, I will try and call out XPATH examples wherever possible. This is more obvious in some of the driver walkthroughs, since there I need to explain what the drivers XPATH example is doing.

Let me reiterate the single most important thing you need to know about XPATH in Identity Manager! The current context is the event node. That is, even though you start with a document that looks something like this (totally contrived example pulled from Simulator for a rule I was working on):

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product version="?.?.?.?">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <add class-name="Group">
      <add-attr attr-name="RoomInfo">
        <value type="string">Section 12b</value>
      </add-attr>
    </add>
  </input>
</nds>
```

If you just read the W3Schools type pages, then the XPATH to select the `<value>` node in its simplest form might be:

```
nds/input/add/add-attr/value
```

However, the engine takes care of those first three steps for the `nds`, `input`, and `add`; and it sets the current context to `nds/input/add`. Sort of like a current directory. The actual XPATH for an Identity Manager context to select the `<value>` node would be more like `add-attr/value` at its simplest.

Of course, you probably should use some predicates to be sure you pick the more appropriate nodes. Maybe this is a little better:

```
add-value[@attr-name='RoomInfo']/value
```

This way if there had been more than one `<add-value>` node in the event, you would only get the RoomInfo example. (The `attr-name` is an XML attribute that you select with the `@` sign. The square brackets `[]` are a predicate like a `WHERE` clause in SQL and help tighten down the focus.)

For an *If XPATH* condition test, a node test returns true or false (as opposed to a test like `count(add-attr)>1`, which still returns true or false based on the calculation being performed). If there is something returned from the specified node, it is true; and if it finds nothing there, it is false. If there is some evaluation done in the XPATH statement, then, that determines if the answer is true or false.

You actually get a nice helping hand, in terms of getting your condition test quite right, since the token has an operator choice of true or false. This can help when you have an easy way to

define the true case, but not such an easy time defining the false case. No problem. Just define the true test and set the operator to not true.

What is nice is that this token is way more versatile than just node tests. It can do way more than that. There are a series of string and math functions available in XPATH. Some make sense, of which, several are listed here.

- `substring-before()`
- `substring-after()`
- `contains()`
- `starts-with()`
- `concat()`
- `normalize-space()`
- `string-length()`
- `translate()`

The basic math functions are much as you would expect with the standard conventions of +, -, /, and * for add, subtract, divide, and multiply.

For example, in a string example, you might want to test the XPATH.

```
contains(add-value[@attr-name='RoomInfo']/value,'12b')
```

This will check if the string, at the node `add-value[@attr-name='RoomInfo']/value`, contains the value `12b`. In this case, it will be true. Remember that `contains()` is case sensitive. If the value had been `12B`, it would not be true.

Variables can be used in XPATH, by preceding the name with a single dollar sign. That could be much more simple if you chose to define some variables and use them like this:

```
contains($ATTR,$VALUE)
```

There are some useful string functions in XPATH, like `substring-before()` and `substring-after()`. These are great if you need to pick apart an email address or other such values. Here is an example to make sure an email address is within your domain.

```
substring-after($EMAIL,'@')=domain.com
```

Of course, this is a case sensitive compare. Probably your best bet would be not to do this all in one step; rather, use the XPATH noun token with the same basic XPATH to set the domain name into a variable. Then use an *If Local Variable* compare since you have the power of choosing your equality method there. Regular Expressions might be sufficient to do what you need anyway. Again, some contrived examples are fun.

You might need to do some math, in which case XPATH is perfect. Perhaps a value you get in an attribute is a time stamp in CTIME format (a count of seconds in a 32 bit integer, since the beginning of the year 1970, so Jan 1 starting at midnight), and you need to see if the date is in the past or perhaps is within 90 days? Your XPATH test could be:

```
(number(add-value[@attr-name='SomeTimeAttr']/value) - $NOW) > 0
```

The value you got, minus a variable with the current time in CTIME being greater than zero, means the timestamp is in the future. A more common example might be:

```
($TimeVariable - (60 * 60 * 24 * 90)) > $NOW
```

This will help you understand if your time variable is within 90 days. You could use a variable or GCV for any of those, so the 90-day part might be stored in a NumDays GCV or variable. You can easily change your policy about 90 days to 45 or 180 just by changing a GCV value. Then it would look like:

```
($TimeVariable - (60 * 60 * 24 * $NumDays)) > $NOW
```

A local variable and a global variable are both referenced the same way, as `$VariableName`, in XPATH. (To contrast elsewhere in policy where it would be `$VariableName$` instead.)

Of course, an XPATH expression can do more than node tests, math, and string manipulations. It can also call out to Java and ECMA Script. You could call a Java function to perform a test as long as it returns something vs nothing. Then nothing will be false and something will be true. It would be nicer if the function called returned booleans, but we'll take what we can get.

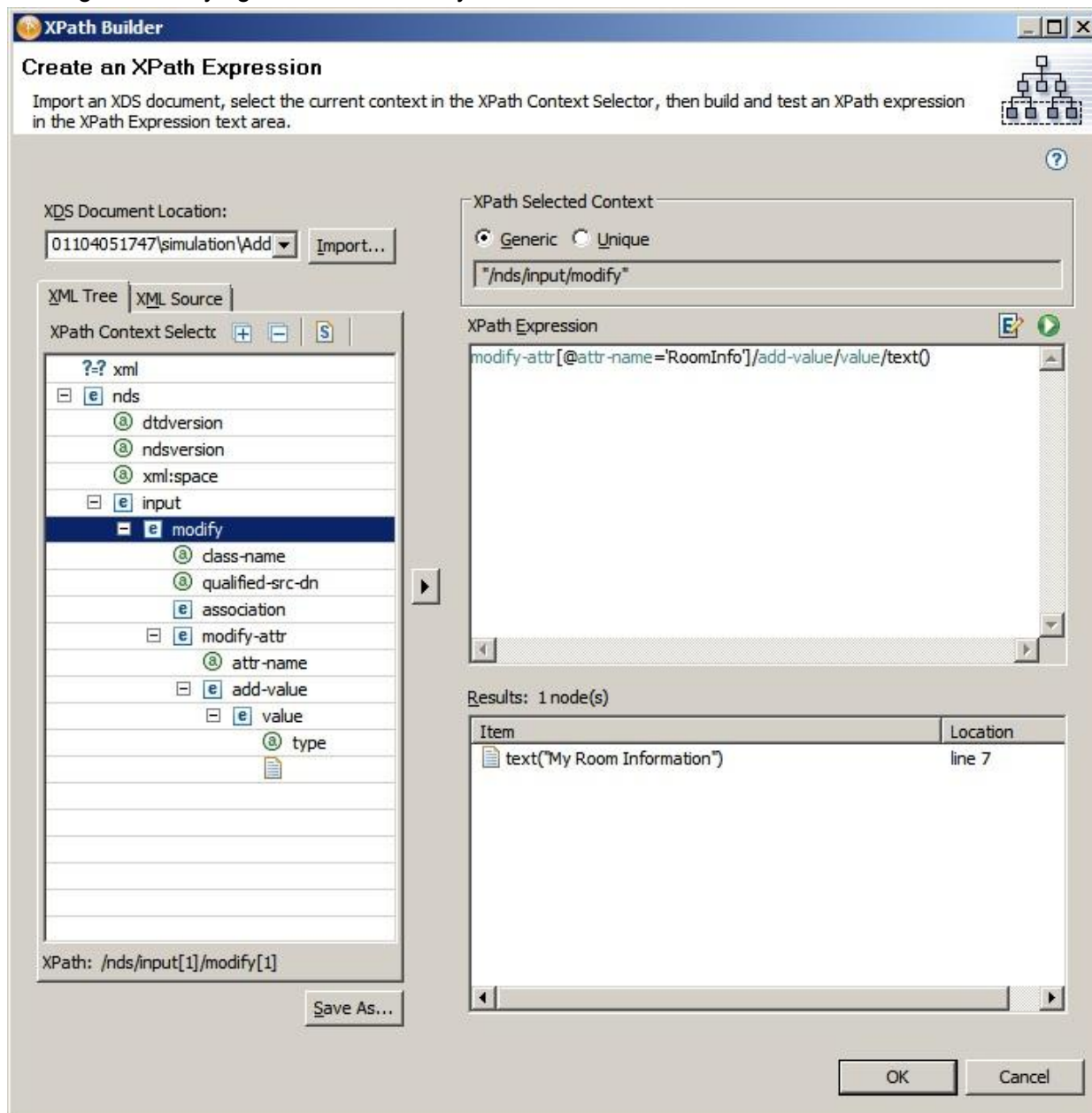
With the addition of ECMA, the possibilities get even wilder. Anything you can do in ECMA, which can also call Java, can be performed in this step. You would just have to define the ECMA function first in an ECMA Resource object, link it to the driver object, and you are good to go.

To make life even easier, Designer offers 4 little icons to help you along. They are the XPATH editor, the ECMA builder, Namespace Editor, and Variable Selector as shown in the image below:



The XPATH editor was initially one of the most frustrating things in Designer in terms of learning to use XPATH. This should be my gateway to XPATH greatness, right? A GUI tool to help me!

I never could get it to work and prove that my expressions worked. Except, of course, while clicking around trying stuff and suddenly it would work.



It turns out, like the general rule for XPATH, it's all about the current context. In the XPATH builder, you get to supply some sample XML on the left side (XML Tree window) and then build an expression in right panel (XPath Expression). Underneath you see the results (Results window). Well, of course, I am writing in the context of Identity Manager so I might use something like:

```
modify-attr[@attr-name='RoomInfo']/add-value/value/text()
```

It turns out that the XPATH editor is a generic XPATH editor. Therefore, if you have not selected any context on the left-hand side (A click on a node in the XML tree is all it takes to select a context.), then your default context is the root of the document, and it is expecting the leading `nds/input/modify` part. Of course, if you add the full path, then the XPATH tool works; but when you try it in the engine, it fails. How frustrating is that? Well, the solution is so simple, yet, it's not called out in giant letters 100 feet tall in the docs, which I find maddening. Just click on the `<modify>` node to set the current context to match what the engine is expecting. Then, the current context in the XPATH builder tool now matches the current context that the driver will be working from, and things suddenly get much better.

Suddenly, the XPATH builder starts working, and it turns out to be really helpful. A neat and useful tool.

You can load some sample events since the Simulator function in Designer ships with a bunch of example documents. The path for these docs is:

```
%INSTALL_DIR%\plugins\com.novell.idm.policy_4.0.0.201104051747\simulation
```

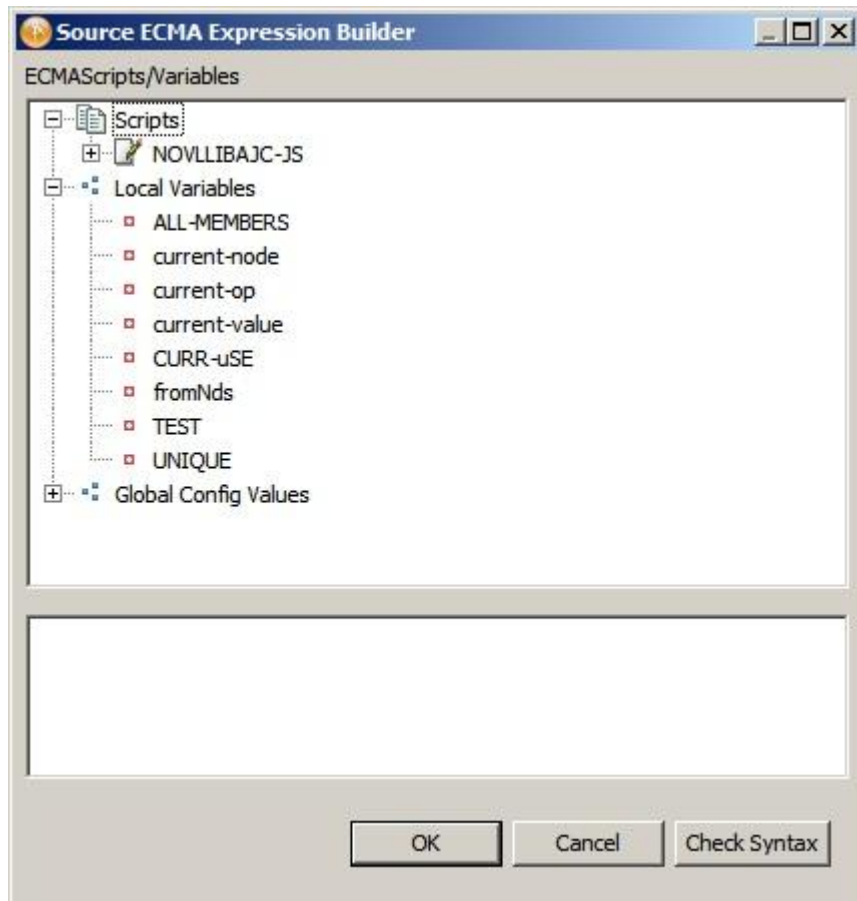
Watch out that each update (like Auto Update 2 that came out in December 2011) may update this directory. Since the package has a date stamp in it, look at your version focusing on the `com.novell.idm.policy` part, the `_` then version string (4.0.0) followed by a date stamp. The first bits, before the date stamp, will not change.

In principle you could modify the files in this directory, or add your own into the set. The problem is that, as soon as an Update touches this package, your change will be lost.

However, you can browse to anywhere to load in a document. I usually just paste into the XML Source tab on the left side of the screen. Then I make any changes I need for my test example.

Now, in the XPATH Builder screen, there is an icon for an ECMA editor. This is the same as the second icon on the **Value** line of the *If XPATH Expression* condition token. It is meant to help you insert ECMA expressions that make sense in the context of an XPATH expression.

This has been enhanced in Designer 4.0, and I did not notice when it happened! It is much nicer now than in older Designer versions. In the latest versions, you now get shown an ECMA editor that looks more like the User App/Provisioning Request Definition style ECMA editor (which is not focused on writing a new ECMA function, but, rather, on using existing ECMA functions).



This makes sense as you cannot really define ECMA functions here; rather, you need to call existing ones. (Almost all ECMA expressions in the User App side of the house allow you to write a function and then call it in the same editor.)

What is nice about this editor is that it shows you all the available ECMA scripts that you have linked to your driver. If you linked in the LIBAJC-JS, you would see a reference to that object. When expanded, it would show you close to 100 functions.

There is a item for Local Variables and Global Config Values. These are very helpful to avoid typos in variable names. Alas, unlike the standard GCV selector, you cannot make a new one via this interface, but I will take what I can get in this context. Interestingly, it just shows the display name of the GCV, unlike the usual variable selector that shows three things, the GCV name, the display name, and the value on the selected server. Being shown the value is a nice way to be sure you got the right GCV. Alas, this selector is weaker in that respect.

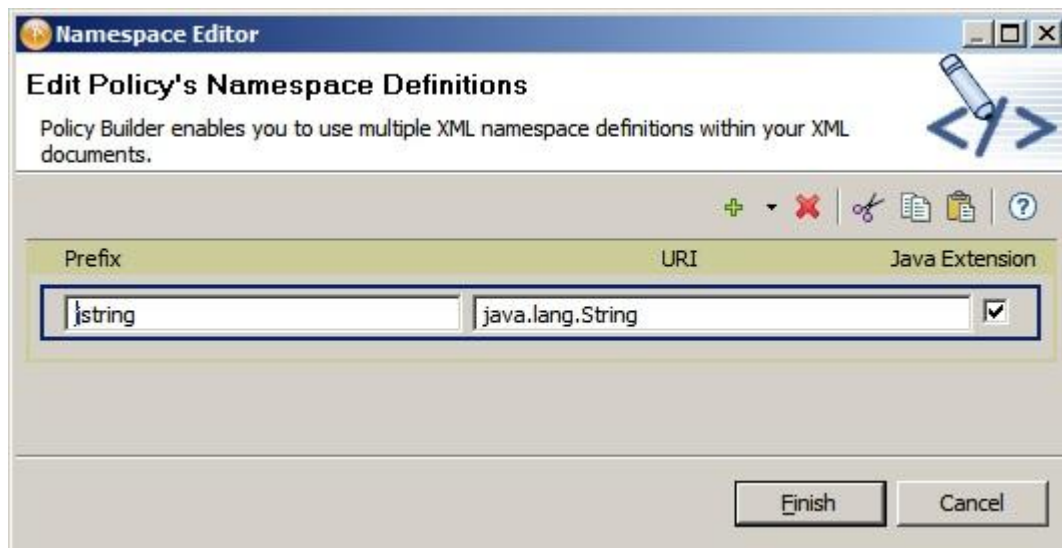
On the whole, I think I like this somewhat new editor. It will definitely make adding an ECMA function a lot easier, thus, avoiding all sorts of typos I usually make. There is even a Check Syntax button that validates the raw syntax. It helps catch silly typos as well.

All ECMA functions in XPATH are called via a predefined, namespace declaration for `es:` to be available.

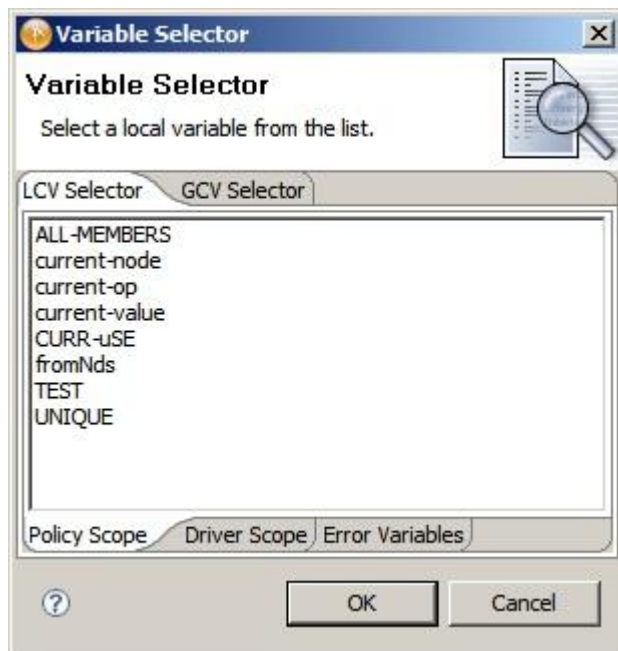
Speaking of which, the third icon on the Value line is sort of a pencil drawing in between angle brackets. This is meant to be a namespace editor. This is nice since it clears up any syntax confusion. To edit the namespace declaration by hand, they would need to be set in the `<policy>` node at the root of the Policy object's XML. This is the same namespace editor that every Policy window will show you in the upper right-hand corner.



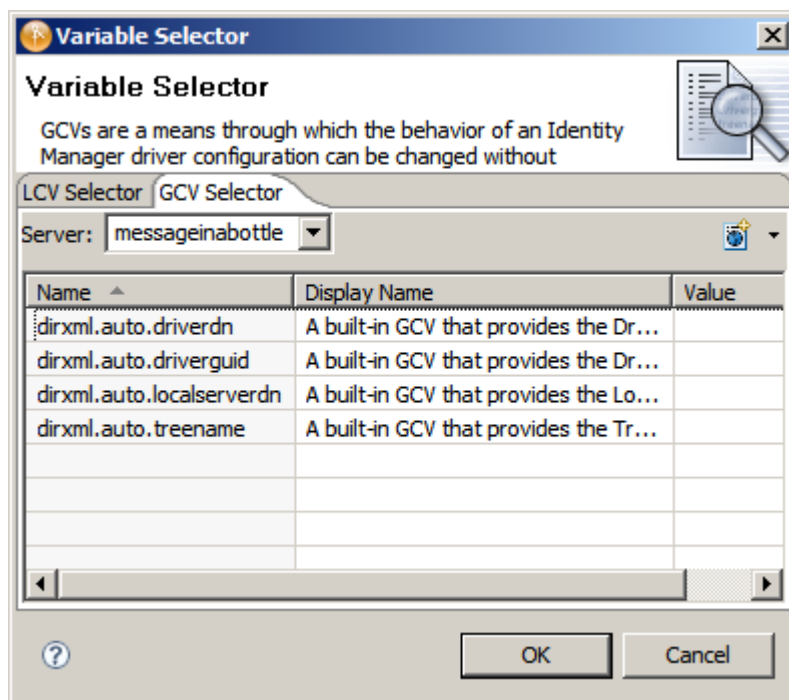
There you will see a bunch of icons; the fifth being namespace editor. (The icons are Insert new icon, Deploy, Compare, Simulator, and then Namespace Editor. The following arrows are used to move policies and items around. The jagged lightning bolt is to change trace settings, followed by help, and then a drop down with some simple Expand All, Collapse All options.)



Finally, the last icon on the Value line is the standard variable selector we have seen many times before.



This variable selector along the top has tabs for Local Variables (LCV Selector) and Global Variables (GCV Selector). Along the bottom, there are different tabs within the set of LCVs. In this example, there are Policy scoped variables, Driver scoped, and error variables as discussed elsewhere. When the GCV Selector tab is chosen, the view changes entirely as below:



That about wraps up the *If XPATH Expression* condition token. It is the last of the Condition tokens.

Action Tokens

After the Noun and Verb tokens, there were the Condition tokens. The last set of tokens are the Action tokens. There are 57 of these, and they allow policy to modify the current event document in many different ways. Some are often part of a family of tokens. Where one adds a value; the other removes it. These will be grouped together for simplicity.

Add Association

Remove Association

These two tokens are basically opposite sides of each other. Normally, the shim ought to be handling the adding and removing of associations. However, there are lots of cases where perhaps you would like to modify the association value provided by the driver in the flow. Or perhaps your driver shim is not adding it, and you need to provide it.

An example would be the Loopback driver. These drivers typically do not add an association value. You need to add it in your policy if you expect to get an association.

In the first case I mentioned, where perhaps you would like to modify the association value the shim adds, you could strip the current value with a *Strip By XPATH Expression* action (We shall see it later in this series.). The destination XPATH would simply be `association`, since the `<association>` node is a child of the current context, and should just get rid of it. As I was writing this, I initially thought this would just add in an `<association>` node. After some testing, I realized it actually generates an `<add-association>` event. In that case, to modify the association value you could still *Strip by XPATH Expression*, then *Append XML Element* with `association` as the target, then *Set XML Text* with `association` as the target, and the string you want for the association as your value.

For more on the various values that different drivers use in the `DirXML-Associations` attribute, you can read this one:

<http://www.novell.com/communities/node/1696/open-call-idm-association-values-edirectory-objects>

If you look at the token, you get three lines for input: Select Mode, Specify DN, and Specify Association.

Modes for writing events

The Mode line is important and pretty common to a lot of these tokens. There are four modes.

- add to current operation
- add before current operation
- add after current operation
- write directly to destination datastore

This gives you the flexibility to have some measure of control over when this additional action will take place.

Add to current operation tries to add it to the current event document. I will note that this one is a bit tricky since, if you have used some other document in the process before this step, the engines focus as to what is the 'current operation' may be changed.

This is why I say, "some measure of control". It is not quite as absolute as I would like. I am pretty sure I understand this one, but I have never seen this behavior documented. I might be wrong. It appears that if you have an `<add>` event, and then you chose to do an action with one of these modes, perhaps "add to current event", you should be fine. It will get tacked onto the end (usually) of the `<add>` document. I say, "usually" since, of course, the association is usually at the top. In this case, the token adds an actual additional event of `<add-association>` or `<remove-association>` right after the current event. In the specific add association case, sort of by definition, the current operation seems closer to the 'add after current operation' choice. Most other action tokens would just add it to the current event as one more item in event document.

The mode of 'add before current operation' will try to add this item before the current operation. This makes a fair bit of sense, since it is a stand alone event being generated. In later examples, we will see *Set Destination Attribute* where 'add before current operation' would imply adding a stand alone `<modify>` event, with one `<modify-attr>` node, for the attribute you are trying to set. In other words, it will makes its own event to cover what it needs to succeed. (Usually, this is a case discussed in the noun tokens where the `dest-dn` or `src-dn` might not be available for the *Destination DN*, *Destination Name*, *Source DN*, and *Source Name* tokens to use.)

Add after the current event will generate an event after. Like in the before case, a modify event will be generated.

You will often see this in drivers where an `<add>` event comes through followed by a `<modify>` of the same object (If groups or other objects with referential attributes are involved, sometimes a `<modify>` event for the Group object.). This is why you often see multiple `<status>` events when you add a User. In fact, the add event spawned two or three more modify events. All of which are technically independent of each other to a point and can succeed or fail independently. I say, "to a point" because like everything else in this product, there is a quibble. If the first event fails, the `<add>` event and the three following `<modify>` events have a `dest-dn` that points at the object created in that first `<add>` that failed. Then, of course, the following `<modify>` events will fail.

This is actually a pretty common error pattern and can freak someone out who is just getting used to what is happening. In trying to create a user in Active Directory, you can get 5 error messages in failure and, thus, think there must be some catastrophic failure. In fact, it was some simple issue in the first event that caused all the following (and dependant) failures.

You can see some examples of these sorts of errors in some articles I wrote. In this article I am trying to explain error messages you might see in the section on multiple cascading errors.
<http://www.novell.com/communities/node/8228/active-directory-driver-error-messages-part-2>

You might see this behavior in either channel, just the error messages will differ. In eDirectory the error would be -601 for Object Not Found as the cascaded (second or more) error after the initial root error. For Active Directory the specific error message ("LDAP_NO_SUCH_OBJECT") is different, but it is the same basic issue.

The "Write directly to destination datastore" mode is kind of interesting. It sort of pauses the current process, after this rule completes, and sends the event to the target data store immediately. Interestingly, on the *Set Source Attribute*, there is no such choice. They all basically happen as "Write directly to destination datastore". Of course, you cannot add a modify of source to the current event heading towards the destination. The only option is to write directly.

What you will see is, once the rule completes, it will look like a new event just occurred with whatever you are sending. Usually the trace will show "Direct command from Policy". These help identify what is happening and clear up some of the confusion. These direct commands will process through the Input or Output transforms (but not the Event or Command Transforms), and it will look like the engine just started doing something unrelated when a direct write happens. However, it will return to where it left off when completed. This makes reading the trace a little bit harder, but if you can keep track of what is going on, it is not so difficult.

One other interesting note is, if you look in the XML of the tokens (the actual DirXML Script language) that use these modes, you will see that for this token, which is represented as an XML node, the initial parent node looks something like:

```
<do-add-association/>
```

This example would use the current object, as you did not specify any modifiers.

If you try 'add after the current operation' or 'add before the current operation', you get an XML attribute named `when='after'` or `when='before'` like:

```
<do-add-association when="after"/>
```

If you select 'write direct to datastore', you might expect something like `when='direct'` or `when='now'`, but instead you get a different attribute, `direct='true'`.

I do not know the actual history of this functionality, but my guess is that the addition of after/before came at a different time (possibly later) than the direct approach.

```
<do-add-association direct="true"/>
```

The next line in the token is Specify DN. There is even a neat note that says, "Leave the DN field blank to use the current object". This is sort of interesting in that the UI differs from the usual one we will see where the options are Current Object, DN, or Association. This kind of makes sense. It would be silly to use a UI that allows you to specify the target by Association in an *Add Association* token! Since they had to redo the UI just for this one case, the current approach makes a lot of sense. It is just interesting to notice the difference and then consider the reason why they changed it in this case.

If you leave the DN field blank, then I noticed something interesting in testing this with Simulator. Since the `<add-association>` node in the event is actually an event, of the same level as a `<modify>` or `<add>`, it needs to be able to specify a target. Thus, you need to have some info in the original event to base that off. When the shim does it, I assume it remembers the `src-dn` to use in the returned `<add-association>` event.

When you use this token, you have to make sure you have a `dest-dn` in your event document for it to actually work, just like the *Destination DN* or *Destination Name* noun tokens. If not you will get an error that no target for the operation is available, even though you neglected to specify an explicit DN. Thus, the engine should be trying to use THIS object, please. Alas, there needs to be information on THIS object to be usable in this fashion. If there is no information on this event, you get an error about no target found for the destination of the event.

Specifying the DN will add an `<arg-dn>` structure under the `<add-association>` of the token (in the XML view of the policy), and you can use all the Noun and Verb tokens discussed in the beginnings of this series to build a string that you would like. This is the target DN that would otherwise be taken from the `dest-dn` value of the current event.

The next line is for building an association value. You get the full facilities of the Argument Builder again, so it can be almost anything you need it to be. As mentioned earlier, I have an article explaining the various values that are used in different drivers for the `DirXML-Associations` value. Most important is to remember to make sure it is unique.

For example, you might have a GUID from your application, let's say, Active Directory. Well, objectGUID in Active Directory comes across as a binary value, thus, is base64 encoded to be handled in XML.

Now, the format for a GUID is actually defined and has a strange patterning. Some bytes are little endian, others are big endian, and it is mixed up. There is even an interesting article on Wikipedia about it. Who knew GUIDs could be so interesting?

http://en.wikipedia.org/wiki/Globally_unique_identifier

There is an ECMA function, in the Advanced Java Class ECMA library that now comes with most drivers in IDM 4 as a Package, that will decode the data into the proper format. In fact, there are two of them. Some drivers like to enclose the value in curly braces {}, and others like slightly different spacing and dashing. You can easily call an XPATH expression to run the ECMA function to do the dirty work in Argument Builder. The calls would be something like:

- `es:guid2Association($GUID)`
- `es:guid2string($GUID)`

The functions expect that the GUID variable be base64 encoded. It will handle it all, return the NDS 2 NDS driver GUID format with the first function, and the Active Directory style GUID with the second function.

Or you can try to build any string that makes sense in your use case. I will qualify that. You MUST make sure it is unique! If you add an association to an object that is the same value as any other object, then every event afterwards, on one of those objects, will fail until you fix the association. It MUST be unique or else bad things happen.

What I mean by bad things is the next event that comes through on that user will throw a bad error. Bad in the sense that the event will stop at that point. There is no easy recovery in the current event, since the engine stops it pretty cold at that point. If the match is found as part of a Find Matching object token, then there is an error variable that would have the result that would be usable. But if it is found on the Publisher channel as the association processor starts to resolve the association, I do not think that error is 'catchable', and you will be out of luck. I suppose this should generate an audit event, but I have not specifically checked to see.

Add Destination Attribute Value

Add Source Attribute Value

Set Destination Attribute Value

Set Source Attribute Value

These four tokens are pretty much all the same. They differ mainly in their focus of direction (source vs destination), and add method (add vs set), otherwise they are much the same.

The main difference between an add and set example is that when doing a set, a `<remove-all-values>` node is sent in as part of the `<modify>` event under the `<modify-attr>` node. A call to *Clear Destination* (or *Source*) *Attribute* would send the `<remove-all-values>` by itself.

This is most useful when focusing on a single valued attribute. If you try to add an attribute to a single valued attribute that already has something in it, you get an error appropriate for whatever your target system uses for its errors. Thus, if you know you are dealing with a single valued attribute, use the *Set Destination Attribute*. However, if you want to add another value to a multivalued attribute, then the *Add Destination* (or *Source*) *Attribute* token should be fine.

Sometimes, even in the multivalued case, you want to reset the complete bunch of values. In which case, *Set Destination Attribute* is the way to go.

It also happens that the `<remove-all-values>` token is a child of `<modify-attr>` and is not available in an `<add>` event. Therefore, if you should use one of the 'set attribute' tokens during an `<add>` event, then you will get a second event appended with a `<modify>` for the target user and just one attribute. Thus the current event will consist of two events, an add and a modify. Further attempts to *Add Destination Attribute Value* calls would append to the second event, the modify, rather than initial add event. I wish it was a tad bit smarter and would strip out any existing values in the `<add>` document case, but it does not.

What this also means is that in the `<modify>` case, if the attribute of interest is already in the document as a remove value, you will not actually get the existing values removed. Rather, you will just get tacked onto the end of your current event `<modify-attr>` with a `<remove-all-values>` token. I am not certain if the engine looks at an event like that and tosses away the preceding values in its 'head'; or if it basically processes them one by one, removes a value, adds a value, processes the `<remove-all-values>` node to clear the value, and then adds the next one. This would be highly inefficient if it actually processed them in order and wrote to the target that way. But I have no easy way of knowing. (That would be remove, add, clear, and add. Four write events, which are usually the slowest things in the system aside from really foolish queries.)

All these tokens have five or six lines to fill in. This gives you some much needed control. The Source versions have five lines. Destination versions have six lines since the source token is always executed after the policy object completes.

Set Destination Attribute below:

Do **set destination attribute value** ?

Specify attribute name: *   

Specify class name:   

Select mode:

Select object:

Specify value type:

Enter string: 

 Action modified. Click OK to update the policy or click Cancel to discard changes. * Required

Set Source Attribute below:

Do **set source attribute value** ?

Specify attribute name: *   

Specify class name:   

Select object:

Specify value type:

Enter string: 

 Action modified. Click OK to update the policy or click Cancel to discard changes. * Required

First up is Attribute name. I highly recommend using the picker on the right because it helps prevent foolish spelling mistakes. In fact, once you have selected the field for typing, you can tab to highlight the attribute name selector, then spacebar to activate it, and then start typing. It will try and complete your attribute name. You can get pretty quick at this using the keyboard. I hit the Insert key to add a new action, start typing the name, cursor up or down to fine tune that selection, tab tab (first tab is usually the help icon selected), and then, as I just explained, hit tab spacebar to open the attribute selector.

Remember where you are in the policy flow. If you are not in the Input or Output transform, then you are on the eDirectory side of the Schema Map rule, so you want to use eDirectory schema names most of the time.

If, however, you wish to write to an attribute in the target system that is not schema mapped or in the filter, you can name it in the application namespace, and it will make it through just fine. The schema map won't touch it since it is not one of its named attributes. As I think about it, the filter will touch it depending on how you do it.

If you write to source vs destination, I do not think the filter will touch it on a write to source. If you do this after the filter, of course, then the filter will not affect it.

Attribute name can, in fact, come from a variable or from a GCV using the Variable selector. This is immensely useful since you might be looping (using a *For Each* token, we will get there in this series eventually) over a List GCV that has a list of attribute names. Then you could easily do an add destination attribute, where the attribute name is `$current-node$`. Thus, you will get the attribute name, from the variable, for this iteration of the loop.

You might have like the SAP HR driver, a way of calculating the attribute name, which seems crazy; but in the SAP HR case makes a fair bit of sense. The SAP HR system has this notion of Relationships between objects (in some ways like DN references in eDirectory but, basically, a single multivalued example for every possible relationship). There are a couple of modifiers where one links forwards to an object and the other is a backlink from another object. Another modifier is the object class and a relationship type value, which is a three digit integer that defines the type of relationship. So for the SAP HR drivers there is actually an attribute, `DirXML-sapO-R-B-S-003`, where this stands for an SAP O (organization) object. The first part of the value, `DirXML-sapO`, stands for the driver name. To indicate this is a relationship linking attribute, they threw in -R- (dash R dash). Also the driver is a B type to an S object (S being position since P was taken to mean person already), thus, B-S- (B dash S dash). Finally, the specific attribute is representing a 003 type relationship.

Thus, you can quite usefully calculate the name of the attribute you want to write to. Having it ready in a variable to use is great since you do not have to hardcode each case; rather, you handle generating the name in policy.

You will also see a middle button in between the Schema browser and Variable Selector. This allows you to try and pick a name from the Application Schema namespace. That is, if you are in the Input or Output transform, you might want to write in the Application namespace. Here you can select from a list.

For this to work, the driver needs to support the `getSchema()` function call, which you will see your driver do the first time it starts up. If you ever select Refresh Application Schema from iManager or Designer, heads up, it will actually restart the driver (which I find bothersome since it does not warn you that it will be restarting the driver) and make this call. The data retrieved is stored as XML in the `DirXML-ApplicationSchema` attribute on a driver. Interestingly, this can be exported in the driver configuration, thus, saving some work later, but it's still better to have fresh local data.

Not every driver supports `getSchema()`. Take the SOAP driver, for example. I once tried implementing `getSchema()` for Salesforce.com and gave up. I think I could have done it, but the schema in SFDC was huge. I thought it would be a fun perk but too much work in the end to justify the effort. If your driver supports it and you have gotten a successful schema refresh imported into Designer, then you can view the application schema using the schema browser. Thus, I suppose I am trying to explain why you might NOT see any values in this selector, and why you should not be entirely too concerned. I have not seen a line item, while doing a driver compare, that implies the application schema is being read. Somehow it works and can be exported and browsed.

In fact, if your driver supports sending enough meta data about the schema, then you actually can use the `DirXML-ApplicationSchema` data to decide if the attribute is multivalued in your destination data source. In the following article, you can read about a very clever approach to use this in the Active Directory driver.

<http://www.novell.com/communities/node/9413/generic-single-valued-schema-enforcement>

Now let's talk about a tricky item, **Specify class name**. Depending on context, you may or may not need this item. The Active Directory driver is pretty picky and requires that you specify the class. I have seen the event come in, get to the driver shim (usually a Remote Loader instance), look good, be processed, and then nothing happens. The Active Directory driver will silently drop the event unless the class name is specified.

eDirectory does not seem to be so picky as it will try to write the attribute as long as the target object exists.

There is some additional confusion amongst folk whether we should mention the base class in this field. Base classes include items like User, Group, and Organizational Unit. For example, if I want to add the attribute `DirXML-sapO-R-B-S-003` that is part of the `DirXML-sapO` auxiliary class, which class should I specify for the **Specify class name** field? In this case, it should always be the base class. It is possible that your attribute is not part of the base class, rather, it is part of an auxiliary class. If the event is there before the Command Transform, the engine will notice, check if the auxiliary class is already present on the object, and if it is missing, add it. Alas, if your attribute is part of three auxiliary classes in the schema, then the engine will actually add all three since it does not know which you intended. However, I find it a better practice to consider adding the auxiliary class you need (make to ADD not SET) by writing to the *Object Class* attribute as part of the first time you add such an attribute. Of course, you need to test if the object class is on the user already, which is easily enough performed. Thus, do not get confused by the Specify Class Name line to think if you specify an aux class, it will add it for you since your attribute is included in that aux class. Rather, it is intended as an identifier and is focused on the base class.

Next up is Select Mode. This is the same as discussed in the previous argument where you can add this to the current event, after this event, before this event, or directly to the data store. See

the Mode section at the beginning of the Action section for more on this line. Also, this is only available on the *Destination* tokens. It wouldn't make a lot of sense as writes to the source are always out of band and direct.

Select Object is available with three choices: Current object, DN, or Association. For the latter two options, you get the argument builder and can figure out the DN or association value any way you need.

In fact, a common approach is to do references. Like in a Work Order case I will set source attribute of the object stored in the DirXML-nwoContent attribute. This would be a DN pointing at a Group. Then I read back the Owner of that Group. Ultimately, the object I reference is the owner of the group that the Work Order object points at. That would look something like this:

```
<do-set-dest-attr-value class-name="User" direct="true" name="OwnerExample">
  <arg-dn>
    <token-src-attr name="Owner">
      <arg-dn>
        <token-op-attr name="DirXML-nwoContent"/>
      </arg-dn>
    </token-src-attr>
  </arg-dn>
  <arg-value type="string">
    <token-text xml:space="preserve">SomeValue</token-text>
  </arg-value>
</do-set-dest-attr-value>
```

You can see from the XML that the <arg-dn> nodes get nested inside another token, <token-src-attr>. This token also has a <arg-dn>, which comes from an operational attribute.

You can keep hopping along till the cows come home. At some point it just gets silly, although the above is not an irrational example.

Next up you can specify the value type. Most of the time we end up just using string, but there are many other value types. The most interesting value types are structured. For example, the Path syntax attribute. In that case, you need to specify the 'component' names, and the values. The builder is pretty easy though. For Path you set nameSpace, volume, and path components with corresponding values.

That might look something like this:

```
<do-set-dest-attr-value class-name="User" direct="true" name="StructuredExample">
  <arg-value type="structured">
    <arg-component name="nameSpace">
      <token-text xml:space="preserve">1</token-text>
    </arg-component>
    <arg-component name="volume">
      <token-text xml:space="preserve">O\OU\OU\CN\geoffc</token-text>
    </arg-component>
  </arg-value>
</do-set-dest-attr-value>
```

```
<arg-component name="path">
  <token-text xml:space="preserve">Some String value. nameSpace is an integer, volume is a
DN, and path is a string. So do not violate syntax rules!</token-text>
</arg-component>
</arg-value>
</do-set-dest-attr-value>
```

Finally, you provide the value itself. In most cases it is a simple value. In the Structured case, it is more complex as the above example shows. But you have the full set of Noun and Verb tokens to use as your tools. These tokens were talked about in the first eight articles in this series. So you ought to be able to accomplish whatever it is you require.

That is about it for these tokens. I am sure I can say more. You might find it helpful to read the following articles with more information on the topic if you are interested:

<http://www.novell.com/communities/node/5818/different-attribute-options-identity-manager>
<http://www.novell.com/communities/node/5920/ism-more-thoughts-sourcedestinationoperation-attribute-tokens-identity-manager>
<http://www.novell.com/communities/node/13144/various-attribute-tokens-idm-policy>

Set Local Variable

This token is immensely powerful in Identity Manager, much like variables are in most programming languages. I have actually written a fair bit about this token before, and ways you can use it in many contexts. Most recently and specifically in this article about things beginners to IDM need to know:

<http://www.novell.com/communities/node/13383/common-mistakes-newcomers-idm-make-part-9>

Some of the things addressed in that article are more about how variables can be used. In this book we focus more on how you could use the *Set Local Variable* action token. For example, in the article Common Mistakes newcomers to IDM Make part 9, I explain how you can add a local variable to itself in a string context to build up a larger and larger string.

In that article, I discuss the differences between using a \$ sign in XPATH, a leading and trailing \$ sign in policy, and leading and trailing tildes (~) characters for a GCV.

But this is the token where much of the magic happens, because here is where you get to set values into a variable. There are four lines for entering information.

Do: **set local variable**

Enter variable name: *

Select scope: **policy**

Select variable type: **String**

Specify string:

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

Interestingly enough, this value could be a variable itself using variable replacement with the leading and trailing dollar signs.

Now, that seems kind of crazy to use a variable to define a variable name and perhaps it is. There are some fun tricks you can do since this is possible. Two examples come to mind. One I use all the time, and another I have seen used in a clever fashion.

The one I use would be similar to an example I used in the previous article about the Add or Set Destination (or Source) Attribute value actions. You might have a List GCV with a list of names in it, and for some reason, you may wish to have a variable named after the name. Thus, you could loop through the node-set that the GCV produces, and when it comes time inside the loop to set a local variable, you would just use the name `$current-node$`. It would generate a variable named after the value of the variable. That example makes a bit more sense with using the variable for the attribute name rather than variable name, but the same basic principle applies.

The other example is actually much more clever, and I saw it in the SIF driver that I reviewed recently. In it they have a structured GCV that allows you to define information about students and staff. However, the conditions for the object class students and staff are identical. This means that you cannot use the schema map to map one class to a student and another to a staff member. So they use hints in the data (attribute values) to decide which it is.

In a very clever generic rule, they get the values of the GCV that holds the stuff for the current `sifClass` by using a variable `lv_sifClass`, and then generating a new variable to hold the stuff specific to it as `lv-someName-$lv-SifClass$`. That one rule can build the variables they need for all the `sifClass` values they have. Very clever.

2. Select scope

The default scope setting is `policy`, which means the value of the variable goes away when the current policy object is finished processing. If you need to carry data from one policy object to another, you should probably (officially) use an operation property. Operation property is a `name='value'` XML attribute added to the `<operation-data>` node at the end of the event document. The other scope choice is `Driver`. That is, once this is set, it remains until you clear it (set it to nothing) but it will still be 'available'. Be careful as an *If Local Variable* condition will return true,

but the value is actually empty giving you false results. Also when the driver restarts the value is cleared. This seems like a great idea for carrying data around the driver, but the thing to consider is that the driver has two channels. The two channels act independent of each other. It is possible, if you are trying to carry data across channels, that an event in the other channel is reading the value from your current event. This would be bad because your current event might have its value changed by an event in the other channel.

Interestingly enough, there is an implicit additional scope for things like `current-node`. This variable is available inside a *For Each* loop and only for that *For Each* loop. A nested *For Each* loop will generate its own local value of `current-node`. When you return out of that loop, the outer `current-node` will still be available.

I often do use driver scoped variables to hold data I need for the entire duration of the driver. It is easy to see if the value has been set with a *If Local Variable* equals a regular expression compare of `.+`. Now you know if it has been set or if you need to set it. Lots of the new base packages in IDM 4 come with stuff like this. The Managed System Gateway and Data Collection Service drivers both use it. You can read more about those drivers in these articles if you are interested:

Data Collection Service

<http://www.novell.com/communities/node/12336/data-collection-service-driver-walkthrough-part-1>
<http://www.novell.com/communities/node/12343/data-collection-service-driver-walkthrough-part-2>
<http://www.novell.com/communities/node/12347/data-collection-service-driver-walkthrough-part-3>
<http://www.novell.com/communities/node/12358/data-collection-service-driver-walkthrough-part-4>
<http://www.novell.com/communities/node/12359/data-collection-service-driver-walkthrough-part-5>
<http://www.novell.com/communities/node/12473/data-collection-service-driver-walkthrough-part-6>
<http://www.novell.com/communities/node/12476/data-collection-service-driver-walkthrough-part-7>
<http://www.novell.com/communities/node/12488/data-collection-service-driver-walkthrough-part-8>

Managed System Gateway

<http://www.novell.com/communities/node/12304/trying-understand-managed-system-gateway-driver-idm-4-part-1>
<http://www.novell.com/communities/node/12310/trying-understand-managed-system-gateway-driver-idm-4-part-2>
<http://www.novell.com/communities/node/12314/trying-understand-managed-system-gateway-driver-idm-4-part-3>
<http://www.novell.com/communities/node/12315/trying-understand-managed-system-gateway-driver-idm-4-part-4>
<http://www.novell.com/communities/node/12330/trying-understand-managed-system-gateway-driver-idm-4-part-5>
<http://www.novell.com/communities/node/12332/trying-understand-managed-system-gateway-driver-idm-4-part-6>

Often driver-scoped variables are used when reading stuff back from the driver, usually things like configuration info, schema map, or a table-like set of data.

Using it to initialize a value only once is also pretty common even if the task is not very 'slow', but is something you only ever need to run once per driver start. Sometimes this variable is a table of data read from an object and used often but only needs to be loaded once.

3. Select variable type

There are three variable types: String, Node-set, and object.

A. String is the most commonly used example and the default, and is much as you would expect out of a string.

B. Node-set is much more interesting and offers some interesting possibilities. For example, some GCVs behave differently if you treat them like a string or as a node-set. The List and Structured GCV types quickly come to mind. If you treat them like a string, you get a delimited string. If you treat them like a node-set, you get an XML node-set of the data. Depending on what you need, it is neat to be able to get it either way. You can read more about how totally cool the Structured GCV type is in this article:

<http://www.novell.com/communities/node/11480/structured-global-configuration-values-idm>

Now, in the common mistake series (part 9, cited above), I discussed how you might construct a node-set of your own. You could loop through something and in each iteration set a local variable to itself. This would cause the `current-node` local variable to contain only a string value and you would lose the XML structure. Now you can only use XPATH on it positionally since it is not XML. By positionally I mean that something like `$ValuesNodeSet[3]` would return the third value in the list. This is probably of marginal use only. However, if you use a *Split* verb token on, say, a comma delimited string, you could store that in a node-set variable and then get the fields by their position number. Interestingly, `$valueNodeSet[$Counter]` does not seem to work very well. I think this is because the notation of a number inside square brackets is actually a shortcut for `[position()=3]`, thus, you could use a variable for a counter in this format `$valueNodeSet[position()=$Counter]` instead.

However, once you have a node-set of values or an XML node-set that contains strings, the *If Local Variable* test can test the many to one case (*If Local Variable MyValues equal SingleValue*). It will return true if any values in the node-set match and false if none match. This is immensely more efficient than looping through it and testing each value.

Thus, you could collect data about something, and then test to see if it has what you need in it at the end.

Perhaps you need to see if a user is a member of a group. Store the group's membership from the Group object in a node-set variable, and then test if your single value is in it. Systems like Active Directory do not store group memberships in the user object. So using a node-set variable is a great way to get around this.

In eDirectory you might do it the other way. Store the Users Group Membership attribute in the variable, and a simple test to see if the target group is in that list.

What is interesting is that the trace looks funny. Initially you will think it is incorrect when you do this sort of test. In fact, it is the sure way to prove it is working. The trace, while looping through

each node, will show the variable name as `$VariableName$` until a match is found. It may look wrong, but that is because the variable cannot be expanded to a value. If there were five and it finds it on the third node, then it might look something like this trace snippet showing the same expansion three times:

Test PT:	Action: do-if().
Test PT:	Evaluating conditions.
Test PT:	Expanded variable reference '\$VariableName\$' to 'myAttributeName'.
Test PT:	Expanded variable reference '\$VariableName\$' to 'myAttributeName'.
Test PT:	Expanded variable reference '\$VariableName\$' to 'myAttributeName'.
Test PT:	(if-local-variable 'OBJ-CLASS' equal "\$VariableName\$") = TRUE.

Had there been no match, it would have shown the expanded line five times in this example and then returned FALSE for the test. In this case, the rule was testing to see if an object already had an object class (list of the objects current object class values perhaps using the Source Attribute token stored in the variable OBJ-CLASS) to know if it needed to be added. This could be done in a loop, over the values, and testing each one. From a performance perspective, this is much faster, particularly, on very large node-sets.

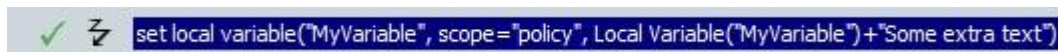
With an XML node-set, you can use XPATH upon it to select nodes as you see fit in the standard XPATH fashion. The XPATH statement would just start with `$VarName` to tell the XPATH processor that the context to start looking at is inside a variable, as opposed to the current operation node. It should be noted that an XML node-set is better than a node-set of values. If you built a node-set of values, instead of an XML node-set as discussed above, the XPATH options are pretty much limited to positional testing.

C. Object is meant to be a Java object. This is the sort of thing you could pass to a Java class expecting an object in return. You might set a local variable to type object, and the **Specify value** consists of an XPATH call to a Java function that would return an object. If you have any XSLT stylesheets that make Java calls and wish to port them to DirXML Script policy, then you likely will need to use Object variables for each intermediate step. In fact, at the end you might even need to call a function to convert the object into a node-set before policy can continue to use it. This is all for a large side topic about XPATH and Java.

4. Specify “Something”

It changes its name to match the variable type selected. Here you have the full facilities of the Argument Builder interface with all its associated nouns and verb tokens that were discussed earlier.

As discussed just a little up above, if you want to build a long string that keeps getting values added onto it as you loop through some data (commonly things like Member list of a Group), you can use the Local Variable noun token to indicate to start with the current data, and then append whatever you like. Perhaps like this policy example:



```
<do-set-local-variable name="MyVariable" scope="policy">
  <arg-string>
    <token-local-variable name="MyVariable"/>
    <token-text xml:space="preserve">Some extra text</token-text>
  </arg-string>
</do-set-local-variable>
```

However, if you are looking for a node-set of values, then you would have declared the variable to be a node-set type from the outset; and when you use the Local Variable noun token and something else, you get that something else added as the next node. This one is a little tricky, as in the String case, you get to build a string of as many pieces as you like. They just get concatenated in order onto the variable. In the node-set case, each item you add (that would count a verb with a nested noun as one item which is the same as just a noun) would be added as a value of the node-set. In the tenth article in the series on things beginners to IDM need to know:

<http://www.novell.com/communities/node/13486/common-mistakes-newcomers-idm-make-part-10>

I wanted to discuss and demonstrate why when using the *Append XML Element* style actions, you need to specify your XPATH to be something like `modify-attr[last()]` with the predicate using the `last()` function. To demonstrate what happens when you do not, I needed to provide a node-set of values. The easy way might have been a List GCV to loop through, but I was being lazy. So instead, my *For Each* loop is over three noun tokens, and it treats each of the three as a single value in the node-set. Thus, if you wanted to append a string, you need to build a node-set. Then build the local string first to append it with a *Local Variable* noun token. Otherwise, if you tried to build it in the *Set Local Variable* for the node-set, each piece of it would become a node.

When using the string variable type, it is interesting. If you just start typing, it works in the specify string line. There's no need to pop open the Argument Builder. Pretty clever on their part. A nice way to make things that much simpler and work nicely.

Add Destination Object

Add Source Object

These are useful tokens as they allow you to start a new object based on some event coming out of policy. As discussed in the *Set Destination Attribute Value*, if the object you specify is not the current object, you will get a fresh `<modify>` event for that object. Well, if the object does not yet exist, this will not end well. You will get a -601 error that the object does not exist when writing back to eDirectory. (Active Directory and other systems will return a different specific error but, of course, meaning the same thing.)

A common example where this might be used would be with the Work Order driver. The Work Order driver is something of a strange beast. In principle all it does is scan at every polling cycle to see if there are any work orders due for processing or deleting. Who makes the Work Orders in the first place? As they say in school, that is an exercise left to the student. What do you do with a work order when it comes due? The shim returns a Work Order To Do down the publisher channel. What happens with that object? That's your problem.

If you need to make a Work Order somewhere in policy, while it is easy to modify one as we have seen before, these tokens allow you to start a fresh object from whole cloth.

Now, this is one of those cases where the interactions with the *Add* and *Set Destination/Source Attribute Value* tokens gets a bit wonky. It seems like there is some kind of focus issue within the policy. Such that once you use this token to create a new object, you take the focus away from the current event and move it to this new event for the duration of this policy objects processing flow. For example, if after using this token you want to add an attribute to the current object, in the original event, you will end up getting a `<modify>` event document added on. This might explain why you will often see an event come in as a `<modify>`, and when done processing, you might have three or four events in the document.

The list of events might consist of something like the original `<modify>` then an `<add>` for a Work Order object that was spawned to process future dated events. Then maybe another `<modify>` event when you tried to add a value after the focus moved away. Thus, you can easily pick up three or more events.

The next policy object you move into will process the three events separately. The first `<modify>`, if you need to add stuff to it, will now have the focus centered back on the first event so that will work fine. Then on the second event the same, and the third event. Once you change focus, I am not sure how you can get it back to the original event. The result will be the same if your attributes change in one event document or two (with an exception mentioned below). Whatever you do, please do not create any of these. They are a bad idea for many reasons.

An auxiliary class with mandatory attributes is one exception I ran into. I remember it was with `posixAccounts`. LUM requires this to enable a user or make it usable from a Linux plugin perspective. That is, Linux does not identify users by name so much as it does by `uidNumber`. The object is referenced by the Object ID, which means they never store the name or full path. Thus, a move or rename event is easy as it does not really affect these attributes. Anyway, if you are adding `posixAccount` to a user, you also need to provide five mandatory attributes. Otherwise the event fails with a Missing Mandatory (-609) error. The problem is if the object class add value for `posixAccount` is not in the same `<modify>` event. It will fail even if the mandatory attributes are in the next event following it. In other words, the mandatory attributes must exist inside the same `<input>` document. This would not include two entirely separate events.

Thus, you can see the pain of an auxiliary class with mandatory attributes. In LDAP this is not a problem as you can have one LDIF with all the values on the user. In ConsoleOne you can try to do this on the Other tab. But, of course, the names of the attributes are not available until after you add the object class value of the auxiliary class, and then click on the OK button, except if it has mandatories. Please, if you are making your own aux classes, don't add attributes as mandatory.

Add Role

These tokens are basically wrappers around a SOAP call. That is, they let you call function in SOAP web services via these preconfigured tokens. This is very obvious since you need to specify a User Application URL, Authorized User DN, and password to connect with. The downside is that these are basically asynchronous as you kick them off, and they go off and do their own thing in User App.

Remove Role

These tokens are basically wrappers around a SOAP call. That is, they let you call function in SOAP web services via these preconfigured tokens. This is very obvious since you need to specify a User Application URL, Authorized User DN, and password to connect with. The downside is that these are basically asynchronous as you kick them off, and they go off and do their own thing in User App.

Start Workflow

The *Start Workflow* token has a significant shortcoming. A Workflow could do much more work than you ever could imagine, and all the *Start Workflow* token does, when it succeeds, is begin the workflow. Thus, success or failure of these tokens only means that the first step in the process began or failed.

The *Add Role* or *Remove Role* are a little simpler since they make a single web services call that either succeeds or fails. The point is that they call the webservice directly so that you have a direct connection into where the work is being done. This gives you direct access to any errors that are thrown so a little better, whereas, *Start Workflow* calls any workflow you might create and then disconnects not knowing if the workflow finished or not.

One of the key, unclear things about the following tokens is that they return error variables.

- error.do-add-role
- error.do-remove-role
- error.do-start-workflow

Right after one of these tokens, within the same policy object, you can look at the contents of the variables to see if the function call to start the process was successful or not.

Roles in IDM are part of the Roles Based Provisioning Module and an entire concept on their own. They are a thing that can be applied to users, groups, or containers. They can contain other Roles and have assigned Resources. (Resources map to Entitlements. It's an extra abstraction at that point.)

Thus, if a Role is added to a User, you get the implied set of Resources that the Role carries with it. The Roles and Resources Service Driver is what actually implements the actual Role change and assignments when requested via the User Application interface. The interface supports the *Add Role* token or other SOAP clients. Anyway, when the call is made, an object buried under the User App driver in eDirectory in the AppConfig container gets created, carrying the information about the Role request. Then the RRSd driver does whatever the object says to do.

The *Add Role* function is a bit more than just simple attribute or few attribute changes. You probably could reproduce this in Policy by 'hand', but I have yet to validate that it will really work that way. Officially, it is unsupported to do it any other way but through the User App. Normally, this would bug me a lot. The more I look at the complexity and ease of using the tokens, I have come to learn to live with it.

Remove Role is just the flip side of this, to remove a Role assignment from an object. The object can be a user group or container since those are all legal things to be assigned a role. The docs, of course, neglect to mention this minor detail.

In greater detail, you need to provide the DN in LDAP format for the call to succeed. I am not entirely sure how the driver handles getting that value for the 'current user' option, but it seems able to resolve the DN. When I test in Designer, I get errors about being unable to determine the target object. It seems likely that the safest bet to use this token is to make sure you get and provide the LDAP DN in eDirectory yourself. This is done by querying for the object into a node-set variable, say *lv-query*. You can then get the value with an XPATH of *\$lv-query/@qualified-src-dn* and apply the *Parse DN* token to convert from qualified slash to LDAP format.

Something like this perhaps:

```
<do-set-local-variable name="lv-query" scope="policy">
  <arg-node-set>
    <token-query datastore="src" scope="entry">
      <arg-dn>
        <token-src-dn/>
      </arg-dn>
    </token-query>
  </arg-node-set>
</do-set-local-variable>
<do-set-local-variable name="lv-ldap-dn" scope="policy">
```

```
<arg-string>
  <token-parse-dn dest-dn-format="ldap" src-dn-format="qualified-slash">
    <token-xpath expression="$lv-query/@qualified-src-dn "/>
  </token-parse-dn>
</arg-string>
</do-set-local-variable>
```

Same thing for *Start Workflow*. When you reference an object, you need to specify it in full LDAP DN format.

The Roles container exists in `cn=RolesDef,cn=RoleConfig,cn=AppConfig,cn=User App Driver,cn=Driversset` followed by wherever the rest of the tree specific info is. The object browser is nice since it starts you at the Driver set level, so you can browse down and select the right User App driver (there can be as many as you like). Then drill into the Role Catalog to find the right object. For a *Start Workflow*, it is much the same, except you are looking into the Provisioning Request definition container to find your workflow.

In the RolesDef container, there will be `cn=Level10`, `cn=Level20`, and `cn=Level30` containers depending on the type of role you are referencing.

The User Application URL is not the roles service web service endpoint which is <http://userAppIP:port/IDMProv/role/service>. Rather, it's just the User App URL, and the token figures out the rest. So something like <http://userAppIP:port/IDMProv> instead.

The Authorized User DN is also the LDAP DN of an object in eDirectory, that is, a Roles Administrator. By default, a Provisioning Administrator has the rights for the *Start Workflow* token. There is a setting you can modify (It's buried in the docs.) to relax the requirement of being a Provisioning Admin to start a workflow. I am not sure if that applies to the *Add/Remove Role* tokens or not.

The password is best to come from a *Named Password*. In fact, if you leave a clear text value (type the password in), Designer is nice enough to warn you this is a bad plan. Very clever! One other option is to ensure the admin user you specify has a Universal Password, and then just read back the `nspmDistributionPassword` from it. This way there is no need to manage or store Named Passwords.

The **Select Object** line is the usual, and I have never tested how it gets the current object DN. When you use an association, it tries to query for the object that will return the `qualified-src-dn`. I assume it knows to convert that value.

Specify Strings is interesting as this is pretty dynamic depending on circumstances. When specifying the *Add/Remove Role*, there are four strings, three of which are there by default. The fourth `sod-justification` is there in the drop downs, but you would need to add an additional string to use it. When you call *Start Workflow*, you get the Request form field names read from workflow.

However, if you update the workflow after you added the token, the token does not seem to re-read them. So be careful if you edit it later and make the additional field names available.

To start a workflow, you usually need to provide data in all the fields or else it errors. It is every field marked required on the start form or mapped in the pre-activity action of the Start item in the Workflow view.

For the *Add Role*, two of the strings are `effective-time` and `expiration-time`. This token is nice in that it does not make you figure out LDAP formatting of the string. Just provide it in CTIME, which is how eDirectory stores time. In fact, if you use any other format, it will error out on you.

I have written a bit about the *Start Workflow* token that you can read more about here:

<http://www.novell.com/communities/node/11611/introducing-start-workflow-token-part-1>
<http://www.novell.com/communities/node/11612/introducing-start-workflow-token-part-2>
<http://www.novell.com/communities/node/12187/using-soap-terminate-running-workflow-part-1>
<http://www.novell.com/communities/node/12200/using-soap-terminate-running-workflow-part-2>

One of these is about a clever idea to create a *Stop Workflow* token. You can do this by using the Provisioning web service endpoint in an Integration Activity. A lot of the User Application functionality is exposed via SOAP and is available in the WSDL. The User Application equivalent of calling a SOAP function is called the Integration Activity, and it bears no resemblance to these tokens whatsoever. It is the last part of the Silverstream Composer project left behind, which had its own Designer product. That product has since gone to pasture and the Integration Activity is all that remains.

Append XML Element

Using this token you can construct arbitrary XML. Maybe add some XDS content or another `<modify-attr>` node. In many ways, these are meant to allow you to replace most XSLT functionality with tokens.

Append XML text

Using this token you can construct arbitrary XML. Maybe add some XDS content or another `<modify-attr>` node. In many ways, these are meant to allow you to replace most XSLT functionality with tokens.

Set XML attribute

I had a client that would send an XML snippet of data in some dialect back to their HR system when a user was successfully added. Initially they did this in XSLT, and it was interesting stuff.

But upon reviewing it, I was able to reproduce it in just seven actions in a Policy. Both ways are perfectly valid.

I wrote fairly extensively about these tokens in this article for newcomers to IDM:

<http://www.novell.com/communities/node/13486/common-mistakes-newcomers-idm-make-part-10>

In an XDS document like:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <input>
    <add qualified-src-dn="o=o" src-dn="o\ou\cn">
      <association>abcd</association>
      <add-attr attr-name='something'>
        <value>Else</value>
      </add-attr>
    </add>
  </input>
</nds>
```

The XML elements are: `<nds>`, `<input>`, `<add>`, `<association>`, `<add-attr>`, and `<value>`.

- You would select these in XPATH by name and path.
- Keep in mind how IDM sets the current context. This would mean that you start selecting from the event node, which in this case is `<add>`.

The XML attributes are: `dtdversion`, `ndsversion`, `qualified-src-dn`, `src-dn`, and `attr-name`.

- You would select these in an XPATH with the `@` sign. For example you would choose `@src-dn`, `@qualified-src-dn`, or `add-attr/@attr-name`

The XML text is in between the `association` nodes or the `value` nodes.

- You normally select these in one of two ways. You describe the path to them, and either let it assume it is a text node or else you use the `text()` function. So by example you would choose `association`, `association/text()`, `add-attr[@attr-name='something']/value` or `add-attr[@attr-name='something']/value/text()`

It is possible to build this whole document just using the tokens we are talking about, but there is usually no need.

In all three of these tokens, you need to specify your target via an XPATH expression. So that might be something that describes where you want to insert it. One key point I explained in the mentioned article above is that you need to be careful that your XPATH specifically describes one location, or it gets inserted in every matching location. Thus, the use of the `last()` function is

helpful. You could use `add-attr[last()]/value[last()]` to be sure you are touching the last value node in the last `add-attr` node in the document; otherwise, you would be adding it to every `add-attr/value` node. There could be many `add-attr` nodes, and each one could be a one to many value.

The other important use case is that you can build an XML document in a variable. If you define a node-set variable, I usually start by XML Parse'ing the string `<firstNode/>` into it when I define it as a node-set, like this:

```
<do-set-local-variable name="XML-DATA" scope="policy">
  <arg-node-set>
    <token-xml-parse>
      <token-text xml:space="preserve"><firstNode/></token-text>
    </token-xml-parse>
  </arg-node-set>
</do-set-local-variable>
```

In that variable my XML looks like this:

```
<firstNode/>
```

This gives me a node-set with a parent node that it is easy to hang XML off of. Like *Append XML Element*, named `secondNode` with the target XPATH of `$XML-DATA/firstNode`

```
<do-append-xml-element expression="$XML-DATA/firstNode" name="secondNode"/>
```

Now my XML would look like:

```
<firstNode>
  <secondNode/>
</firstNode>
```

Now I can embellish with *Set XML Attribute* `firstAttr` to the value 'set', like this:

```
<do-set-xml-attr expression="$XML-DATA/firstNode/secondNode" name="firstAttr">
  <arg-string>
    <token-text xml:space="preserve">set</token-text>
  </arg-string>
</do-set-xml-attr>
```

Now my XML will look like:

```
<firstNode>
  <secondNode firstAttr='set'/>
</firstNode>
```

I can add some XML text to that secondNode, like this:

```
<do-append-xml-text expression="$XML-DATA/firstNode/secondNode">
  <arg-string>
    <token-text xml:space="preserve">SomeText</token-text>
  </arg-string>
</do-append-xml-text>
```

Now my node-set would look something like this if I used trace with *XML Serialize* to show it after each step:

```
<firstNode>
  <secondNode firstAttr='set'>SomeText</secondNode>
</firstNode>
```

A couple of other notes about the tokens themselves. Each time you have an XPATH expression line, you get a couple of tools in little icons to the right. You get an XPATH builder, an ECMA Builder, Namespace Editor, and the variable selector. These are all the same as discussed in earlier segments on XPATH.

The key thing with the XPATH builder is to remember it is a generic XPATH builder. It probably came from Eclipse not Novell. It does not know where the current context is in relation to IDM. So if you just paste your XML document into the source view, it will likely want you to specify an XPATH of `/nds/input/add/add-attr`. This would be a valid XPATH expression. However, since the engine starts you at the event node to use it in the engine, your correct XPATH is actually `add-attr` alone.

Sounds like the editor is useless then, right? Not exactly! You may set the context to match the Engine if you take the XML source document, and on the XML Tree view, select/highlight the `add` node. You have just now set the current context for this builder to match the engine's. Now the `add-attr` is the correct XPATH. This is usually an enlightening moment for anyone who has tried using it and not quite figured out why it barely seems to work.

Alas, getting access to variables in here is not an option in terms of reading content out of them. Trying to test my `$XML-DATA/firstNode/secondNode` as a XPATH statement. It will not work since it does not know what `$XML-DATA` is. Instead I could paste my XML source of:

```
<firstNode>
  <secondNode firstAttr='set'>SomeText</secondNode>
</firstNode>
```

into the XML source view. Be sure to leave the current context at the root and then try the XPATH of `firstNode/secondNode`. It's kind of weird doing all of this knowing when I transpose it back, I need to add back the variable reference. Still, it is good practice and causes thoughtful coding.

Probably the most common place to use these tokens in shipping driver configurations (or packages) is to build an XML structure under the `<operation-data>` node. If you used the *Set Operation Property* token, then an XML attribute of the `<operation-data>` node will get created with the specified value. In the Password Synchronization packages, the NetIQ folk decided to carry more information than simple XML attributes would allow, so they append the elements and content into the node using these tokens. This allows more flexibility for storing and using information about the event.

Break

This is a fun token, but could support a few additional options. I have submitted an enhancement request (you can too at <http://www.novell.com/rms>) requesting that the break token support different levels of break.

This token only has one level of breakery (That's what happens when you try to use this in a bakery.) at the entire policy object. You will often see policy objects where the first rule does a test *If Class Name* not equal to **User**, then, *Break*. This is a nice way of saving time in the flow since you could have every rule in the policy object. Just have an *If Class Name* equal User test. It's better to do it once at the very beginning, and then you don't worry about it again.

I would love to be able to break out of a loop and maybe break out a different number of levels, but the token does not support this. Overall, it's very useful but could be better if we had some more fidelity in the breakage level.

This differs from *Veto*. *Break* stops the current running Policy object from processing any further. *Veto* stops the entire event at this point.

Clear Destination Attribute Value

I probably should have covered these in my section on the *Add Source/Destination Attribute* and *Set Source/Destination Attribute* tokens.

Actions

Define new action below

Do **clear destination attribute value** ?

Specify attribute name: *

Specify class name:

Select mode: add to current operation

Select object: Current object

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

This just inserts a `<modify-attr>` node with a `<remove-all-values>` node. Alas, it is NOT smart enough to do what seems obvious, which would be to also strip out any operational attributes for this attribute as well. That is left to the user. I suppose this is better in the grand scheme of things, since this way you now can do it either way. Just remember to do a *Strip Operation Attribute*, and then a *Clear Destination Attribute Value* if that is what you desire or not as the use case may dictate.

Clear Source Attribute Value

Clear Source Attribute Value does not require the same *Strip Operation Attribute Value*. Since we are not acting upon the current event document.

Actions

Define new action below

Do **clear source attribute value** ?

Specify attribute name: *

Specify class name:

Select object: Current object

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

Additionally, if you use this on an `<add>` event, you will NOT remove any operational attributes with it; and, in fact, it will add a second event that is a `<modify>` changing this attribute as above.

Specify attribute Name

As usual the attribute name can be provided in a variable. For example, if you had a list of attributes to clear in Active Directory when a user is disabled (so they do not appear in an Exchange GAL perhaps), then you might have a List GCV with all the variable names. You could loop with a *For Each* over the GCV noun token, call *Clear Destination Attribute Value* for the attribute named `$current-node`, and then you have a simple method where to add clearing another attribute or removing a few is not a code change, rather, it's a configuration change in a GCV.

Specify class Name

Class name is the next line, and for eDirectory this is usually a non-issue. For Active Directory, it seems to be sensitive to needing the object class. Other systems may be the same or differ. It's hard to know without trying.

As always this is the base object class. You might consider it likely that if you are removing an attribute that is in an auxiliary class, you should specify the auxiliary class name here, but do not be led astray. It is the base class of the object you need to specify (User, Group, etc) and not the extension. Of course, if you try to remove a mandatory attribute, expect a failure unless you also remove the object class in the same operation.

Mode

Mode is the same as in the *Add Role* example earlier, and you get add to current operation, add after, add before, and write directly to datastore.

Clear Source Attribute Value, of course, does not have the Mode choice since it is always write directly to the data source as we have no current event to add before or after.

As usual you also have the ability to target the current object or another object based on DN or association value.

In all drivers but the eDirectory to eDirectory driver, you use the association in the target data store. That is, there is only one association value. The eDir to eDir driver is a very confusingly two-driver system. You want to consider which association you mean. There are two association values based on where you are pointing and what you want.

In a typical driver, you will have an object. Either you have the DN in one system, or the single association value. You can resolve both in either direction. In other words, an association can be used to resolve a DN in the target system or eDirectory. Using the *Resolve* token, a DN in either system can be resolved to the single association.

However, in the eDir to eDir case, you need to consider the other eDirectory system as the target application. By so doing you can resolve a DN in the source eDir to an association in the destination eDirectory. Just be careful with which way you point as you do it.

As always, you will have attribute and class pickers as needed to avoid typos on the various selection lines.

Clear Operation Property

An operation property is an XML attribute on the `<operation-data>` node of an event. The engine removes it as the event is submitted to the shim and put back on the event as it returns from the shim. This way you can carry a Source DN or Event type value through the shim, so on a failure `<status>` event, where `@level='error'` you could have a rule that catches it, *XML Serializes* the `current-op` local variable to get the error message, but the event will not have a DN or event type to let you know what caused it.

Thus, in your Subscriber Command Transform, you might add an operation property `src-dn` and maybe `event` with the events info. Then you have it all available to be used when the event returns on the other end of the shim.

Clear SSO Credential

Set SSO Credential

Set SSO Passphrase

I wish I had more to say about these tokens. When you have them working, they can do wonderful things with Secret Store. The Secret Store allows you to securely store 'secrets' on a user. The Secret Store is more than a password store. You can store arbitrary strings and many numbers. Additionally, only the user can get them out. If an Admin tries to get at them, they can be locked out so the end user knows they have been tampered with.

However, every time I use them I have to rediscover the exact usage. Probably the hardest part is knowing the strings you need to specify that are stored inside the credential. You need the Secret Store plugin to iManager to be able to even begin to figure this out.

Actions

Define new action below

Do **clear SSO credential** ?

Specify credential repository object DN: *  

☒ Set DN relative to policy

Specify target user DN: * 

[Populate the following from an application object](#)

Specify application credential ID: * 

Specify login parameter strings: 

 Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

Actions

Define new action below

Do **set SSO credential** ?

Specify credential repository object DN: *  

☒ Set DN relative to policy

Specify target user DN: * 

[Populate the following from an application object](#)

Specify application credential ID: * 

Specify login parameter strings: 

 Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

Actions

Define new action below

Do: **set SSO passphrase** ?

Specify credential repository object DN: * 🔍

☒ Set DN relative to policy

Specify target user DN: * 📅

Question string: 📅

Answer string: 📅

ⓘ Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

It's good to know that Secure Login can use the Secret Store for workstation based SSO (Single Sign On). Novell and NetIQ have other products like, NetIQ Access Manager, that can also use the Secret Store for SSO. This can be very powerful, since your IDM system could generate crazy passwords for some application that the user never knows but is stored in Secret Store. When a user connects via Access Manager or Secure Login, it is silently pushed into the login fields without their noticing.

However, I do not have concrete examples on how to use these, and the docs are lousy on the topic. So I may have to take a pass on these for now. I have, however, written one article in the past, but it is not as helpful as I would have liked.

<http://www.novell.com/communities/node/2716/using-secret-store>

Clone By XPATH Expression

This is a fun token. Like the *Append XML Element* style tokens, you can do some cool things with it. For example, if you had followed my idea of building up some XML in an attribute and now have a variable XML-DATA that has a nice XML structure in it. You might *XML Serialize* the data into a string and send it to the application.

But perhaps, in the case of a SOAP driver, you want to copy the SOAP envelope you just built in a variable into the event document. Well, *Clone By XPATH Expression* is the tool for you.

This token needs a source and destination XPATH expression for what you are copying, and where you are copying it to.

For example, rather than using XSLT to change your XML from XDS to some SOAP document that your application needs, you could build the SOAP document in a node-set variable. Then *Clone by XPATH expression* the source of `$XML-DATA` and then the target of as simple as a period (`.`) the current context. Then you could *Strip By XPATH* the current event.

As always with using XPATH as a source or target, you need to be very very specific. If you were to clone a value to `modify-attr` as a target, EVERY `<modify-attr>` node would then get a copy. You have to be more specific. Perhaps use a target of `modify-attr[@attr-name="What You Want"]/value[last()]` or maybe even just `add-attr[last()]` to more specifically identify the target. The good news is that Simulator is your friend here, and you can test pretty quickly with a bit of thinking. The source as well needs to be specific, as you might wish to only clone a segment of the variable. In which case, your XPATH should be specific enough to get just what you are looking for.

There are so many interesting ways to use this token, manipulating XML under the covers. Most people underestimate its power.

One interesting oddity about this token is that it often does not like creating a node-set as a target out of whole cloth. That is, *Clone By XPATH Expression* from `$VAR-NAME` to `$OTHER-VAR` will work fine if `$OTHER-VAR` is already defined as a node-set. Using a Set Local Variable token can do this. Specify the type as a node-set and then set the content as an *XML Parse* applied to a text string of nothing. Thus you have the needed empty node-set variable to clone into, instead of implicitly expecting this token to create it for you.

Clone Operation Attribute

This is a useful token, and you will see it used in the Active Directory driver. It is used in the Publisher Command Transform when it sees a `Member` attribute on a Group change.

A quick diversion about eDirectory groups to explain that. A group in eDirectory has a `Member` attribute that lists members. On the user, there is a matching attribute `Group Membership` (LDAP directories usually call this `memberOf`). However, in eDirectory the granting of rights via security equivalence is NOT done through these two attributes but, rather, through a second pair that apply to all objects when they need to made security equivalent.

The Group contains `Equivalent To Me` (A list of objects that are equal to this object.), and the User contains `Security Equals`.

This ability to have a group that is or is not a security group can be useful. Active Directory has a different model. In Active Directory, the group type indicates if it is a security group or not.

However, to further complicate things, Active Directory only stores the member list on the group. It does not have the reciprocal attribute on the User object. Wait, you say. When I look in Active Directory Users and Computers, I see a `Member` tab on users and a list of their groups. Well, yes you do. However, that list is actually generated in a query on the fly. Earlier versions of Active Directory did not expose a `memberOf` attribute at all; however, around Windows 2008, I noticed that there is a `memberOf` attribute available via LDAP that is read only. Considering how much static waste of space Active Directory already stores, it always amazes me that they tried to save an attribute in this fashion. Regardless, when a Group synchronizes in from Active Directory, you want to add `Security Equals` and `Equivalent To Me` values.

You might think, that's fine, I could just do a *For Each over Operation Attribute* `Member` and *Add Destination Attribute* `Equivalent To Me` with the `current-node` local variable as the value. However, you would then need to do a second loop over *Removed Attributes* to remove the destination attributes as needed.

If you had a remove-all-values in there, you would need to handle that as well. This is basically a special case of *Clone By XPATH* where it knows how to handle an add, or modify event. Clearly this makes no sense in a delete, rename, or move event case. Do note that instance documents do not get processed in the Command Transform. You could use this in the Input or Output Transform if you wanted to. Like the *Reformat Operation Attribute* token that is built to know how to handle an add, modify, or instance document, this one is much more clever under the covers than you might suspect. It is only when you try to reproduce the results by hand that you realize how much work it is really doing under the covers.

It is quite a clever token and takes an existing function, and exposes it wrapped to make our lives easier.

Delete Destination Object

This is a very simple token and is useful to be able to get rid of objects on demand.

Delete Source Object

This is a very simple token and is useful to be able to get rid of objects on demand. A common use for this is when a user is disabled in the Identity Vault. In the Subscriber channel, you might set *If Operation Attribute Login Disabled* is changing to true, *Delete Destination Object*.

Now, it is worth mentioning that you should really remember to also *Remove Association* when you do this. Thus, I usually remove the association first then delete the object. From a licensing point of view, you pay by associated user, so getting rid of unneeded associations is a good practice.

Now, if you are deleting the object in eDirectory, it does not matter if you remove the association or not since the object carrying the association is deleted.

Do be careful if you consider vetoing events. The most common example is to veto delete events. You want to be careful to scope it properly. It actually is possible to *Veto* the driver startup event causing the driver to fail to start! Thus, you would want to make sure you specify a class or operation type specifically. The common mistake is *If Operation* not equal to delete then *Veto*. Well, that will often stop the driver from starting up. It may look funny in some systems when an object is deleted. Active Directory, for example, will show the DN of deleted object already moved into the tombstone area of Active Directory and named by the object GUID instead of the previous name. Additionally, the class is lost and not in the event at this point. So scoping to, say, class of User or Group just won't detect it. You can read more about these scoping concerns in these articles:

<http://www.novell.com/communities/node/2052/avoiding-startup-vetos-scoping-rules>

<http://www.novell.com/communities/node/1858/more-about-veto-command-scoping-events>

<http://www.novell.com/communities/node/4224/scoping-delete-events-lotus-notes-driver-idm>

Find Matching Object

This is a very interesting token with some unexpected behaviors. First up, you can chain them. That is, you can do six different types of matching attempts. As soon as one returns a matched value, the others just skip doing anything. Upon success of finding a match, it will add the `dest-dn` XML attribute to the `<add>` node. At which point, the other tokens stop doing any searching and very quickly process through.

Thus, you should order stacked *Find Matching* tokens in the order you want them to take precedence.

```
<do-find-matching-object scope="subtree">
  <arg-dn>
    <token-global-variable name="base-dn"/>
  </arg-dn>
  <arg-match-attr name="uniqueID">
    <arg-value type="string">
      <token-local-variable name="PROPOSED-NAME"/>
    </arg-value>
  </arg-match-attr>
</do-find-matching-object>
```

The docs now have some information about special values that might get set in certain error cases and are not recorded in an error variable (like the *Add Role*, *Remove Role*, *Start Workflow*, *Send Email*, SSO ops do) but, rather, in funny Unicode values in the destination DN value.

You can test for these with a Regular Expression using the `\u` (Backslash u) and then the Unicode value. This way you can detect these cases.

An example might look something like this:

```
<if-dest-dn mode="regex" op="equal">\uFFFC</if-dest-dn>
```

I wrote about this token before in the article:

<http://www.novell.com/communities/node/6178/new-idm-36-enhancement-do-find-matching-objec>

Just to complicate things, you also get the funny error variable named `error.do-find-matching-object`, which will hold the DNs of any error objects. If more than one is found, it should hold a list of the DNs. If an already associated object is found, you should get the DN in the variable.

Select Scope

You get a couple of options to scope the search depth. It supports the usual options of:

- **entry** - for just the specific DN you provide in the next line
- **subordinates** - to search only those directly under the specified DN
- **subtree** - to search to the bottom

These are basically the same as for the *Query* noun token since, of course, this is just a special case of a *Query*.

Then, you have to specify the DN to be used. I strongly recommend scoping this and using a GCV to store the value. You should provide a DN in the format the target system expects and not in eDirectory format (the backslash format). For Active Directory or LDAP drivers, you want LDAP format DN's. The query DN would not get converted based on an association, so you need to be sure to have the right value provided.

Specify match attribute

This specifies what attribute the object should contain. The little box on the side brings you to Match Attribute Builder, which is really just the String Builder. In this wizard you specify the Variable to match against (Variables are allowed using \$VarName\$ format). Then you can choose to use the operational attribute value of that attribute. In other words, if you selected Full Name, then you might want to use the current value of Full Name in the <add> event. But often that is not the case. Perhaps you are not sure the Full Name will always be in the event at that time. Instead of "Use Values from Current object", use Other Value and define it using the full suite of tools you get to build any string value.

You can stack as many match criteria as you want in here. At least one Attribute must match for a successful match.

For example, you could match on three things like Given Name, Surname, and preferredName. The first match could match against all three. Then if that fails, you could try on just Given Name, and Surname. These are examples as Given Name, Surname, and preferredName are just about the worst possible to use.

How many Inigo Montoya's (You killed his father. Prepare to have your account die! How many fingers do you have again?) are in your organization? In the real world, I was at a client with about 40,000 users, and they named users uniquely in eight characters. They had a user with the name "Swilla" up to number 33! That was a bit nutty but clearly a common naming pattern in that neck of the woods.

Better matching criteria are immutable things. Maybe employee number? Unless, of course, you use SAP for your HR system. In this case, PERNUM (what looks like your employee ID,

and by default is mapped to workforceID) has several valid cases and might change over the course of your employment. Then, of course, SAP's answer is to use the Central Person number (CP number) except that every event comes linked to your current PERNUM. You would have to go check what the CP number is with extra work. I know they have good reasons, but it is really annoying to work with. Or perhaps you have merged two companies, and some of the employee numbers overlap. This is another dreadful situation.

So think carefully about your matching criteria. If you are licensed for IDM 4 Advanced or Standard edition, you will get a copy of Analyzer for Identity Manager. This is also an Eclipse RCP application like Designer, with its own database. You can import data via LDAP, CSV, and existing IDM drivers to analyze. You can then perform data cleansing and matching exercises to figure out what is the best matching criteria. The Analyzer gives you a good idea how your drivers might perform in the real world.

For a weird bit of info. In the trace, the status of what object matched will be shown. There was a bug recently that would show the DN when duplicate matches are found. Before it did not show the DN's of the matched objects, which was silly since it clearly had them. Why show them in the trace, since they are already found?

If you watch in the trace, you will see the query the event generates, and you do need to be careful with wildcards. Not all attributes will support matching by wildcards. Some do, some don't. Documentation does exist, but is buried in the schema reference (not in the eDirectory reference, or at least not where I know how to retrieve it), so I always have to search to find the right schema reference doc. Or else use the LogicSource for eDirectory. I got a copy on a CD ten years ago, when I was still young. Now the reference ships in NPRS, the program that mails you DVDs with all the Novell/NetIQ products every month. One neat example is if perhaps you matched on a telephone number syntax field. Crazy example I know, but you would be amazed. In that particular syntax type, the rule is that it ignores the round brackets, dashes, spaces, etc. Instead, the match must be an exact numeric match.

For Each

There are the two loop structures in IDM *For Each* and *While*. These are great and so darn useful.

For Each needs a node-set to loop over. That is, you have to provide a node-set of some kind, and many kinds and sources are allowed. Then you perform actions on each iteration of a loop. Inside the loop, there will be a local variable `current-node` made available that will have the attribute name in the current iteration. If the node-set is an XML node-set, then `current-node` will have the current node and its children as XML and XPATH can be used against the value. If you are looping over a node-set of values, then you can treat it as a string. If you are looping over, say, a List GCV, then `current-node` will actually have `<item>CurrValue</item>`. This is actually a node-set. If you treat it as a string, you get the text element as a literal string of "CurrValue". Both of these

methods are valid. As always when casting a node-set to a string, you actually get the concatenated strings as the result. If there is only a single text string, this works fine. It is when there is more than one that you get odd results.

If you nest a *For Each* loop inside another *For Each* loop, the inner loop uses `current-node` locally, and you will lose access to the outer `current-node` for the duration. Thus, if you need it, remember to store it in a variable before the inner loop. When you return and complete the inner loop `current-node` should be back and available, and then the next iteration of the outer loop begins all over.

The node-set can be an XML node-set, in which case, you can use XPATH to specify the starting point to loop. For example, take the current event (say a modify) in its entirety and set it in a local variable either using XPATH of `.` [just a period] or from the built-in local variable

`current-op`. Now you can loop over all the `<modify-attr>` nodes or you could use an XPATH of `$LOOP/modify/modify-attr`. In fact, you could do that more simply with XPATH of `$current-op/modify/modify-attr`. Another simpler XPATH of `modify-attr` would retrieve the value from current.

One of the very cool things about this token is that you can use a *Query* noun token that was discussed in the section earlier about Noun tokens. If you specify a value for `max-results`, then the *Query* will page its results and query for the number you specified in as many pages that it takes.

Thus, if you said `max-results` of 30, then it would not be the first thirty results and then nothing else as many folk initially assume. Rather, if there were 100 values to be returned, you would get results in pages of 30, 30, 30, and then 10 in successive iterations.

If you are setting a local variable with this *Query* token, you get all of the results one page at a time (size defined by `max-results`). However, if you use it as the target node-set in this *For Each* context, then the loop asks for the first page of results, loops over them, then asks for the next page, and so on.

`Max-results` actually makes great use of memory. It would require large amounts of memory to process ten thousand attributes as one big chunk. The default memory setting for IDM is 64 Megs, which is usually plenty for IDM. I tried to estimate the kind of memory usage you might see from a query like this in these articles:

<http://www.novell.com/communities/node/6176/another-node-set-size-discussion>

<http://www.novell.com/communities/node/6085/more-thoughts-size-node-set-identity-manager>

What is interesting is how far that default of 64 Megs will get you! Most people run with the default until one day they run out. They are often surprised at how long the default worked for them. Alas, usually a large group membership event is what pushes them over the edge.

Regardless, using a *Query* token with a `max-results` as the node-set is a great way to handle this. It turns out, however, that there is a big exception to this idea. The best example is Group membership where a large node-set will need to be handled. The attribute `Group Membership` is a multi-valued attribute on one object with potentially thousands of values; therefore, a `query-ex` will still return it in one page. This is because the paging is at the returned object level and not at the returned attribute level. In other words, if you query one group for its `Member` list that is ten thousand members long, it is really a single result returned and not paged. However, if you queried for a `max-results` of 10 and there were 15 groups with ten thousand members each, you would get two pages of groups. Each group would potentially have thousands of attributes.

Now, when you use a *Query* like that, the node-set you are looping over is an XML node-set. You can use XPATH upon the `current-node` local variable if you want. This is one of those instances where you can treat the variable as a node-set and use XPATH upon it, or as a string and get the string version of the node-set.

Another neat reason to use a *For Each* loop would be if you are not sure how many results you will get. A *For Each* loop over the data will loop 0 or x number of times as needed with no testing to check for values required. This way you do not need a *If Operation Attribute* is available. You can *For Each* loop over the *Operation Attribute* token safe in the knowledge that if it is single valued, great; multivalued, great; or absent, no problem. The same code handles all three cases. Of course, how you handle a multivalued attribute probably differs from a single valued, but you get the idea.

In fact, before IDM 3.5 introduced the nested *If* action token, the easiest way to simulate an *If-Then* test was to make a *For Each* loop over an XPATH condition that implemented your test. If the test returned a value, it looped once. If it failed, it did not loop at all. Thus, you get an implicit *If-Then* test.

Some interesting XPATH ideas to loop over would be things like:

XPATH of: `//@attr-name`

This will loop over each attribute name in an add, modify, or instance document and allow you to consider each attribute name in turn. At that point, if you decide to do something with it, then `current-node` is the attribute name. So you might want to get the *Operation Attribute* for `$current-node` into a variable to do something with it, and then you can *Strip Operation Attribute* `$current-node`, and then *Add Destination Attribute* `$current-node` with that value you stored.

There are two other interesting items to loop over that tie into that example:

XPATH of: `add-attr`

XPATH of: `modify-attr`

These two options will loop over all the attribute nodes in the current event document. Now, you can get the attribute name with XPATH of `$current-node/@attr-name`, and then check for a `<remove-all-values>` with a test of if XPATH is true `$current-node/remove-all-values` and so on.

Note that you need a different XPATH for an add event versus a modify event. This is where the *Operation Attribute* token is better since with one token it will return the needed node-set, regardless of whether it is an add or modify event.

There is a major distinction to be made between the node-sets returned by two different types of queries. The first is the result of a *Query* token, and the second would be the result of *Attribute* noun tokens (*Attribute*, *Operation Attribute*, *Source Attribute*, and *Destination Attribute*).

Returning to the earlier *Query* example for group members as the node-set, the `current-node` in that case would be the `<instance>` document returned. So you would likely need two loops in the group object and `Member` attribute case. The first outer loop would be over the *Query* results looping once per group object returned. Then if you needed to do something with the `Member` attribute, you could loop a second time over the node-set XPATH of `$current-node/attr[@attr-name='Member']/value`, and now inside that inner loop the `current-node` is the DN value of the actual member.

If you use the noun tokens *Attribute*, *Operation Attribute*, *Source Attribute*, or *Destination Attribute* from Argument Builder, then your loop will be over values of the attribute since, of course, only one object is returned (The one you specified, which is either the current object or the one whose DN or association was provided as a target object.). Then the node-set will be the set of values returned (0, 1 or many), and the variable `current-node` will hold a value. You can treat the `current-node` variable as a node-set or as a string at this point. Thus, you can treat it differently based upon what you need. Let's assume the returned node looks like:

```
<value type='dn'>SomeDN\O\OU\CN</value>
```

As a node-set, you might be able to use an XPATH token of `$current-node/@type` if the type was of interest. As a string, just the *Local Variable* token of `current-node` would return the string value. Of course, an XPATH token of `$current-node/text()` would be the same since treating a node-set as a string returns any text values in the node-set.

In the IDM 3 and lower days, before the nested *If-Then* condition test was available, *For Each* was the best way to fake it. If you can do an XPATH test that returns zero, one, or more results, you can *For Each* over that XPATH. If it is false, it won't execute (0 loops); if it is true, you perform your action. Since IDM 3.5, we thankfully no longer have to do that workaround, but it shows how some clever folk worked around limitations. I tried to address other such features in IDM 3.5 that are not there when I have to go back to an older IDM 3.0 project in this article:

<http://www.novell.com/communities/node/6299/working-idm-301-after-using-351-while>

One interesting feature of the *For Each* over a node-set is that if you want to, you could build the node-set on the fly in the token itself. That is, each noun token you add in the Argument Builder will be treated as an item to be looped over. I did this in an example rule I published in this article:

<http://www.novell.com/communities/node/13486/common-mistakes-newcomers-idm-make-part-10>

```
<rule>
  <description>Buildnodeset using a loop and last()</description>
  <conditions>
    <and/>
  </conditions>
  <actions>
    <do-for-each disabled="true">
      <arg-node-set>
        <token-text xml:space="preserve">One</token-text>
        <token-text xml:space="preserve">Two</token-text>
        <token-text xml:space="preserve">Three</token-text>
      </arg-node-set>
      <arg-actions>
        <do-break/>
      </arg-actions>
    </do-for-each>
  </actions>
</rule>
```

You can see that the node-set (<arg-node-set> part) is really just three *Text* tokens. Each of those is treated like an instance to loop over. Silly example, but it does show the point. That is, each token in this context is treated as an individual node. Whereas, in a string they would be concatenated together into one string. When treated as a node-set, they each become a node. This is, of course, the same behavior you get when you treat a node-set as a string in the converse.

Additionally, I was, at one point, wanting to loop over the results of three queries in a single *For Each* loop. Not knowing any better, I set a local variable (node-set) to the three queries to get all the data and then looped over the local variable. This does work, but that can be one mighty big variable that will eat up a lot of your Java memory heap. Even worse, you do not get the memory benefit of paged results via the <query-ex> functionality.

There is a much simpler way to handle this. Use three *Query* tokens in the node-set definition for the loop not using the intermediate variable, and each will be processed in sequence and paged according to the `max-results` setting. They will come in pages to save memory. That approach is so much better!

Generate Event

This is a poorly named action token, since we talk about events in IDM as a <modify> or <add> event. But in this case, they really mean an Audit Event that gets sent to the audit subsystem.

Copyright© 2013 Ninja-Tools

Now, initially this used NSure Audit, which amusingly was derived from the audit model used in Netmail (a highly scalable mail system that Novell sold off to Messaging Architects and is still available). This uses an audit collector on each box, and the IDM engine sends the event to the collector. Then the collectors forward the events on to the audit system. Then Novell bought eSecurity for its Sentinel product and wrote a collector in Sentinel to accept these Audit events from the NSure Audit subsystem. This nicely allowed current systems to just change where their collectors send events to move from one to the other. This allowed a migration from NSure Audit to Sentinel in a reasonable fashion.

Sentinel now has several different versions. There is Sentinel Log Manager that is designed to collect events and let you look at them historically or search as needed. Then there is the original Sentinel that is meant more for real time examining of the data and correlating things. There is Sentinel RD (Rapid Deploy) that replaced some of the commercial (expensive license costs) components (the Message Queue, Business Objects) to more affordable Open Source alternatives that are similar in functionality. With Sentinel 7, the non-RD version goes away, and there is just one Sentinel version.

What Sentinel and RD can do is watch events from all the various sources and recognize that Event #1 and Event #2 are not very suspicious. When they come one after another, to the same destination or from similar source locations, they are suddenly much more suspicious. Even cooler, it has the notion of a sliding time window, so you can trivially specify that Event #1 and #2 occur within 30 seconds of each other.

With IDM 4, the logging subsystem has been modified again, and the remaining NSure Audit components have been replaced with OpenXDAS collector instead. You can see they are slowly moving pieces around and through abstraction layers keeping all the bits coordinated.

If you turn on Logging, which is different than Trace, and have an NSure audit collector or the OpenXDAS collectors ready to go, then the events it collects get sent on to where the collectors point based on how you configure the Log settings.

What if you want only specific events or only certain events with some specific criteria instead of all modify events to be sent on to the Audit system? Well, you can use this token to send your own custom audit event. Now, these not being the standard out-of-the-box ones means you need to update your Sentinel or NSure Audit side to accept and process the new event type you are sending in.

Jeff Johnson wrote a nice article on how you might do this for the NSure Audit days. It is still the same process. This was a bit harder to find than I expected. It is from 2006 and still valid!

<http://www.novell.com/cool solutions/feature/18622.html>

Do generate event ?

Specify ID: *

Select level: informational

Specify strings:

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

*Required

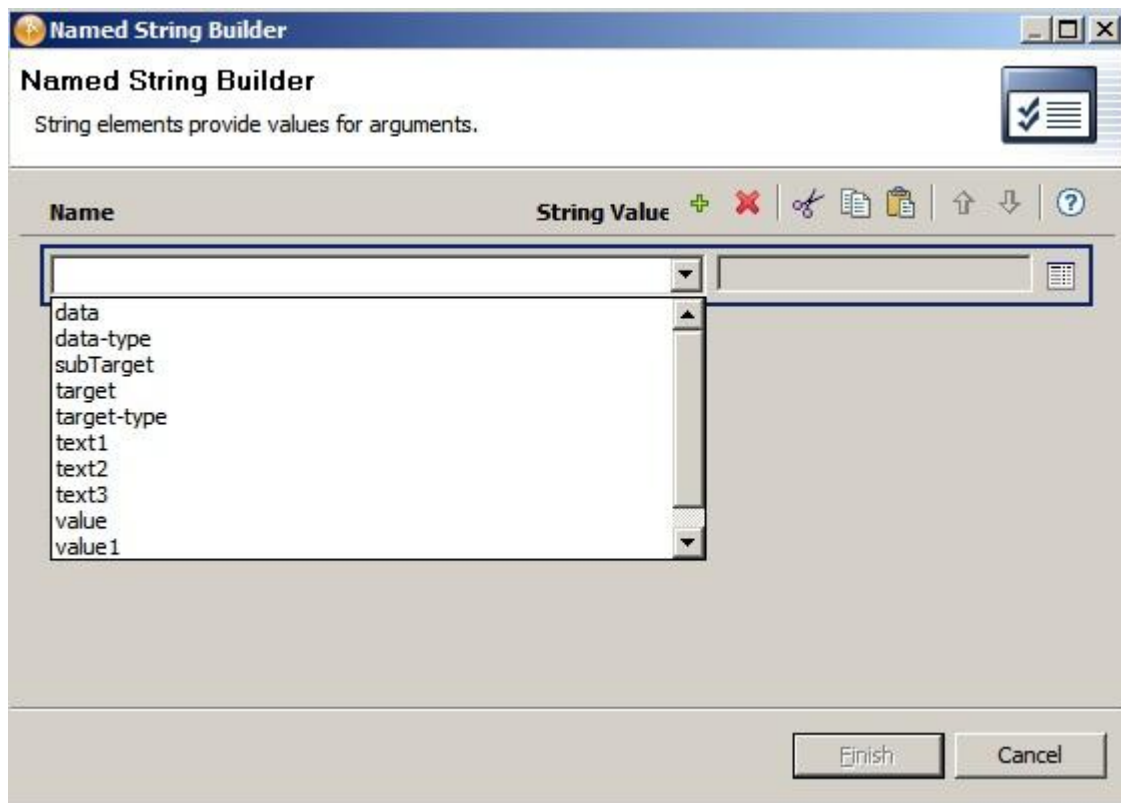
The key issue is formatting the data being sent to your Sentinel or Audit system in the manner it is expecting. Conversely, making sure your Sentinel or Audit system knows what data it is getting. Events have ID's; thus, the first line is to select a number. They have to be integers between 1000 and 1999, so there is lots of room for custom events. This goes into the **Specify ID** field.

Then you can **Select level** from the following choices:

- informational
- alert
- critical
- debug
- emergency
- error
- notice
- warning

I won't waste time on these since the docs do a good job of explaining the distinctions.

Next for **Specify strings** you get a Named String builder where you can send 0 to 10 different strings, and probably more if you wanted to. These are the ten that are expected in an event.



Like other tokens, the Named String Builder gets a pre-populated list of available strings, which helps avoid typos, and makes sure all the options are presented. Tokens like *Start Workflow*, *Add Role*, and *Send Email from Template* do this as well, which can be quite helpful.

Now, the exact structure and values are somewhat unclear and really depends on what you want to do with it in your audit system. The data you send, depends on the data you need in your audit system.

The docs are actually pretty good on explaining that you get text1, text2, and text3 as text strings 256 bytes long. You get value (value1 will work as the same one) and value3, which are integers. There is a data field that can take up to 3K of payload, and data-type is used to indicate what type of data is in the data field. The fields subTarget and target are broken up to allow you to specify an attribute as the subTarget and target as the object itself. Then target-type is used to indicate what format the target DN is written in: None (0), slash (1), dot (2), or LDAP (3).

The issue really becomes what the audit system is expecting in each of those fields. As this token is used for sending custom audit events, there is less of a standard. Each of these, when received, must be supported in the audit system. Thus, you are in control of both ends, so send what makes sense for your use case.

If

This token is the greatest thing since sliced bread! Before this arrived in IDM 3.5, if you had to use a multiple conditions in your policy, you needed to set the variables and then use another Rule with its condition block to perform the *If-Then* testing.

With a nested *If-Then* token, life is just plain better. Alas, now we have the possibility of nesting so deep it takes 30 clicks to get down to the bottom to edit. That is a price I am willing to pay, since I can always go the XML view and do my edits in there.

In fact, what I find most painful is that it is hard to select a group of tokens, copy them, go into an *If* token, and paste them in. You have to do it one at a time, and in Designer the main Rule view does not like you doing the copy with the *If* token open for editing and pops up a warning each time. Interestingly enough, the editor that lets you add stuff to the Then or Else clause, once you are inside a nesting at least one level deep, will let you copy with the next nested *If-Then* open for editing. So I do not know why it complains at the top level or, conversely, why it is inconsistent and does not complain at all levels.

What I usually do is get my nested *If-Then* ready to go. My action inside it is something silly but unique, like a *Break* token. If those are actually in use, I often drop a *Text* token in with 42 since that makes it easy to find. I go to the XML view, copy the XML I need to move around, and then paste it over my `<token-text>` or `<token-break>` node since that shows me where the Then (or Else) clause I wanted resides. With all that nesting, it is sometimes hard in the XML editor to see where the level you want is located.

All the rules about conditions and using the Condition tokens discussed earlier in the Conditions section apply, just the same as if you were in a Conditions block.

Beyond that, this is a pretty simple token. You do not actually HAVE to fill out any of the three sections (Conditions, Then, and Else), and if you do not, you get a simple skeleton that does nothing.

```
<do-if>
  <arg-conditions/>
  <arg-actions/>
  <arg-actions/>
</do-if>
```

You can use this as a simple XML template, if you want to build your own conditions in XML by hand, which can be useful. Start with the empty token, and then copy in the XML for the conditions and actions as needed. Sometimes this is much faster than using the Policy Builder interface, specifically, when some block of code needs to be moved into a *If-Then* test.

Implement Entitlement

This is an interesting and confusing token. One of the features of the Entitlement process in the IDM engine is to track what entitlement related actions have occurred. Thus, the main function of this token is when an Entitlement is determined to be applicable in your driver to record what was done next. That is, your driver will do whatever is appropriate, which should be wrapped in this token. When all is said and done, all that extra work is so the `DirXML-EntitlementResult` will be updated as appropriate by the engine. Entitlements are stored in `DirXML-EntitlementRef` attribute, and each action related to entitlements is meant to be recorded as a value of the `DirXML-EntitlementResult` attribute.

The dynamic help in Designer suggests, "Designates actions that implement an entitlement so that the status of those entitlements can be reported to the agent that granted or revoked the entitlement."

Thus, the way that the status is reported is not explained, but the `DirXML-EntitlementResult` attribute is the carrier of this information.

You could just do a test for *If Entitlement SomeGroup* is available, and then add to member of a group. But then there is no feedback to the system if it succeeds or fails. (Now, that may be what you want.)

Define new action below

Do: implement entitlement ?

Specify node set: *

Specify action: *

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

If you wish to have an update written to the `DirXML-EntitlementResult` on the object, then the node-set it requires in **Specify node set** is the Entitlement node-set. You can get this a couple of ways. One would be the *Entitlement* noun token (Or *Removed Entitlement*, same basic idea), which actually returns a node-set of the Entitlement. This way it is able to parse it to get the entitlement name and the state (0 or 1, for revoked or granted). The other would be the *Removed Entitlement* token, which would do much the same, but will *For Each* over the revoked entitlements in the event document.

The **Specify Action** item will bring up an Action builder just like you would get for a *For Each* or *If-Then* token. Inside there, you can add actions and further nested tokens as needed to do whatever magic is expected to implement your entitlements actions. If the event that gets tagged into succeeds, the success will be added as a value of `DirXML-EntitlementResult` and would

look something like the exchange of events shown below. Here an *Implement Entitlement* token was used with the Entitlement being added in this modify event:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product edition="Advanced" version="4.0.1.0">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <modify cached-time="20121105203711.688Z" class-name="User" event-id="GApp#Publisher#01:c059238c-922f-4240-acd7-769d3a30c67c" qualified-src-dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src-dn="\ACME\edu\acme\People\jsmith" src-entry-id="1977252" timestamp="1352147831#8">
      <association state="associated">https://apps-apis.google.com/a/feeds/collab-dev.its.acme.edu/user/2.0/jsmith</association>
      <modify-attr attr-name="DirXML-EntitlementRef">
        <add-value>
          <value timestamp="1352147831#8" type="structured">
            <component name="nameSpace">0</component>
            <component name="volume">\ACME\services\IDM\DriverSet\Google Apps 2\NOVLGGLEUSER-Account</component>
            <component name="path.xml">
              <ref>
                <src>NRF</src>
                <id/>
                <param>{"ID":"acme.edu"}</param>
              </ref>
            </component>
          </value>
        </add-value>
      </modify-attr>
      <modify-attr attr-name="Login Disabled">
        <remove-all-values/>
        <add-value>
          <value type="string">>false</value>
        </add-value>
      </modify-attr>
      <operation-data>
        <entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-id="1977252" state="1">{"ID":"collab-dev.its.acme.edu"}</entitlement-impl>
        <entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-id="1977252" state="0">{"ID":"acme.edu"}</entitlement-impl>
      </operation-data>
    </modify>
  </input>
</nds>
```

The *Implement Entitlement* token, after being used, adds on in the `<operation-data>` node the following XML:

```
<entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-
dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-
id="1977252" state="1">{"ID":"collab-dev.its.acme.edu"}</entitlement-impl>

<entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-
dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-
id="1977252" state="0">{"ID":"acme.edu"}</entitlement-impl>
```

In this example, there actually were two actions taken just to confuse everyone. The first was to grant, and the second was to revoke the entitlement. You can tell the difference by the `state="0"` (revoke) or `state="1"` (grant).

There are many things you can see in that sample. The Entitlement in the original event has the `path.xml` component with a payload of this value:

```
<ref>
  <src>NRF</src>
  <id/>
  <param>{"ID":"acme.edu"}</param>
</ref>
```

The values for `<src>` can be provided by the User Application with a value of AF, the Entitlements Service Driver (sometimes known as Roles Based Entitlements) with a value of RBE, or the Roles and Resources Serviced Driver (RRSD) with a value of NRF.

This entitlement is using the new IDM 4 version of the `<param>` node, which is based on JSON, versus the older two approaches. Originally, the `<param>` node was just a single value, but in IDM 3.61, support was added to give the SAP UM fanout driver a way to represent multivalued parameters using the pipe symbol to separate them. With IDM 4 and higher, a more flexible approach using JSON, which is a set of key value pairs, was adopted to provide more flexibility.

The `<status>` event that returns a success, error, warning, or fatal level, also contains the `<operation-data>` node with the `<entitlement-impl>` nodes underneath. Thus, the engine silently, in trace (at least), updates the `DirXML-EntitlementResult` with either success or failure.

That event might look like this:

```
<nds dtdversion="3.0">
  <source>
    <product build="20120207_0907" instance="Google Apps 2" version="4.0.1.3">GoogleApps Driver</product>
    <contact>Novell, Inc.</contact>
  </source>
  <output>
    <status event-id="GApp#Publisher#01:c059238c-922f-4240-acd7-769d3a30c67c" level="success" type="driver-
status">
```

```

<operation-data>
  <entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-
dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-
id="1977252" state="1">{"ID":"collab-dev.its.acme.edu"}</entitlement-impl>
  <entitlement-impl id="" name="NOVLGGLEUSER-Account" qualified-src-
dn="dc=edu\dc=acme\OU=People\uniqueID=jsmith" src="NRF" src-dn="\ACME\edu\acme\People\jsmith" src-entry-
id="1977252" state="0">{"ID":"acme.edu"}</entitlement-impl>
  </operation-data>
</status>
</output>
</nds>

```

Now, having said all that, if you were to go look at the NetIQ provided packages for IDM 4, you will likely not find the *Implement Entitlement* token in use in much (if any!) of their policies. It turns out that one of the hidden features of the *For Each* token, when using one of the entitlement noun tokens (*Entitlements*, *Removed Entitlements*), is to implicitly do the work of the *Implement Entitlement* token.

Move Destination Object

Move Source Object

These are fairly simple tokens that do what they say, that is, move an object in either the source or destination data store. Thus, this works in both channels and in both directions, assuming the target system supports a move event.

In something like the Bidirectional Unix driver, there is no move support on the Subscriber channel since the structure is entirely flat.

In a Lotus Notes system, on the other hand, a move from one Organizational Unit (hereafter referred to as “OU”) to another requires use of certifier certificates to resign the private key of the object with the new parent certifier. So a move is quite complicated and needs to get some extra stuff added (names of Certifier files, passwords, etc.) onto the event.

One interesting thing to look out for in Active Directory (I imagine an LDAP driver might see something similar.) is that a move, whether initiated by hand in the Active Directory Users and Computers tool or looping back by the engine, will send two events down the Publisher channel, a move and a rename. This is because a rename or move modify the *Distinguished Name* attribute. In LDAP terms, this would be a *moddn* action. A rename changes the leaf most nodes value, and a move changes something other than the leaf most node. For example, in the Active Directory DN, *cn=geoffc, ou=users, dc=acme, dc=com* there would be a change of the *cn=geoffc* part in a rename and of the *ou=users, dc=acme, dc=com* part in a move. Put another way, the naming component changes in a rename, but the placement component changes in a move.

This is because both cases change the DN attribute, and the driver shim itself does not know if it was a move or a rename. The default Active Directory driver configurations, since probably IDM 2 in the Publisher Event policy set, have a policy object named “Handle Move and Renames” that uses the DirXML-ADContext attribute to figure out which part has changed and vetoes the other event. You can read way more than you probably ever care to know in this series on how DirXML-ADContext is used, and how you might change it to use a plural version so you can support more than one Active Directory driver in your Identity Vault without issues:

<http://www.novell.com/communities/node/12388/managing-multiple-active-directory-domains-one-idm-system-part-1>
<http://www.novell.com/communities/node/12390/managing-multiple-active-directory-domains-one-idm-system-part-2>
<http://www.novell.com/communities/node/12393/managing-multiple-active-directory-domains-one-idm-system-part-3>
<http://www.novell.com/communities/node/12399/managing-multiple-active-directory-domains-one-idm-system-part-4>
<http://www.novell.com/communities/node/12434/managing-multiple-active-directory-domains-one-idm-system-part-5>
<http://www.novell.com/communities/node/12448/managing-multiple-active-directory-domains-one-idm-system-part-6>
<http://www.novell.com/communities/node/12467/managing-multiple-active-directory-domains-one-idm-system-part-7>

This is relevant to a move initiated out of the engine because if you send a move into Active Directory on the Subscriber channel, it will be processed. The shim will detect the change in AD, and send it into the Publisher channel as a move and rename event, which is called a loop back. The Active Directory driver shim cannot tell who performed an action in AD; therefore, every event you send into AD loops back. Usually, optimize modify discards it as a non-event.

Define new action below

Do **move destination object** ?

Specify class name:

Select mode: **add to current operation**

Select object to move: **Current object**

Select container to move to: **DN**

Specify DN: *

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes. * Required

Specify class name is not always required but good form to provide.

As usual you get to **Select the object to move** with the usual choices of Current object, provide a DN, or provide an Association value; and for destination object, you get to **Select mode** in terms of when to execute the move. The usual options for **Select mode** of add to current operation, after, before, and direct are available.

One thing that often messes people up is that when you move an object, you do not provide the final DN; rather, you provide the container to move the object into. You can specify the target by DN or Association, the usual methods.

I discussed this more in this article I wrote on common mistakes newcomers to IDM make:

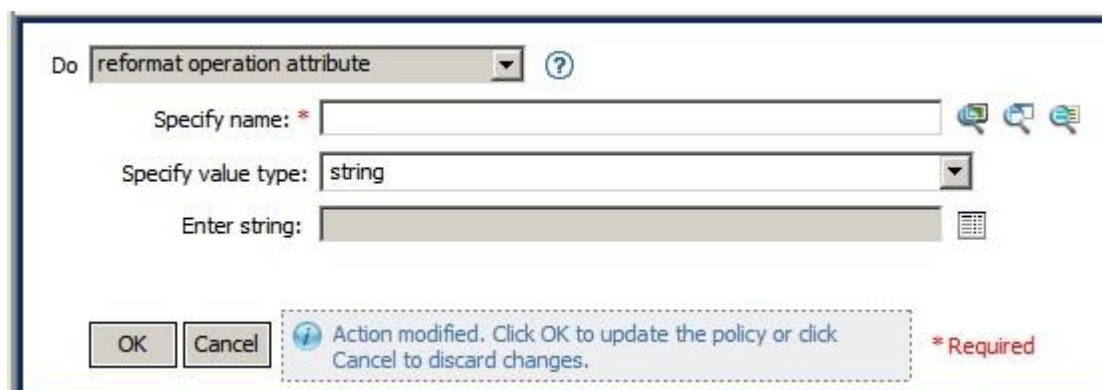
<http://www.novell.com/communities/node/13057/common-mistakes-newcomers-idm-make-part-2>

You can read the rest of the series to see if I hit any of the things you ran into. If there is something that bothers you or had puzzled you, let me know. I would like to keep that series going to help people out, but I am running out of ideas for further content.

Reformat Operation Attribute

This token seems simple but is really quite powerful and clever. For example, it works on a `<modify>` document, an `<add>` document, and on an `<instance>` document. In all three cases the XML that holds the attribute data is a little bit different. When you go about trying to implement it yourself, this is a lot more painful than you would expect.

Also there is a special local variable here, `current-value` (Current-node is used inside a *For Each* loop, and `current-op` is the entire event document at any time you ask for it.), that allows you to implicitly loop your work. If you need to do some work on a value of an attribute, the token here does not obviously allow for a loop. How would you handle a multivalued attribute? Well, the good news is that by using `current-value`, the *Reformat Operation Attribute* token's work is performed on all values in the event document. Very cool.



Also, you will notice that the middle item in the token's UI is a selection for **Specify value type**. If you had a DN syntax attribute coming in, say, `Member` from a Group but you were only syncing groups and not user objects in this driver (imagine a Group only AD driver for some reason), you might have a problem. The engine would normally see the `<value type='dn'>` and try to resolve the associations to convert the DN from the connected systems DN's to the matching object in

eDirectory. Well, if you have not associated the objects to this driver, then the engine will just drop each non-associated object from the event when the type is DN.

With a *Reformat Operation Attribute* token where the **Enter string** value is just local variable `current-value` and the **Specify value type** is left as string, the end result will be to change the `<value type='dn'>` on each entry to `<value type='string'>`. Once that is done, the engine will no longer try to resolve the associations and drop the values.

Or perhaps you needed to sync them to `memberUid` attribute, which some Unix variants prefer to group membership (In Unix, usually the namespace for users and groups is pretty flat so a `Uid` is a better identifier than a full DN.). You would want the trick that changes them from type DN to type String, at the same time, you could apply a *Parse DN* verb token (start of -1, length of 1) to the local variable `current-value` to get the naming attribute (leaf most node). In one operation your conversion is done. Of course, this works on modify, add, and instance events with no changes needed.

This also means that, in a query, it will be reformatted properly as well, which is great and helps keep the data looking consistent throughout the system.

I have written about this token. You can read it here:

<http://www.novell.com/communities/node/3034/reformat-operation-attribute>

Rename Destination Object

Rename Source Object

Much like the move tokens from the previous article, these are used to rename the object. A rename is a special case of modify. Some systems will let you just change the naming attribute without a care in the world, whereas others (usually directories) make a big stink about doing a rename. eDirectory has to run through the Obituary process on a move or rename, which used to be a bigger deal and nowadays pretty much just works.

One other tricky bit is understanding what constitutes a rename. In Active Directory, for example, there are many attributes that might be considered naming. There is the DN itself that has to be unique for its complete name. That is, while there can be more than one user whose `cn=geoffc` in the system, in any given container there can only be one at a time with `cn=geoffc`.

There is `sAMAccountName`, which must be unique within the domain, but if you have multiple domains in the same forest, there can be the same `sAMAccountName` values in two domains. (This is a bad idea as it will probably cause you pain later, but it can be done.)

Finally, there is User Principal Name, which is supposed to be unique within the forest, but I do not think this is enforced in any of the tools, whereas `sAMAccountName` uniqueness is enforced by the tools. Of course, this is likely an example of laziness. A query to the current domain is easy to ensure uniqueness before proceeding, whereas a query to first figure out all the domains in the forest, then query each of them (which may not be available directly on the network) for the current proposed User Principal Name, is a lot of work. It may not even work anyway if one of the domains is unavailable. Though it might use the Global Catalog or some other such construct to handle that.

So which is the one being renamed? Well, in the Active Directory driver, the default configuration supports a couple of models for handling these values. All of which are controlled by GCVs. This means, without any code changes, you can switch or select different modes. Now, of course, running with one mode and changing at a later date to another is not so simple. There is likely to be lots of existing objects that would need updating. With that in mind, here are the three modes:

- Full Name Mapping
- Logon Name Mapping
- UpnMap

Global Configuration Values

Global configuration values for server:

Note: Please add new Global Configuration Values to the 'Active Directory Driver' tab.

Configuration Entitlements Password Synchronization Account Tracking Managed System Information **Active Directory Driver**

Synchronization Settings

Domain DNS Name *

Subscriber Channel Placement Type

Active Directory User Container *

Publisher Channel Placement Type

Name Mapping Policy

Show name mapping policy

Full Name Mapping

Logon Name Mapping

User Principal Name Mapping:

Follow Active Directory e-mail address
Follow Identity Vault e-mail address
Follow Identity Vault name
None

Full Name Mapping

This allows you to name the object in Active Directory (the CN= part of the full DN) as cn=Geoffrey Carman, though, Active Directory folk much prefer Last Name, First Name. That sorts so much nicer in Active Directory Users and Computers (ADUC). This mode assumes that the Full Name attribute in eDirectory is populated and will use that value. There is a rule that has moved around a bit in various configs that will try to build the missing Full Name value for Given Name, a space, and then the Surname attribute value. In earlier configurations of the Active Directory (pre-IDM 4.x), there used to be a *Veto if Operational Attribute is not available* token for the Full Name attribute. There was a similar veto action if the nspmDistributionPassword attribute was missing and was the most likely cause of users failing to synchronize from eDirectory to Active Directory. As

this is one of the most common causes for failure, to synchronize in the most common driver, it is worth showing an example of the trace.

For the Universal Password missing, it might look something like this, and then the event would abruptly end:

Test :	Applying rule 'Veto Testing'.
Test :	Action: do-veto-if-op-attr-not-available("nspmDistributionPassword").

For the Full Name missing, it might look something like this, and then the event would abruptly end:

Test :	Applying rule 'Veto Testing'.
Test :	Action: do-veto-if-op-attr-not-available("Full Name").

I prefer not to use Full Name mapping, since there is no testing (out of the box) for uniqueness within the destination container.

Logon Name Mapping

This uses an approach where the CN in Active Directory is based on the naming attribute from eDirectory so that it might look like, cn=geoffc. In fact, it does this two slightly different but similar ways to accommodate the option of 'flat' vs 'mirror' placement.

There is another common placement option in many drivers. This option allows you to select a placement model in the default configuration without any code changes and select between flat and mirrored placement. Flat means specify a target DN in Active Directory, and all objects get created in that one container. Mirror implies that the OU's object in your eDirectory tree would be mirrored over to the Active Directory side. The mirroring is somewhat clever, and takes a moment to think about. It begins in the source side as all OU objects deeper than the specified container to look from. An example makes this much more obvious. If the specified User container in eDirectory that is set in the configuration is com\MyOU\MyCompany, then the *Unmatched Source DN* token is used to get the path of the current user deeper than the specified path. Thus, a user in the container

com\MyOU\MyCompany\Users\North\Boston\Boylston\Geoffc would generate an *Unmatched Source DN* of Users\North\Boston\Boylston\Geoffc (which, as discussed earlier, is set in modern policies into an operation property named unmatched-src-dn, but this is NOT a feature of the *Unmatched Source DN* token). The Mirror placement mode would use this stuck on the end of the base DN provided in the configuration for the Active Directory side user container. Thus, while the initial example was to say it would be the geoffc part of CN=geoffc used to name the object in Active Directory, in fact, it would actually be whatever the naming attribute in eDirectory is set to. This matters as User objects in eDirectory support at least two naming

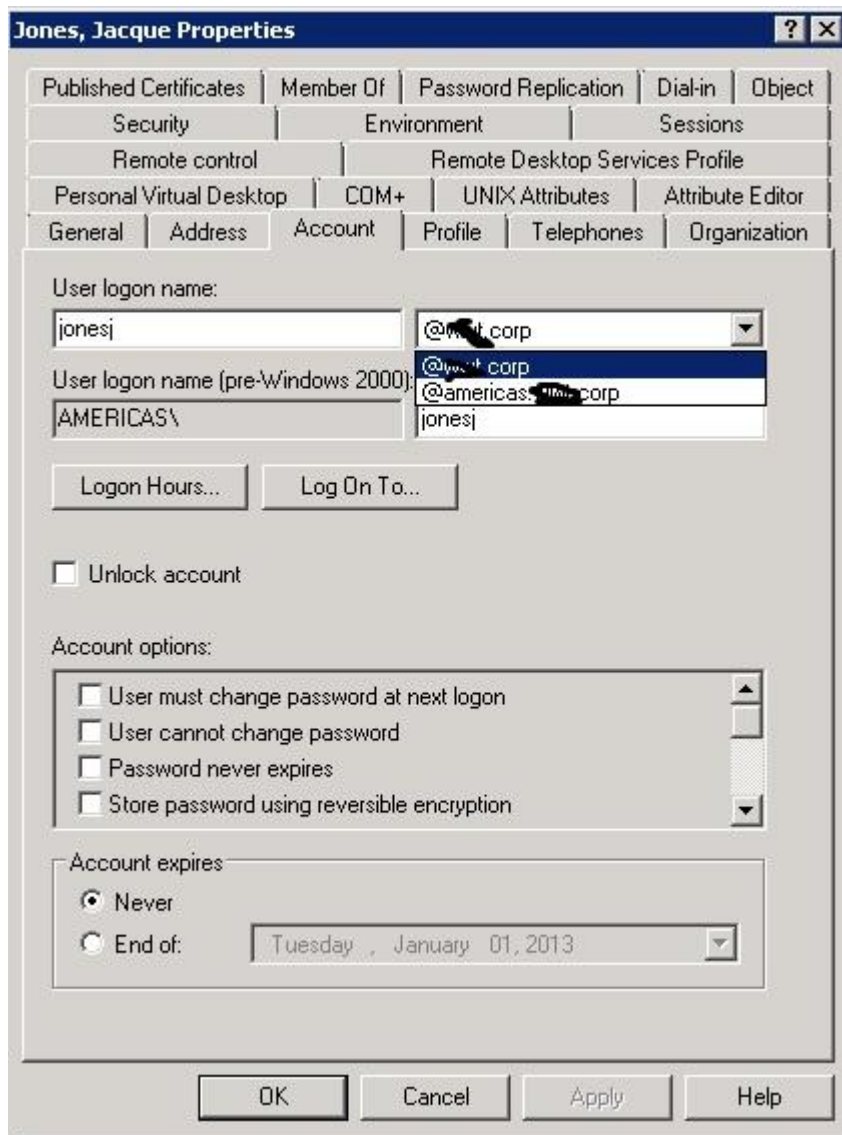
attributes, CN and UniqueID (uid in LDAP). Regardless of which is in use (or a third custom choice, which is allowed), it will work.

For Flat placement, it uses the *Source DN* token and its built in *Parse DN* like functionality to get the naming component of the DN.

UpnMap

This allows you a number of options to set the User Principal Name in the format of geoffc@domain.com, or to the users email address from eDirectory, or the email address from Active Directory. This is a little distinct from the previous two mapping modes. Whereas the other two were used for object naming in Active Directory (its CN attribute value), this is more focussed on a sort of parallel attribute, `userPrincipalName`, which can be based on the same attribute data or not.

The User Principal Name looks a lot like an email address being a name @ a domain-like value. However, it does not have to be a valid email address, nor does it appear to even have to be related to the current Active Directory domain name (At least in terms of the driver setting it. I imagine this would break all sorts of possible functionality in Active Directory if you started using random values.). The Active Directory Users and Computers MMC snapin only allows you to select from some preset values in its user interface; however, that is just a UI thing enforcing it. Making the change to the attribute directly via the driver or via an LDAP command will actually allow you to enter anything. It may not even require an @ sign.



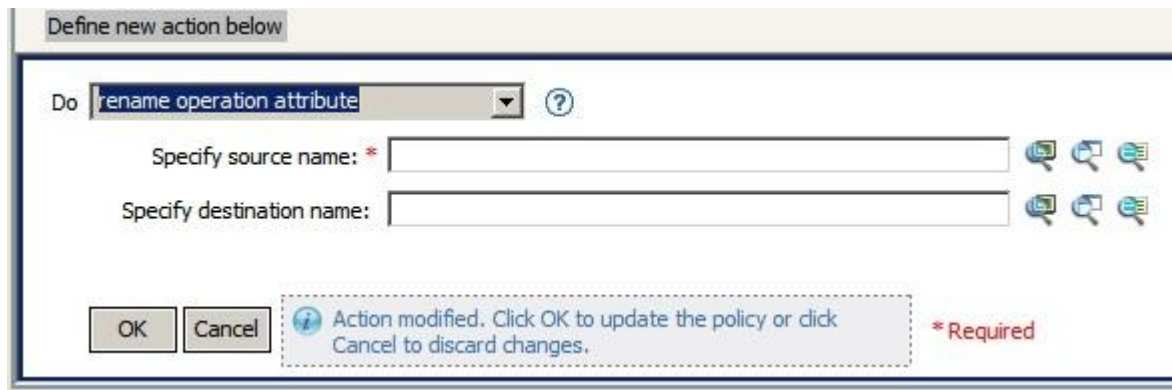
These are the sort of things you need to think about when you do handle a rename to Active Directory. The rename really is just the naming attribute, but depending on the target system, you might need to consider other issues.

In Domino, for example, when you rename an object, they usually leave the old name behind in the CN attribute. That grows cluttered as time goes on.

Databases rarely like renames since the naming attribute in the database could be the Primary Key in the table, and you should never really change that. It just depends on the system you are working with, so be careful!

Rename Operation Attribute

This is an interesting token. Using it, you can basically re-implement the Schema Map rule if it was needed. The only driver I ever really saw use this approach was in the GroupWise driver, which has a number of five-digit attribute names that changed a little between GroupWise revisions. Conditional mapping is not something the basic Schema Map can easily handle. Map 12345 to Full Name if the GroupWise system is 5.x, or map it to 12349 or something silly like that if it is GroupWise 8.x.



Define new action below

Do: rename operation attribute ?

Specify source name: *

Specify destination name:

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

If you import a GroupWise driver into Designer, you can see a Policy object linked into the Schema Map policy set that uses the GCV that specifies which GroupWise system is the target, and maps the attributes by *Rename Operation Attribute* tokens as needed.

Like *Clone Operation Attribute* discussed earlier, you could potentially use this to morph attributes. In the *Clone Operation Attribute* case, I would think about copying the *Equivalent to Me* attribute on a Group to Member or perhaps the other way around. But you might just want to rename it instead. I suppose it would depend on how you were using it.

As always the **Specify source name** and **Specify destination name** fields can take variables, which is nice. This means you could, in theory, loop through a List type GCV for attributes you need to rename, and perhaps *If Operation Attribute* `$current-node$` is available. Then you know you need to remap it. Since a Schema Map works well as a map, you might have a mapping table that holds the counterpart value. You could then *Map* the source value to get the destination attribute name and be able to **Specify source name** to `$current-node$` for the current instance of your loop. Your Mapped value as the **Specify destination name** field.

This basically re-implements most of the schema map rule. Of course, the schema map is probably the better way to go unless you have some conditionals you need to handle (like the GroupWise driver).

It is nice having the option available. If we did not have this token, you would have to *Clone By XPATH* perhaps the old value to the new value (or maybe just *Clone Operation Attribute*), and then *Strip By XPATH* or maybe *Strip Operation Attribute* to get rid of the old attribute name. It is much cleaner with a simple token that can do these steps all in one.

Send Email

Send Email From Template

IDM has at least two available mail sending code paths. There is this pair of tokens, and the other is in the User Application where there is a Workflow action to send email. Depending on how finicky your email system is, you might not be able to use these. (I know of clients where User App works, and the engine emailer does not. Often it is an issue that the mail server wants some form of authentication this token does not support.)

Send Email token:

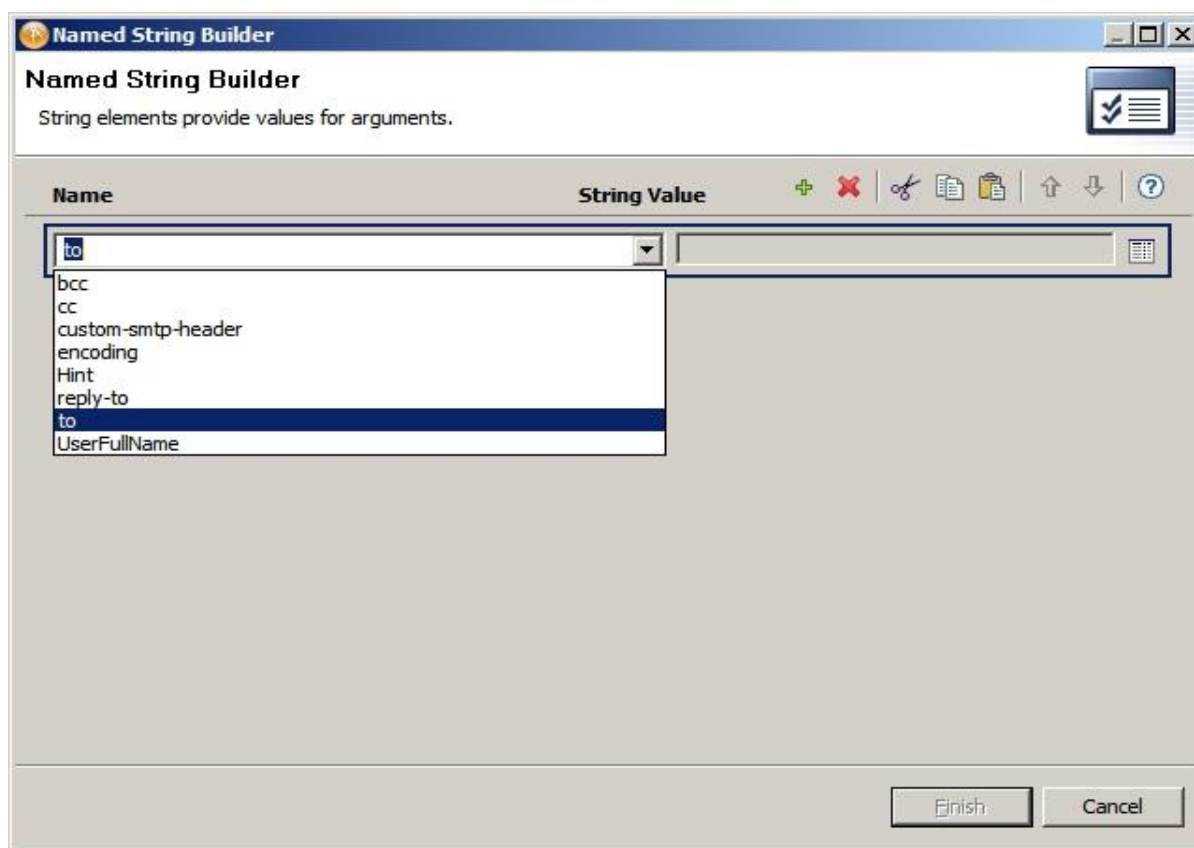
The screenshot shows a dialog box titled "Define new action below". The "Do" dropdown menu is set to "send email". Below this, there are five input fields: "Specify ID:" (empty), "Specify server: *" (empty), "Select message type:" (set to "text"), "Specify password:" (empty), and "Specify strings:" (empty). Each field has a search icon to its right. At the bottom left are "OK" and "Cancel" buttons. At the bottom right is a message box that says "Action modified. Click OK to update the policy or click Cancel to discard changes." and a red asterisk followed by the word "Required".

Send Email from Template:

The screenshot shows a dialog box titled "Define new action below". The "Do" dropdown menu is set to "send email from template". Below this, there are four input fields: "Specify notification DN: *" (set to "Security\Default Notification Collection"), "Specify template DN: *" (set to "Security\Default Notification Collection\Forgot Hint"), "Specify password:" (empty), and "Specify strings:" (empty). Each field has a search icon to its right. At the bottom left are "OK" and "Cancel" buttons. At the bottom right is a message box that says "Action modified. Click OK to update the policy or click Cancel to discard changes." and a red asterisk followed by the word "Required".

The core difference between the two ways to send email has to do with how you generate the body of the message. Both tokens expect a user ID (**Specify ID**), if you expect to connect, and password if your mail server needs it. You need to specify the server, which I always replace with a GCV on the driver set object because that always changes on me, and then one change of the GCV is all it takes to uniformly change the SMTP server settings in your drivers. You can choose (**Select message type**) to send an HTML or text email, which affects how the message body is formed. The address of the mail server is required to be provided (**Specify server**) for the *Send Email* token; whereas, the *Send Email from Template* gets the server address from the Collection object's attribute `notfSMTPEmailHost`.

There are strings available for you to specify out-of-the-box stuff: To:, Cc:, Bcc:, Message Body, reply-to, encoding, Subject, From, custom-smtp-header, and so on.



When you use the *Send Email from Template* version, you specify the Template object in the Security container you wish to use. You get a lot more flexibility with a template since you can build a nice, complex, pretty (or not) message. To customize it, you can set data fields that you can provide as Strings when you submit the token. For example, you might have the date in a string to be inserted (Perhaps the date + 90 days warning that the account has been inactive and will be deleted if there is no activity before that 90 day deadline.). The email template would have a `$expiryDate$` string in it, and then, when using the token, a string named `expiryDate` can be set to provide the value.

As you can see in the example screen shot above, the last string is named `UserFullName`, which was read from the Password Hint template selected. This is very useful as it becomes trivial to send very customized emails based on events and keep a consistent look and feel.

The SMTP server settings, in that case, are read from the Email Notification Collection object (which, by default, in every tree is the object `Default Notification Collection.Security`) that has the SMTP server and user information specified already. There is no need to repeat in the token. You just need to provide the password (**Specify password** which has the full set of String builder tokens like *Named Password*), which Template you wish to use (**Specify template DN**), and any strings it might need. You get all the usual preset strings from the *Send Email* token, plus any you define in your email template object.

This can let you do all sorts of neat things in email. Specifically, since you do not have to build a complicated document as a long string via Argument Builder, rather, you can have a predefined Email template with fields you can provide data to fill in. You can generate much classier emails. Compared to the string approach, in an Email template, it is much easier to make sure it looks good and appropriate. I find it reasonably tricky to ensure I get the strings just perfect. Usually I forget a leading space or trailing space in *Text* tokens surrounding a local variable, for example. In a template, it is much easier to see how it is all going to look, and make sure your inserted data is in roughly the right place. You can use any HTML editor, and paste the results into your token. Just try and keep it reasonably simple.

Additionally, the Email Template objects are actually used in a couple of different places. For example, emails sent from the User Application via a Provision Request Definition (PRD) workflow use the same Email Template objects. Warning messages sent by IDM about bad password changes use these templates (which, to be fair, is using the *Send Email from Template* token in the Input Transform rule).

Alas, I have yet to figure out exactly how you are supposed to send attachments. There has been an answer in the forum, and it was detailed enough. It requires that the attachment be in a specific file system location, and not everyone was able to get it to work. There is an enhancement request in asking for this functionality, so please go in and add your own such request. They use the enhancement system (www.novell.com/rms) to try to decide how much interest there is in a feature to prioritize development resources. Multiple requests can indicate levels of desire for a particular feature.

The *Start Workflow* token is no better at attachments. Your best bet is to reference them in an HTML email with a link, which is an interesting thing to bring up.

With IDM 3.6, support in the *Send Email* token was added for HTML content. If you want to pass HTML into this token, the forums revealed that you need to enclose it in the tags `use-html` like:

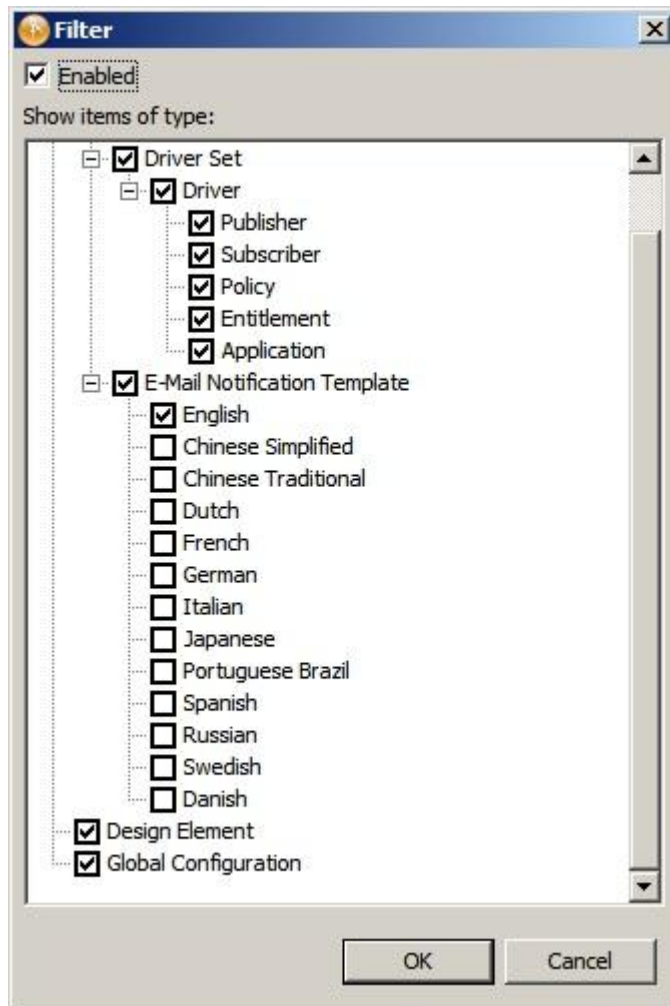
```
<use-html><title>Cool Title Content</title></use-html>
```

It also needs to be well formed HTML, that is, more like XML rules so all nodes have to be closed. For example, if you want to use `<br>` or `<p>` tags, you would need to make sure to close them. Like `<p>Some Text</p>` or `<br/>` rather than the way HTML lets you leave them unclosed and not quite properly valid per XML rules. The engine is really uptight about your HTML being well formed, much more so than any web browser. Basically, it treats them like XML and follows similar validation rules.

If you want your entire email message to be in HTML, you can define that in the Template, but there is an option of Type for the message in a simple *Send Email* token call. Your choices are text and HTML.

You can actually localize the templates and have an English, French, Italian, German, Japanese, etc. versions just by naming a primary Email Template. Then making copies in different languages, ending the object name with `_XX`, where XX is the two letter language code usually used. You can see this in eDirectory or Designer where you might already have all the localized templates. In Designer, if you right click the Default Notification Collection item, and select Add All Templates, you will get all the available templates your version of Designer knows of and in all the different languages. You may NOT wish to do this in a real system unless you intend to use them. This is not because it slows down eDirectory, which scales nicely to billions of objects, but because it slows down Designer as the number of objects in the project is related to the performance of Designer.

If you are not seeing the localized versions of the email templates, consider right clicking in that area and select Edit Filter. This pops up the following box:



This shows an example where only the English language templates are being shown. But you can enable or hide the other languages as needed.

Set Default Attribute Value

This is a truly uncommon token. Honestly, I had not even known about it until recently when I was working through a driver configuration writing a Walk Through guide and ran into it. You can find the driver walkthrough series of articles at this link:

http://wiki.novell.com/index.php/Detailed_driver_walk_through_collection

Basically, this is only valid on an Add event. If you use it, it will add the value you specify to the event document if it is missing. So using it can save you an *If Operation Attribute* available or equal to something test. Then, if the operation attribute is not available, it gets added to the event document. However, if this is a modify event, this token does not do anything, which is

really annoying. Thus, it is best used in the Match, Create, or Placement rules rather than in the Event or Command Transformation policy sets. If you wish to use it in a modify event, make sure to wrap it in a test for the correct event type (add only).

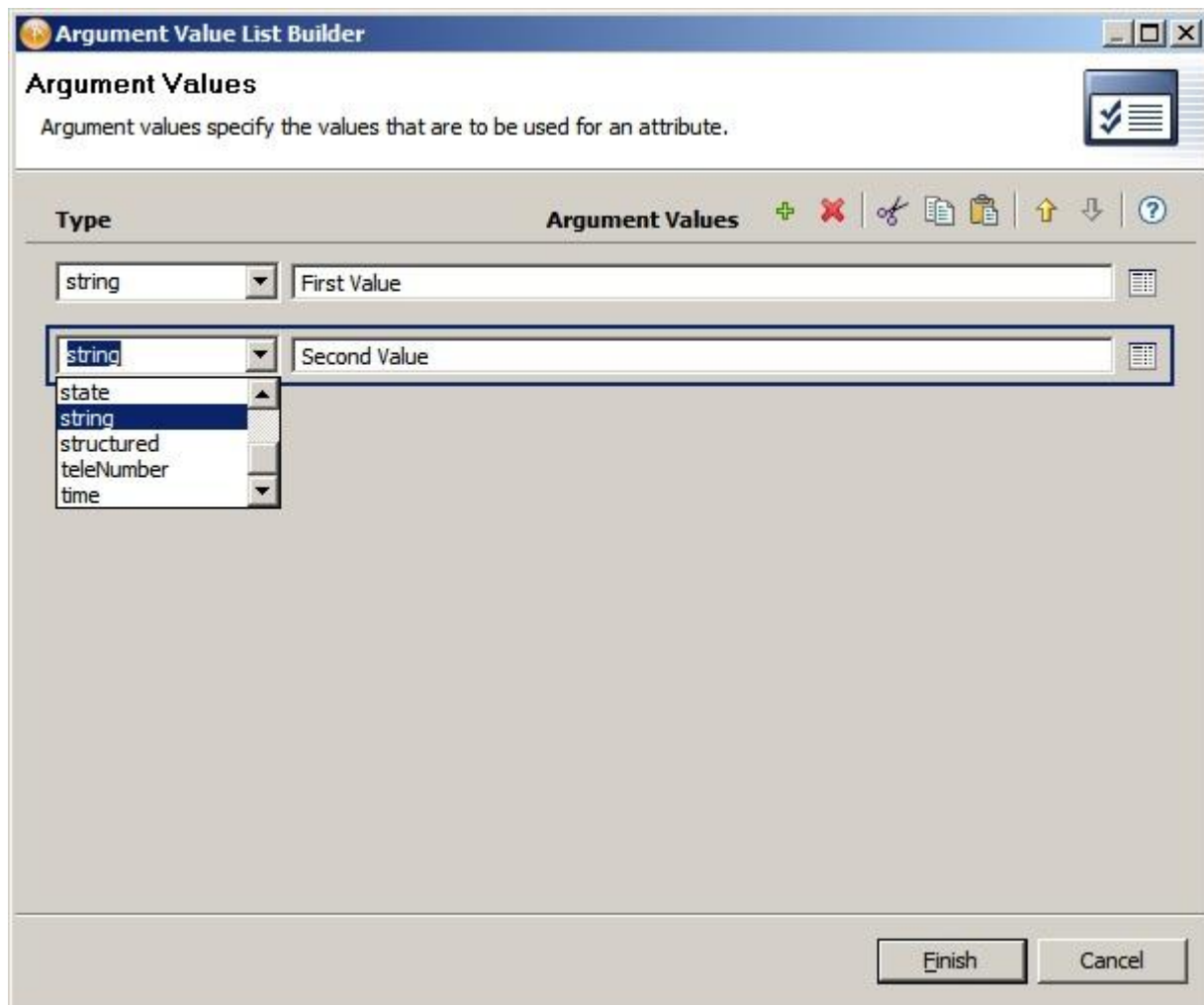
The screenshot shows a dialog box titled "Define new action below". Inside, there is a "Do" dropdown menu with "set default attribute value" selected. Below this are three fields: "Specify attribute name:" with an empty text box, "Write back:" with a dropdown menu set to "false", and "Specify argument values:" with an empty text box. At the bottom left are "OK" and "Cancel" buttons. At the bottom right is a message box that says "Action modified. Click OK to update the policy or click Cancel to discard changes." and a red asterisk followed by the word "Required".

There is a neat extra twist, called **Write back**, which interestingly defaults to false. If set to true, it will also write back, onto the source object, the value you are adding to the destination object being added.

This way, in one token, you test if the attribute is present. If not, you add it to the destination. It already knows it is not present on the source object (or else it would be in the add event already) and will add the attribute back to the source object if needed. This is a clever bundling of functionality for a reasonably common event. I personally have never used it. The place where I found it left the Write back to false, so it basically was the same as a *Set Destination Attribute* token call. When I asked the driver developer, he said he forgot to change it back. There is no real difference so it is not something worth wasting much time on clearing up.

A good place to use this would be in the Active Directory driver regarding how the DirXML-ADAliasName attribute is added in a rule in the Create policy set. As a side note: this attribute at one time mapped to sAMAccountName and later was changed to map to userPrincipalName. It might have made sense to use this token with Write back set to true to save some work in the driver since on a user create it needs to be written in both directions.

What is also kind of neat about this is how it handles the possibility that you might want to set multiple values as your default. You get the String Builder interface, when you click on the editor button for the **Specify argument values**, where you can specify one value or as many as you need just like you would in a *Send Email* or *Start Workflow* token.



In the above example, the values were simple strings. Of course, they could have been variables, GCVs, attribute data read from the source or destination, or anything the String builder allows.

By adding more than one value line (little plus sign upper right), you can add more than value as the default values, and the type of attribute value can be the usual set that eDirectory schema supports.

Set Destination Password

Set Source Password

This token will generate a `<modify-password>` event. This matters as you need to consider where you use these tokens. If you use it after the Password transformation rules in the Command Transform policy set, you may not get the results you expect. That is because the way IDM

does passwords is slightly non-intuitive, but it makes perfect sense once explained. You can read about how the Password transform rules work in these two articles:

<http://www.novell.com/communities/node/1474/password-transformation-rule-sets>

<http://www.novell.com/communities/node/9661/password-transformation-rules-publisher-channel>

Define new action below

Do **set destination password** ?

Specify class name:

Select mode: **add to current operation**

Select object: **Current object**

Specify old password:

Specify new password: *

OK Cancel

Action modified. Click OK to update the policy or click Cancel to discard changes.

* Required

The `<modify-password>` event is usually what is sent by the engine into the shim. This is meant to call any special application APIs needed to change a password, recognizing that the password is usually a special case. The shim will also generate a `<modify-password>` event if it is able to detect password changes to send down the Publisher channel. However, what is written to eDirectory is likely different. A `<modify-password>` making it into eDirectory will change the RSA key pairs (the NDS password as it were), which is usually not what is desired. The GCVs in the Password Synchronization page (for Designer, right click on the driver line, and it is called Server Variables in iManager) control how this event should be processed, whether to allow this `<modify-password>` event to pass unchanged or to convert it to a `<modify>` of `nspmDistributionPassword`. The NMAS processes, following the password policy, will then synchronize the password inside eDirectory to the Universal Password (which is really the `nspmPassword` attribute). It does seem somewhat convoluted (and it is, but for a reason), but it works.

Thus, if you are writing a policy in the Publisher channel, and after the Password policy rules in the Command Transform have fired, and you execute this *Set Destination Password* token, you will probably end up resetting the NDS password (via a `<modify-password>` event) and not the Universal Password that you likely desired.

Conversely, using *Set Source Password* may skip the Command Transform rules, and you should be careful where you use it.

Interestingly, while this is a set action, you can optionally provide the old password value, which the docs note is an IDM 3.6 and higher function. Specifically, the Notes driver requires this. To change a Notes password, you need the old one to decrypt the private key stored in the ID file, re-encrypt it with the new password, and return the file. This leads to the bizarre situation in Lotus Notes. You can have four copies of the Notes ID file. Each copy encrypted with a different password, and each of the four will let you login as long as you provide the correct matching password.

If you are generating passwords, do remember the *Generate Password* noun token in the Argument Builder that was discussed earlier, which can be pointed at a Password Policy and generate a random value that meets the policies rules. This is great as writing your own password generator can be fun but not worth the effort. You would have to customize it for each set of password rules. In the *Generate Password* token, you have a nice GUI for defining the password rules (Password Policy editor in iManager), and then a simple token to call them in IDM Policy.

Set Operation Association

This is a useful token, and I probably should have discussed it together with the *Add Association* and *Remove Association* tokens earlier. Those tokens generate an `<add-association>` or `<remove-association>` event for the selected user.

This token modifies the `<association>` node of the current event. This is useful if you are manufacturing your own Association value. Now, usually the shim provides the association value based on some rule and unique value in the connected system. For example, ObjectGUID in Active Directory, UNID (Universal ID) in Domino, Primary Key in JDBC, GUID in eDirectory, and so on are used as the unique value in those systems.

However, some shims leave this up to you to provide the association value in policy. A good example would be the Loopback driver. There is literally nothing in the Loopback shim, since all it does is send back the event it received. However, you might want to keep associations on objects, so you can modify them, instead of going through a synthetic add every event, which takes time.

To do this, you need to decide what makes sense as an association value, and set it using this token.

Now, you do not have to define the state or the driver it refers to since both are implicit when using it. Obviously, the driver is the current one, and the state will be 1 for associated. Now, since `DirXML-Associations` is just an attribute, if you wanted to set the value on another object, you certainly could manipulate them with *Set Destination Attribute* or *Set Source Attribute* tokens, though, you should think long and hard about why you are doing that first.

One valid example would be the case of a password only driver. A problem that large IDM solutions will come up with is that password change events have no priority in the queue. If you have a backlog of events being processed (say, HR just updated a ton of stuff), and someone does a password change, it could be delayed until its turn in the queue is reached. Usually, this is not an issue until you reach a point where the backlog of events happen often enough to notice. Something worth considering is adding a second driver to your connected system, and configuring it to ONLY do password synchronization, no creates, no modifies, just passwords. Do remember to block deletes as well or else amusing things can happen.

If you do this, you have an issue of matching again. While you could go through the process of migrating all your users to get association values on them, it is potentially easier to LDIF out all of their associations (or use a tool like DAModifier), edit the driver name, and just add them back in. That is, copy the association values from one driver to another.

Going forward, you can add some code in the Input Transform right around where the `<add-association>` is detected for handling either group entitlements or else adding `DirXML-ADContext` in the AD driver case. When it sees an `<add-association>` event, it also writes a copy of that value to the `DirXML-Associations` attribute. This would entail using the *Add Destination Attribute Value* token, specifying the value type is Structured. Then the three components needed are `nameSpace` set to 1, `volume` set to the DN (Slash format) of the second password only driver, and `path` set to the value being added. (XPath of `.` will treat the event as a string, and since the only string value is the association value, that will work.) This is because `<add-association>` event looks a little bit funny. A simple example looks like this:

```
<nds dtdversion="4.0" ndsversion="8.x">
  <source>
    <product version="???.?">DirXML</product>
    <contact>Novell, Inc.</contact>
  </source>
  <input>
    <add-association dest-dn="acme\users\geoffc">SomeAssoc Value</add-association>
    <add/>
  </input>
</nds>
```

Thus, if the current event is treated as a string, the only string in there is the value of the association. If treated as a node-set, then the XPath of `./text()` would suffice.

Set Operation Class Name

This is a token you would think is not needed. After all, we have the schema map to do this for us. Well, like the *Rename Operational Attribute* token talked about earlier, this is the other piece needed to do Schema Mapping by 'hand' in Policy vs doing it in the Schema Map rule.

As before, if you have some conditional tests needed to decide how to map your attributes, then that is more than the simple Schema Map can handle, so you might as well use these tokens to do it.

This is a fun way to morph the object class on the fly. Do be aware that mandatory attributes and schema compliance are your problems to worry about. The engine will not offer you any short cuts here. If you break schema, you bought it.

One example might be if you have Users from AD coming in, but you need them to be Contact class objects so you can sync them to GroupWise as External Users. In that case, you probably cannot map all Users to contacts, as most of the user objects in your AD are probably users. However, there might be some attribute or container where these Users in Active Directory should be Contacts when they sync into eDirectory. In that case, you can use this token once you know the class needed. Of course, this is really just another conditional schema mapping example.

One older use case was for use in the *Unique Name* noun token. Earlier versions of IDM did not have the ability to check all object classes for names, only the current case. It was possible to have perhaps Contacts and Users with a shared namespace. If you, then, used *Unique Name* on an event for either, it would only check for uniqueness amongst the current object class. Thus, you could get the suggestion of a name value from the token as, say, a User; then *Set Operational Object Class* to be a Contact; test for *Unique Name* again; then when finally done, *Set Operational Object Class* back to User. Thankfully, this was all fixed in later IDM versions, where there is an option to check across all object classes. See the earlier section on the *Unique Name* noun token for more information.

Set Operation Destination DN

This token is most famously used in the Placement policies. In fact, for an `<add>` event, if you do not generate a destination DN, the engine will actually end the event with a message that no destination DN was generated by placement. It will not process it further. After all, you cannot add something somewhere without specifying where. (You can't have everything; where would you put it? Same idea.)

What is neat about this token is how it works in a similar fashion to the *Find Matching Object* token. When you chain a series of *Find Matching Object* tokens one after another, you want it to stop after the first match, not wasting any time doing additional queries. Once the destination DN is set by a *Find Matching Object* token actually matching an object, the next *Find Matching Object* tokens do not do any queries or work. You can, thus, use this token to short circuit the later *Find Matching Object* calls if you needed to. Now, it is an interesting question whether using *Set Operational Destination DN* to short circuit the Matching policy set would cause a merge event as well or whether it is something special about a *Find Matching Object* token succeeding.

Otherwise, in the Placement policy you use this to specify the value for the destination DN you have calculated.

If you wanted to, you could do this with a *Set XML Attribute* token with the destination XPATH of `.` (just a period, i.e., the current context, which is the event node, the `<add>` or `<modify>` node), and the name `dest-dn` with the value you want.

If you wanted to redo a Match, say, to let another *Find Matching Object* run or remove the value for some reason, a similar *Strip by XPATH* of `@dest-dn` would suffice as long as it was done before the end of the Matching policy set.

Set Operation Property

Operation properties were discussed in the Noun tokens, in the conditions, and now here is where we get to set a value into it. These are stored as XML attributes of the `<operation-data>` node. They follow the event until it hits the engine or the shim, whereupon they are removed for submission and returned onto the `<status>` event that comes back.

I often use these in the Subscriber Command policies to set a couple of operation properties. I would set an operation property named `op` to the current operation with a value of, say, add or modify, and then add `src-dn` with the current objects Source DN. I usually will set the `dest-dn` if I know it.

This way when I get back a `<status>` document with a level of success or error in the Input Transform, I can look at the event, and if I need to, do something with it (perhaps email, using the *Send Email* token, or perhaps *Generate Event* for our Audit system); and at least I know that it was an add for the user `com\acme\Users\geoffc` from the operation properties.

The reason this matters is that the `<status>` document does not have XML attributes or other information to tell you what the object with the error was in the source or the destination (Unless the error message itself includes that information, but it usually does not.). Thus, you get an error but no real idea of which object the error is about. You would need to look in the trace file, find the error, and search backwards until you find the previous event. Of course, by the time you try to have a rule process this event, it is too late.

This might be a case where in Active Directory you sent in an `<add-value>` for a single valued attribute that already had a value, and there was no `<remove-all-values>` going with it. (You did an *Add Destination Attribute* instead of *Set Destination Attribute*.) That will return an Active Directory error that tells you of the schema violation, often with the attribute name (It depends on the error if you get that back or not.); but it will not tell you which object it was. By adding these operation properties, you can store that info and get it back on the `<status>` documents.

The Noun token, *Operation Attribute*, is great for reading the current value of the operation property as you might in a *Set Local Variable* token, and you would use the *Operation Attribute* Noun token in Argument Builder for the **Specify Value** line. The *If Operation Attribute* condition token would allow you to use it as a test.

Of course, like most things in IDM 3.5 and higher, you can use a variable for the name of the Operation Property you would like to set. The value can be set with all the tools available in Argument Builder, with all the nouns and verbs discussed in the first bunch of articles in this series.

Set Operation Source DN

This is a token I did not know existed, which made this effort all the more worthwhile. It is the counterpart to *Set Operation Destination DN*.

You could, of course, replicate this with a *Set XML Attribute* token call, since source DN (actually `src-dn`) is just a simple XML attribute. But I have never really had a use for this one.

As always you have the usual tools to build or calculate the source DN in Argument Builder.

Set Operation Template DN

When creating an object, it is possible to use a Template object. These are an eDirectory object that used to be used in NWAdmin or Console One when you created a new user, and you wanted them to have a bunch of attributes filled out by default. When writing to eDirectory, you can still do the same thing in a driver.

I have not really used this token, so I do not know (and the docs do not say) if you need to specify a Template object or if a User object, or really any object, will suffice. In fact it does not say if a Template object will even work. A Template object can have file system and eDirectory permissions assigned, which would be very useful for creating new user objects.

I think this ties in with a setting in the Filter, on the Object Class, called Track Member of Template. When you used a Template object in NWAdmin days (Wow! That is a long time ago now. How long ago did Console One come out?), it would add an attribute `Member` onto the template with the DN of the user.

I never did examine what you could do with that. I suppose it would have been cool to modify the template and apply to all members, but I do not really think it worked that way. Of course, with IDM it would be easy to implement that if you are tracking Members of the Template. When a change is made to the template in an attribute in the filter, you would just loop through the `Member` attribute and apply the change to each object in the list.

The next few tokens were all discussed earlier with their related tokens: *Set Source Attribute Value*, *Set Source Password*, *Set SSO Credential*, *Set SSO Passphrase*, *Set XML Attribute*, and *Start Workflow* .

Status

This token lets you generate a `<status>` event. For example, you might wish to end your event with a veto, or you might like to send a status event. Now, you get to control the level of the status event where you can set values of:

- error
- fatal
- retry
- success
- warning

This can be quite helpful. For example, if you need to stop a driver in policy, then you can detect the event that made you want to stop the driver, then generate a `<status level='fatal'>`, and then the driver will stop itself. If you wanted to start a driver in policy, it is possible, but only from another driver. You can make an XPATH call to a Java function from the *dxcmd* tools Java classes or use the `java.Runtime execute` function to run it as if it were a command line utility; however, these are both pretty contrived cases.

It is possible you need to retry, something that can be common in a SOAP driver, where the web service may or may not respond in time; you might want to issue a Retry status. This waits a specified amount of time, and the event gets resubmitted from the beginning. You control the amount of time in an Engine Control Value (ECV) that is stored as an attribute of the driver object, named `dirxml.engine.retry-interval`, and it is a count in seconds. Usually this defaults to 30 seconds.

An additional use case might be when moving an object in policy, you may have to wait for the move to complete. Thus, if you can detect the move case and know that you need to do something after the move, you can generate this retry to buy yourself a pause for the move to complete. Alas, this does hold up the entire queue events behind it. It's probably not the greatest approach, but it is useful at times.

Error, Warning, and Success are really just informational messages, and you can use them to send a message back to the driver. As discussed in the *Set Operation Attribute* token, these are the sorts of events where setting an operation property with the event type, the source DN, or other information can enrich and make more useful to provide responses. That is, by the time you get the status event back, the original events information is not available. Thus you should append the information of interest as operation properties while you still have it handy.

As usual the string value can be whatever you like. You can provide as much or as little information as you see fit. Use Argument Builder, and you can generate whatever you need in a string to send along.

Strip Operation Attribute

This is useful for removing an operation attribute from an event document. This is a little more specialized than a *Strip By XPATH* token, since an operation attribute in an `<add>` event looks different than in a `<modify>` event. It's still different from a query response in an `<instance>` document.

An Add would look something like:

```
<add src-dn='o\ou\cn'>
  <add-attr attr-name='Something'>
    <value>Some Value</value>
  </add-attr>
</add>
```

A modify would look more like:

```
<modify src-dn='o\ou\cn'>
  <modify-attr attr-name='Something'>
    <remove-all-values>
    <remove-value>
      <value>Some Value</value>
    </remove-value>
    <add-value>
      <value>Some Other Value</value>
    </add-value>
  </modify-attr>
</modify>
```

Whereas, if you were to query and get a response in an Instance document, it might look like:

```
<instance src-dn='o\ou\cn'>
  <attr attr-name='Something'>
    <value>Some Value</value>
  </attr>
</instance>
```

Thus, to do this in a *Strip By XPATH* you would have to keep all these cases in mind and ensure you handle them all. But with the *Strip Operation Attribute*, you do not have to worry about it. Similar to the *Reformat Operation Attribute* handling all three cases as well.

Conversely, with *Strip By XPATH*, if you wanted to, you could strip only the modify case and not the instance case or whatever you needed. If you would want to strip out the `remove-all-values`

node only, you could do that. The *Strip By XPATH* token has a lot more control, limited only by your XPATH-fu skills.

Strip By XPATH

Quite a bit of discussion above in the *Strip Operation Attribute* is relevant to this token, but it is more powerful than just for removing an operation attribute. You can easily remove XML attributes, like `src-dn` with `@src-dn` and so on.

Even more interesting, you can manipulate XML in a node-set variable. So you could have a node-set variable with some SOAP XML in it, and you could strip out specific nodes, perhaps with an XPATH of something like:

```
$XML-DATA/SOAPENV/SOAPBODY/createResource/entityId
```

This would remove the `entityId` nodes in the variable under the `SOAPENV`, `SOAPBODY`, and `createResource` nodes.

Recently a colleague noted an XPATH function I was totally unaware of that is actually rather useful in the SOAP example. The `comment()` function will select a comment, and with *Strip by XPATH* you could actually do something like `$XML-DATA//comment()` to remove any comments in the variable `XML-DATA`. This is nice since the WSDL import for a web service usually is chock full of comments, and being able to easily remove them is quite handy. (They both help during development and hinder during debugging in terms of reading the event.)

You can see more about tricks with node-sets in XPATH in this article:

<http://www.novell.com/communities/node/5686/cool-tricks-using-xpath-nodesets>

I was discussing an issue I had. I needed to process the ACL attributes for thousands of users and find the ones that mattered to me. The size of the data set was pretty big, and my first approach was basically looping through it. However, using a simple *Strip By XPATH* approach on a node-set variable holding the data, I was able to very quickly clean up and remove thousands of entries in a single action. This made the rest of the process of looping over the remaining values much faster and freed up lots of memory.

```
$ACLS//value[(component[@name = 'protectedName'] = 'WM:Registered Workstation')]"
```

My variable `ACLS` in this case had a query result for the ACL attribute of all users. Since there had been ZenWorks 3.x in the system, there were a bunch of permissions allowing writes to the `WM:Registered Workstation` attribute, so I needed those excluded. Also there was the annoying ACL in the User object template (This is in schema and mildly painful to change.) that grants

everybody write rights to their Print Job Configuration and Login Script attributes. (P.S.: It is worth considering fixing this in schema since it adds two ACL entries to every single object, making the database bigger.)

I was looking for permissions to report on, but I did not care about these. I could have looped through all with permissions of interest and then tested for these three cases, discarding them; but with just one or three *Strip By XPATH Expression* statements, I was able to get rid of the undesirable ones before I began working with the data. This approach is a very powerful and efficient tool. In the example in that article, my initial brute force approach took over 40 minutes. After using the strip approach and some other tricks, I got it down to 2 minutes to process tens of thousands of users.

Trace Message

This token lets you trace something to the DStrace file. If you are not familiar with DStrace and are working on IDM, stop right now. Read the following articles:

<http://www.novell.com/communities/node/5681/capturing-and-reading-novell-identity-manager-traces>

<http://www.novell.com/communities/node/9677/comprehending-idm-traces-part-1>

<http://www.novell.com/communities/node/11166/comprehending-idm-traces-part-2>

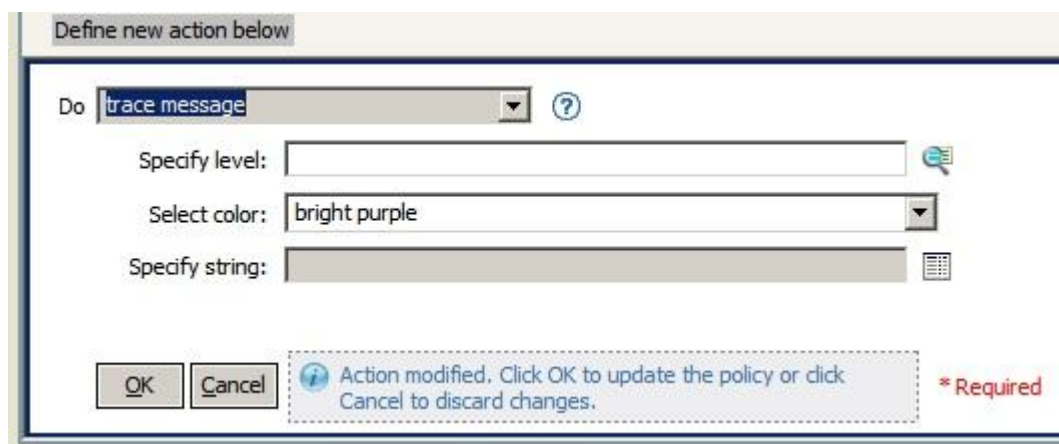
These were written by Fernando Freitas of Novell Support, and he has done one of the best jobs of explaining how to read the DStrace output from IDM that I know of. I took a swing at it, and Fernando's is so much better. I no longer talk about mine.

Now, you should be configured in your driver to output your trace to a file (as opposed to trying to follow it in iManager's DStrace facility, or the command line *ndstrace*) so you can use tools like 'less' on Linux to find stuff and read it. I have a funny 'less' story. I was working on a system with 1.5 million objects and after a weekend roll out of a new driver, I was asked to look at a particular error that occurred. They had a 36 GB log file on a Linux server and said around line 550 million this error occurred. I 'less'ed the file and did an *r* to jump forward 550,000,000 and it jumped me to line 550,000,000 or so, and I was able to use the search tools (/ for forward search and ? for backwards search). But it all worked, (I admit the jump to line 550 million took about 5 minutes, but ,hey, it worked!) and I was able to find the error, identify the cause, and resolve it. Good luck touching a file that size on most other platforms.

One of the other benefits of tracing to a file is that the *ndstrace* command line tool and the iMonitor web based tool, if you enable the DXML and DVRS trace tags, will show all the drivers trace interleaved; and it is almost impossible to read. It shows all those with trace enabled. However, if in each driver configuration you specify a path to a log file (Specific to the platform, the engine is running on /var/log/novell/driver.log for Unix variants, c:\novell\nds\logs\driver.log on a Windows server, and sys:\logs\driver.log on a Netware host.), then just that driver's output is sent to the file. You can have 11 drivers in a system writing to 11 different files and not have to worry about interleaving of the output.

The system traces lots of stuff and at different levels. NTS needs level 3 most of the time unless they are trying to stall you and say, while Level 3 is good, really for this issue we need level 5; and then you do it and see no difference at all.

For the engine level trace, a setting of 3 is usually all that is needed. At the shim level, in a remote loader or local, each shim is different; and for some, level 10 or 25 can be needed. The JDBC driver, for example, in triggerless mode queries the database every polling interval for the timestamps of all the data in the table. Then it reads through them to see which has changed lately. Well, if you turn on level 25, in trace you get shown every item it considers and discards. It turns out, in a 30-second polling interval on a large enough database, it cannot trace out all those decisions before the next polling cycle begins. It is a disaster and barely works in that mode. Of course, running in that mode is insane and only done to try to capture some wonky error.



So when you look at this token, you will see a line (**Specify level**) where you can specify which trace level (and higher) to output your message to. I usually leave it blank, and it shows up in the current trace in Simulator. If I really want it to show up, I set it to level 3, since that is where I always go with my drivers for trace.

The **Select color** line takes a bit of explaining. If you use the NDS iMonitor trace facility that is through the web based iMonitor and its Dstrace option, with the DVRS and DXML tags enabled, then you get a coloured version of Dstrace. The *ndstrace* command line on Linux or Unix and the tracing to a file (which I just recommended for several reasons) do not show the colourized version, alas.

Thus, I usually leave the default colour, since I never use the iMonitor view, where colour would help or matter. Having said that, it does make reading trace much easier with colour on. Alas, it is not worth it to me, after enough time getting experienced at reading trace, considering the trade offs with files. Besides I am a bit colour blind, and the red and green are basically indistinguishable in iMonitor to my eyes.

This token is useful, since at times you may need to see the value you received. Perhaps you did some calculations in a variable and want to see that the result is correct. The calculations, if done in XPATH, will not show the result in the trace by itself. Probably the most common case I use it for is when I am working with a node-set, since the way trace renders a node-set is not the way I would like. I would like to be shown the whole thing, but trace sort of shows the parent node. If the node-set is a bunch of siblings, then it shows just the list of siblings; for example, here a *For Each* is looped over a node-set variable with four instance nodes inside it:

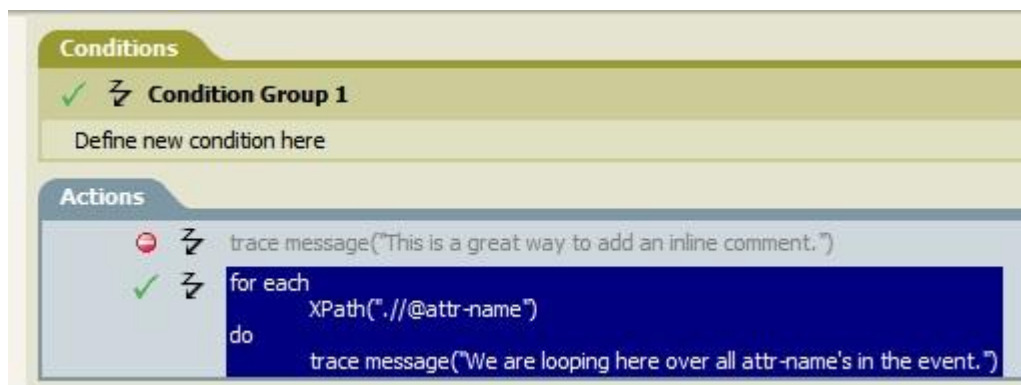
```
Action: do-for-each(arg-node-set(token-local-variable("BIG-GCV"))).
  arg-node-set(token-local-variable("BIG-GCV"))
    token-local-variable("BIG-GCV")
    Token Value: {<instance>,<instance>,<instance>,<instance>,<instance>}.
```

You can see the last line shows how a node-set is rendered and why it would be useful to see what it really contains in more detail.

Thus, I very commonly use trace with *XML Serialize* when building node-sets in variables for use (perhaps in a SOAP driver, or maybe to build an XML payload to send to an application, or maybe to build the path.xml component of a DirXML-EntitlementRef attribute) where I need to see my results, step by step.

In that case, a trace with a **Specify string** using *XML Serialize* applied to my local variable does the trick nicely; and I get to see pretty XML in the trace, look at it, and try to figure out where I screwed up.

One of the most clever uses for this token that I had never considered until someone submitted it in the Excellence in DirXML Script coding contest from a year or two ago was to use the *Trace* token for inline comments on code. That is, write your comments in the *Trace* token and then disable it. This way it never even wastes CPU time, but you now have an inline comment!



I am a huge fan of good comments on policies and have on occasion had so much commentary on a rule that it found all sorts of amusing Designer bugs in the UI. One thing I really like to do now is when I have a complicated rule that I expect I might ever need to tweak in the future, I

snag a copy of the input event that triggers it and paste it into the Comments field of the rule. Sometimes I will even put in what the output should look like. This makes returning a month (or a year, or even a DAY!) later and being able to work right away possible, since I do not have to try to remember (or figure out again) what sort of event should really trigger this rule. Even more so when the event changes, I have something to compare in trace so I can understand how it changed to better update the rule.

With inline comments, using a disabled *Trace* token, I can more accurately pinpoint my comments, since I can leave the token right next to where the comment is of interest. I have quite enjoyed this approach since I learned of it.

Veto

I contemplated discussing this token next to *Break*, but at some levels it is sufficiently different to deserve a section of its own.

A *Veto* will stop the current event and stop processing it. The `<status>` will probably (depending on where the *Veto* occurs) mention that the operation was Vetoed, possibly by the Policy Set it was vetoed in (like Create, Placement, Matching, etc.).

There is a subtle twist in how this affects other events you might have sent during the flow. If you had a single `<modify>` event come into a single Policy object, then the first rule does a *Set Source Attribute* and then a *Veto*. That write to the source will not be blocked. Only the current event will be stopped. I am not sure if you could catch and process that write back to source event at this point anyway.

Now, imagine the same basic case, but this time your first rule did a *Add Destination Attribute* with the mode set to add to current event. It probably became part of the first `<modify>` event and will be vetoed.

One more time through the same basic case, but now your *Add Destination Attribute* is set to either add before or add after the current event. I am pretty sure that in this case the *Veto* will block it. However, if in a second Policy object if you had a rule that detected the first event and vetoed it, the other event your *Add Destination Attribute* generated would survive. This is because each event in the event document (the `<modify>` or `<add>` or `<instance>` or even `<status>` event) is run through the Policy object, one at a time. You may see in trace something like:

Applying rule to Status #1

Then you might see all the rules in the Policy set processed. It seems to repeat, but this time you see a line like:

The repeat occurs in the case where there were two `<status>` documents in the current event. This is how multiple events in a single document are represented in trace as they are processed sequentially.

Now, if your first Policy object generated a second event (which the mode of when = before or when = after will do), each will get processed in all the following Policy objects one at a time. However, during the first Policy object where these extra events are generated, they are not yet full-fledged, independent events, and the entire Policy objects set of rules will not be applied to them; thus, the *Veto* in that first Policy object will still cancel them all. But a *Veto* in the second Policy object will only apply to whichever of the events is being run through the rules. You could work around this by having the first rule detect the conditions, add the events, and in a second Policy object, reuse the same conditions to detect the case, and then *Veto* it. This, of course, assumes you can discern the first event from the secondary events you just generated in your policy.

In your first Policy object, if you had set the mode to Write Direct to Datasource then the *Veto* will not apply at all, and I think you could only touch it in the Output Transform policy set.

That is why it is a bit trickier to explain *Veto* than it should seem at first. The whole notion of how multiple events in a single document are processed is not really discussed well in the documentation. (To be fair, I am not really sure where it even makes sense to try to explain the concept in the current documentation model.) Using this token takes a bit of observation to figure out. On top of that, the process by which an event becomes a full-fledged event is not entirely clear or obvious, so understanding how it interacts with *Veto* adds a layer of confusion on top of that.

Veto If Operational Attribute Not Available

This token is basically another one of those shortcut tokens that was added because it was quite commonly used in the Create Policy Sets. Basically, this token is a short cut for the following test.

```
<do-if>
  <arg-conditions>
    <and>
      <if-op-attr name="SomeAttr" op="not-available"/>
    </and>
  </arg-conditions>
  <arg-actions>
    <do-veto/>
  </arg-actions>
</do-if>
```

That is, if the operational attribute you specify is not available, *Veto* the event. This is one of the main functions performed in the Create Policy set; and since this token predated the *If* token, it was a very handy convenience at the time. Alas, it does not have any additional functionality over the 'available' test itself.

It is worth noting that, like the *If Operation Attribute* token itself, the test Changing implies an `<add-value>`, `<remove-all-values>`, or a `<remove-values>` node is present; whereas, Available implies only that an `<add-value>` node in the case of a `<modify>` event, or `<add-attr>` in the case of an `<add>` event, is present.

Thus, they wisely made sure this token only catches events where there is value being added. You can apply this to an add or modify event, but it was originally intended to be used on add events only.

Prior to the *If* token (pre IDM 3.5 days) the way you would have reproduced this would have been with a single rule with the condition and a *Veto* inside it. This token nicely allows us to get away from that model. With the ability to use an *If* statement, it is not that interesting.

A more common *Veto* case would be if a specific attribute had a specific value rather than just the absence or presence of a value. Perhaps a nice enhancement might have been to extend this token with some equality style tests so you could replicate such a common case. Alas, it does not. Doing it yourself in policy step by step is so easy to implement that it is probably not worth it.

While

This is the last token in the list and the second of the two loop structures. (The first, of course, was *For Each*.) I initially considered including this with the *For Each* token, but I had so much to say about the *For Each* that I decided not to.

I have occasionally used the *While* token since it is useful to be able to end a loop when you want to. This is an issue with the *For Each* or perhaps more correctly with the inability of the *Break* token to have more than one level of 'breakage'. The only way out of a *For Each* loop is to finish the node-set, veto the event, or break out of the policy set entirely.

With a *While* loop, you specify conditions that must be true, and the loop conditions continue until that changes. You could use a variable to control it, like *If Local Variable STOP not equal true*, then within the *While* loop, when you decide you are done, you just *Set Local Variable STOP to true*.

Usually, you loop through some sort of data perhaps in a node-set or a GCV or somewhere else and then when you run out of data you stop. I worked on this approach with a Mapping table when it came to load balancing users across GroupWise Post Offices in this series of articles:

<http://www.novell.com/communities/node/6906/load-balancing-user-placement-groupwise-post-offices>

<http://www.novell.com/communities/node/6908/load-balancing-user-placement-groupwise-post-offices-part-2>

In that case, I used a counter variable *I* (Of course, it HAD to be *I*, right?) that was incremented inside the loop and used that counter as an index in the mapping table. My source column in the mapping table was the index, which was sequential and did not skip any digits out of order. Thus, each time through the loop I could map from the table with a source column of index with my value of *\$I*, since the *Map* token lets you use variables in the fields and then reads back a Post Office name to get the needed contact information to query for a user count via the GroupWise driver. You can then figure out which has the fewest and add the new user to that selected domain.

In hindsight, I probably could have just done a *For Each* over a List GCV with domain names and used that in the mapping table as the source column, but then I would have been maintaining two independent sets of configuration data instead of doing it all in one table.

I had a funny encounter with the method discussed above. I was helping a client out, and they mentioned that they liked my approach to load balancing and used it in their system. While I was working, I went and looked, and they used it literally verbatim! They did not even change any variable names. I got a good laugh out of that one and teased the guy about it! What was more subtle was I wrote that approach for IDM 3.6.1 where the *Map* token has a neat extra feature called Default value. This means if you try to map a value and the value for the source column data you feed in does not exist, in IDM 3.5 and earlier you got an empty value back. With a default value specified, you get back the default value if the source data does not match anything in the table. I had a silly value, *XXYY*, as the default value to know when my loop was finished. That does not work in IDM 3.5.1 (which is the latest version available if your engine is running on Netware). The guys who used my example figured out a trivial solution. They just made sure the last value in the table always returns *XXYY* and there are no code changes needed. Since the extra XML attribute in the token is unknown to an IDM 3.5.1 engine, it ignores it. They did not even have to remove the extra bit of information. Very clever.

The docs do give a simple example, which also shows how you would increment a counter. That actually makes it a really good example, because while technically demonstrating the *While* token usage, it also demonstrates one of the things you pretty much need to do to make the token actually work. We could do with more wonderfully useful examples!

In IDM Policy, if you need to do math, you use the *XPATH* noun token. So *Set Local Variable I* to the *XPATH* of *\$I + 1* works. I would usually wrap the *\$I* in a *number()* function just to be safe. If you initially set it looking like a string, the engine will generate a *NaN* event (Not A Number) and that ends in an ugly fashion. So it is simpler to just remember to do it as: *XPATH* of *number(\$I) + 1*.

You can read more about XPATH and math in this article:

<http://www.novell.com/communities/node/6109/xpath-and-math>

Now, you do not have to just use a counter. As I suggested earlier, if your end conditions are detected, you could set a local variable STOP to true. Of course, since the Condition block of the *While* token is the usual interface allowed for Rules condition or nested *If* conditions, you can have more than one end criteria. You could have your main criteria and then AND a condition block to suggest that *If Local Variable* STOP is not equal to true. Thus, you can end it either way as needed.

You could also use an XPATH that perhaps looks at a node-set of data. Inside the loop, you could *Strip By XPATH Expression* and remove the data you just processed and then have an *If XPATH* is true test for `count($Variable) > 0`. When you get rid of all the nodes in the data set, it is time to end. Of course, if you completely empty the node-set, you could probably test even simpler as *If XPATH* true `$Variable` since the variable as a node test is false when it is empty.

In a more real-world case, you might have an XDS event document, and you want to loop over all the `<modify-attr>` nodes. You could have an XPATH test for your *While* condition of `$Variable/modify-attr`, which will be true as long as there are `modify-attr` nodes. Then inside the loop use the *Strip by XPATH Expression* to remove the attribute you just worked on. Thus, the variable being looped over gets smaller.

I have honestly not had much call for this token as I usually find myself using a *For Each* loop. Having some flexibility, is quite useful. Specifically, the ability to get out of a loop when you need to and not end the entire policy set.

Filter and Schema

I suppose I could continue for a short period talking about the DTD used in the Filter or Schema Map. While these are not technically tokens, they are things used in Policies even if they do not fit the DirXML Script model.

Schema Map

In the Schema Map policy object, you would see XML that looks something like this:

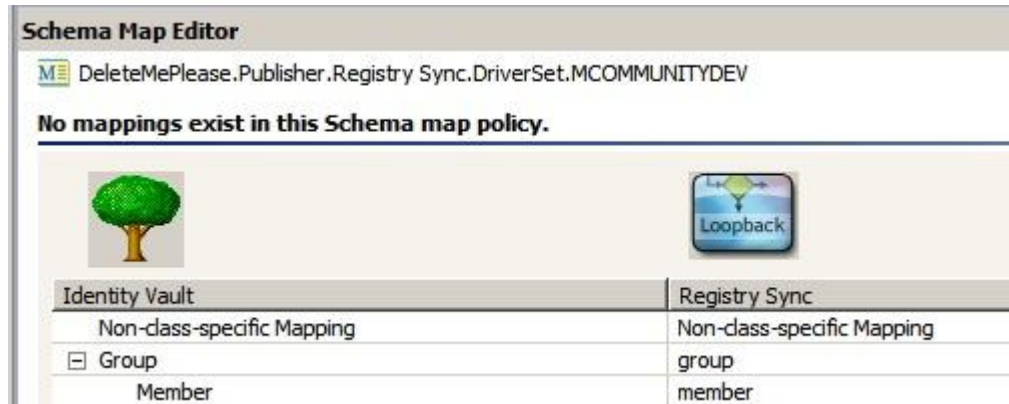
```
<attr-name-map>
  <class-name>
    <nds-name>Group</nds-name>
    <app-name>group</app-name>
  </class-name>
```

```

    <attr-name class-name="Group">
        <nds-name>Member</nds-name>
        <app-name>member</app-name>
    </attr-name>
</attr-name-map>

```

The GUI would render it something like this:



That is a simple example that shows much of the functionality. The `<class-name>` token lets you map classes together where you provide the `<nds-name>` and `<app-name>`. The `<attr-name>` token lets you map attribute names together. You provide the `<nds-name>` and `<app-name>`.

There is an interesting twist. If you look at the `<attr-name class-name="Group">`, the XML attribute `class-name` is how the GUI view knows to tie this as a child of the Group object. Imagine, if you will, the Eclipse plugin selects from this document in a loop over the `<attr-name>` nodes an XPATH, like `@class-name`, and builds the view we see.

So what happens if you do not have a `class-name` XML attribute? Well, that is what is shown as the first class in the GUI view; Non-class-specific Mapping.

I am not aware of much more in the way of Schema map tricks. So on to the Filter whose XML is more complex.

Filter

The XML for the filter, which you can see at any time via the XML view in the editor, looks something like this:

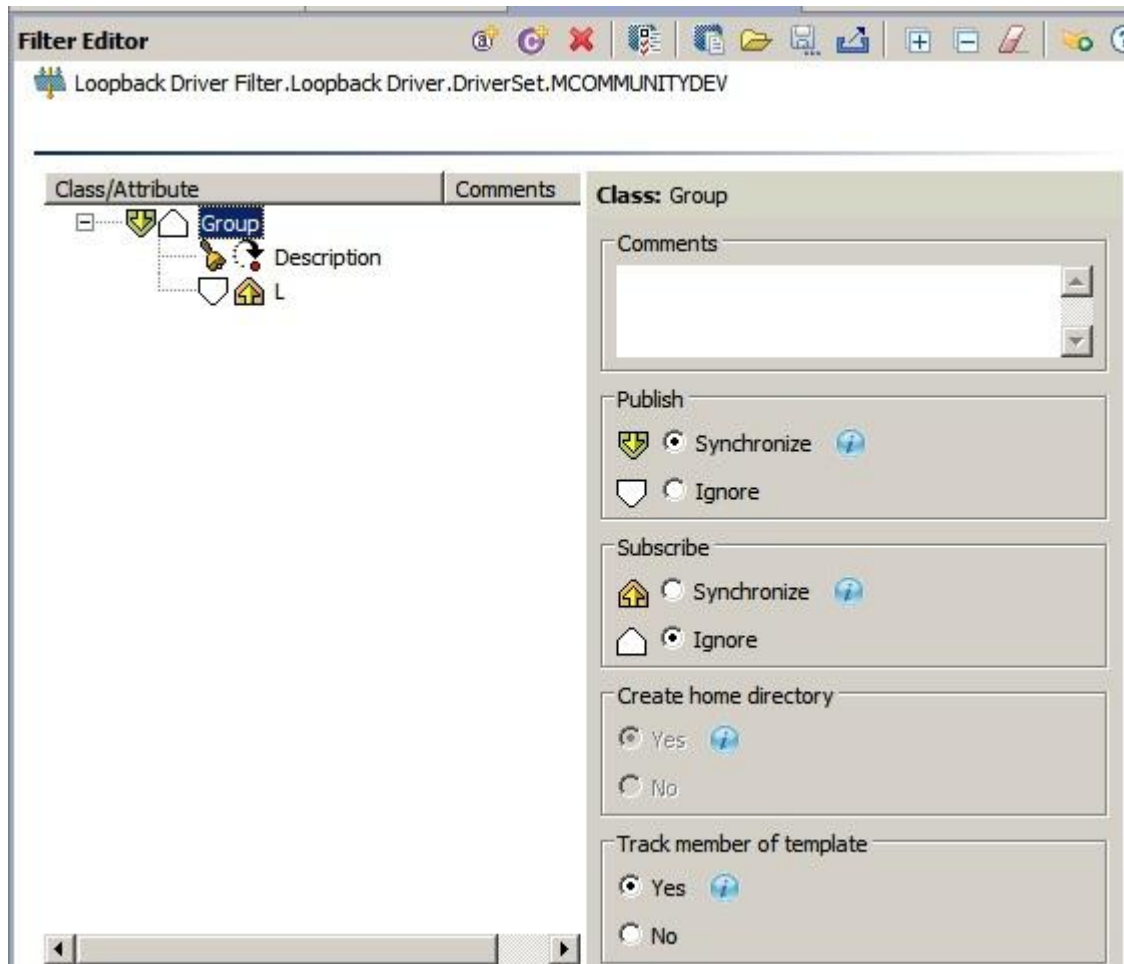
```

<filter>
    <filter-class class-name="Group" publisher="sync" publisher-track-template-member="true" subscriber="ignore">
        <filter-attr attr-name="Description" merge-authority="app" publisher="notify" publisher-optimize-
modify="false" subscriber="reset"/>
        <filter-attr attr-name="L" merge-authority="edir" publisher="ignore" subscriber="sync"/>
    </filter-class>
</filter>

```

```
</filter-class>  
</filter>
```

The GUI view for that example would look something like this:



You can see how the various settings in the GUI are represented as XML attributes on the node. One important trick to learn is that the GUI will only let you add attributes and classes that it knows about. Designer learns the schema from the Identity Vault by importing the schema or from the application via a Refresh Application schema action. However, you might wish some other value. Well, add an attribute into the filter with a simple name, set it up the way you want, flip over to the XML view, find it, and just edit the `attr-name` to be the name you wanted. Then flip back into the Filter editor, and you can see it and manipulate it like any other attribute.

Conclusions

Having gone through all that, I hope this is a useful resource and offers additional information that the official documentation does not provide.

Appendix A: ECMAScript functions

With IDM 3.6 and higher, a number of driver configurations came with an ECMA Script library called the Advanced Java Class (AJC). With IDM 4, there is a package named Advanced Java Class in the Common category and ECMAScript group that delivers the library to the driver set.

A number of tokens were not available until recently in DirXML Script, and many of these functions were written in Java for the Novell Consulting group to use to save time in implementations. With the release of IDM 3.5 and support for ECMA Script, these functions were ported from Java to ECMAScript. With IDM 4 and Packages, this gets even better as now the object can be version controlled. Look in a Library object in the driver set for an ECMA Script object named LIBAJC-JS in IDM 3.6 and higher. The packaged version would be named NOVLLIBAJC-JS.

Many of the functions have since been replaced by tokens and are often much more flexible and easier to use. However, these remain, and some of them are still useful. This is a summary of the list of functions with some basic explanations of their functionality.

Function Reference

print(arg)

Uses the proper function to output the argument.

println(arg)

Uses the proper function to output the argument, followed by a new line character.

getUniqueTimeString()

As the name suggests, generates a reasonably unique time value in milliseconds. If the value it generates is the same as the last time it was called, it adds one to make it unique.

appendLog(log, append, delimiter, b64, maxsize)

Handling large strings might not be most efficient in XPATH, the commentary in this function suggests. Thus, there is this function to append some text to a potentially large string. The

Copyright© 2013 Ninja-Tools

parameters are **log**, the string that is large and will be returned with the **append** string appended to it. The delimiter value lets you specify what the line feed character is so as not to cut off the log early. The **b64** parameter is a boolean to let you know if the log value is base64 encoded and if it needs to be decoded, have the **append** value appended, then re-encoded. Finally, **maxsize** is the length in bytes for the maximum size you might return. I guess to keep you from overflowing some value.

checkStringDiff(oldValue, newValue, orgValue, gcValue)

This takes four values as parameters and says it returns them as:

```
'old-new-org:org', '-new-org:org', 'old- -org:org', '-new- :new', 'old- - :old', 'old-new-old:new', 'old- -old:old'
```

This one is beyond me as to what you would use it for, but there it is.

stripChars(input , validChars)

This basically strips out any illegal characters based on their ASCII code values. The second parameter, **validChars**, is not implemented yet per the comments. It basically loops over all the characters in the **input** string and any that are in a list of illegal character's ASCII values are ignored.

The ASCII values that are ignored are:

32

45

39

65 to 90

97 to 122

192 to 214

216 to 246

248 to 255

What is nice, is that if you know the ASCII value of any particular character you need to get rid of, you could very easily modify this for your own use.

getDocumentFromString(xml)

This function takes a string that looks like valid XML and turns it into a DOM node-set. This is replaced by the *XML Parse* token.

serialize(node, indent)

This function takes a DOM **node** (just a node, not a node-set, which comes later) and serializes it to a string either indented or not based on the value of the second parameter, **indent**.

getStringFromNodeSet(nodeset, indent)

This function takes a DOM **nodeset** and serializes it into a string supporting indenting the output or not. It also calls the previous `serialize()` function on each node to do its work. This is replaced by the *XML Serialize* token. However, the ability to NOT indent the resulting string is missing in the token but probably not that important.

readImage(imageFileName, wrap)

This function is used to read an image from a file and return the content as Base64 encoded string. There is a delimited text driver configuration that can be used to read in JPG images to the photo eDirectory attribute and I imagine it either uses this call, or something equivalent. There is no simple token that can be used to replace this one.

rollTime(time, units, unitID)

This is a helper function that takes a **time** in milliseconds, **units** for the number of units to add, and then **unitID** for the type of unit to add. For example, send in a time, a number of items to be added, and then the specific type of item to be added.

This can be done simply in XPATH by using a time in CTIME (perhaps generated or converted by the *Time* or *Convert Time* tokens) and then adding as many seconds as needed. If you wish to add days, then just add 86,400 seconds multiplied by the number of days.

Additionally, the *Convert Time* token has an offset value that will let you add seconds, minutes, or days as needed as part of the time conversion process.

rollDays(time, days)

This is a special case of `rollTime()` that just adds the number of **days** (or subtracts if negative). This can be completely replaced by the *Convert Time* token.

rollHours(time, hours)

This is a special case of `rollTime()` that just adds the number of **hours** (or subtracts if negative). This can be completely replaced by the *Convert Time* token.

rollSeconds(time, seconds)

This is a special case of `rollTime()` that just adds the number of **seconds** (or subtracts if negative). This can be completely replaced by the *Convert Time* token.

dayOfWeek(yyyy, mm, dd)

This function is meant to get the day name of a given date using the System locale. If called with no arguments, then it uses today's date. This can be completely replaced by the *Convert Time* token.

getXmlHeader()

This just returns a string of "<?xml version='1.0'?>" likely used by other functions to generate node-sets or the like.

replaceLeft(name)

This replaces any less than signs in the argument passed in with `<` (which is the HTML/XML escaped version of the less than sign).

getSeconds(time, pattern)

This function is completely replaced by the *Time* and *Convert Time* tokens and lets you pass in a **pattern** of a date string and get the CTIME equivalent. If there is no time value passed in, then it defaults to the current time. If there is no **pattern** specified, it assumes a pattern of `yyyy-MM-dd kk:mm:ss.SSS`

getTime(seconds, pattern)

This function is completely replaced by the *Time* and *Convert Time* tokens and lets you pass in a **pattern** for an output date string, and the time in **seconds** (CTIME), and get the date in your pattern. If there is no time value passed in, then it defaults to the current time. If there is no **pattern** specified, it assumes a pattern of `yyyy-MM-dd kk:mm:ss.SSS`

numericCompare(x, y)

It is not clear why this was ever needed. It is probably because, at some point in ancient history, the *If Local Variable* condition token did not support numeric compares. The commentary on the function says "0 if x == y; -1 if x < y; 1 if x > y". Thus it returns 0 if the values are equal. It returns -1 if the first value is less than the second value. And finally 1 if the first value is greater than the second value. Of course, this could also be done in simple XPATH as well.

sortNumbersAscending(numbersIn, delimiter)

This function is quite useful even today as it takes a list of **numbersIn** as a string, then a **delimiter**, and returns them sorted. There is no equivalent in the set of tokens out of the box.

moveFile(oldname, newname)

File manipulation functions are not very well handled in DirXML Script or, to be fair, at all. This is a useful function to move a file around. If the new file name exists, it will return an error message that the file already exists.

doWait(interval)

This waits the specified number of seconds. I always just called the Java function it uses internally `java.lang.Thread.sleep(interval)`. This could be easier to call I suppose but not by all that much.

readLineFromFile(filename, encoding)

Like other file handling functionality, DirXML Script is very weak in this regard. Being able to read a file from a file is very handy and remains very relevant. What is odd about this function is it only reads the very first line from a file and nothing else.

makeSpecName(name)

This seems to escape a single apostrophe as a double apostrophe. That sort of makes you wonder what use case this really has in the real world. The example given in the comments is O'Donnell being escaped to O''Donnell, but there are not many name fields where an apostrophe is the escape character; usually it would be a backslash as the escape character.

getUTPTime(time)

This just returns CTIME for the specified time value in an unspecified format. The *Convert Time* and *Time* tokens are much better choices than this function.

match(str, pattern)

This is a very useful function that I still use today. This allows a Regular Expression compare in XPATH. XPATH includes `contains()` function, but it is case sensitive and an exact match only. A Regular Expression allows much more flexibility in finding something specific in a string. By

default, this is case sensitive. To make it case insensitive, you would change the main call inside the function from:

```
var re = new RegExp(pattern);  
return re.test(str);
```

to

```
var re = new RegExp(pattern,'i');  
return re.test(str);
```

The extra parameter with 'i' means case insensitive. There are a number of other options like: s=single line mode, m=multiline mode, and x=free space mode. For more options, see this page: <http://www.regular-expressions.info/modifiers.html>.

octet2AsciiHex(s)

Convert a Base64 encoded octet string value to an ASCII HEX string. This will be used by the GUID converting functions.

guid2Association(s)

guid2string(s)

These two functions convert a binary GUID attribute to one of two formats. There is a common format used in DirXML-Associations values for the eDirectory driver. The difference between the two is mostly that the Association version is surrounded by curly braces.

getHostname()

This function calls a Java function to get the local servers hostname. Until IDM 4.0, there was no easy way to know which server the policy is being executed on. With IDM 4.0, there was a new GCV added, dirxml.auto.localserverdn, that returns the LDAP DN of the driver objects from which it is possible to read the Network Address attribute, find the TCPIP value, and decode its format. Of course, this function is much simpler. Functions to decode the Network Address values are later in the list of functions.

dn2xml(dn, delimiter)

This function converts an LDAP DN to a backslash formatted DN. The *Parse DN* token is a much better choice with better control and more functionality. This simply splits on the comma in an LDAP DN, which is the simple approximation, but has a fatal flaw when a DN includes an escaped comma. For example, in Active Directory it is very common to change the DN of an object to be `cn=LastName, FirstName`, in which case this function fails to handle that internal comma properly. If it is properly escaped as `cn=LastName\, FirstName` (Note the backslash comma), then *Parse DN* can handle it properly.

Base64encode(str, encoding)

Base64decode(b64, encoding)

These two functions are completely replaced by the *Base64 Encode* and *Base64 Decode* verb tokens.

execString(eStr)

This function is a wrapper to the Java call to run a local system executable. It requires the system to have the file present that is specified. It will work on Windows or Linux; but, obviously, the name passed in will have different path formatting depending on the operating system.

xml2dn(slashDN, type1, type2)

This function is an early attempt at the wonderful things the *Parse DN* token can perform. It is not even worth considering it once *Parse DN* is available. It tries to convert from a backslash formatted DN to an LDAP formatted DN. There is a fundamental problem as the naming attribute at each level (`ou=`, `cn=`, and the like) do not exist in the backslash format. Thus, it offers two parameters, **type1** for the first node value type, and **type2** for all the following nodes. Which, of course, fails in the now most common cases where `ou=`, `o=`, and `dc=` are usually mixed into a DN.

NodeSetToString(nodeset)

This function will serialize a node-set to a string that is completely replaced by the *XML Serialize* token.

fixLengthRight(inStr, size, padStr)

This function will pad (via right justification) or truncate a string to a given length. This can still be useful as it would take a number of steps to accomplish the same thing. If the string is short,

it will add more values (**padStr**) until it reaches the size specified. If the string is long it will be truncated to the **size**.

traceMsg(msg)

This function will trace the message specified, but I am not sure where it will show up. I do not think in Dstrace.

xdsQschedule(secs)

This function is confusingly named as it is a time conversion function, not used to schedule XDS events. The xds and schedule part are related to the format submitting a job to the XDS Command processor would need. This converts a CTIME value to the format YYYYMMDDhhmmss. As with all the other time conversion functions in this library, the *Convert Time* token replaces it as it is better, more flexible, and easier to read in code.

interString(input)

This function will try to replace extended Latin characters in string with closest ASCII equivalent. There is no equivalent token to support this directly.

getTimeString()

This function returns the current time in the format YYYYMMDDHHmmss. This is completely replaced by the *Time* token. It is better, more flexible, and easier to read in code.

createPassword(input)

createPassword2(minLength)

createPassword3(minLength)

createPassword4(passwordLength, numOfAlphabets, numOfDigits)

CreatePasswd(input)

This set of five password functions are all better replaced by the *Generate Password* noun token. These were short cuts to help out before the token was added. The *Generate Password* token is a huge improvement over these functions.

sendMail(from, to, subj, msgText, host, debugging)

sendMailWithAttachment(from, to, subj, msgText, file, host, debugging)

sendMailC(from, to, subj, msgText, host, debugging, charSet)

sendMailCWithAttachment(from, to, subj, msgText, file, host, debugging, charSet)

It looks like the original Java class had one function, `sendMail()`, that could be sent with different parameters for different events. Thus, this is broken down into multiple functions to handle the *Email* and *Send Email From Template* possibilities. The ability to send an email with an attachment is a big improvement over the *Send From Template* tokens. Those tokens do not provide any mechanism for attaching files. Of course, the files must be written to the local servers filesystem to work.

makeAscii(s0)

This function will convert a string to pure ASCII. This is one of those functions that would be pretty hard to do in Policy and is better done here in ECMA.

makeAsciiAlpha(s0)

Like `makeAscii()`, this function will convert a string and remove any numeric values from the string.

writeLog(filename, msg, encoding)

In order to write to a file, you need to open a file for writing, send the data, and close the file. This function does all that in one function call. I am told it is not thread safe, such that if multiple threads were to try to write to the same file at the same time you might have an issue. However, in a driver context, this mostly matters if your Subscriber and Publisher threads both contemplate writing to the same log file.

upperString(input)

This function has been replaced by the *Uppercase* verb token. It returns the uppercase version of the input string.

normalizeSpace(input)

This function is an ECMA implementation of the `normalize-space()` function available in XPATH. It removes white space characters. You are better off using the XPATH function if needed.

capitalizeWords(input)

Converts every word in a sentence to have a first character as uppercase. You could reproduce this with policy, but it would take a fair number of steps. If needed, this is probably a simpler way to accomplish the task.

lowerString(input)

This function has been replaced by the *Lowercase* verb token. But it returns the lowercase version of the input string.

createUsername(s1, c1, s2, c2, delimiter)

A common task is to generate a username for an object. Often this is based on name parts. For example, an initial of the first name followed by the last name. The parameters are **s1** for the first string, and **c1** for how many characters to take of that string. The parameter **s2** is the second string, and **c2** specifies how many characters to take of that string. An optional delimiter can be used in between.

However, this makes no guarantee of uniqueness, which the *Unique Name* noun token would do a much better job at.

convertDate(inDate, inFormat, outFormatString)

Like the earlier time formatting functions that are completely replaced by the *Convert Time* verb token, this one is the date equivalent; and the *Convert Time* token is a much better option for almost all cases.

getPrefix(input, delim)

This function is meant to get the first part of a DN using the `delim` value to specify how they are separated. This is much better performed using the *Parse DN* token.

getSuffix(input, delim)

This function is meant to get the last part of a the DN, and is much better performed using the *Parse DN* token.

getSubstring(input , start, end)

This function can be useful but is already present as part of XPATH in the `substring()` function. The DirXML Script *Substring* token cannot do a range the same way as it only allows a start and length value.

patternCount(input, pattern)

This function counts the occurrences of a specified pattern in a string. This can be done in Policy, but would be a fair bit of work and looks to be better off done in ECMA.

fixLength(input, len, pad)

This function will append the **pad** character until the **len** value is reached. Useful for making sure a string is a fixed length.

replaceChar(inString, oldChar, newChar)

removeChar(inString, oldChar)

removeApostrophe(inString)

removeQuote(inString)

removeLesser(inString)

removeGreater(inString)

replaceString(inString, oldString, newString)

All these functions are variations of the *Replace All* or *Replace First* tokens in DirXML Script. The *Replace* tokens are actually much more powerful as they support Regular Expressions; whereas, these functions are all literal strings. Some of functions are special cases that are useful like; `removeQuote()`, `removeLesser()`, `removeGreater()`, and `removeApostrophe()` to remove some common characters.

reverseString(inString, delimiter)

This function is useful as it would be pretty hard to implement in policy if needed. The order of the words, separated by the **delimiter**, is returned in reverse. This is partially need for some of the functions that are replaced by *Parse DN*.

convertDateBetweenLocales(ds, style, fromLC, toLC)

convertTimeBetweenLocales(ts, style, fromLC, toLC)

convertTimeOrDate(useTime, style, ds, fromLC, toLC)

convertTimeDateBetweenLocales(ds, style, fromLC, toLC)

convertDateToDirFormat(ds, fromLC)

simpleDateFormat(ds, fromLC, format)

All these functions are replaced by the *Convert Time* token. These are more of the various shorthand functions that were very useful before the advent of the *Convert Time* token.

getPortFromHexNetAddr(hexStr)

getIPFromHexNetAddr(hexStr)

This set of functions is useful if you have a *Network Address* attribute value (as discussed in *getHostName* above). Then, pick it apart since eDirectory stores the values in a particular format, that is non-trivial to decode. These functions make it easy to get the information out of that attribute, and, in fact, were created for IDM 4. The Managed System Gateway driver uses these to figure out connection information about the various drivers in the system.

getProtocolFromURL(urlStr)

getPortFromURL(urlStr)

getIPFromURL(urlStr)

These three functions are used to take a URL and get the three parts out of it. Consider the example of: `http://myserver.domain.com:8443`

The protocol would be http.
The port would be 8443.

The IP would be myserver.domain.com.

getEntParamField(param, field)

getEntParamFieldBackwardsCompatible(param, idm4field, legacyfield)

getIDM4EntParamField(jsonstr, field)

getParamValue(paramString, paramName)

With the release of IDM 4, a new format for Entitlements payload was supported. Initially in the DirXML-EntitlementRef attribute, the path.xml component held some XML text. The <param> node inside it was the value of the entitlement returned by the *Entitlement* tokens. With IDM 3.61, a model of using the pipe (|) symbol within the <param> node was used to indicate multiple values. With IDM 4 and higher, the <param> node contains JSON strings, and these functions are used to determine the values.

JSON is a key-value type representation. The parameters are the entire value (param), and then the key (field) returning the value.

These functions can handle both models of multiple values and are used in IDM 4 for entitlement value processing.

hex2dec(hexStr)

This function converts a hexadecimal string to decimal. There is no easy equivalent in DirXML script, so this can be useful.

endsWith(str1, str2)

If **str2** is the end of **str1** this function returns true. I am not entirely sure what the use case for this would be, but it is easier doing it in ECMA than in Policy.

tokenize(source, delimiter, escapeChar)

dump(data) internal function

hex(data) internal function

decodeAsciiHex(s) internal function

encodeAsciiHex(abyte0) internal function

Appendix B: Article reference

The collection of articles written by the author are maintained in a Wiki page at:

http://wiki.novell.com/index.php/Geoffrey_Carman%27s_personal_collection

At the time of publication, this collection consists of the following articles sorted by topic:

General Articles

<http://www.novell.com/communities/node/6301/power-community>

<http://www.novell.com/communities/node/1996/response-tracking-admin-passwords>

<http://www.novell.com/communities/node/4778/convertng-speed-values-backup-vendors>

OS Related

Netware

<http://www.novell.com/communities/node/3218/using-inetcfg-manage-your-nic-cards>

<http://www.novell.com/communities/node/3225/tbxnlm>

<http://www.novell.com/communities/node/3252/shared-scsi-two-node-cluster>

<http://www.novell.com/communities/node/3325/tips-installing-service-packs-several-servers>

<http://www.novell.com/communities/node/3326/avoiding-timeout-when-entering-ip-address-during-netware-65-installation>

<http://www.novell.com/communities/node/3600/clusters-migration-wizard-and-backups>

<http://www.novell.com/cool solutions/trench/617.html>

Linux

<http://www.novell.com/communities/node/3096/i810-video-drivers-update>

<http://www.novell.com/communities/node/6177/basic-troubleshooting-tips-edirectory-linux>

Windows

<http://www.novell.com/communities/node/1460/solving-rdp-issues-remote-loader>

Macintosh

<http://www.novell.com/communities/node/3217/using-universal-password-mac-clients>

<http://www.novell.com/communities/node/4288/using-universal-password-with-mac-clients>

<http://www.novell.com/communities/node/3224/kanaka-troubleshooting>
<http://www.novell.com/communities/node/7080/example-troubleshooting-afp-nmas-issues>

iPrint

<http://www.novell.com/communities/node/3328/iprint-troubleshooting>
<http://www.novell.com/communities/node/1440/allow-public-printing-iprint>

Data Center

<http://www.novell.com/communities/node/10279/data-center-migrations-made-easier>

eDirectory

<http://www.novell.com/communities/node/2803/managing-edirectory-database-cache>
<http://www.novell.com/communities/node/1458/handling-port-collisions-multi-instance-edirectory>
<http://www.novell.com/communities/node/3219/setting-uniqueid-edirectory-users>
<http://www.novell.com/communities/node/1855/more-about-uniqueid-edirectory>
<http://www.novell.com/communities/node/3216/controlling-imonitor-session-length>
<http://www.novell.com/communities/node/3220/file-system-factory-identity-based-file-system-management>
<http://www.novell.com/communities/node/3999/role-based-services-collection-ownership>
<http://www.novell.com/communities/node/4142/bulk-deleting-user-objects>
<http://www.novell.com/communities/node/3262/restoring-edirectory-with-dsrepair-rc-and-dsrepair-du>
<http://www.novell.com/communities/node/3018/controlling-memory-allocation-multiple-edirectory-instances-0>
<http://www.novell.com/communities/node/6174/system-generated-indexes-edirectory>
<http://www.novell.com/communities/node/6450/interesting-schema-syntaxes-edirectory-identity-manager-perspective-part-1>
<http://www.novell.com/communities/node/6457/interesting-schema-syntaxes-edirectory-identity-manager-perspective-part-2>
<http://www.novell.com/communities/node/8438/troubleshooting-pamldap-against-edirectory-issues>
<http://www.novell.com/communities/node/8445/examples-jim-willekes-dump-tool>
<http://www.novell.com/communities/node/12578/pki-server-startup-dstrace-pki>

LDAP as a service of eDirectory

<http://www.novell.com/communities/node/9625/troubleshooting-external-ldap-application-sap-grc>

IDM Overview

<http://www.novell.com/communities/node/10022/novell-idm-and-secure-login-make-life-easier>
<http://www.novell.com/communities/node/11222/using-novell-identity-manager-make-life-easier>
<http://www.novell.com/communities/node/11983/what-does-it-mean-be-cloud-when-talking-about-idm>

IDM 4 Specific

<http://www.novell.com/communities/node/12165/what-has-changed-between-identity-manager-versions-part-1>
<http://www.novell.com/communities/node/12166/what-has-changed-between-identity-manager-versions-part-2>
<http://www.novell.com/communities/node/12177/what-has-changed-between-identity-manager-versions-part-3>
<http://www.novell.com/communities/node/12198/more-whats-new-idm-4>

<http://www.novell.com/communities/node/12398/new-object-type-idm4-ds-object-and-how-you-might-generate-info>
<http://www.novell.com/communities/node/12943/new-features-idm-4-and-401>
<http://www.novell.com/communities/node/12944/new-features-idm-4-and-401-part-2>

IDM Engine

<http://www.novell.com/communities/node/3329/troubleshooting-synchronization-identity-manager-installations>
<http://www.novell.com/communities/node/5921/more-common-identity-manager-errors>
<http://www.novell.com/communities/node/6299/working-idm-301-after-using-351-while>
<http://www.novell.com/communities/node/5033/reading-and-displaying-value-java-heap-identity-manager-rules>
<http://www.novell.com/communities/node/6176/another-node-set-size-discussion>
<http://www.novell.com/communities/node/6085/more-thoughts-size-node-set-identity-manager>
<http://www.novell.com/communities/node/5846/changes-do-not-require-driver-restart-and-errors-starting-identity-manager-driver>
<http://www.novell.com/communities/node/3077/testing-destination-container-explaining-why-it-works>
<http://www.novell.com/communities/node/6306/certificate-expired-error>
<http://www.novell.com/communities/node/2206>
<http://www.novell.com/communities/node/2913/ldap-password-changes-and-versions-security-services>
<http://www.novell.com/communities/node/3002/things-i-think-idm-needs-support>
<http://www.novell.com/communities/node/2290/auxiliary-classes-and-identity-manager>
<http://www.novell.com/communities/node/2052/avoiding-startup-vetos-scoping-rules>
<http://www.novell.com/communities/node/1589/solving-license-activation-issues-idm-35>
<http://www.novell.com/communities/node/3099/somewhat-obscure-idm-error-codes>
<http://www.novell.com/communities/node/2995/solving-9063-matching-error>
<http://www.novell.com/communities/node/1478/open-call-rules-need-updating-when-migrating-idm-35>
<http://www.novell.com/communities/node/3505/another-thought-upgrading-drivers-idm-35x>
<http://www.novell.com/communities/node/4141/identity-manager-performance-example>
<http://www.novell.com/communities/node/4641/upgrading-idm-1x2x-3x-issues-and-concerns>
<http://www.novell.com/communities/node/4860/discussing-gcvs-comments-field-identity-manager>
<http://www.novell.com/communities/node/6343/importing-driver-export-file-without-password-prompts>
<http://www.novell.com/communities/node/6937/novell-identity-manager-high-availability-concepts>
<http://www.novell.com/communities/node/8878/open-call-idm-move-event-documents>
<http://www.novell.com/communities/node/9465/diagnosing-6038-error-identity-manager>
<http://www.novell.com/communities/node/11558/convert-entitlements-resources-more-details>
<http://www.novell.com/communities/node/12760/talking-about-entitlements-part-1>
<http://www.novell.com/communities/node/12908/talking-about-entitlements-part-2>
<http://www.novell.com/communities/node/12909/talking-about-entitlements-part-3>
<http://www.novell.com/communities/node/13019/migrating-users-idm>
<http://www.novell.com/communities/node/13027/talking-about-entitlements-part-4>
<http://www.novell.com/communities/node/13326/tip-using-ecma-script-objects-designer>
<http://www.novell.com/communities/node/13394/password-tunneling-model-identity-manager>

Designer Packages

<http://www.novell.com/communities/node/12208/lets-talk-some-more-about-packages-designer-4-part-1>
<http://www.novell.com/communities/node/12218/lets-talk-some-more-about-packages-designer-4-part-2>
<http://www.novell.com/communities/node/12241/lets-talk-some-more-about-packages-designer-4-part-3>
<http://www.novell.com/communities/node/12246/lets-talk-some-more-about-packages-designer-4-part-4>
<http://www.novell.com/communities/node/12251/lets-talk-some-more-about-packages-designer-4-part-5>
<http://www.novell.com/communities/node/12267/lets-talk-some-more-about-packages-designer-4-part-6>
<http://www.novell.com/communities/node/12275/lets-talk-some-more-about-packages-designer-4-part-7>
<http://www.novell.com/communities/node/12294/lets-talk-some-more-about-packages-designer-4-part-8>

<http://www.novell.com/communities/node/12303/lets-talk-some-more-about-packages-designer-4-part-9>

Dstrace

<http://www.novell.com/communities/node/4418/the-many-faces-dstrace>
<http://www.novell.com/communities/node/4428/guide-reading-dstrace-output-identity-manager>
<http://www.novell.com/communities/node/5831/how-dstrace-can-affect-identity-manager-performance>
<http://www.novell.com/communities/node/1893/finding-last-item-dstrace-live>
<http://www.novell.com/communities/node/3035/using-grep-find-things-idm-trace-log>

DirXML Script Tokens

<http://www.novell.com/communities/node/6178/new-idm-36-enhancement-do-find-matching-objec>
<http://www.novell.com/communities/node/4337/parsedn-token-identity-manager-and-some-its-limitations>
<http://www.novell.com/communities/node/5687/examples-using-parsedn-token-identity-manager>
<http://www.novell.com/communities/node/5818/different-attribute-options-identity-manager>
<http://www.novell.com/communities/node/5920/ism-more-thoughts-sourcedestinationoperation-attribute-tokens-identity-manager>
<http://www.novell.com/communities/node/4906/the-query-token-identity-manager>
<http://www.novell.com/communities/node/2209/unique-name-token-functionality-idm-35>
<http://www.novell.com/communities/node/3034/reformat-operation-attribute>
<http://www.novell.com/communities/node/3279/open-call-what-can-you-do-xslt-you-cannot-do-dirxml-script>
<http://www.novell.com/communities/node/2572/using-time-tokens-idm-35>
<http://www.novell.com/communities/node/1903/idm-35-update-new-features>
<http://www.novell.com/communities/node/1858/more-about-veto-command-scoping-events>
<http://www.novell.com/communities/node/2198/troubleshooting-if-then-else-blocks-idm-code>
<http://www.novell.com/communities/node/1698/good-grammar-and-idm-rules>
<http://www.novell.com/communities/node/4844/mapping-tables-and-render-browsed-dn-relative-policy-option>
<http://www.novell.com/communities/node/8439/open-call-useful-ecma-functions-use-identity-manager>
<http://www.novell.com/communities/node/9332/dirxml-script-elegance-coding-contest>
<http://www.novell.com/communities/node/9351/join-and-split-tokens-identity-manager-35-and-higher>
<http://www.novell.com/communities/node/9356/using-and-copying-email-templates-identity-manager>
<http://www.novell.com/communities/node/11319/parsedn-custom-dn-delimiter-example>
<http://www.novell.com/communities/node/11344/explaining-gcvs-part-1>
<http://www.novell.com/communities/node/11471/explaining-gcvs-part-2>
<http://www.novell.com/communities/node/11480/structured-global-configuration-values-idm>
<http://www.novell.com/communities/node/11611/introducing-start-workflow-token-part-1>
<http://www.novell.com/communities/node/11612/introducing-start-workflow-token-part-2>
<http://www.novell.com/communities/node/11718/dirxml-excellence-coding-contest-winners>
<http://www.novell.com/communities/node/13144/various-attribute-tokens-idm-policy>

XPATH

Concepts

<http://www.novell.com/communities/node/4833/some-thoughts-xpath-novell-identity-manager>
<http://www.novell.com/communities/node/6175/xpath-and-context-node>
<http://www.novell.com/communities/node/6109/xpath-and-math>
<http://www.novell.com/communities/node/6179/using-string-compares-xpath-statements>

<http://www.novell.com/communities/node/6910/another-attempt-explaining-xpath-context-node>
<http://www.novell.com/communities/node/9214/example-walk-through-using-xpath-identity-manager>
<http://www.novell.com/communities/node/12261/using-xpath-reproduce-map-token>
<http://www.novell.com/communities/node/12361/examples-using-xpath-identity-manager>

Cool tips

<http://www.novell.com/communities/node/5845/using-xpath-examine-association-values>
<http://www.novell.com/communities/node/5686/cool-tricks-using-xpath-nodesets>
<http://www.novell.com/communities/node/4825/using-global-configuration-values-xpath>
<http://www.novell.com/communities/node/6276/using-xpath-get-position-node-node-set>
<http://www.novell.com/communities/node/11637/xpath-do-schema-mapping-rule>

IDM Tutorials

<http://www.novell.com/communities/node/6308/toolkit-rules-identity-manager-part-1>
<http://www.novell.com/communities/node/6310/toolkit-rules-identity-manager-part-2>
<http://www.novell.com/communities/node/6316/toolkit-rules-identity-manager-part-3>
<http://www.novell.com/communities/node/6440/toolkit-rules-identity-manager-part-4>
<http://www.novell.com/communities/node/6441/example-use-toolkit-rule-identity-manager>
<http://www.novell.com/communities/node/6514/another-toolkit-rule-use-example-bad-attribute-value-cleanup>
<http://www.novell.com/communities/node/13053/common-mistakes-newcomers-idm-make-part-1>
<http://www.novell.com/communities/node/13057/common-mistakes-newcomers-idm-make-part-2>
<http://www.novell.com/communities/node/13058/common-mistakes-newcomers-idm-make-part-3>
<http://www.novell.com/communities/node/13125/common-mistakes-newcomers-idm-make-part-4>
<http://www.novell.com/communities/node/13126/common-mistakes-newcomers-idm-make-part-5>
<http://www.novell.com/communities/node/13302/common-mistakes-newcomers-idm-make-part-6>
<http://www.novell.com/communities/node/13316/common-mistakes-newcomers-idm-make-part-7>
<http://www.novell.com/communities/node/13347/common-mistakes-newcomers-idm-make-part-8>
<http://www.novell.com/communities/node/13383/common-mistakes-newcomers-idm-make-part-9>
<http://www.novell.com/communities/node/13486/common-mistakes-newcomers-idm-make-part-10>

Drivers

General

<http://www.novell.com/communities/node/1474/password-transformation-rule-sets> (Subscriber channel)
<http://www.novell.com/communities/node/9661/password-transformation-rules-publisher-channel>
<http://www.novell.com/communities/node/1696/open-call-idm-association-values-edirectory-objects>
<http://www.novell.com/communities/node/8548/identity-manager-troubleshooting-toolkit>
<http://www.novell.com/communities/node/9200/drivers-vs-driversets-vs-servers-identity-manager>
<http://www.novell.com/communities/node/9205/lets-talk-about-dirxml-associations>
<http://www.novell.com/communities/node/9215/idm-613-invalid-syntax-error-troubleshooting>
<http://www.novell.com/communities/node/12517/what-do-various-idm-driver-object-attributes-represent-part-1>
<http://www.novell.com/communities/node/12520/what-do-various-idm-driver-object-attributes-represent-part-2>
<http://www.novell.com/communities/node/12528/what-do-various-idm-driver-object-attributes-represent-part-3>
<http://www.novell.com/communities/node/12529/what-do-various-idm-driver-object-attributes-represent-part-4>
<http://www.novell.com/communities/node/13018/talking-about-dirxml-policies-attributes>

AD

<http://www.novell.com/communities/node/4335/how-gcvice-a-driver-part-1-subscriber-channel>
<http://www.novell.com/communities/node/4336/how-gcvice-a-driver-part-2>
<http://www.novell.com/communities/node/1765/ad+driver+default+rule+oddiy>
<http://www.novell.com/communities/node/1424/sub-error-codes-ldap-error-49>
<http://www.novell.com/communities/node/5729/watching-group-membership-changes-coming-ad-driver>
<http://www.novell.com/communities/node/6307/active-directory-driver-and-setting-attributes-active-directory>
<http://www.novell.com/communities/node/7048/active-directory-driver-catch-move-subtrees-block-and-reset>
<http://www.novell.com/communities/node/7051/active-directory-services-2008-prevent-accidental-deletion>
<http://www.novell.com/communities/node/7702/active-directory-driver-error-messages-part-1>
<http://www.novell.com/communities/node/8228/active-directory-driver-error-messages-part-2>
<http://www.novell.com/communities/node/8304/active-directory-driver-error-messages-part-3>
<http://www.novell.com/communities/node/8551/active-directory-driver-error-messages-part-4>
<http://www.novell.com/communities/node/9272/active-directory-driver-error-messages-part-5>
<http://www.novell.com/communities/node/9372/idm-36-remote-loader-and-windows-2000-servers>
<http://www.novell.com/communities/node/12388/managing-multiple-active-directory-domains-one-idm-system-part-1>
<http://www.novell.com/communities/node/12390/managing-multiple-active-directory-domains-one-idm-system-part-2>
<http://www.novell.com/communities/node/12393/managing-multiple-active-directory-domains-one-idm-system-part-3>
<http://www.novell.com/communities/node/12397/when-active-directory-driver-will-not-synchronize-attributes>
<http://www.novell.com/communities/node/12399/managing-multiple-active-directory-domains-one-idm-system-part-4>
<http://www.novell.com/communities/node/12434/managing-multiple-active-directory-domains-one-idm-system-part-5>
<http://www.novell.com/communities/node/12448/managing-multiple-active-directory-domains-one-idm-system-part-6>
<http://www.novell.com/communities/node/12467/managing-multiple-active-directory-domains-one-idm-system-part-7>

AS400

<http://www.novell.com/communities/node/1666/solving-account-disabled-issues-as400-bidirectional-driver>
<http://www.novell.com/communities/node/1591/association-problems-when-updating-i5os-bidirectional-driver-idm-35>
<http://www.novell.com/communities/node/1585/limitation-as400-driver-support-idm>
<http://www.novell.com/communities/node/7079/as400-bi-directional-idm-driver-adding-additional-attributes>

Composer

<http://www.novell.com/communities/node/2727/about-idm-integration-modulecomposer>

Data Collection Service

<http://www.novell.com/communities/node/12336/data-collection-service-driver-walkthrough-part-1>
<http://www.novell.com/communities/node/12343/data-collection-service-driver-walkthrough-part-2>
<http://www.novell.com/communities/node/12347/data-collection-service-driver-walkthrough-part-3>
<http://www.novell.com/communities/node/12358/data-collection-service-driver-walkthrough-part-4>
<http://www.novell.com/communities/node/12359/data-collection-service-driver-walkthrough-part-5>
<http://www.novell.com/communities/node/12473/data-collection-service-driver-walkthrough-part-6>
<http://www.novell.com/communities/node/12476/data-collection-service-driver-walkthrough-part-7>
<http://www.novell.com/communities/node/12488/data-collection-service-driver-walkthrough-part-8>

Delimited Text

<http://www.novell.com/communities/node/9338/thoughts-delimited-text-driver>

eDirectory

<http://www.novell.com/communities/node/9264/error-codes-edirectory-driver-identity-manager-part-1>
<http://www.novell.com/communities/node/9347/interesting-edirectory-driver-startup-problem>

Google Apps

<http://www.novell.com/communities/node/13431/walking-through-idm-4-google-apps-driver-part-1>
<http://www.novell.com/communities/node/13433/walking-through-idm-4-google-apps-driver-part-2>
<http://www.novell.com/communities/node/13434/walking-through-idm-4-google-apps-driver-part-3>
<http://www.novell.com/communities/node/13481/walking-through-idm-4-google-apps-driver-part-4>
<http://www.novell.com/communities/node/13482/walking-through-idm-4-google-apps-driver-part-5>
<http://www.novell.com/communities/node/13483/walking-through-idm-4-google-apps-driver-part-6>
<http://www.novell.com/communities/node/13484/walking-through-idm-4-google-apps-driver-part-7>
<http://www.novell.com/communities/node/13487/walking-through-idm-4-google-apps-driver-part-8>
<http://www.novell.com/communities/node/13488/walking-through-idm-4-google-apps-driver-part-9>

GW

<http://www.novell.com/communities/node/6906/load-balancing-user-placement-groupwise-post-offices>
<http://www.novell.com/communities/node/6908/load-balancing-user-placement-groupwise-post-offices-part-2>
<http://www.novell.com/communities/node/8437/troubleshooting-groupwise-driver-external-user-creation>

JDBC

<http://www.novell.com/communities/node/4446/error-codes-novell-identity-manager-driver-jdbc-part-1-4>
<http://www.novell.com/communities/node/4447/error-codes-novell-identity-manager-driver-jdbc-part-2-4>
<http://www.novell.com/communities/node/4448/error-codes-novell-identity-manager-driver-jdbc-part-3-4>
<http://www.novell.com/communities/node/4449/error-codes-novell-identity-manager-driver-jdbc-part-4-4>
<http://www.novell.com/communities/node/2786/calling-stored-procedures-idm-jdbc-driver>
<http://www.novell.com/communities/node/2569/using-jdbc-driver-and-direct-sql>
<http://www.novell.com/communities/node/2567/using-jdbc-idm-driver-custom-descriptor-files>
<http://www.novell.com/communities/node/2292/idm-jdbc-driver-and-third-party-jdbc-driver-versioning>
<http://www.novell.com/communities/node/5078/identity-manager-jdbc-driver-state-files>

Jobs

<http://www.novell.com/communities/node/4857/identity-manager-new-feature-jobs>
<http://www.novell.com/communities/node/5907/ism-even-easier-way-call-job-policy>

Linux/Unix Bidir

<http://www.novell.com/communities/node/2217>

Managed System Gateway

<http://www.novell.com/communities/node/12304/trying-understand-managed-system-gateway-driver-idm-4-part-1>
<http://www.novell.com/communities/node/12310/trying-understand-managed-system-gateway-driver-idm-4-part-2>
<http://www.novell.com/communities/node/12314/trying-understand-managed-system-gateway-driver-idm-4-part-3>
<http://www.novell.com/communities/node/12315/trying-understand-managed-system-gateway-driver-idm-4-part-4>
<http://www.novell.com/communities/node/12330/trying-understand-managed-system-gateway-driver-idm-4-part-5>
<http://www.novell.com/communities/node/12332/trying-understand-managed-system-gateway-driver-idm-4-part-6>

Notes

<http://www.novell.com/communities/node/1335/windows+remote+loader+tasks+lotus+notes+driver>
<http://www.novell.com/communities/node/1534/setting+notes+id+password+idm+3.5>
<http://www.novell.com/communities/node/1521/upgrading+notes+driver+shim+idm+3.5>

<http://www.novell.com/communities/node/1326/configuring+lotus+notes+group+types+idm>
<http://www.novell.com/communities/node/1823/setting-multiple-mailfile-path-idm-notes-driver>
<http://www.novell.com/communities/node/1615/setting-passwords-notes-driver-remote-share>
<http://www.novell.com/communities/node/1355/signing-dsrepcfgntf-file-domino-server>
<http://www.novell.com/communities/node/1945/mapping-cn-full-name-notes-driver>
<http://www.novell.com/communities/node/1357/handling-e-mail-addresses-objects-idm-notes-and-edirectory>
<http://www.novell.com/communities/node/4224/scoping-delete-events-lotus-notes-driver-idm>

Password Notifier

<http://www.novell.com/communities/node/3017/password-notifier-driver-and-some-possible-issues>

Reporting

<http://www.novell.com/communities/node/12742/adding-idm4-reporting-support-pre-idm4-drivers-part-1>
<http://www.novell.com/communities/node/12761/adding-idm4-reporting-support-pre-idm4-drivers-part-2>
<http://www.novell.com/communities/node/12778/adding-idm4-reporting-support-pre-idm4-drivers-part-3>

Remote Loaders

<http://www.novell.com/communities/node/2994/many-faces-remote-loaders-idm>
<http://www.novell.com/communities/node/1425/troubleshooting-remote-loader-upgrade-issues>
<http://www.novell.com/communities/node/4780/updating-remote-loaders-identity-manager>

SAP

<http://www.novell.com/communities/node/9471/sap-driver-family-idm>

SAP Business Logic

<http://www.novell.com/communities/node/11551/sap-business-logic-idm-driver-walkthrough-part-1>
<http://www.novell.com/communities/node/11552/sap-business-logic-idm-driver-walkthrough-part-2>
<http://www.novell.com/communities/node/11563/sap-business-logic-driver-walkthrough-part-3>

SAP HR

<http://www.novell.com/communities/node/4562/troubleshooting-idoc-issues-sap-hr-driver-identity-manager>
<http://www.novell.com/communities/node/2792/idm-sap-hr-driver-where-get-jconnect-libraries>
<http://www.novell.com/communities/node/6351/sap-hr-driver-and-organizational-management-part-1>
<http://www.novell.com/communities/node/4144/decoding-idocs-idm-sap-driver>
<http://www.novell.com/communities/node/4482/using-sap-hr-driver-idm-a-64-bit-os>
<http://www.novell.com/communities/node/9271/error-codes-sap-hr-driver-identity-manager-part-1>
<http://www.novell.com/communities/node/9512/sap-hr-cmp-integration-driver>
<http://www.novell.com/communities/node/9537/sap-hr-cmp-integration-driver-walkthrough-part-1>
<http://www.novell.com/communities/node/9547/sap-hr-cmp-integration-driver-walkthrough-part-2>
<http://www.novell.com/communities/node/9556/sap-hr-cmp-integration-driver-walkthrough-part-3>
<http://www.novell.com/communities/node/10001/sap-hr-cmp-integration-driver-walkthrough-part-4>
<http://www.novell.com/communities/node/11274/sap-hr-cmp-integration-driver-walkthrough-part-5>
<http://www.novell.com/communities/node/11313/sap-hr-cmp-integration-driver-walkthrough-part-6>
<http://www.novell.com/communities/node/11481/sap-hr-cmp-integration-driver-walkthrough-part-7>
<http://www.novell.com/communities/node/11517/sap-hr-cmp-integration-driver-walkthrough-part-8>
<http://www.novell.com/communities/node/11518/sap-hr-cmp-integration-driver-walkthrough-part-9>

Secret Store

<http://www.novell.com/communities/node/2716/using-secret-store>

SOAP

<http://www.novell.com/communities/node/11435/using-operation-data-soap-idm-driver>
<http://www.novell.com/communities/node/11778/getting-started-building-soap-driver-idm-part-1>
<http://www.novell.com/communities/node/11807/getting-started-building-soap-driver-idm-part-2>
<http://www.novell.com/communities/node/11810/getting-started-building-soap-driver-idm-part-3>
<http://www.novell.com/communities/node/11850/getting-started-building-soap-driver-idm-part-4>
<http://www.novell.com/communities/node/11982/starting-soap-driver-idm-part-5>
<http://www.novell.com/communities/node/12022/getting-started-building-soap-driver-idm-part-6>
<http://www.novell.com/communities/node/12052/getting-started-building-soap-driver-idm-part-7>
<http://www.novell.com/communities/node/12138/getting-started-building-soap-driver-idm-part-8>
<http://www.novell.com/communities/node/12144/getting-started-building-soap-driver-idm-part-9>
<http://www.novell.com/communities/node/11852/ssl-certificate-expires-your-soap-driver>
<http://www.novell.com/communities/node/12152/error-codes-soap-driver-part-1>
<http://www.novell.com/communities/node/12158/error-codes-soap-driver-part-2>

Student Interoperability Framework (SIF)

<http://www.novell.com/communities/node/13159/walking-through-sif-driver-part-1>
<http://www.novell.com/communities/node/13160/walking-through-sif-driver-part-2>
<http://www.novell.com/communities/node/13327/walking-through-sif-driver-part-3>
<http://www.novell.com/communities/node/13336/walking-through-sif-driver-part-4>
<http://www.novell.com/communities/node/13381/walking-through-sif-driver-part-5>
<http://www.novell.com/communities/node/13382/walking-through-sif-driver-part-6>
<http://www.novell.com/communities/node/13384/walking-through-sif-driver-part-7>

User Application

<http://www.novell.com/communities/node/3477/solving-614-error-user-app>
<http://www.novell.com/communities/node/11580/introduction-workflows-and-prds-identity-manager-part-1>
<http://www.novell.com/communities/node/11582/introduction-workflows-and-prds-identity-manager-part-2>
<http://www.novell.com/communities/node/11905/interesting-user-app-not-starting-error>
<http://www.novell.com/communities/node/11931/user-app-soap-integration-activity-interesting-error>
<http://www.novell.com/communities/node/12187/using-soap-terminate-running-workflow-part-1>
<http://www.novell.com/communities/node/12200/using-soap-terminate-running-workflow-part-2>
<http://www.novell.com/communities/node/12426/user-app-roles-failing-apply-administrators>

Work Order

<http://www.novell.com/communities/node/2295/using-work-order-driver-idm>

Tools

<http://www.novell.com/communities/node/4339/review-a-great-regular-expression-development-tool-regex-buddy>

Designer

<http://www.novell.com/communities/node/1522/designer+and+xml+validation>

<http://www.novell.com/communities/node/2739/moving-idm-designer-location-linux>
<http://www.novell.com/communities/node/4826/using-global-configuration-editor-designer-30>
<http://www.novell.com/communities/node/9418/designer-35-compare-deploy-issue>
<http://www.novell.com/communities/node/11743/first-look-some-designer-40-new-features>
<http://www.novell.com/communities/node/12494/using-simulator-designer>
<http://www.novell.com/communities/node/12502/using-edirectory-browser-designer>
<http://www.novell.com/communities/node/13380/things-set-when-you-install-designer>

iManager

<http://www.novell.com/communities/node/2204>
<http://www.novell.com/communities/node/2682/workaround-404-error-imanager>
<http://www.novell.com/communities/node/3073/imanager-and-404-frameservice-not-found-error-part-2>
<http://www.novell.com/communities/node/4905/identity-manager-dashboard-imanager>
<http://www.novell.com/communities/node/4777/driver-cache-stats-idm-36-imanager-plugins>
<http://www.novell.com/communities/node/13278/finding-and-dealing-idm-driver-icons-imanager>

DXCMD

<http://www.novell.com/communities/node/2725/using-dxcmd-script>
<http://www.novell.com/communities/node/2732/another-reason-use-host-dxcmd>

ConsoleOne

<http://www.novell.com/communities/node/2985/solving-search-issues-consoleone>
<http://www.novell.com/communities/node/3327/another-workaround-consoleone-templates>

Zenworks

<http://www.novell.com/communities/node/1410/quick+workaround+flashing+desktop+bug>

ZAM

<http://www.novell.com/communities/node/1344/collecting+data+file+info+zenworks+asset+management>
<http://www.novell.com/communities/node/1367/scanning+registry+key+settings+zenworks+asset+management>
<http://www.novell.com/communities/node/1413/open+call%3A+troubleshooting+zenworks+asset+management+connectivity>
<http://www.novell.com/communities/node/1681/upgrading+zenworks+asset+management/inventory+server+7+7.5>
<http://www.novell.com/communities/node/1711/collecting+data+file+info+zenworks+asset+management>
<http://www.novell.com/communities/node/3215/inventory-multiple-machines-once-zenworks-asset-management>

ZCM

<http://www.novell.com/communities/node/4145/setting-zenworks-control-center-timeout-value>

ZfH

<http://www.novell.com/communities/node/6945/zenworks-handheld-management-figuring-out-console-one-snapins>